

—— // ZTERMINAL // ——

zterminal

v0.29.9

THE COMPANION

the grid · the renderer · the extensions



MENKE
TECHNOLOGIES

Copyright and Attribution

zterminal — the grid, the renderer, the extensions

Copyright © 2026 MenkeTechnologies. All rights reserved.

No part of this book may be reproduced, distributed, or transmitted in any form or by any means without the prior written permission of the publisher, except for brief quotations in reviews.

Published by MenkeTechnologies.

The trademarks and product names referenced in this book are the property of their respective owners. This book is an independent work: it is not affiliated with, authorized by, sponsored by, or otherwise endorsed by any terminal emulator, terminal-standards body, or other project named in these pages.

The software this book describes

`zterminal` is proprietary MenkeTechnologies software. Its `LICENSE` states that the software “is proprietary and confidential”, that it is “part of the MenkeTechnologies paid stack and is distributed under the SPDX identifier `UNLICENSED`”, and that no license — express or implied — is granted to use, copy, modify, merge, publish, distribute, sublicense, or sell copies of it without the prior written permission of MenkeTechnologies.

Reading this book conveys no license to the software it describes.

`zterminal` implements published terminal behaviour — the ECMA-48 control sequences, the DEC private modes, and the de-facto extensions the ecosystem has converged on. Those specifications and conventions belong to their own bodies and authors; the implementation described here is original, and no other terminal's source is vendored or derived from.



MTPUBLISHING



MTAPP

Contents

Copyright and Attribution	1
Before You Begin	6
A Terminal-Emulator Primer	8
Chapter 1 — The Pitch	14
Chapter 2 — Lineage and Honesty	16
Chapter 3 — The Grid	18
Chapter 4 — The Renderer	21
Chapter 5 — Damage Tracking and Frame Scheduling	23
Chapter 6 — Escape Sequence Support	26
Chapter 7 — The Input Path	28
Chapter 8 — The Configuration Model	32
Chapter 9 — Window	36
Chapter 10 — Font	38
Chapter 11 — Grapheme Clusters, Shaping, and Ligatures	40
Chapter 12 — Colors	42
Chapter 13 — Cursor, Bell, Selection, Scrolling	45
Chapter 14 — Terminal and Mouse	48
Chapter 15 — Hints	49
Chapter 16 — Keyboard and Bindings	52
Chapter 17 — Debug	54
Chapter 18 — Vi Mode and Search	56
Chapter 19 — The Control Panel	59
Chapter 20 — The Settings Tab	62
Chapter 21 — Profiles	65
Chapter 22 — The Dashboard	66

Chapter 23 — Command and History Palettes	68
Chapter 24 — Native Splits and Layouts	70
Chapter 25 — Exposé, MRU, and the Quake Dropdown	72
Chapter 26 — Inline Graphics	74
Chapter 27 — Shell Integration	77
Chapter 28 — The tmux Client	80
Chapter 29 — Session Restore and Persistence	82
Chapter 30 — tmux Broadcast, Search, and Process Tree	85
Chapter 31 — Native Panes versus tmux Panes	86
Chapter 32 — Env Vars, Snippets, Triggers, Recent Dirs	89
Chapter 33 — IPC	91
Chapter 34 — The Inventions	94
Chapter 35 — The Process Model	96
Chapter 36 — The Event Loop	98
Chapter 37 — Performance	101
Chapter 38 — Selection, Clipboard, and Copy	104
Chapter 39 — Security and Trust Boundaries	106
Chapter 40 — Terminfo and Capability Advertisement	108
Chapter 41 — Building and Installing	110
Chapter 42 — Migrating an Existing Setup	112
Chapter 43 — The Line Discipline	116
Chapter 44 — Encoding, Malformed Bytes, and the C1 Range	120
Chapter 45 — Backpressure and the Shape of a Firehose	123
Chapter 46 — Nesting: SSH, tmux, Containers, and Capability Attenuation	126
Chapter 47 — Colour, Contrast, and Legibility	129
Chapter 48 — Writing a Program That Behaves Well Here	132
Chapter 49 — What a Terminal Cannot Do	135
Chapter 50 — Rasterization: Coverage, Gamma, and the Weight of a Stem	138
Chapter 51 — Presentation: the Swap, the Compositor, and the Idle Window	141
Chapter 52 — Locale, Ambiguous Width, and Text That Runs the Other Way	144

Chapter 53 — Windows: the Pseudoconsole and the Chapters That Do Not Apply	147
Chapter 54 — Conformance: How You Know an Emulator Is Right	150
Chapter 55 — Accessibility, and the Surface a Screen Reader Cannot See	153
Chapter 56 — Failure, and the Blast Radius	155
Chapter 57 — Where It Sits in the Stack	158
Chapter 58 — The Configuration Derive Macro	160
Chapter 59 — The Command Ledger	166
Chapter 60 — Retroactive Relational Queries Over Command Output	170
Chapter 61 — Fan-Out Divergence	174
Chapter 62 — ztmux-core: the Wire, Byte by Byte	178
Chapter 63 — The Test Corpus, Concretely	182
Chapter 64 — The Build Pipeline	186
Chapter 65 — The Grid’s Storage: a Ring, a Zero, and Lazy Rows	189
Chapter 66 — Damage, Line by Line	192
Chapter 67 — The Text Pipeline: Atlas, Instances, and Two Passes	194
Chapter 68 — The Parser You Did Not Write	197
Chapter 69 — The Scanner in Front of the Parser	200
Chapter 70 — OSC 133 in Practice	203
Chapter 71 — Selection and the Clipboard, Concretely	206
Chapter 72 — Scrollback, Reflow, and Search	210
Chapter 73 — The Split Tree	214
Chapter 74 — Keys, Modifiers, and the Bytes They Produce	217
Chapter 75 — The Eight, Part One: A Carriage Return That Is Not There	221
Chapter 76 — The Eight, One at a Time	224
Chapter 77 — The Reporter That Panics	227
Chapter 78 — What Forty-Five Recordings Actually Pin	231
Chapter 79 — Reads That Land in the Wrong Place	234
Chapter 80 — The Parser’s Limits	238
Chapter 81 — Resize, Reflow, and the Column That Never Comes	241
Chapter 82 — The tmux Client That Is Not a Client	246

Chapter 83 — Things That Pass	249
Appendix A — Command-Line Reference	252
Appendix B — Configuration Reference	253
Appendix C — Default Keybindings	256
Appendix D — Escape Sequence Support	258
Appendix E — The ~/.zterminal Directory	261
Appendix F — Glossary	262
Appendix G — Troubleshooting	264
Appendix H — Worked Configurations	267
Appendix I — The Escape Sequence Cookbook	270
Appendix J — Environment Variables	272
Appendix K — An Annotated Session	275
Colophon	278
Reproducing the Book’s Claims	279
Also from MenkeTechnologies	280

Before You Begin

This is the book about `zterminal` — the one you read in a chair, front to back, to understand not only *what* `zterminal` does but *why* it is shaped the way it is. It is not a man page. The man pages — `zterminal(1)`, `zterminal-msg(1)`, `zterminal(5)`, `zterminal-bindings(5)`, and `zterminal-escapes(7)` — enumerate every flag, every configuration field, every default binding, and every escape sequence the emulator answers, and you reach for them when you need one fact fast. This book is the other thing: the continuous account of the program, from the byte that arrives on the PTY to the pixel that lands on the screen, and of the layer of features built on top of that core that make `zterminal` a terminal *environment* rather than a terminal *window*. It is also exhaustive where the man pages are terse: every setting in `zterminal.toml` is walked here with its type, its values, its default, and its effect, and every feature in the extension layer gets a chapter or a section of its own.

`zterminal` is a fast, GPU-accelerated, cross-platform terminal emulator and part of the MenkeTechnologies stack. It runs on Linux, BSD, macOS, and Windows. Two sentences describe it accurately and together explain everything that follows. The first: its core is a faithful, high-performance terminal emulator in the Alacritty lineage — a `vte`-based xterm-compatible VT parser feeding an OpenGL ES 2.0 glyph-atlas renderer, with truecolor, scrollback, a built-in vi mode, search, and hints. The second: on top of that minimal, fast core MenkeTechnologies builds an integrated extension suite — an in-process control-panel GUI built entirely from the shared `zgui-core` component library, native i3-style tiling, inline GPU graphics in three protocols, OSC 133 shell integration, a direct tmux wire-protocol client that *live-edits* a running tmux server, profiles, snippets, output triggers, a telemetry dashboard, and more — none of which the core emulator it descends from has, and all of which ship inside the one binary.

This book is honest about that division, because the honesty is the point. The emulator core is not reinvented from a specification; it is built on the Alacritty terminal lineage, the same way the other books in this series credit their foundations — vimlrs ports Neovim’s eval engine, zshrs ports zsh’s C, `zterminal` builds on Alacritty’s VT and renderer. The repository’s own inventions file is explicit: “`zterminal` is an Alacritty derivative; base VT/grid/search/ hints are **not** claimed.” The configuration format, the escape-sequence support, the default keybindings, and the vi-and-search model are that inherited core, documented faithfully and completely. The original work — the work that makes `zterminal` a MenkeTechnologies product rather than a rebuild — is the extension layer, and the book devotes its second half to it, ending with a chapter that walks every documented invention with its source basis.

Everything here is grounded in the program and its documentation. Every configuration snippet is the real TOML the emulator parses; every escape code is one the parser actually handles, at the support level the escape-sequence table records; every keybinding is a default the binding table ships; and every claim about the control panel and the inventions is grounded in the running app or in a cited source file. `zterminal` is young and moves fast — its version is `v0.29.9` at the time of writing — and where a number or a behaviour is provisional this book marks it rather than rounding up. `zterminal` is the work of MenkeTechnologies, built on the foundation of the Alacritty authors and the VT and xterm tradition before them, and this book is grounded in that code.

Provenance. Every figure in this edition was measured on `zterminal` commit `0f885f23b1efe54651dba8deb759865059ea88f3` (`zterminal/Cargo.toml` version `0.29.9`), on macOS aarch64, with the parser figures taken from `vte 0.15.0` as resolved by that commit’s `Cargo.lock`. Recover the tree and re-run any measurement in the book with:

```
git -C zterminal checkout 0f885f23b1efe54651dba8deb759865059ea88f3
```

The platform matters for the test counts: four `#[test]` attributes are gated to Windows and Linux and do not compile here, which Chapter 63 works through.

One unnumbered section comes first. Everything after it assumes you know what a terminal emulator actually is — the pty and who holds each end of it, the byte stream and the escapes inside it, the grid, the alternate screen, and the frame — so if you have used a terminal for years without ever being told what the program between your keyboard and your shell is doing, read the primer; if you already know, skip straight to Chapter 1.

Let's start.

A Terminal-Emulator Primer

Everything after this section assumes you know what a terminal emulator *is* — not how to use one, but what the program is doing between your keyboard and your shell. If you already know what a `pty` is, which end of it the emulator holds, and why an emulator draws a grid rather than a document, skip to Chapter 1. If you do not, this is the half hour that makes the rest of the book legible, because the internals chapters assume every noun below without redefining it. It is deliberately not a reference: Appendix D is the escape-sequence support table, Appendix F the glossary, Appendix I the cookbook of sequences you can paste at a prompt.

One disclosure first, because a primer that invents its evidence is worse than one with none. `zterminal` opens a window, so nothing here drives it interactively and no screenshot or session transcript has been imagined. What was run, on macOS aarch64, is every `console` block below, verbatim: `tty`, `infocmp`, four `tput` capability queries, two `printf` pipelines through `cat -v` and `od -c`, two `zsh/zpty` pseudo-terminals spawned to show `tty`, `stty size` and `stty -a` from the far side of a real `pty`, and the installed emulator's `--version`, which does run headless — that bundle reports `0.29.8` where the tree this book measures is `0.29.9`, so the version line trails the provenance commit by a patch and is quoted only to show a headless invocation exists. Everything else said about `zterminal` below is sourced from the source tree with the file cited inline, not from a running window.

The problem it solves, and why “emulator”

A terminal used to be furniture: a physical device with a keyboard, a screen, and a serial cable, wired to a computer elsewhere in the building. It had two jobs. Press a key, it puts a byte on the wire. A byte comes back, it either prints a character at the current position or — if the byte is a control code, or the start of a short escape-prefixed command — does something structural: move the cursor, erase to end of line, scroll up a row, switch to reverse video. That is the whole contract, and it is worth noticing how small it is: the terminal had no idea what program was on the other end, and the program had no idea what the screen looked like.

The hardware is gone; the contract is not, because every program written for it still speaks it — your editor, your pager, your shell's line editor, `htop`, a package manager's progress bar. A *terminal emulator* pretends to be that vanished furniture well enough that those programs cannot tell: it draws a rectangle of characters, accepts the same byte stream, obeys the same commands, and sends back the same key codes. Chapter 2 is the account of which parts of that contract `zterminal` inherits and from where; Chapter 54 is how you would prove an emulator gets it right.

Two ends of one kernel object

The wire is the piece people miss. There is no serial cable, so the kernel provides a stand-in: a **pseudo-terminal**, universally shortened to *pty* — a pair of file descriptors that are two ends of the same object, so whatever is written into one end is read out of the other. The emulator holds the **primary** end and never gives it away. The **secondary** end is handed to the program the emulator launches — your shell — as its standard input, output, and error, and it *is* a terminal as far as the kernel is concerned. That asymmetry is the entire architecture: the shell believes it is talking to a terminal because from its side it is, the descriptor answering terminal ioctls, reporting a size, carrying a device name; the emulator is simply the process on the other end, reading what the shell writes and writing what the keyboard produces.

A process whose standard input is a pipe has no terminal at all:

```
$ tty
not a tty
```

A process spawned onto a freshly created pty does:

```
$ zsh -f -c 'zmodload zsh/zpty; zpty p "tty; stty size"; zpty -r p'
/dev/ttys015
0 0
```

`zsh/zpty` allocates a pty, spawns the command on the secondary end, and lets the calling shell read the primary end — the shape of a terminal emulator, minus the window. The `0 0` is the second lesson: a pty has a *window size* in rows and columns, but the kernel does not invent it. The holder of the primary end declares it, and here nobody did. In a real emulator that declaration happens twice — when the pty is created, and again every time you drag the window edge, at which point the emulator writes the new dimensions down and the kernel raises `SIGWINCH` in the foreground program so a full-screen application can relay out. In `zterminal` that is `openpty` with a `Winsize` at `zterminal_core/src/tty/unix.rs:196` and the `TIOCSWINSZ` ioctl at `zterminal_core/src/tty/unix.rs:414`. The same file makes the child a session leader with `setsid` and claims the pty as its controlling terminal with `TIOCSCTTY`, which is what makes `Ctrl+C` in that window reach that shell and no other. Chapter 35 is the process model in full.

One more occupant of the pty is neither the emulator nor the shell: the kernel's **line discipline**, a filter on the secondary end. It is why a bare `cat` lets you backspace over a typo, why `Ctrl+C` becomes a signal rather than a byte, and why a password prompt shows nothing as you type. Its settings are visible from inside the pty:

```
$ zsh -f -c 'zmodload zsh/zpty; zpty p "stty -a|head -4"; zpty -r p'
speed 9600 baud; 0 rows; 0 columns;
lflags: icanon isig ixtext -echo echoe -echok echoke -echonl echoctl
        -echopr -altwerase -noflsh -tostop -flusho -pendin -nokerninfo
        -extproc
```

`icanon` is canonical mode — the kernel buffers a whole line and does the editing itself. `isig` turns the interrupt character into a signal. `-echo` is off only because `zpty` leaves it off unless given `-e`; under an interactive shell it is on until the shell's line editor switches it off to take over. The `9600 baud` is a fossil with no meaning on a pty. None of this is emulator code, and Chapter 43 exists largely so you stop blaming the terminal for it.

The byte stream, and the escapes inside it

What flows through the pty is one undifferentiated stream of bytes. Printable text and commands share it, and commands are marked by starting with the escape character, `0x1b`. That is all “escape sequence” means: a run of bytes introduced by escape, terminated by a byte saying the command is over, with parameters between. Here is text with a colour command embedded in it, printed with the escapes made visible instead of obeyed:

```
$ printf 'A\033[31mB\033[0m\n' | cat -v
A^[[31mB^[[0m
```

`^[` is how `cat -v` writes the escape byte. The stream is: the letter `A`; then `ESC [ 3 1 m`, “set the foreground colour to index 1”; then the letter `B`; then `ESC [ 0 m`, “reset attributes.” Two characters of text and two commands in one stream, distinguishable only by the escape byte. Piped to a terminal instead of `cat -v`, the commands vanish from the display and the `B` turns red. At byte level there is nothing else going on:

```
$ printf '\033[2J\033[H' | od -c
0000000 033  [  2  J 033  [   H
0000007
```

Seven bytes: erase the whole display, then move the cursor home. That is what `clear` sends.

The family starting `ESC [` is CSI — Control Sequence Introducer — covering cursor movement, erasing, scrolling regions, colours, and mode switches. A second starts `ESC ]` and is OSC, Operating System Command,

carrying strings rather than numbers: the window title, a clipboard payload, a hyperlink, a working-directory announcement. A third, DCS, carries bulk data such as an image. The component that walks these bytes and decides which is which is the **parser**, and it is a state machine rather than a pattern match, because the stream arrives in arbitrarily sized chunks and a sequence can be split across two reads. `zterminal` does not write its own: it uses the `vte` crate, pinned at `0.15.0` in `zterminal_core/Cargo.toml:29`. Chapter 68 is that parser, Chapter 69 the scanner `zterminal` puts *in front* of it, Chapter 6 and Appendix D the catalogue of what the emulator answers.

Terminfo: how a program knows which bytes to send

Something has to tell a program which sequences *this* terminal understands, because real terminals disagreed. That is **terminfo**: a compiled database, one entry per terminal type, keyed by the `TERM` environment variable, mapping capability names to the bytes that invoke them.

```
$ infocmp xterm-256color | head -6
#   Reconstructed via infocmp from file: /usr/share/terminfo/78/xterm-256color
xterm-256color|xterm with 256 colors,
    am, bce, ccc, km, mc5i, mir, msgr, npc, xenl,
    colors#256, cols#80, it#8, lines#24, pairs#32767,
    acsc=``aaffggiijjkkllmmnnnooppqqrrssttuuvvwwxxyyzz{||}~~,
    bel=^G, blink=\E[5m, bold=\E[1m, cbt=\E[Z, civis=\E[?25l,
```

`bold=\E[1m` says “to embolden, send escape, bracket, one, m.” `colors#256` is a number rather than a string — a fact about the terminal rather than an instruction. `tput` is the command-line front end to the same lookup, turning a capability name plus arguments into the exact bytes:

```
$ tput -Txterm-256color cup 5 10 | cat -v
^[[6;11H
$ tput -Txterm-256color setaf 1 | cat -v
^[[31m
```

`cup 5 10` — row 5, column 10, counting from zero — came out as `6;11`, because the wire protocol counts from one. That off-by-one is not a bug in either layer; it is the seam between a library’s convention and the VT protocol’s, and the kind of detail terminfo exists to hide. `zterminal` ships an entry of its own, `extra/Zterminal.info`, and identifies as `xterm-256color` when it is not installed. Chapter 40 covers that, and the modern alternative: rather than trust a database that may be stale on the far side of an SSH hop, a program can ask the terminal what it supports and read the reply back through the same `pty`.

The grid, the cursor, and the second screen

A terminal emulator does not hold a document. It holds a **grid**: a fixed number of rows by a fixed number of columns, every position a cell, every cell holding one character’s worth of content plus its attributes — colours, bold, underline. There is no notion of a paragraph, a word, a line that wraps because it is too long, or a line still being edited. There are cells, and there is a **cursor**, the coordinate the next printed character lands on. Nearly every escape sequence is an operation on those two things: move the cursor, print here and advance, erase to the end of the row, scroll rows 3 through 20 up by one. When a full-screen program repaints it is not sending a picture; it is issuing a few hundred of those operations. Chapter 3 is the grid as a data model and Chapter 65 how it is actually stored — a ring buffer, so scrolling costs a pointer move rather than a copy of the screen.

Scrollback is the history above the visible area: rows that scrolled off the top and were retained instead of discarded. It belongs to the emulator, not to any program — the shell has no idea it exists and cannot read it — which is why scrolling up shows output from a command that already exited. Chapter 72 covers it, along with what happens when the window is resized and stored rows must reflow into a new width.

The **alternate screen** is a second grid, and it explains a behaviour everyone has seen without explaining. The primary accumulates scrollback and is where your shell lives; the alternate is a scratch screen of exactly

the visible size, with no scrollback, and switching to it leaves the primary untouched. Full-screen programs switch on entry and back on exit, which is why quitting `vim` or `less` restores your prompt and history exactly as they were, and why you cannot scroll back through what was on screen inside them. The switch is one mode toggle, which terminfo names:

```
$ tput -Txterm-256color smcup | cat -v
^[[?1049h
$ tput -Txterm-256color rmcup | cat -v
^[[?1049l
```

`1049` is the private mode number; `h` sets it and `l` clears it. That is the whole mechanism.

The shell is not the terminal

This distinction causes the most confusion, so state it flatly. The terminal emulator draws a grid and moves bytes. The **shell** is an ordinary program running inside it — `zsh`, `bash`, `zshrs` — with no privileged relationship to the emulator beyond being the process it happened to launch. Tab completion, command history, `Ctrl+R` search, `$PATH` lookup, globbing, pipelines, job control, your prompt: all shell. The emulator never sees a command; it sees the keystrokes you typed and the bytes the shell chose to echo back. Font, colours, window size, scrollback, selection, copy and paste, the title bar, and which byte sequence the Home key produces: all emulator. The shell cannot change your font and the terminal cannot complete a filename.

The middle ground is small and explicit: the shell can ask the emulator for things by writing OSC sequences into its own standard output — set the title, put this on the clipboard, mark where the prompt starts. That last is OSC 133, the shell-integration protocol, and it is the only way an emulator can know which region of the grid was a command's output rather than a wall of undifferentiated cells (Chapters 27 and 70). Absent such a sequence, the emulator's knowledge of your session is exactly: bytes arrived, cells changed.

What a terminal emulator is not responsible for

Three boundaries, because the internals chapters assume them.

It is **not a multiplexer**. Sessions that survive a closed window, one screen split among several programs, attaching from another machine — that is `tmux` or `ztermux`, a separate process holding its own ptys, with your terminal reduced to a view onto it. An emulator can implement splitting natively, and `zterminal` does, but that is an added feature and not part of the contract; Chapter 31 is about the difference between the two kinds of split.

It is **not the font engine**. The emulator decides which glyph goes in which cell and how wide a cell is; turning an outline into coverage values is a rasterizer's job and choosing which glyphs represent a run of text is a shaper's job. It owns the cell grid and the atlas of already-rasterized glyphs; it does not own hinting or gamma (Chapters 10, 11, 50).

And it is **not a general graphics surface**: its addressing unit is a cell. Inline image protocols exist, and `zterminal` implements three (`zterminal_core/src/graphics/sixel.rs`, `kitty.rs`, `iterm2.rs`), but they place pixels *over* a rectangle of cells the grid still accounts for. Chapter 49 is the honest list of what the model cannot express at all.

Why any of this needs a GPU

A grid of characters sounds like the least demanding thing a computer could draw, and per frame it nearly is — but the load is not in one frame. An 80×24 grid is under two thousand cells; a full-screen window on a modern display is easily 250 columns by 70 rows, seventeen thousand cells, each a textured quad with a foreground colour, a background fill, and possibly an underline. Redrawn sixty or a hundred and twenty times a second, that is a few million small draw operations per second whose only real requirement is that they all be issued

cheaply, which is what a GPU is for. The mechanism every fast emulator uses is a **glyph atlas**: every distinct glyph, at the current size, is rasterized once into a texture, and a frame becomes a list of instances — for each visible cell, which atlas rectangle, at which screen position, in which colour. The CPU builds the list, the GPU composites it, and the rasterizer runs only when a glyph is seen for the first time. Chapter 4 is zterminal's renderer, Chapter 67 the atlas and the two-pass draw.

Damage tracking and frame scheduling

The GPU handles the wide case. Two disciplines handle the frequent one, and both are about *not* drawing.

Damage tracking is the observation that the renderer should not ask “what does the grid look like now” but “what changed since the frame already on screen.” A prompt with a blinking cursor changes a single cell; a log scrolling one line changes one row plus the shift; an editor rewriting its status bar changes that bar. Tracking dirty rows and the changed column span within them means a redraw uploads a few dozen cells instead of seventeen thousand. Only a resize or a switch between the primary and alternate screens forces the whole grid to be reconsidered, and both are rare enough to be allowed to cost something. zterminal keeps this in [zterminal/src/display/damage.rs](#); Chapter 5 is the concept, Chapter 66 the implementation.

Frame scheduling answers a question that catches people out: if a program writes a thousand times in the span of one display refresh, how many frames should the terminal draw? One. Bytes from the pty mutate the grid and accumulate damage continuously, but the *draw* is deferred to the display's cadence, so all damage since the last frame is coalesced and drawn once. Most intermediate states of the grid are never drawn at all — overwritten before a frame is due — and that is correct, because they were never on screen long enough to be seen. It is why `cat` of an enormous file is bounded by how fast the grid can be mutated rather than by how many times the screen could have changed. zterminal's timers, including the `Frame` topic, live in [zterminal/src/scheduler.rs](#); Chapter 5 covers the policy, Chapter 51 what happens afterwards at the compositor.

What is different about zterminal specifically

Three things are worth knowing before Chapter 1, and all three follow from the architecture above rather than being bolted to its side.

The core is inherited, deliberately, and the book says so. The grid, the VT parsing, the renderer, search, and hints come from the Alacritty lineage, and the parser is the `vte` crate at `0.15.0`. Chapter 2 is the full accounting, and the repository's own inventions file states that the base VT, grid, search and hints are not claimed.

The additions sit inside the same loop, not beside it. A scanner runs ahead of the VT parser and peels the three inline-image protocols and OSC 133 shell marks out of the byte stream before the parser sees them, because `vte` discards APC sequences and its DCS hooks are no-ops ([zterminal_core/src/graphics/scanner.rs](#), Chapter 69). Native splits own several ptys and scissor each pane into a region of the one GL surface (Chapters 24 and 73). The tmux client is a real wire-protocol client — `crates/ztmux-core`, its own repository consumed as a path dependency — with no `tmux` subprocess anywhere (Chapters 28 and 62). This is Chapter 1's “one process, one loop, one config” restated at the level of the mechanism this primer described.

It is a GUI program that exposes a non-GUI way in. The binary answers headlessly:

```
$ /Applications/Zterminal.app/Contents/MacOS/zt --version </dev/null
zterminal 0.29.8 (7c9b6e1332)
```

and a running instance is addressable over an IPC socket through the `msg` subcommand, which creates windows and reads and writes live configuration without touching a file. Chapter 33 is that surface; Appendix A is the flag reference.

Where to go from here

You now know the nouns the rest of the book uses without introduction: pty, primary and secondary ends, line discipline, byte stream, escape sequence, CSI and OSC, parser, grid, cell, cursor, scrollbar, alternate screen, damage, frame. Chapter 1 states what zterminal is and why it exists; Chapters 3 through 5 are the grid, the renderer, and the frame loop this primer sketched, in full; Chapters 35 and 36 are the process model and the event loop that hold them together. Appendix F is the glossary if a word gets away from you, and Appendix K walks one real session from keystroke to pixel — the best thing to read if you want to see all of it moving at once.

Chapter 1 – The Pitch

A terminal emulator is, on paper, a solved problem. The VT100 is older than most of the people writing software today; the escape sequences that move a cursor and set a colour were standardized while mainframes still filled rooms; and every operating system ships a terminal that works. The temptation, looking at that, is to conclude that a new terminal emulator can only ever be a marginal thing — a little faster, a little prettier, a slightly different set of defaults. For most new terminals that conclusion is correct, and the MenkeTechnologies project bar says so bluntly: a faster clone of an existing tool is not worth building. “World’s fastest at a duplicate” is dead on arrival.

zterminal is not pitched as a faster terminal. Its core *is* fast — that is inherited, and the inheritance is deliberate — but speed is the floor, not the pitch. The pitch is the integration. Consider what a working terminal user actually assembles around a bare emulator: a multiplexer for splits and sessions (tmux), a separate tool or protocol for inline images (kitty, or a sixel viewer), a shell-integration script for prompt marks (the iTerm2 or FinalTerm snippets), a session-restore plugin (tmux-resurrect), a dropdown-terminal daemon (a Quake-style hotkey window), an output-trigger tool, and a pile of configuration spread across all of them. Each of those is a separate program, with its own config language, its own failure modes, and its own seams where it does not quite talk to the others.

zterminal’s argument is that this assembly is itself the problem, and that the pieces belong inside the emulator, sharing one process, one config file, one event loop, and one GPU surface. It tiles natively, so splits are the emulator’s own panes rather than a multiplexer’s redraw of a single grid. It renders images itself, in all three common protocols, as GPU textures blitted over the cells. It reads the tmux server’s wire protocol directly — and goes further, live- editing that server’s options, paste buffers, and keybindings from its own GUI, which no other terminal does. It parses OSC 133 prompt marks out of the byte stream, so prompt-jumping and output-selection work without a plugin. It snapshots the whole terminal-plus-tmux state into switchable profiles. And it puts all of it behind a single in-process control panel and a searchable command palette, so none of it requires memorizing a keybinding.

That is the world’s-first-and-fastest test applied honestly. The *fastest* leg is the inherited core: a GPU renderer and a tight VT parser that already compete with the fastest emulators. The *first* leg is the combination and several of its pieces individually — a terminal that live-edits a running tmux server, that stores tmux state in profiles, that ships a real telemetry dashboard. The repository’s inventions file claims those three as outright firsts, and the broader stack’s inventions catalogue lists more than a dozen additional candidates. No single program in the terminal ecosystem occupies the same intersection. The rest of this book is the detailed account of both legs: the core it inherits and documents faithfully, and the layer it adds and owns.

The assembly tax, itemized

It is worth making the “assembly” argument concrete, because the cost it names is easy to wave at and easy to underestimate. A terminal power user’s real environment is not one program; it is a stack of them, each with its own configuration language, its own update cycle, and its own seams:

- a **multiplexer** (tmux) for splits, sessions, and persistence — its own `.tmux.conf`, its own key prefix, its own copy-mode;
- an **image protocol or viewer** (a sixel tool, `kitten icat`) for inline graphics — which the multiplexer then filters unless passthrough is configured;
- a **shell-integration script** (the iTerm2 or FinalTerm snippets) for prompt marks — sourced per shell, guarded per terminal;

- a **session-restore plugin** (tmux-resurrect, tmux-continuum) — its own save format, its own restore whitelist;
- a **dropdown-terminal daemon** for a Quake-style hotkey window;
- an **output-trigger tool** for reacting to matched text;
- and a **pile of glue** — aliases, keybindings, environment exports — holding the above together.

Each item is a separate failure mode and a separate place the pieces do not quite talk. The image tool's escapes get eaten by the multiplexer. The restore plugin does not know about the shell marks. The dropdown daemon does not share the multiplexer's sessions. zterminal's thesis is that this list is not a rich ecosystem to be celebrated but a *tax* to be collapsed: every item above has a chapter in this book's second half where it is instead a feature of the one process, sharing the one config file and the one event loop. The pitch is not that any single one of these is novel in isolation — several are not — but that assembling them out of separate programs is the problem, and integrating them is the product.

What zterminal is, precisely

zterminal is a Rust terminal emulator. It opens a window, allocates a pseudo-terminal, spawns your shell into it, and runs an event loop that does three things forever: read bytes from the PTY and feed them to a VT parser that mutates a grid of cells; read input events from the window and translate them to bytes written back to the PTY (or to actions); and, when the grid is dirty, upload the changed cells to the GPU and draw a frame. That loop is the whole of a terminal emulator, and zterminal's core runs it the way the Alacritty lineage does — minimally, with the parsing in `vte` and the drawing in OpenGL ES 2.0.

Onto that loop zterminal grafts the extension suite. The grafts are not afterthoughts bolted to the side; they sit in the same loop. The split tree owns multiple PTYs and scissors each pane into a sub-viewport of the same GL surface. A single scanner sits *ahead* of the VT parser, peeling graphics and OSC 133 sequences out of the same byte stream before the parser sees them. The tmux client opens a second socket to the tmux server and folds its events into the same loop. The control panel is an embedded WebView, served over a custom `zterminal://` protocol and built only from the shared `zgui-core` component library, that reads and writes the same live configuration the emulator hot-reloads. One process, one loop, one config.

Who it is for

zterminal is built for someone who lives in the terminal — specifically, for the MenkeTechnologies development workflow, where the shell is the IDE, the build system, the deploy tool, and the orchestrator, and where many synchronized shell instances run at once. Anything that slows the terminal slows that whole pipeline, which is why the core is the fast inherited one and why the extensions are integrated rather than shelled out to subprocesses. The terminal is the host for the rest of the stack — `zshrs` runs inside it, `zpw` paints inside it, the GUI applications embed the same shared terminal component — and zterminal is the window all of it lives in. Chapter 57 returns to that placement; the chapters between here and there are the terminal itself, in full.

Chapter 2 – Lineage and Honesty

Every faithful book in this series opens its second chapter by naming its foundation, because faithfulness to a foundation is only meaningful if you say what the foundation is. vimlrs names Neovim's `eval/*` tree and calls the C source its specification. zshrs names zsh. zterminal's foundation is the Alacritty terminal lineage, and this chapter says what that means, what is inherited from it, and what is not — because the rest of the book's originality claims only hold if the inheritance is stated plainly first.

The evidence of descent

You do not have to take the lineage on assertion; it is legible in the shipped artifacts. The configuration file is TOML with exactly the table structure — `[general]`, `[window]`, `[scrolling]`, `[font]`, `[colors]`, `[bell]`, `[selection]`, `[cursor]`, `[terminal]`, `[mouse]`, `[hints]`, `[keyboard]`, `[debug]` — that the Alacritty lineage uses, down to field names like `option_as_alt`, `draw_bold_text_with_bright_colors`, and `ipc_socket`. The escape-sequence support table in `zterminal-escapes(7)` is the Alacritty support matrix: the same `IMPLEMENTED` / `PARTIAL` / `REJECTED` vocabulary, the same charset caveat, the same set of private modes under `CSI ? h`. The man pages still credit the upstream maintainers in their `MAINTAINERS` sections. The renderer is OpenGL ES 2.0 with a glyph atlas; the parser is `vte`. The core crate is named `zterminal_core`. This is not a terminal written from the VT spec; it is a terminal carried forward from a specific, excellent open-source lineage, and the honest framing — the framing this series always uses — is that the core is *ported and inherited*, not invented.

That matters for the project's own rules. The MenkeTechnologies bar kills “X but faster” and “X but in Rust” outright; a faster Alacritty would be the awkrs trap, dead on the first test. zterminal survives that test only because of the second half of the book — the extension layer that does not exist upstream. So the lineage chapter is not a hedge; it is the setup for the originality claim. You cannot credibly say “this is the first terminal that live-edits a tmux server” while pretending the VT core underneath sprang from nothing. Name the foundation, then show what was built on it. The repository's own `docs/INVENTIONS.md` does exactly this, opening with the disclaimer that “base VT/grid/search/hints are **not** claimed.”

What faithful porting means here

“Inherited” is a precise word in this series, and it is worth saying what it does and does not mean for zterminal. It does not mean a fork left to drift. It means the emulator core — the VT parser's escape handling, the grid's semantics, the renderer's glyph-atlas architecture, the configuration schema, the vi-and-search model — is carried forward from the Alacritty lineage as the *specification* to be matched, the same way vimlrs treats Neovim's C as the spec and zshrs treats zsh's. The support table's exactness (Chapter 6) is the evidence: a terminal that half-implemented its inheritance would not be able to state, sequence by sequence and parameter by parameter, precisely what it handles and what it rejects. That precision is only possible because the behaviour is faithful to a known reference rather than approximated from a vague memory of “what xterm does.”

The discipline has a direct payoff for the reader of this book. Because the core is faithful, the first fifteen chapters can document it *as the reference documents it* — the same table names, the same field names, the same escape statuses — and a fact learned here transfers to the broader VT and xterm tradition rather than being a zterminal-only quirk. And because the inheritance is stated rather than hidden, the originality claims of the second half can be sharp: every invention names its basis (Chapter 34), and none of them has to pretend

the ground it stands on was also invented. Faithful porting is not a smaller ambition than rewriting; it is the thing that lets the ambition — the extension layer — rest on a floor that is known to be correct.

What is inherited

These are inherited from the Alacritty lineage and documented faithfully in Chapters 3–18. They are not zterminal's inventions, and the book does not claim them as such: the `vte`-based VT parser and xterm escape handling; the OpenGL ES 2.0 glyph-atlas renderer and built-in box drawing; the TOML configuration format and every table in it; the built-in vi mode and search; terminal hints; the keyboard/mouse binding model; and the `zt msg` IPC mechanism with its migrate subcommand.

What is original

These are the MenkeTechnologies extension layer, documented in Chapters 19–34 and catalogued with their source basis in Chapter 34. They are zterminal's own work, integrated into the one binary: the in-process `zgui-core` control panel and its managers; the live tmux-server editor (options, buffers, keybindings); profiles; the telemetry dashboard; the command, shell-history, and prompt palettes; native i3-style tiling with saveable layouts; Exposé and cross-pane search/process tree; inline GPU graphics in three protocols plus a config-set background image; OSC 133 shell integration with failed-command gutter marks; native tmux session save/restore and cross-session broadcast; a GUI env-var editor that hot-injects into running shells; per-pane output triggers; snippets; a recent-directories tracker; the bundled `zterminal-icat`; and the glassy overlay compositing.

Hold those two lists in mind. The chapters do not re-litigate the division each time, but it is always there: the first half documents a faithful, fast inherited core, exhaustively; the second half documents the original layer, exhaustively.

Chapter 3 – The Grid

Strip every extension away and a terminal emulator is a machine that maintains a two-dimensional grid of character cells and mutates it in response to a byte stream. Understanding that machine is the prerequisite for everything zterminal layers on top, because every extension is ultimately expressed as either a change to the grid or a thing drawn over it.

Cells, not characters

Each cell in the grid holds a character (a Unicode scalar, possibly a zero-width combiner attached to its neighbour), a foreground colour, a background colour, and a set of attributes — bold, italic, underline, strike, dim, inverse, hidden. The grid has a fixed number of columns and rows, derived from the window's pixel size divided by the cell size, which is itself derived from the font metrics (Chapter 4). When the window resizes, the column and row counts change and the grid reflows; `window.dimensions` can pin an initial column/row count, and `window.resize_increments` can constrain resizing to whole cells so the grid never has a partial row or column of pixels.

The parser

Bytes arriving from the PTY are not all characters to print; interleaved among them are control sequences — the escape codes that move the cursor, change colour, clear a region, scroll, switch to the alternate screen, and a hundred other things. zterminal's core uses the `vte` parser to tokenize this stream into printable characters and parsed control sequences, and a terminal state machine applies each one to the grid. The four families — ESC codes, CSI sequences, OSC sequences, and the narrow DCS support — are the subject of Chapter 6 and Appendix D; the point here is that the parser is faithful and that its faithfulness is what makes vim, tmux, htop, and arbitrary ncurses applications behave correctly.

The two screens and the scrollbar

A terminal has a primary screen and an alternate screen. The primary screen scrolls: lines that move off the top are pushed into a scrollbar buffer, sized by `scrolling.history` (default `10000`, capped at `100000`, `0` to disable). The alternate screen does not scroll — it is the fixed-size canvas full-screen applications paint on, and switching to it (private mode 1049) saves the primary screen and cursor, restored when the application exits. The scrollbar is what vi mode (Chapter 18) navigates, what the OSC 133 marks (Chapter 27) anchor to by absolute line number, and what cross-pane search (Chapter 30) greps.

Truecolor and palettes

Each cell's colour can come from three sources: the 16-colour ANSI palette (`colors.normal`, `colors.bright`), the 256-colour indexed palette (the first 16 being the ANSI colours, then a 6×6×6 cube and a greyscale ramp, with overrides via `colors.indexed_colors`), or a direct 24-bit RGB value from an SGR truecolor sequence. zterminal carries all three; truecolor is why a modern colour scheme renders exactly rather than quantized to 256 steps. The default palette is a base16 scheme — background `#181818`, foreground

`#d8d8d8` — and Chapter 12 covers replacing it field by field, while Chapter 20 covers replacing it through the control panel's theme picker.

Width, combiners, and the wide-cell problem

Not every character occupies one cell, and the grid has to know which do not. A CJK ideograph, a wide emoji, and a handful of symbols are *double-width*: they claim two horizontal cells, the second of which is a spacer that holds no glyph of its own. Zero-width combining marks are the opposite — an accent or a variation selector that attaches to the preceding cell's character rather than advancing the cursor. The grid resolves each incoming scalar to a width before placing it, using the East-Asian-width rules, so that a full-width character advances the cursor by two columns and a combiner advances it by none. Getting this wrong is not cosmetic: a single mis-measured wide glyph shifts every following cell on the line, and a TUI that assumed its own layout desynchronizes from the terminal's. The faithfulness of this width accounting is part of what the inherited core buys, and it is why an emoji-laden prompt or a Japanese `man` page lines up under `zterminal`.

Reflow

When the window widens or narrows, the column count changes, and the terminal must decide what happens to lines that were longer or shorter than the new width. The primary screen *reflows*: a logical line that had been wrapped across several rows is rejoined and re-wrapped to the new width, so widening the window un-breaks paragraphs rather than leaving ragged fragments, and narrowing re-breaks them cleanly. The alternate screen does not reflow — it is a fixed canvas the application owns, and on resize the application is simply told the new size and repaints. `window.resize_increments` (Chapter 9) constrains resizing to whole cells so a reflow never has to contend with a partial column of pixels. Reflow is also why the OSC 133 marks of Chapter 27 anchor to *absolute line numbers* rather than screen rows: a mark pinned to a row would drift the moment the buffer re-wrapped, while a mark pinned to a logical line survives the reflow intact.

Erasing, scrolling, and the scroll region

Between printing characters and moving the cursor, the third thing a byte stream does to the grid is clear and shift regions of it, and these operations are worth naming because they are what a full-screen application spends most of its sequences on. Erasing comes in three granularities: erase-in-line (`CSI K`, Chapter 6) clears from the cursor to the end of the line, the start, or the whole line; erase-in-display (`CSI J`) does the same for the screen and can additionally clear the scrollbar; and erase-characters (`CSI X`) blanks a count of cells in place without moving anything. Scrolling shifts whole rows: when the cursor is on the bottom line and a newline arrives, the top line moves into the scrollbar and every row shifts up one (Chapter 3's primary-screen behaviour). The *scroll region* (`CSI r`) is the refinement that makes fixed headers and footers possible: it sets a top and bottom margin, and thereafter scrolling happens only *within* those margins, leaving the rows outside them untouched. This is how a TUI keeps a status bar pinned to the bottom while the content above it scrolls — it sets a scroll region that excludes the last line, and the terminal honours it. Origin mode (above) then lets the application address rows relative to that region. These operations are the grid's verbs; printing is only the most visible of them.

The cursor and its saved state

The grid has a cursor — a row/column position where the next printed character lands — and the cursor is more than a coordinate. It carries the *current pen*: the foreground colour, background colour, and attribute set (Chapter 3's cell fields) that every printed character inherits until an SGR sequence changes them. It also carries two mode bits that change how printing behaves at the edges of the grid, discussed below. `ESC 7` (save cursor) and `ESC 8` (restore cursor) — implemented, Chapter 6 — push and pop the whole of this state:

not just where the cursor is, but the pen and the character set it was using. This is why a program can save the cursor, move away to draw something, and restore, and find its colours and position exactly as it left them. The cursor is the grid's write head and its style register at once.

Tab stops

The grid maintains a set of *tab stops* — columns the cursor jumps to on a horizontal tab (`\t`). By default a stop sits every eight columns, the ancient teletype convention, but the stops are individually settable: `ESC H` (or `CSI g` with a parameter, Chapter 6) sets a stop at the cursor's column, and `CSI g` clears one or all. Tab stops are grid state, not display state, which is why a program that lays out columns with tabs — the output of `ls -C`, a `column`-piped table — depends on the terminal and the program agreeing on where the stops are. `zterminal` carries the standard eight-column default and honours the set/clear sequences so that tabular output aligns.

Origin mode, autowrap, and the edges

Two mode bits decide what happens at the grid's boundaries, and both are private modes the application toggles (Chapter 6). **Autowrap** (private mode 7, on by default) decides what happens when a character is printed in the last column: with autowrap on, the cursor wraps to the start of the next line and the line is marked *wrapped* so reflow (below) can rejoin it; with autowrap off, further characters overprint the last column in place. **Origin mode** (private mode 6) decides whether cursor-addressing coordinates are measured from the top of the screen or from the top of the current scroll region (`CSI r`, Chapter 6) — so a program that has set a scroll region and turned on origin mode can address rows relative to that region without arithmetic. These two bits, small as they are, are why full-screen applications that set margins and print into them land their text where they intend; getting either wrong is a class of “the bottom line is in the wrong place” bug, and the inherited core carries both faithfully.

Chapter 4 – The Renderer

The grid is data; the renderer turns it into light. zterminal's core renderer is OpenGL ES 2.0, and its central data structure is a glyph atlas — the technique that lets a terminal redraw tens of thousands of cells per frame without the CPU ever touching a pixel.

The glyph atlas

Rasterizing a glyph — turning a font outline into a grid of coverage values — is expensive, and a terminal would be unusably slow if it rasterized every glyph every frame. The glyph atlas solves this by rasterizing each glyph *once*, the first time it is needed, and packing the resulting bitmap into a large GPU texture. After that, drawing a cell is just drawing a textured quad: the renderer emits two triangles at the cell's screen coordinates, textured from the glyph's rectangle in the atlas, tinted by the cell's foreground colour, over a quad of the cell's background colour. A frame is thousands of such quads in a few draw calls, and the GPU does the work the CPU used to. This architecture makes the core fast, and it is inherited from the Alacritty lineage.

Cell metrics and the grid's pixel size

The font's metrics define the cell. A monospace font has a fixed advance width and a line height; the cell width is the advance plus `font.offset.x` (letter spacing), the cell height is the line height plus `font.offset.y` (line spacing). The grid's column count is the window's content width divided by the cell width; the row count is the content height divided by the cell height. `font.glyph_offset` nudges where the glyph sits inside its cell. These numbers are why changing the font size reflows the whole grid: a larger cell means fewer columns and rows in the same window.

Built-in box drawing

Fonts disagree about box-drawing and block characters — the lines, corners, shades, and powerline separators in the Unicode ranges `U+2500–U+259F`, `U+1FB00–U+1FB3B`, and `U+E0B0–U+E0B3`. A font that draws them a pixel short leaves gaps in a TUI's borders. zterminal's core can draw these itself with a built-in geometric font, so lines meet exactly regardless of the text font; `font.builtin_box_drawing` controls it and defaults to `true`. This is what makes a `zpw` status bar or a `tmux` border render seamlessly.

Damage, frames, and synchronized output

A terminal does not redraw when nothing changed. The renderer tracks *damage* — the cells changed since the last frame — and `debug.highlight_damage` visualizes it. Synchronized output (private mode 2026) lets an application say “hold the frame until I am done,” so a complex repaint shows up as one atomic update rather than a tear. `debug.render_timer` overlays per-frame draw time, and `debug.renderer` forces a backend (`glsl3`, `gles2`, `gles2pure`) when the automatic choice misbehaves. These are troubleshooting tools, documented as unstable, but they are the windows into the frame loop.

Compositing: transparency, blur, background image, overlays

The renderer composites, and that composite path is the hook for several extensions. `window.opacity` makes the background translucent; `window.blur` asks the compositor (macOS and KDE Wayland) to blur what shows through; `colors.transparent_background_colors` extends opacity from the default background to every cell's background. The inline-graphics extension (Chapter 26) draws its images and its config-set background image as textured quads in this same pass, ordered by a z-index that can place an image behind the text. And the control-panel and palette WebViews (Chapter 19) composite *over* the live GL terminal as translucent glass, by driving the native window's alpha — the inherited renderer keeps drawing underneath. The background image being just another texture in the existing compositing pass is the clearest small example of the whole program's method: original features built on the inherited core's machinery.

The atlas is finite: packing and eviction

The glyph atlas is a fixed-size GPU texture, which raises a question the happy path hides: what happens when more distinct glyphs are seen than the atlas can hold? Each newly-rasterized glyph is packed into free space in the texture — the renderer keeps a simple allocator that finds a rectangle for the new bitmap and records where it went — and in steady state the atlas fills with the few hundred glyphs a session actually uses and then stops growing, because every subsequent character is already resident. The pathological case named in Chapter 37 is an application that cycles through an unbounded parade of *distinct* glyphs — an emoji rain, a font-showcase tool — faster than the atlas has room, at which point the renderer must evict resident glyphs to make space and re-rasterize them if they reappear. That is the one workload where the amortization of Chapter 4 breaks down, and it is vanishingly rare because real terminal text reuses a tiny alphabet relentlessly. The design bets on that reuse, and the bet almost always wins.

What one draw call actually contains

It helps to be concrete about the “few draw calls” claim. The renderer does not issue one draw per cell — that would be tens of thousands of GPU calls per frame, the exact overhead the architecture exists to avoid. Instead it builds, per frame, a single batch describing every damaged cell: for each, the screen rectangle, the atlas rectangle of its glyph, its foreground tint, and its background colour. That batch is uploaded once and drawn in a handful of GPU calls — the backgrounds as one set of quads, the glyphs as another textured from the atlas. The CPU's job per frame is therefore to *assemble a list*, not to paint pixels, and the GPU's job is to rasterize thousands of pre-described quads in parallel, which is precisely the work GPUs are built for. This is why the renderer scales to a large, dense grid without the frame time growing with the cell count the way a CPU blitter's would.

Chapter 5 – Damage Tracking and Frame Scheduling

Chapter 4 introduced the renderer and named damage in a paragraph. This chapter takes that paragraph apart, because the discipline of *not drawing* is where a terminal spends most of its cleverness, and because the same machinery — a set of dirty cells, a frame clock, and a small pile of timers — underlies the performance floor of Chapter 37 and several of the extensions above it. All of it is inherited from the Alacrity lineage, and none of it is claimed as zterminal's; it is described here because a reader who understands the frame loop understands why the terminal stays responsive under a feature suite that, done naïvely, would stall it.

What “damage” means, precisely

Damage is the set of cells that changed since the last frame the GPU presented. The word matters: the renderer does not ask “what does the grid look like now,” it asks “what is *different* from what is already on the screen,” and it draws only the difference. In the common case that difference is tiny. A shell sitting at a prompt with a blinking cursor damages exactly the cells the cursor occupies. A `tail -f` scrolling one line damages one row and the rows the scroll shifted. A full-screen editor repainting its status line damages that line and nothing else. The renderer tracks this as line-granular dirty ranges — which rows changed, and within a changed row which span of columns — so a redraw uploads only the altered cells to the glyph-atlas draw path of Chapter 4 rather than re-emitting the whole grid every frame.

Two events force *full* damage, where every cell is considered dirty: a resize, because the grid's dimensions changed and every screen position may now hold a different cell (Chapter 3's reflow), and a switch between the primary and alternate screens (private mode 1049), because the visible contents are wholesale replaced. A colour-scheme change through the control panel (Chapter 20) is a third — re-theming re-tints every cell, so the whole grid is redrawn once. These are rare and allowed to be expensive; the frequent events are cheap. That asymmetry — the frequent case cheap, the rare case dear — is the same principle Chapter 37 identifies in the read loop, and it is not a coincidence that the read half and the draw half of the loop are built on the same bet.

`debug.highlight_damage` (Chapter 17) tints the regions the renderer actually redraws, and running it while a TUI is live is the single most instructive thing in the book. Start `htop`, or scroll a `man` page, or let a prompt blink, and watch how little of the screen lights up: the terminal you thought was “redrawing” is touching a few hundred cells out of tens of thousands. The debug flag is unstable API (Chapter 17), but the behaviour it visualizes is the load-bearing one.

Coalescing: many writes, one frame

The PTY does not deliver its bytes on the display's schedule. A program can emit a thousand tiny writes — a progress bar redrawing its percentage, a log streaming lines — in the span of a single display refresh. If each write triggered a draw, the terminal would try to present a thousand frames the monitor cannot show, burn the GPU doing it, and fall behind. The frame scheduler refuses that. Bytes read from the PTY mutate the grid and accumulate damage, but the *draw* is deferred: the scheduler coalesces all the damage that arrived since the last presented frame and draws it once, at the display's refresh cadence. A thousand writes in one frame's worth of time cost one frame, not a thousand. The visible consequence is that a firehose of output

looks smooth rather than frantic, and the invisible consequence is that the CPU and GPU are doing a bounded amount of work per second no matter how noisy the program on the other end of the PTY is.

This is why the traditional torture test — `cat` an enormous file — is bounded by how fast the grid can be mutated and one frame drawn per refresh, not by how many times the screen “could” have changed. Most of those intermediate grid states are never drawn at all; they are overwritten by the next batch before a frame is due. The terminal is, in effect, sampling the grid at the refresh rate, and the sample is all the eye could have seen anyway.

Synchronized output: handing the batch decision to the application

Coalescing is the terminal’s own decision about when to draw. Synchronized output (private mode 2026, Chapters 6 and 38) hands a piece of that decision to the application. Between a `CSI ? 2026 h` (begin) and a `CSI ? 2026 l` (end) the terminal holds the frame: it keeps applying the byte stream to the grid but does not present a frame until the end mark arrives, so a complex multi-sequence repaint lands atomically instead of tearing through several partial frames. A status-bar tool that clears a region, moves the cursor a dozen times, and rewrites several coloured segments wants all of that to appear at once; without synchronization the user can catch a frame mid-repaint, with the bar half-cleared. The DCS form of the same idea is rejected in favour of the `CSI` toggle (Chapter 6, Appendix D) — one mechanism, cleanly, rather than two that half-overlap.

The relationship between coalescing and synchronization is worth stating exactly: coalescing is the terminal batching *because the display cannot show more*, while synchronization is the application batching *because a repaint is not yet coherent*. The first is always on and invisible; the second is opt-in and driven by the program. They compose — a synchronized repaint is still subject to the refresh-rate ceiling — and together they are why a well-behaved TUI never flickers under `zterminal`.

The timers that also wake the loop

Not every frame is provoked by PTY bytes. Several features run on their own clocks, and each is a timer that can mark damage and request a frame without any input arriving:

- **Cursor blink.** `cursor.blink_interval` (default 750 ms, Chapter 13) toggles the cursor’s visibility, damaging only the cursor cells each half-period. `cursor.blink_timeout` (default 5 s) stops the blink after a period of inactivity so an idle terminal is not waking the loop forever, and `Never/Always` versus `Off/On` decide whether an application may override the blink at all (Chapter 13).
- **The visual bell.** `bell.duration` and `bell.animation` (Chapter 13) run an easing curve over the configured flash colour; while the animation plays, each frame damages the flashed region, and when it completes the timer stops.
- **Inline-image animation.** A Kitty-protocol animated image (Chapter 26) plays its frames on a timer, damaging the cells the image covers as each frame advances — the one case where the graphics extension, not the text grid, drives the frame clock.

The important property is that all of these are *bounded* and *self-terminating*: the blink stops on timeout, the bell animation stops when the curve completes, an idle terminal with no timers pending draws nothing and consumes no frames. The event loop is not a busy render loop spinning at the refresh rate; it sleeps until either PTY bytes arrive, an input event occurs, or a timer comes due, and only then wakes, applies the change, marks damage, and — if damage exists and a frame is due — presents. An idle `zterminal` window is genuinely idle.

Why the extensions stay off this path

Chapter 37 makes the general argument that the extension layer is built to stay off the hot path; the frame loop is where that argument is concrete. The control panel is a `WebView`, and a `WebView` repainting on the render thread would be a disaster — so it does not (Chapter 19). It composites over the still-live GL terminal by driving the native window's alpha, which means the terminal's own frame loop keeps running underneath at its own cadence and never blocks on the panel's rendering. The pre-`vt` scanner (Chapters 26–27) adds one linear pass over the incoming bytes but marks damage exactly as the parser does, so an image or an OSC 133 mark is just more damage in the same coalesced frame. The `tmux` client's events (Chapter 28) fold into the same event loop as another wake source, not a separate render path. Every extension that touches the screen expresses itself as damage in the one frame loop, which is precisely why adding them did not cost the terminal its responsiveness — there is only ever one draw, and everything that wants to be seen has to earn its cells in it.

Chapter 6 – Escape Sequence Support

This chapter is the narrative companion to Appendix D and to `zterminal-escapes(7)`: what it means that zterminal supports a given escape sequence, why the support table is worth reading rather than assuming, and what the whole inherited support surface is. The support is inherited from the Alacritty lineage, and so is the discipline behind it.

The three statuses

Every sequence in the table carries one of three statuses. **IMPLEMENTED** means fully handled. **PARTIAL** means some parameters or modes are handled and the rest are not yet. **REJECTED** means deliberately not handled — and “rejected” is a feature, not a gap: a terminal that answers a sequence it does not really implement is worse than one that cleanly ignores it, because applications probe capabilities and adapt. The man page’s note that an **IMPLEMENTED** sequence which is “partially implemented” means “all other features were rejected” captures the philosophy: implement a thing fully or reject the remainder cleanly, never half-answer.

The four families

- **ESC codes** — `ESC 7/ESC 8` (save/restore cursor), `ESC D/ESC M` (index/reverse index), `ESC E` (next line), `ESC H` (tab set), `ESC =/ESC >` (keypad mode), `ESC # 8` (DEC alignment test), `ESC (/)*/+` (charset selection, charsets B and O only), and `ESC Z` (identify). All implemented.
- **CSI sequences** — the large `ESC [` family: cursor motion (`CSI A–CSI H`, `CSI a–CSI f`), erase (`CSI J`, `CSI K`, `CSI X`), insert/delete (`CSI L`, `CSI M`, `CSI P`, `CSI @`), scroll (`CSI S`, `CSI T`), the SGR colour-and-attribute setter (`CSI m`), device status (`CSI n`), tab control (`CSI g`), scroll regions (`CSI r`), cursor save/restore (`CSI s`), cursor style (`CSI SP q`), the keyboard-protocol family (`CSI u`, `CSI ? u`, `CSI = u`, `CSI < u`, `CSI > u`), and the private-mode toggles (`CSI ? h/CSI ? l`).
- **OSC sequences** — `ESC ]`: window title (`OSC 0/OSC 2`), palette set/query (`OSC 4`, `OSC 10/11/12`, and the reset forms `OSC 104/110/111/112`), hyperlinks (`OSC 8`), cursor shape (`OSC 50`, CursorShape only), and clipboard (`OSC 52`, clipboard and primary selection only, gated by `terminal.osc52`).
- **DCS** — narrow by design; the synchronized-output `DCS = s` is rejected in favour of `CSI ? 2026 h/l`.

The partials, precisely

The table is exact about its partials, and the precision is the inherited core’s most underrated quality. `CSI m` accepts SGR parameters `0–9`, `21–25`, `27–49`, `58`, `59`, `90–97`, `100–107`, and explicitly rejects `11–19` and `51–55`. `CSI h/CSI l` handle modes `4` and `20`. `CSI ? h/CSI ? l` handle the modes applications actually use: `1` (application cursor keys), `3` (132-column), `6` (origin), `7` (autowrap), `12` (cursor blink), `25` (cursor visibility), the mouse-reporting family `1000/1002/1005/1006/1007`, focus reporting `1004`, `1042`, the alternate-screen `1049`, bracketed paste `2004`, and synchronized output `2026`. `CSI t` answers window-manipulation parameters `22` and `23` and rejects the geometry-manipulation ones. When an application misbehaves under zterminal, this table is the first place to look, because it names exactly which sequence and parameter is handled and at what status; Appendix D reproduces the full matrix.

A worked example: reading an SGR sequence

The abstraction becomes concrete the moment you read a real sequence off the wire. Consider the bytes a shell prompt emits to draw bold cyan text on a dark background and then reset:

```
ESC [ 1 ; 38 ; 2 ; 117 ; 181 ; 170 m   the pen: bold, truecolor fg #75b5aa
ESC [ 48 ; 5 ; 236 m                   the pen: 256-palette bg, index 236
Hello                                   printed with the current pen
ESC [ 0 m                               reset the pen to default
```

Every one of those is a `CSI m` (SGR) sequence, the colour-and-attribute setter of this chapter, and reading it is a matter of splitting the numeric parameters on `;` and applying each. `1` sets the bold attribute. `38` introduces a foreground colour, and its sub-parameters say *how*: `38;2;r;g;b` is a 24-bit truecolor value (the `2`), here `#75b5aa`; `38;5;n` would be a 256-palette index (a `5`). `48` is the same for background — `48;5;236` selects palette index 236, a dark grey from the 6×6×6 cube (Chapter 3). `0` resets everything to the defaults of Chapter 12. The parameters zterminal accepts here are exactly the ones the partials table names — `0–9`, `21–25`, `27–49`, `58`, `59`, `90–97`, `100–107` — and a sequence carrying `11–19` or `51–55` is cleanly ignored rather than half-answered, per the REJECTED discipline above.

A worked example: a full-screen application's setup

The other place the table earns its keep is the handshake a TUI performs on startup and teardown, which is a small liturgy of the private modes:

```
ESC [ ? 1049 h   switch to the alternate screen (save the primary)
ESC [ ? 1 h      application cursor keys (arrows send SS3, not CSI)
ESC [ ? 25 l     hide the cursor while painting
ESC [ ? 1006 h   SGR mouse reporting (report clicks as CSI < ... M)
ESC [ ? 2004 h   bracketed paste (Chapter 38)
...the application runs...
ESC [ ? 2004 l   disable bracketed paste
ESC [ ? 1006 l   disable mouse reporting
ESC [ ? 25 h     show the cursor
ESC [ ? 1 l      normal cursor keys
ESC [ ? 1049 l   restore the primary screen
```

Every mode in that exchange is one the `CSI ? h/CSI ? l` partials row lists as handled (Appendix D), which is precisely why vim, htop, and any ncurses program enter cleanly and — the underrated half — *leave* cleanly, restoring your prompt and scrollbar exactly as they were. A terminal that mishandled `1049` would leave you staring at the application's last frame after it exited; one that ignored `25` would blink a cursor over the application's own. The support table is not bookkeeping; it is the contract that makes the alternate-screen round trip invisible.

Chapter 7 – The Input Path

Chapters 3 through 6 followed bytes in one direction: from the PTY, through the parser, into the grid, onto the screen. This chapter follows them the other way. A terminal is a bidirectional device, and the return path — from a physical key or a mouse button to bytes written into the PTY — is the half that users feel most directly and that documentation most often skips. It is also where a disproportionate share of “my terminal is behaving strangely” symptoms live, because the encoding of input is a negotiation between the terminal and the application, conducted through the very modes Chapter 6 catalogued, and a negotiation that ends in the wrong state leaves arrows printing `^[A`.

The whole path is inherited from the Alacritty lineage, like the output path, and it is described here for the same reason: the extension layer sits on top of it, the binding model of Chapter 16 is a stage within it, and several of the troubleshooting entries in Appendix G resolve to a specific step in it.

The stages, in order

A key press becomes PTY bytes through a short pipeline, and knowing the order explains why some keys never reach the shell and others do:

1. **The window system delivers an event.** The platform's window layer (winit, the same library whose logs `ZTERM_EXTRA_LOG_TARGETS` can widen, Chapter 17) reports a key with its physical identity, its produced text if any, and the modifier state. Composition — dead keys, an IME's multi-keystroke sequence — has already happened at this layer; what arrives is the composed result, so the terminal writes what the platform hands it rather than reimplementing the input method.
2. **The binding table is consulted.** `keyboard.bindings` (Chapter 16) is checked against the key, the modifiers, and the current *mode* — the `AppCursor`, `AppKeypad`, `Search`, `Alt`, and `Vi` guards. A match runs an `action`, writes `chars`, or forks a `command`, and the key is consumed.
3. **Unmatched keys are encoded.** A key with no binding falls through to the encoder, which turns it into the bytes the terminal tradition specifies for that key under the currently-active modes.
4. **The bytes are written to the PTY.** The primary end of the pane's pseudo-terminal (Chapter 35), where the kernel's line discipline may transform them again before the shell ever sees them.

The two escape hatches of Chapter 16 are positions in this pipeline rather than special cases. `action = "ReceiveChar"` removes a binding at stage 2 so the key proceeds to stage 3 and is encoded normally, which is how you reclaim a key the defaults took. `action = "None"` consumes the key at stage 2 and encodes nothing, which is how you make a key do literally nothing — useful when a muscle-memory chord from another application would otherwise send something disruptive.

Encoding an ordinary key

Most keys are trivial. A printable character is written as its UTF-8 bytes: `a` is `0x61`, `é` is the two bytes of `U+00E9`, an emoji is four. There is no escape sequence and no negotiation, which is why typing works in every terminal ever built regardless of what modes are set.

Control keys are the first place a convention appears. `Ctrl` combined with a letter produces the control byte in the `0x01–0x1A` range corresponding to that letter's position in the alphabet: `Ctrl+A` is `0x01`, `Ctrl+C` is `0x03`, `Ctrl+Z` is `0x1A`. This is not a terminal invention; it is the ASCII design, and it has a consequence that Appendix G names as a symptom. Because `Ctrl+I` maps to `0x09` and the `Tab` key *also* sends `0x09`, the

two are indistinguishable in the byte stream. The same collision joins `Ctrl+M` to `Enter` (`0x0D`) and `Ctrl+[` to `Escape` (`0x1B`). An application receiving `0x09` cannot know which key you pressed, and no amount of care in the terminal fixes it, because the information was destroyed by the encoding itself. This is the specific deficiency the Kitty keyboard protocol below exists to repair.

`Alt` (and, on macOS, `Option` when `window.option_as_alt` says so, Chapter 9) follows a different convention: the key's ordinary bytes prefixed with `ESC`. `Alt+p` is `0x1B 0x70` — an escape byte followed by `p`. Readline, zsh's line editor, and every emacs-derived keymap read that prefix as a meta modifier, which is why `Alt+b` moves back a word at a shell prompt. The `option_as_alt` field matters precisely here: with it unset, macOS's Option key composes characters (`Option+p` produces `π`), and with it set to `OnlyLeft`, `OnlyRight`, or `Both`, the chosen physical key stops composing and starts sending the `ESC` prefix. You cannot have both behaviours on one key, which is why the field enumerates individual sides rather than being a boolean — one Option composes, the other modifies.

Arrows, function keys, and the two cursor-key modes

Cursor keys have no ASCII byte, so they are sent as escape sequences, and *which* sequence depends on a mode the application sets. In the default state the up arrow sends `ESC [ A` — a CSI sequence, the same family Chapter 6 catalogued on the output side. When an application enables application-cursor-keys mode (private mode 1, `CSI ? 1 h`) the same key sends `ESC O A` instead: an SS3 sequence rather than a CSI one. Both are ancient DEC conventions; full-screen applications typically ask for the SS3 form because their own keymaps were written against it.

This mode is the single most common source of a terminal that has “gone weird.” The mode is *terminal* state, not application state, so an application that sets it and exits without resetting it leaves the terminal sending `ESC O A` to a shell whose line editor expects `ESC [ A`. The shell does not recognize the sequence, prints the parts it cannot interpret, and you see `^[A` or `^[OA` where a history recall should have happened. Nothing is broken; a mode is simply still on. Resetting the terminal (`reset`, or `tput rmkx` to clear specifically this mode) restores the default encoding, and the mode is also what the `mode = "AppCursor"` binding guard of Chapter 16 tests, so a binding can deliberately do different things depending on which encoding is active.

The numeric keypad has an exactly parallel pair, `ESC =` and `ESC >` (Chapter 6), switching the keypad between application and numeric encodings, with the same failure mode when an application leaves it set.

Modified special keys and the parameter form

The plain sequences above carry no modifier information: `ESC [ A` is “up,” full stop, with nowhere to say that Shift was held. The xterm tradition extends them with a parameterized form — the modifier travels as a numeric parameter inside the same CSI sequence, so a modified up arrow becomes `ESC [ 1 ; <mod> A` where the first parameter is the (unused) repeat count and the second encodes the modifier set. The convention is a bitfield offset by one: shift adds 1, alt adds 2, control adds 4, and the sum is incremented, so an unmodified key is 1, `Shift+Up` is `ESC [ 1 ; 2 A`, and `Ctrl+Shift+Up` is `ESC [ 1 ; 6 A`.

This covers the special keys adequately and does nothing for the ASCII collisions above, because a key that was encoded as a bare control byte has no sequence in which to carry a parameter. That asymmetry — modifiers expressible on arrows but not on letters — is what makes the legacy encoding a patchwork rather than a scheme, and it is the reason a modern protocol was proposed at all.

The Kitty keyboard protocol, in depth

The `CSI u` family (Chapter 6 lists all five forms as implemented) replaces the patchwork with one uniform encoding. Its central idea is that every key event is reported as a CSI sequence carrying the key's codepoint and its modifiers as explicit parameters, terminated by `u`. Under it, `Ctrl+I` and `Tab` are different sequences

because the codepoint reported is the key's own, not the control byte the old encoding collapsed it into. Modifiers are unambiguous because they are a parameter rather than an implied transformation. Depending on the flags an application requests, key *release* events and repeat events can be reported too, which a legacy terminal cannot express at all — the byte stream has no vocabulary for “the user let go of a key.”

The protocol's other design decision is the one that makes it safe to adopt: the mode is a *stack*, not a flag. The five forms divide accordingly.

- `CSI > u` pushes a set of flags onto the stack, enabling the enhanced encoding with the requested features.
- `CSI < u` pops, restoring whatever mode was active before the push.
- `CSI = u` sets the current flags directly.
- `CSI ? u` queries the active flags; the terminal replies with them.
- `CSI u` is the report form the terminal emits for a key.

The stack matters because terminals are nested environments. An editor running inside a multiplexer inside a shell all may want different keyboard behaviour, and each can push its requirement on entry and pop it on exit without knowing or caring what the layer beneath it had set. A flag-based design forces each layer to save and restore the global state correctly, and any layer that fails leaves the terminal in the wrong mode for everyone above it — precisely the application-cursor-keys failure described earlier, generalized. The push/pop discipline makes the common case correct by construction.

The query form is the input half of the capability-advertisement story in Chapter 40. An application does not have to guess whether the enhanced encoding is available: it pushes flags, queries what actually took effect, and adapts. That is the same measure-rather-than-assume shape the graphics support query (`a=q`, Chapter 26) follows on the output side, and zterminal implements both for the same reason — a terminal that answers honest questions honestly lets applications stop relying on `TERM` being right.

The mouse, encoded

The mouse follows the identical pattern: an event, a mode that decides whether it is reported at all, and a second mode that decides how it is encoded. Chapter 14 covered the *policy* — the application owns the mouse when it asks, `Shift` takes it back for one action. This is the mechanism underneath.

What is reported is chosen by the tracking modes. Private mode 1000 reports button presses and releases only. Mode 1002 adds drag — motion events while a button is held — which is what a pane-resizing or text-selecting TUI needs. An application that wants clicks but not the flood of motion events asks for 1000; one that needs to follow a drag asks for 1002.

How it is reported is chosen by the encoding modes, and this is where a real historical limit shows. The original encoding packs the button and the coordinates into single bytes offset by 32, which caps the addressable coordinate at 223 — a column beyond that simply cannot be expressed, and on a wide modern display the right-hand side of the window becomes unclickable. Mode 1005 attempts a fix by encoding the coordinates as UTF-8, which extends the range but leaves the byte stream ambiguous in ways that are awkward to parse. Mode 1006 — SGR encoding — is the one that actually solved it: coordinates are sent as decimal parameters in a CSI sequence, `ESC [ < <button> ; <col> ; <row> M` for a press and the same with a trailing `m` for a release. Decimal parameters have no coordinate ceiling, and the distinct terminator means press and release are unambiguous rather than inferred. This is why the startup handshake of Chapter 6 shows `ESC [ ? 1006 h`: a well-written modern TUI enables SGR encoding, and zterminal supports all of 1000/1002/1005/1006 so that older applications still work.

Mode 1007 is a different kind of convenience. On the alternate screen (Chapter 3) there is no scrollbar to scroll, so a wheel event has nothing sensible to do; alternate-scroll mode translates wheel notches into arrow-key sequences instead, which is why the wheel pages through `less` and `man` even though neither program implements mouse handling. And mode 1004 — focus reporting — has the terminal send `CSI I` when the window gains focus and `CSI O` when it loses it, so an editor can check its files for external modification the moment you return to the window.

Paste is not typing

A paste enters the input path at a different point from a key press, and the difference is load-bearing. Pasted text does not traverse the binding table: the bytes are the payload, not a chord to be interpreted, so a multi-line paste containing what looks like a keybinding does not fire that binding. What the terminal does add is the bracketing of Chapter 38 — when the application has enabled private mode 2004, the paste is wrapped in `ESC [ 200 ~` and `ESC [ 201 ~` so the receiving program can tell the payload from typed input.

The security consequence deserves stating alongside the mechanism. Without bracketing, a paste containing a newline executes at the newline, which is the classic hazard of copying a command from a web page that ends in a hidden `\n` and a second command. With bracketing, a shell that supports it receives the whole payload as inert text and can display it for review instead of running it. The terminal cannot force this — the application must enable the mode and honour the markers — but `zterminal` implements the mode, and modern shells and editors enable it, which is why a multi-line paste into `zsh` or `vim` lands as text.

Where the extension layer touches the input path

The extension layer of Chapters 19 onward writes into this path in exactly two places, and both are worth naming so the injection surfaces of Chapter 39 are legible as mechanism rather than magic. The `env-var hot-inject` (Chapter 32) writes a `export NAME=value` line into the PTY of shells sitting at a prompt: it enters at stage 4, bypassing the encoder entirely, which is why the book describes it as a keystroke injection and why it works only against a shell at a prompt rather than an arbitrary child program. The broadcast overlay (Chapter 30) does the same thing to a chosen set of `tmux` panes, and snippets are the same mechanism with saved text. Everything else the extension layer does is a stage-2 action — a palette entry, a menu item, and a keybinding are three views of one action object (Chapter 19), and all three dispatch through the binding stage rather than around it.

Chapter 8 – The Configuration Model

zterminal runs with no configuration, but it is built to be configured, and the configuration story has one MenkeTechnologies twist on the inherited core: a canonical home under `~/.zterminal` and a GUI that edits it live. This chapter is the configuration model; Chapters 9–17 walk every table and every field, and Appendix B is the field index.

The file and its format

The configuration file is TOML — the format at toml.io — and its canonical location is `~/.zterminal/zterminal.toml`. On first run, when no config exists anywhere, zterminal seeds that file with sensible defaults, including an auto-detected installed Nerd Font as the font family so powerline and icon glyphs render immediately; if no Nerd Font is present it falls back to the platform mono font. Everything zterminal writes — config, logs, layouts, session snapshots, command history, recent directories, triggers, snippets, profiles — lives under `~/.zterminal` (Appendix E maps the whole directory).

Resolution order

The config is resolved from the first match in a fixed search order:

1. `--config-file <path>` on the command line.
2. `~/.zterminal/zterminal.toml` — the canonical file; legacy locations below are migrated here on first run.
3. `$XDG_CONFIG_HOME/MenkeTechnologies/zterminal.toml`
4. `$XDG_CONFIG_HOME/zterminal.toml`
5. `$HOME/.config/MenkeTechnologies/zterminal.toml`
6. `$HOME/.zterminal.toml`
7. `/etc/MenkeTechnologies/zterminal.toml`

On Windows the file is `%APPDATA%\zterminal\zterminal.toml`. The man page documents the inherited XDG-style search (`$XDG_CONFIG_HOME/zterminal/zterminal.toml` and friends); the `~/.zterminal` canonical home and the `MenkeTechnologies/` namespacing are the product's own convention layered on top.

Imports and environment

`general.import` is an array of additional config files, loaded in order with the importing file applied last, so a later import overrides an earlier one and the top file overrides them all. Paths are absolute (`/...`), home-relative (`~/...`), or relative to the importing file. The `[env]` table injects environment variables into every spawned process, including the shell — and the control panel's Env Vars manager (Chapter 32) both writes this table and hot-injects the export into every already-running shell.

```
[general]
import = [
  "~/.config/zterminal/base16-dark.toml",
  "~/.config/zterminal/keybindings.toml",
]

[env]
WINIT_X11_SCALE_FACTOR = "1.0"
```

Live reload, overrides, migration

`general.live_config_reload` defaults to `true`, and it is what makes the control panel feel like a settings app rather than a text editor: saved changes apply to running windows without a restart. A handful of fields are read once at window creation and are restart-only (`window.dimensions`, `window.startup_mode`, and a few others), marked in Appendix B. Two command-line mechanisms sit alongside the file: `-o`, `--option` overrides a single field for one launch (`zt -o 'cursor.style="Beam"'`), and is also how `zt msg config` pushes a live change (Chapter 33); and `zterminal migrate` upgrades an old config to the current schema, with `--dry-run`, `--skip-imports`, and `--skip-renames`. Migration is what lets the format evolve without breaking a file that worked yesterday — the durability discipline the whole stack holds to.

A complete starter config, annotated

The chapters that follow walk every table field by field; it helps to see them assembled first. The following is a realistic `~/.zterminal/zterminal.toml` that touches every major table without exotic values — the sort of file a user actually keeps — so that each later chapter's fields have a home to return to:

```

[general]
live_config_reload = true           # the control panel relies on this (Chapter 8)
import = ["~/zterminal/theme.toml"]

[env]
TERM = "zterminal"                 # advertise the real capabilities (Chapter 40)

[window]
padding = { x = 6, y = 6 }
dynamic_padding = true
opacity = 0.92
decorations = "Full"
option_as_alt = "OnlyLeft"         # macOS: left Option is Alt (Chapter 9)

[font]
normal = { family = "JetBrainsMono Nerd Font", style = "Regular" }
size = 12.0
builtin_box_drawing = true         # crisp borders regardless of face (Chapter 4)

[cursor]
style = { shape = "Beam", blinking = "On" }
unfocused_hollow = true

[scrolling]
history = 50000                    # deep scrollbar; capped at 100000 (Chapter 13)
multiplier = 3

[selection]
save_to_clipboard = true           # every selection also hits the clipboard

[terminal]
osc52 = "OnlyCopy"                 # the security default (Chapters 14, 39)

[colors.primary]
foreground = "#d8d8d8"
background = "#181818"

[keyboard]
bindings = [
  { key = "F", mods = "Command", mode = "~Search", action = "SearchForward" },
]

```

Two things about that file are worth naming because they recur. First, nothing in it is required — delete the whole file and `zterminal` runs with the defaults of Chapters 9–17, seeding a fresh one with an auto-detected Nerd Font (Chapter 8). Second, every value in it is also reachable without editing the file at all: the Settings tab (Chapter 20) is a graphical front end to this same TOML, writing the same keys, and because `live_config_reload` is on, a change made either way lands in the running window immediately. The file and the GUI are two doors onto one configuration, and the config resolution order above decides which file both of them write.

Layering one configuration across several machines

The import mechanism is usually described as a way to keep a theme in its own file, which undersells it. It is a general layering system, and the layering rule is the part worth internalizing: imports are applied in the order they are listed, each one overriding what came before it, and the importing file is applied last and overrides them all. That single rule is enough to express the arrangement most people actually need, which is one shared configuration plus a small set of per-machine differences.

The shape that follows is a base file containing everything that is true everywhere, imported by a thin top-level

file containing only what is true here. The base holds the keybindings, the scrollbar depth, the selection behaviour, the hints — the things whose value does not depend on which machine you are sitting at. The top file holds font size, because a laptop panel and an external display want different ones; `window.opacity` and `window.blur`, because the blur request is honoured on some platforms and not others (Chapter 9); and any `[env]` entries that are genuinely machine-local. Everything else stays in the base, and the base is the file you actually version and copy between hosts.

```
[general]
import = [
    "~/.zterminal/base.toml",      # shared: bindings, hints, scrolling
    "~/.zterminal/theme.toml",    # shared: the whole of [colors]
]

[font]
size = 14.0                      # this machine only

[window]
opacity = 1.0                    # no compositor here
```

Two properties of this arrangement are worth naming because they are not obvious. First, the ordering means a theme import placed *after* the base can override colours the base set, and placed *before* it cannot — so the position of a line in the import array is semantic, not cosmetic. Second, the whole layered result is re-read on reload (Chapter 8's `general.live_config_reload`), including the imports, so editing the shared base file changes every running window without a restart in exactly the same way editing the top file does. There is no separate “reload the imports” step, because there is no separate import cache to invalidate.

The interaction with the control panel is the one thing to plan around. The Settings tab (Chapter 20) writes to the resolved config file — the top one — which means a change made in the GUI lands in the machine-local layer rather than in the shared base. That is usually what you want for a font size and usually not what you want for a keybinding. The practical rule is to treat the GUI as the editor for the local layer and a text editor as the editor for the shared one, and to move a setting down into the base by hand when you decide it should be true everywhere. Chapter 21's profiles solve a different problem — switching whole environments including tmux state — and the two compose rather than compete: profiles capture a complete configuration, imports factor a single configuration into reusable parts.

Chapter 9 – Window

`[window]` governs the frame around the grid. It is the largest of the inherited tables, and `zterminal` adds three fields of its own to it. Each field, in full:

- `dimensions = { columns, lines }` — initial grid size in cells (not pixels). Both must be non-zero to take effect; columns must be at least 2. `{ 0, 0 }` falls back to the window manager's recommended size. Restart-only. Default `{ columns = 0, lines = 0 }`.
- `position = "None" | { x, y }` — startup window position in pixels, or `"None"` to let the window manager place it. No effect on Wayland. Default `"None"`.
- `padding = { x, y }` — blank space inside the window edges in pixels, scaled by DPI, added at both opposing sides. Default `{ 0, 0 }`.
- `dynamic_padding = true | false` — spread leftover padding evenly so the grid stays centered. Default `false`.
- `decorations = "Full" | "None" | "Transparent" | "Buttonless"` — `Full` is borders and title bar; `None` is neither; `Transparent` (macOS) is a title bar with transparent background and buttons; `Buttonless` (macOS) is the same with no buttons. Default `"Full"`.
- `opacity = <float>` — background opacity, `0.0` (transparent) to `1.0` (opaque). Default `1.0`.
- `blur = true | false` — ask the compositor to blur behind a transparent window (macOS, KDE Wayland). Default `false`.
- `startup_mode = "Windowed" | "Maximized" | "Fullscreen" | "SimpleFullscreen"` — `SimpleFullscreen` (macOS) fullscreens without taking a Space. Restart-only. Default `"Windowed"`.
- `title = "<string>"` — window title. Default `"Zterminal"`.
- `dynamic_title = true | false` — let applications change the title via OSC. Default `true`.
- `class = { instance, general }` — X11/Wayland window class; on Wayland `general` is the `app_id` and `instance` is ignored. Linux/BSD only. Default `{ "Zterminal", "Zterminal" }`.
- `decorations_theme_variant = "Dark" | "Light" | "None"` — override the system/GTK/CSD theme variant; `"None"` uses the system default. Default `"None"`.
- `resize_increments = true | false` — resize in whole-cell steps (macOS/X11/Wayland). Default `false`.
- `option_as_alt = "OnlyLeft" | "OnlyRight" | "Both" | "None"` — make the macOS Option key behave as Alt. macOS only. Default `"None"`.
- `level = "Normal" | "AlwaysOnTop"` — window z-order. Default `"Normal"`.

zterminal's window additions

Three fields here are `zterminal`'s own, not in the inherited man page:

- `background_image = "<path>"` — a GPU image drawn behind the cells (Chapter 26). Because it is config-set, not an escape sequence, it works everywhere including inside `tmux`. Reloads live when the path changes; PNG/JPEG/ GIF.
- `background_image_opacity = <float>` — `0.0–1.0`, how strongly the background image shows.
- `overlay_opacity = <float>` — `0.0–1.0`, the alpha of the glassy control-panel and palette overlays composited over the live terminal (Chapter 19); also driven by a live slider in the panel.

A compact example combining the inherited and the added:

```
[window]
padding = { x = 3, y = 3 }
dynamic_padding = true
opacity = 0.85
background_image = "~/Pictures/wall.png"
background_image_opacity = 1.0
```

Decorations, startup modes, and the platform seams

Several fields in this table are qualified with a platform, and the qualifications are not arbitrary omissions — they are places where the window system, not zterminal, owns the behaviour. Naming why each is limited turns a confusing list of caveats into a short set of rules.

`decorations` has four values but only two are universal. `Full` and `None` mean what they say everywhere: draw the platform's title bar and borders, or draw neither. `Transparent` and `Buttonless` are macOS-only because they describe variations on a macOS title bar specifically — a title bar whose background is transparent so the terminal's own background shows through it, and the same with the traffic-light buttons removed. There is no X11 or Wayland equivalent to express, because on those systems a decorated window's title bar is drawn either by the window manager (which zterminal does not control) or by client-side decorations whose styling is a different mechanism entirely. This is also why `decorations_theme_variant` exists as a separate field: on systems using client-side decorations, the light-or-dark styling of a bar zterminal *does* draw is a choice the config has to be able to make, whereas on macOS the system decides.

`startup_mode` is restart-only for a structural reason rather than an implementation shortcut. Whether a window is created windowed, maximized, or fullscreen is an argument to the act of *creating* it; there is no coherent meaning to changing it for a window that already exists, and the actions that toggle fullscreen and maximization at runtime (`ToggleFullscreen`, `ToggleMaximized`, Chapter 16) are the mechanism for that. `SimpleFullscreen` is macOS-only and is the one worth knowing about: ordinary macOS fullscreen moves the window to its own Space, with the animation and the Mission Control consequences that implies, while simple fullscreen expands the window to fill the display in place. For a terminal you flip in and out of constantly, in-place is usually what you want.

`position` and `class` are the two fields most likely to appear not to work, and in both cases the reason is that the platform declined rather than that the setting was ignored. Wayland does not let a client place its own windows — positioning is the compositor's prerogative — so `position` has no effect there by design, not by oversight. `class` is an X11 and Wayland concept with no macOS or Windows analogue; it is how a window manager identifies your terminal for rules, workspace assignment, and icon matching, and on Wayland only the `general` half survives as the `app_id`. If you write window-manager rules, this is the field they match on, and it is restart-only because the identity is fixed when the window is created.

`level = "AlwaysOnTop"` and `blur` round out the set of “the compositor must cooperate” fields. Blur in particular is a *request*: macOS and KDE Wayland honour it, and elsewhere it is silently ignored rather than emulated, because emulating a backdrop blur would mean compositing content the terminal cannot see. Appendix G's transparency entry is the troubleshooting face of this same fact.

Chapter 10 — Font

DPI, scaling, and multiple monitors

Before the fields, a word on the pixels those fields resolve to, because the same `font.size` produces a different cell size on different displays. A point is a physical unit; a pixel is not. The window manager reports a *scale factor* for the monitor the window is on — 2.0 on a typical macOS Retina panel, some fractional value on a scaled HiDPI Linux display — and zterminal multiplies the point-based font metrics by it to get pixel metrics, so 12pt text is the same physical size on a Retina laptop and a standard external monitor even though one has twice the pixels. Two consequences follow. First, dragging a window between monitors of different scale factors re-resolves the cell size and reflows the grid (Chapter 3), exactly as a font-size change would, because the pixels-per-cell changed. Second, the `WINIT_X11_SCALE_FACTOR` environment variable (settable in `[env]`, Chapter 8) overrides the reported factor on X11 when the automatic value is wrong — the one knob for the “everything is too big” or “too small” symptom on a misdetected display. `window.resize_increments` (Chapter 9) interacts with all of this by snapping window resizes to whole multiples of the current pixel cell size, so the grid never carries a partial row or column regardless of the scale factor in play.

`[font]` selects the faces and tunes their placement in the cell. Every field:

- `normal = { family, style }` — the base face. Defaults are platform- appropriate: `monospace/Regular` on Linux/BSD, `Consolas/Regular` on Windows, `Menlo/Regular` on macOS. First-run seeding prefers an installed Nerd Font so icon glyphs render without configuration.
- `bold = { family, style }` — the bold face; family falls back to `normal`'s if omitted. Default style `Bold`.
- `italic = { family, style }` — the italic face; family falls back to `normal`'s. Default style `Italic`.
- `bold_italic = { family, style }` — family falls back to `normal`'s. Default style `Bold Italic`.
- `size = <float>` — font size in points. Default `11.25`. Changed live by the font-size key bindings, which reflow the grid (Chapter 4).
- `offset = { x, y }` — extra space around each glyph: `y` is line spacing, `x` is letter spacing. These change the cell size. Default `{ 0, 0 }`.
- `glyph_offset = { x, y }` — position of the glyph within its cell; increasing `x` moves it right, increasing `y` moves it up. Default origin is the cell bottom.
- `builtin_box_drawing = true | false` — draw box-drawing, legacy- computing, and powerline glyphs with the built-in geometric font (Chapter 4). Default `true`.

Fallback and the Nerd Font default

No single font contains every glyph a terminal will be asked to draw, so the font stack is a *chain*, not a face. When the primary face lacks a glyph — a CJK ideograph, an emoji, a powerline separator, a Nerd Font icon — the rasterizer walks a fallback chain until it finds a face that has it, and caches the result in the atlas like any other glyph (Chapter 4). This is why a prompt studded with devicon glyphs renders even when your text font knows nothing of them: the system's icon and emoji fonts stand behind it. The one seam worth knowing is metric mismatch — a fallback glyph designed for a different cell size may sit high, low, or wide — which is exactly what `font.glyph_offset` and the built-in box-drawing font exist to paper over. zterminal leans into this by preferring an installed Nerd Font as the seeded default face on first run (Chapter 8), so powerline and icon glyphs are in the *primary* face rather than reached through fallback, sidestepping the metric problem for the glyphs most likely to hit it.

Styles are faces, not synthesis

The four `[font]` faces — normal, bold, italic, bold-italic — are real, separately loaded font files, not the bold and slant that a renderer can fake by smearing and shearing the normal face. Synthesized bold thickens strokes unevenly and synthesized italic shears the box-drawing glyphs into gaps; loading the designed face avoids both. When you omit a face's `family`, it falls back to `normal`'s family at the requested style, so a font that ships a genuine bold is used for bold automatically, and only a font that lacks one falls back to the normal weight rather than a synthesized approximation. The interaction with colour is worth recalling here: `draw_bold_text_with_bright_colors` (Chapter 12) couples the bold *weight* to the bright *palette*, a legacy convention that some colour schemes still expect, off by default so bold means heavier rather than brighter.

Chapter 11 – Grapheme Clusters, Shaping, and Ligatures

Chapter 3 established that the grid stores cells, not characters, and that width accounting decides how many cells an incoming scalar claims. Chapter 10 established that the font is a chain of faces, walked on fallback. This chapter sits between them and answers the question those two raise but do not settle: when a run of bytes decodes to something more complicated than one character per cell — a base letter with three combining marks, a family emoji built from five code points joined by zero-width joiners, a flag made of two regional-indicator symbols, a programming-font ligature like `!=` — what does a grid terminal actually do? The answer is inherited from the Alacritty lineage, and it is a set of deliberate choices, not gaps. A grid terminal is not a word processor, and understanding where it draws that line is the difference between a prompt that lines up and one that does not.

The grid is the contract

Everything in this chapter follows from one fact stated in Chapter 3: a terminal maintains a fixed grid, and every application that draws into it — vim, tmux, htop, your shell's line editor — computes its own layout by assuming a specific, stable mapping from characters to cells. When the shell's line editor puts the cursor after your twelfth typed character, it moves the cursor by the *cell width* it believes those twelve characters occupy. If the terminal and the application disagree about that width by even one cell, the cursor lands in the wrong place, redraws smear, and the display desynchronizes. So the terminal's first obligation, above prettiness, is to agree with every application about exactly how many cells each thing occupies. That contract is what constrains shaping, and it is why a terminal cannot freely reshape text the way a browser or a word processor can.

Grapheme clusters: many code points, one cell

A *grapheme cluster* is what a human calls “a character”: a base scalar plus any combining marks that attach to it. The grid resolves each cluster to a width and stores it against a single cell (a wide cluster against a cell plus its spacer, per Chapter 3). The cases the terminal must get right:

- **Combining marks.** `e` followed by U+0301 (combining acute) is one grapheme, `é`, one cell wide, advancing the cursor by one. The combiner is zero-width (Chapter 3); it is stored with the base and rendered composed. A naïve terminal that advanced the cursor on the combiner would push every following cell one column right.
- **Variation selectors.** U+FE0F (emoji presentation) and U+FE0E (text presentation) select whether the preceding scalar draws as a colourful emoji or a monochrome glyph, without advancing the cursor.  versus  is the same base scalar with and without the selector — the terminal must honour the selector's effect on rendering while treating it as zero-width for layout.
- **Zero-width joiners.** A ZWJ (U+200D) sequence fuses several emoji into one: the “family” and “profession” emoji are code-point sequences the Unicode standard says render as a single glyph. Whether the fallback emoji font actually *has* that composed glyph is a font question (Chapter 10); whether the terminal keeps the whole sequence anchored to one grid position, so the cursor advances once, is the terminal's question, and the grid keeps the cluster together.
- **Regional indicators.** A flag is two regional-indicator scalars; the pair is one grapheme, drawn as a flag glyph if the font has it and as two letter-boxes if it does not — but either way occupying the width the pair

resolves to, so layout is stable regardless of whether the flag renders.

The through-line is that grapheme clustering is *layout* work the grid must do, and it does it, because getting it wrong breaks the width contract. What the terminal does *not* promise is that every exotic cluster has a glyph — that is the font's job, and a missing composed emoji falls back to its component glyphs or a tofu box (Chapter 10) without the layout shifting.

Shaping, and why a grid terminal does little of it

Shaping is the process a full text engine performs to turn a run of characters into positioned glyphs: applying the font's substitution and positioning tables to form ligatures, position marks precisely, and adjust spacing (kerning) between letters. A browser shapes every run. A terminal in the Alacritty lineage deliberately does almost none of it, and the reason is the grid contract above. Kerning would pull letters closer than one cell, breaking the monospace assumption every TUI relies on. Contextual substitution that changed a glyph's advance would change how many cells it occupies, desynchronizing the application's cursor arithmetic. So the terminal renders essentially per-cell: each grapheme cluster is rasterized and placed in its cell (via the atlas of Chapter 4), marks are composed onto their base within that cell, and the pen does not carry kerning-adjusted positions across cell boundaries. This is not an oversight; it is the price of the guarantee that the terminal and the application always agree on the grid.

Ligatures: the honest position

Programming-font ligatures — `!=` drawn as a single “not-equal” glyph, `=>` as an arrow, `>=` and `<=` and `===` and `---` collapsed into single marks — are the most requested feature that the grid model resists, and the honest statement is that a per-cell grid terminal in this lineage does not perform them, by the same logic that rules out kerning. A ligature spans two or more cells and replaces their separate glyphs with one; but each of those cells still has to be individually addressable, selectable, and overwritable, because the application believes `!` and `=` are two distinct grid positions it can move the cursor between, erase one of, or colour differently. The moment the cursor lands between the `!` and the `=`, or the application repaints only the `=`, a fused ligature glyph has to come apart — and reconciling “draw as one” with “address as two” is exactly the complexity the Alacritty lineage chose not to carry, in exchange for a simpler, faster, always-consistent renderer. `zterminal` inherits that choice. The practical consequence is small and worth stating plainly so no reader expects otherwise: your `!=` renders as a clean `!` and `=` in adjacent cells, sharp and correctly spaced, rather than as a fused arrow glyph. Users who want ligatures choose a terminal that pays the cost of full shaping; users who want the grid's speed and exactness — the workflow of Chapter 57 — get the per-cell renderer, and this book documents what the program does rather than rounding up to what a different design would.

Where box drawing fits

There is one place a grid terminal *does* substitute glyphs wholesale, and it is instructive because it proves the rule. The built-in box-drawing font (Chapters 4 and 10, `font.builtin_box_drawing`) replaces the font's own box, block, and powerline glyphs with geometry the renderer draws itself. This is allowed — indeed preferred — precisely because those glyphs are *defined* to occupy exactly one cell and to meet their neighbours at the cell edge. Substituting them changes nothing about width or addressing; it only makes the lines meet. It is the one substitution that does not fight the grid contract, which is why it is the one the terminal makes eagerly while declining kerning and ligatures. The box-drawing case and the ligature case are the two poles of the same principle: substitute freely where the grid is preserved, never where it is broken.

Chapter 12 — Colors

`[colors]` is the largest configuration surface and the one the control panel's theme picker most visibly drives (Chapter 20). Colours are `#RRGGBB` hex strings, or — for the cell-relative fields — the special references `CellForeground` and `CellBackground`. The full structure, sub-table by sub-table:

`colors.primary`

- `foreground` — default text colour. Default `"#d8d8d8"`.
- `background` — default background. Default `"#181818"`.
- `dim_foreground` — dim text; auto-derived from `foreground` if unset. Default `"#828482"`.
- `bright_foreground` — used only when `draw_bold_text_with_bright_colors` is `true`; falls back to `foreground` if unset. Default `"None"`.

`colors.cursor` and `colors.vi_mode_cursor`

Each is `{ text, cursor }`, accepting hex or the cell references. The cursor colours apply to the terminal cursor; the vi-mode pair applies in vi mode, falling back to the normal cursor's active value. Both default to `{ text = "CellBackground", cursor = "CellForeground" }`.

`colors.search`

- `matches = { foreground, background }` — non-focused search matches. Default `{ "#181818", "#ac4242" }`.
- `focused_match = { foreground, background }` — the currently focused match. Default `{ "#181818", "#f4bf75" }`.

`colors.hints`

- `start = { foreground, background }` — the first character of a hint label. Default `{ "#181818", "#f4bf75" }`.
- `end = { foreground, background }` — the remaining label characters. Default `{ "#181818", "#ac4242" }`.

The bars and selection

- `line_indicator = { foreground, background }` — the history-position indicator during search and vi mode; `"None"` uses the opposing primary colour. Default `{ "None", "None" }`.
- `footer_bar = { foreground, background }` — the bottom bar used by the search regex input and hyperlink preview. Default `{ "#181818", "#d8d8d8" }`.
- `selection = { text, background }` — drawing colours for selections, hex or cell references. Default `{ "CellBackground", "CellForeground" }`.

The ANSI palettes: `normal`, `bright`, `dim`

Each is the sixteen-colour set keyed `black red green yellow blue magenta cyan white`. `normal` and `bright` ship a base16 palette (for example `normal.red` is `"#ac4242"`, `bright.red` is `"#c55555"`); `dim` is auto-calculated from `normal` if unset, with documented defaults (`dim.red` is `"#712b2b"`). Appendix B lists every default.

Indexed and the two booleans

- `indexed_colors = [{ index, color }]` — overrides for palette indices 16–255; unset entries are filled with sensible defaults. Default `[]`.
- `transparent_background_colors = true | false` — apply `window.opacity` to every cell background, not just the default one. Default `false`.
- `draw_bold_text_with_bright_colors = true | false` — draw bold text in the bright palette. Default `false`.

A complete theme, as one file

Because `[colors]` is large, a theme is usually kept in its own file and pulled in with `general.import` (Chapter 8), so a colour scheme is a single unit you can swap. A full example — the base16 default written out explicitly, the form any imported theme takes:

```
[colors.primary]
foreground = "#d8d8d8"
background = "#181818"

[colors.normal]
black = "#181818"
red = "#ac4242"
green = "#90a959"
yellow = "#f4bf75"
blue = "#6a9fb5"
magenta = "#aa759f"
cyan = "#75b5aa"
white = "#d8d8d8"

[colors.bright]
black = "#6b6b6b"
red = "#c55555"
green = "#aac474"
yellow = "#feca88"
blue = "#82b8c8"
magenta = "#c28cb8"
cyan = "#93d3c3"
white = "#f8f8f8"

[colors.selection]
text = "CellBackground"
background = "CellForeground"

[[colors.indexed_colors]]
index = 16
color = "#dc9656"
```

Three details in that file repay attention. The `dim` palette is absent, and that is deliberate: leave it out and

zterminal auto-derives it from `normal` (the documented `dim.red = "#712b2b"` and friends of Appendix B), so a theme need only supply the sixteen it cares about. The selection uses the `CellForeground` / `CellBackground` references rather than fixed hex, so a selection inverts whatever the cell already shows and stays legible over any content — the reason those special references exist. And the single `[[colors.indexed_colors]]` entry overrides exactly one slot of the 256-palette (index 16, the first colour past the ANSI sixteen) while every other indexed slot keeps its computed default; this is the array-of-tables form the Settings tab's indexed-colour editor (Chapter 20) appends to. Swap the imported file and the whole scheme changes live, because the import is re-read on reload like any other config.

Chapter 13 — Cursor, Bell, Selection, Scrolling

Four smaller tables finish the feedback surface. Each field in full.

[cursor]

- `style = { shape, blinking }` — `shape` is `Block`, `Underline`, or `Beam` (default `Block`); `blinking` is `Never`, `Off`, `On`, or `Always` (default `Off`). The distinction between `Off/On` and `Never/Always` is whether an application may override the blink: `Off/On` set a default an app can change, `Never/Always` force it.
- `vi_mode_style = { shape, blinking } | "None"` — the cursor in vi mode; `"None"` falls back to the normal cursor's active value. Default `"None"`.
- `blink_interval = <integer>` — blink period in milliseconds. Default `750`.
- `blink_timeout = <integer>` — seconds after which blinking stops; `0` never stops. Default `5`.
- `unfocused_hollow = true | false` — draw a hollow box when the window is unfocused. Default `true`.
- `thickness = <float>` — beam/underline thickness as a fraction of the cell width, `0.0–1.0`. Default `0.15`.

[bell]

- `animation` — the visual-bell easing curve: one of the `EaseOut*` family (`Ease`, `EaseOut`, `EaseOutSine`, `EaseOutQuad`, `EaseOutCubic`, `EaseOutQuart`, `EaseOutQuint`, `EaseOutExpo`, `EaseOutCirc`) or `Linear`. Default `"Linear"`.
- `duration = <integer>` — flash length in milliseconds; `0` disables the visual bell. Default `0`.
- `color = "<string>"` — flash colour. Default `"#ffffff"`.
- `command = "<string>" | { program, args }` — a program run on every bell; `"None"` runs nothing. Default `"None"`.

[selection]

- `semantic_escape_chars = "<string>"` — the characters that bound a “semantic word” (the unit double-click and the `Semantic*` vi motions select). Default is a set of punctuation and whitespace.
- `save_to_clipboard = true | false` — copy every selection to the clipboard automatically, not only to the primary selection. Default `false`.

[scrolling]

- `history = <integer>` — scrollbar line count; `0` disables scrolling, capped at `100000`. Default `10000`.
- `multiplier = <integer>` — lines moved per scroll-wheel notch. Default `3`.

Scrollback: what it costs and where the cap comes from

`scrolling.history` looks like a free dial, and it is not quite, which is why it has a cap. Every line of scrollback is retained cells — characters plus their colours and attributes (Chapter 3) — held in memory for as long as the line stays in the buffer, and a pane that scrolls a firehose keeps replacing the oldest lines with new ones once the buffer is full. The default of `10000` lines is a comfortable balance; raising it to the `100000` maximum keeps a very deep history at a correspondingly larger memory cost *per pane*, which matters precisely because each native pane and each tmux pane has its own scrollback (Chapters 24, 28) — deep history multiplied across many panes adds up. Setting `history = 0` disables scrolling entirely, which is the right choice for a pane that only ever runs a full-screen application on the alternate screen (Chapter 3), where scrollback is never consulted anyway. The cap is not arbitrary paternalism; it is the point past which a single pane’s history would consume more memory than any real workflow benefits from, and it is why the field is bounded rather than unbounded.

The bell as a feedback channel

The bell is easy to dismiss and worth a second look, because `[bell]` turns a single event — the `BEL` byte, `0x07` — into a configurable feedback channel. An application rings the bell to say “something happened”: a completed compile, a finished download, a line matched in `less`, a message in an IRC client. By default zterminal does nothing visible with it (`bell.duration = 0` disables the visual flash), but the table lets you route the event three ways at once — a visual flash with a chosen colour and easing curve (`animation`, `duration`, `color`), and a program run on every ring (`command`). The `command` form is the powerful one: a bell can trigger a desktop notification, a sound, or any script, which turns “the build finished and rang the bell” into “the build finished and my notifier told me.” It overlaps deliberately with the output triggers of Chapter 32 — a trigger fires on matched *text*, a bell command fires on the `BEL` byte — and which you reach for depends on whether the program announces itself by ringing or by printing.

`semantic_escape_chars`, and what a word is

`selection.semantic_escape_chars` looks like a minor tuning knob and is in fact the single definition of “word” that several unrelated features share. It is the set of characters that *bound* a semantic word — the characters that are not part of a word, so a word is the run of characters between them.

The features that consult it are worth listing, because the sharing is the point. Double-clicking selects a semantic word (Chapter 38). The `w`, `b`, and `e` motions in vi mode traverse semantic words, while `W`, `B`, and `E` traverse whitespace-delimited ones (Chapter 18). The `Alt+V` semantic selection and the `SemanticSearch*` action family use the same boundaries. So one field decides what the mouse selects, what the keyboard motions skip over, and what the semantic search matches — and because it is one field, those three never disagree with each other, which is the reason it is configuration rather than a hard-coded “whitespace.”

The tuning question is what you want a double-click to grab. The default set is punctuation and whitespace, which means double-clicking inside a path like `/usr/local/bin/zterminal` grabs a single path component rather than the whole path, because `/` is a boundary. Remove `/` from the set and a double-click takes the whole path — convenient for filesystem work, and less convenient in prose or in code where a slash genuinely separates two things you might want individually. The same reasoning applies to `.` (grabbing `example.com` whole versus `example` and `com` separately), `-` (hyphenated flags and identifiers), and `:` (a `host:port` pair, or a `file:line` reference). There is no universally correct set; there is the set that matches the text you actually double-click, and the field exists so you can say which that is.

The interaction with hints (Chapter 15) is worth noting, because the two solve adjacent problems differently. A hint matches a *regex* you wrote, so it recognizes a URL or a git hash precisely regardless of what characters

happen to bound a word. Semantic selection applies one *boundary rule* uniformly to everything on screen. Use the boundary rule to make ordinary double-clicking match your habits; reach for a hint when you want a specific, structured thing recognized exactly.

Chapter 14 — Terminal and Mouse

Two tables cover the shell and the pointer.

[terminal]

- `shell = "<string>" | { program, args }` — the shell to spawn; entries in `args` are passed to it. Default is `$SHELL` or the login shell on Linux/BSD/macOS, `powershell` on Windows. Example: `{ program = "/bin/zsh", args = ["-l"] }`.
- `osc52 = "Disabled" | "OnlyCopy" | "OnlyPaste" | "CopyPaste"` — controls clipboard access via the `OSC 52` escape. Reading the clipboard from a remote application is the abuse risk, so the default permits only writing. Default `"OnlyCopy"`.

[mouse]

- `hide_when_typing = true | false` — temporarily hide the pointer while typing. Default `false`.
- `bindings = [{ mouse, mods, mode, action | chars | command }]` — mouse bindings, sharing the `[keyboard]` model (Chapter 16). `mouse` is `Left`, `Right`, `Middle`, `Back`, `Forward`, `WheelUp`, `WheelDown`, or an integer button; the action set adds `ExpandSelection`. When an application captures the mouse, holding `Shift` suppresses reporting so the binding fires; if no Shift-binding exists, the un-Shifted action is used. Example: `{ mouse = "Right", mods = "Control", action = "Paste" }`.

Mouse reporting and the Shift override

The pointer has two lives in a terminal, and the boundary between them is the source of most mouse confusion. Ordinarily the mouse is the *terminal's* — a drag selects, a right-click expands the selection, a middle-click pastes the primary selection (Chapter 38). But a full-screen application can *capture* the mouse by enabling a mouse-reporting private mode (1000/1002/1006, Appendix D), after which clicks and drags are encoded as escape sequences and sent to the application instead: this is how `tmux` lets you click a pane, how `vim` positions the cursor on click, how `htop` sorts by clicking a column. When an application has captured the mouse, holding `Shift` suppresses the reporting for that action, so the terminal's own binding fires — a `Shift`-drag selects text even inside a mouse-grabbing TUI. The default bindings encode this: `Middle` (~Vi) pastes the primary selection, and the `Right` expand-selection bindings work whether or not an application is grabbing (Appendix C). The rule to remember is simple — the application owns the mouse when it asks to, and `Shift` takes it back for one action — and it is why “I can’t select text in this program” is answered by holding `Shift` while you drag.

Chapter 15 — Hints

Hints turn visible terminal text into actionable targets: find every URL, path, hash, or hyperlink on screen and open it, copy it, select it, or pipe it to a command — by keyboard or mods-click. They are configured in `[hints]`.

`hints.alphabet`

The keys used for hint labels. Default `"jfkdl;ahguriewpq"` — a home-row-biased set so labels are fast to type. The label's first character is coloured by `colors.hints.start`, the rest by `colors.hints.end`.

`hints.enabled`

An array of hint definitions. Each must carry at least one *matcher* and at least one *effect*, plus optional refinements:

- `regex = "<string>"` — a pattern matched against each line.
- `hyperlinks = true | false` — match OSC 8 escape-sequence hyperlinks.
- `post_processing = true | false` — trim characters unlikely to belong to a match (a trailing `.` on a URL); `regex` matches only.
- `persist = true | false` — keep hints visible after one is chosen.
- `action = "Copy" | "Paste" | "Select" | "MoveViModeCursor"` — the built-in effect.
- `command = "<string>" { program, args }` — a program run with the match appended as its last argument.
- `binding = { key, mods, mode }` — the key that starts keyboard hint selection.
- `mouse = { mods, enabled }` — `enabled` underlines the match on hover with `mods` held, making it clickable.

The default and a custom hint

The shipped default opens URIs and hyperlinks with the platform opener (`xdg-open`, `open`, or `cmd /c start`), bound to `Ctrl+Shift+O` and to mods-click, with `post_processing` on. Adding your own is a few lines — a hint that copies a git hash:

```
[[hints.enabled]]
regex    = "[0-9a-f]{7,40}"
action   = "Copy"
binding  = { key = "G", mods = "Control|Shift" }
```

You need not hand-edit the TOML: the control panel's **Hints** tab (Chapter 19) is a graphical editor for this same `[hints]` table — a form for the regex, the built-in action or piped command, the hyperlink / post-processing / persist flags, and the key and mouse triggers, plus the `hints.alphabet`. It rewrites the whole list on every change, and the running windows reload it at once.

How matching and refinement actually work

The two required ingredients — a matcher and an effect — and the optional refinements combine in an order worth understanding, because it explains the otherwise-mysterious `post_processing` flag. When you trigger hints, every enabled definition is matched against the visible text: a `regex` definition tests each line against its pattern, and a `hyperlinks = true` definition matches any `OSC 8` hyperlink on screen (Chapter 6). Each match becomes a labelled target, the label drawn from `hints.alphabet` and coloured by `colors.hints` (Chapter 12). You type the label; the effect — `action`, `command`, or the built-in open — runs on the matched text.

`post_processing` exists because regexes over-match at the edges. A URL regex will happily swallow the trailing `.` of a sentence or the `)` that closed a parenthetical, so the “copy this URL” you meant includes punctuation that is not part of the URL. With `post_processing = true` the matched span is trimmed of characters unlikely to belong to it before the effect runs (regex matches only — hyperlinks are exact by construction and need no trimming). `persist = true` keeps the hint overlay up after you pick one, for selecting several targets in a row rather than re-triggering each time. And a definition can carry *both* a keyboard `binding` and a `mouse` trigger, so the same target is reachable by typing a label or by mods-clicking an underlined match — the two access paths to one hint. The practical upshot is that a good hint is a matcher tight enough to catch the thing, `post_processing` on to clean the edges, and whichever trigger fits your hands.

The alphabet, and why labels are typeable

`hints.alphabet` defaults to `"jfkdl;ahguriewpq"`, and the ordering is not arbitrary — it is home-row-biased, with the strongest fingers first. The reason is arithmetic. With an alphabet of n characters, the first n matches on screen get one-character labels and everything beyond that needs two, so the alphabet's length sets how many targets are reachable with a single keystroke. Its *composition* sets how fast those keystrokes are: a label you have to hunt for on the keyboard costs more than the mouse movement the hint was supposed to replace.

This is also why the first character of a label is coloured differently from the rest (`colors.hints.start` versus `colors.hints.end`, Chapter 12). With two-character labels on screen, you are typing a prefix and then narrowing, and the colouring tells you at a glance which character you are about to consume. If you find yourself regularly seeing two-character labels, the fix is usually a tighter matcher rather than a longer alphabet — fewer, better matches beat more labels.

Pitfalls worth knowing before writing one

Four things bite people writing their first custom hint, and all four follow from the mechanics above.

A loose regex is worse than no hint. A pattern like `[0-9a-f]{7,40}` for a git hash will also match hex colours, checksums, parts of UUIDs, and any lowercase word made of the letters `a–f`. Every one of those becomes a labelled target competing for the short labels, which pushes the thing you actually wanted onto a two-character label. Tighten by context where you can.

Regexes match a line, not the screen. Each pattern is tested against each visible line independently, so a URL that wrapped across two rows is two partial matches rather than one whole one. This is the same line-scoped property triggers have (Chapter 32) and it is worth remembering when a long path or link “does not get a hint” — it did, twice, in pieces.

command receives the match as its last argument, not on stdin. The idiom in Appendix H — `{ program = "sh", args = ["-c", "$EDITOR \"$1\"", "sh"] }` — exists because that is how you get the match into an arbitrary shell expression: `sh -c` takes the script, then a `$0`, and the appended match becomes `$1`. Quote it, since matched text is by definition text you did not choose.

Hints open with the platform opener, and matched text is untrusted. A URL hint hands attacker-influenceable text to `xdg-open`, `open`, or `cmd /c start` (Chapter 39). `post_processing` trimming a stray trailing `)` is as much a correctness aid as a convenience, and the ordinary caution about where a link goes applies exactly as it does in a browser.

What hints are, structurally

Stripped to the mechanism, a hint is three things: a *matcher* over visible text, a *label* the alphabet assigns, and an *effect* — and every one of those parts is shared with something else in the terminal. The effect vocabulary (`Copy`, `Paste`, `Select`, `MoveViModeCursor`) is the action set of Chapter 16. `Select` and `Copy` build a selection and hand it to the clipboard model of Chapter 38. `MoveViModeCursor` positions the vi cursor (Chapter 18), which is how a regex match becomes the starting point for keyboard motions. And `hyperlinks = true` matches the `OSC 8` sequences the parser already recovers (Chapter 6).

That layering is why the feature is small: hints do not implement selection, copying, opening, or cursor movement. They implement *finding*, and delegate the rest — which is the same method the whole extension layer follows, applied to a piece of the inherited core.

Chapter 16 – Keyboard and Bindings

The binding model is the most important inherited surface to understand in detail, because every action zterminal can perform — inherited or original — is reachable as a binding, and the model is more expressive than “key maps to string.”

The shape of a binding

`keyboard.bindings` is an array of tables, each combining a trigger and an effect. The trigger is a `key`, optional `mods`, and optional `mode`; the effect is exactly one of `action`, `chars`, or `command`.

```
[keyboard]
bindings = [
  { key = "N", mods = "Control|Shift", action = "CreateNewWindow" },
  { key = "L", mods = "Control|Shift", chars = "l" },
]
```

- `key` is a named key ("A", "F1", "Enter", the "Numpad..." family), a literal character, or a decimal scancode. The named-key vocabulary is winit's.
- `mods` combine with |: `Command`, `Control`, `Option`, `Super`, `Shift`, `Alt`.
- `mode` requires (or, with a ~ prefix, forbids) a terminal mode: `AppCursor`, `AppKeypad`, `Search`, `Alt`, `Vi`. "`~Vi|Search`" means “not in vi mode, and in search mode.” This is how the same physical key does different things by context.
- `action` runs a built-in action; `chars` writes a literal string to the PTY; `command` forks a program.

Two escape hatches: `action = "ReceiveChar"` removes a default binding by letting the key fall through as input, and `action = "None"` inhibits the key entirely. Multiple bindings can fire on one press, in definition order.

The action catalogue, by group

Seeing the groups is more useful than memorizing the list (Appendix C is the full default table; `zterminal(5)` lists every action):

- **Always available** — `Paste`, `Copy`, `IncreaseFontSize/DecreaseFontSize/ResetFontSize`, the eight scroll actions (`ScrollPageUp/Down`, `ScrollHalfPageUp/Down`, `ScrollLineUp/Down`, `ScrollToTop/Bottom`), `ClearHistory`, `ClearLogNotice`, `Hide`, `Minimize`, `Quit`, `ToggleFullscreen`, `ToggleMaximized`, `ToggleViMode`, `ClearSelection`, `SearchForward/SearchBackward`, `SpawnNewInstance`, `CreateNewWindow`, `ReceiveChar`, `None`.
- **Vi mode** — the motions, selections, and inline/semantic searches of Chapter 18 (`Up/Down/Left/Right`, `First/Last/FirstOccupied`, `High/Middle/Low`, the `Semantic*` and `Word*` family, `Bracket`, `ParagraphUp/Down`, the `Toggle*Selection` family, `SearchNext/Previous/Start/End`, `Open`, `CenterAroundViCursor`, the `InlineSearch*` and `SemanticSearch*` families).
- **Search mode** — `SearchFocusNext/Previous`, `SearchConfirm`, `SearchCancel`, `SearchClear`, `SearchDeleteWord`, `SearchHistoryPrevious/Next`.
- **macOS exclusive** — `ToggleSimpleFullscreen`, `HideOtherApplications`, `CreateNewTab`, `SelectNextTab/PreviousTab`, `SelectTab1–9`, `SelectLastTab`.
- **Linux/BSD exclusive** — `CopySelection`, `PasteSelection`.

A deliberate non-collision

The MenkeTechnologies extension shortcuts (Chapters 19–32) are layered so that *no zterminal binding masks a shell or tmux key on any OS*. macOS uses `⌘`, which the shell never sees; Linux and Windows use `^↑` or route through the command palette so no bare key is consumed. This is a stated design rule, and it is why a `^`-heavy TUI or a tmux session runs under zterminal without the terminal stealing keys. The binding model is inherited; this discipline about how the extensions use it is the product's, and Chapter 34 notes the single-dispatch path (menu, key, and palette all routing the same action object) that enforces it.

Recipes: the model in practice

The three effect kinds — `action`, `chars`, `command` — and the `mode` guard combine into a small vocabulary that covers most real customization. A handful of worked bindings show the range:

```
[keyboard]
bindings = [
  # Write a literal string to the PTY only when NOT in vi or search mode, so the
  # same physical key still drives vi motions when vi mode is active (Chapter 18).
  # A TOML \u escape sends a control byte – here U+000C, form feed:
  { key = "K", mods = "Control", mode = "~Vi|~Search", chars = "\u000C" },

  # Fork a program with the current selection unavailable to it – a pure launcher:
  { key = "T", mods = "Command|Shift", command = { program = "tmux", args = ["new-window"]
    ~ } },

  # Remove a shipped default by letting the key fall through as ordinary input:
  { key = "N", mods = "Control|Shift", action = "ReceiveChar" },

  # Inhibit a key entirely so nothing – not even the character – reaches the PTY:
  { key = "Q", mods = "Control", action = "None" },

  # A binding that only exists inside search mode (Chapter 18):
  { key = "P", mods = "Control", mode = "Search", action = "SearchHistoryPrevious" },
]
```

The pattern worth internalizing is the `mode` guard. Because a binding can require or forbid a mode, the *same* key can mean one thing at the shell prompt, another in vi mode, and a third in search — which is exactly how the inherited core layers its own vi and search keymaps over the pass-through keys without collision, and how you extend that layering without stealing a key from an application. A binding with `mode = "~Vi|~Search"` is the idiom for “only when the terminal is behaving as a plain terminal,” and it is the safe default for any custom key that should not interfere with the modal features.

Reading the default table

Appendix C reproduces the shipped defaults, and two conventions there are easy to misread. The `~` prefix on a mode is a *negation* — `~Vi|~Search` means “not in vi mode and not in search mode,” so the binding is suppressed while either is active. And where two bindings share a key with different mode guards, both are in the table and the guard decides which fires: `Ctrl+L` maps to `ClearLogNotice` always and to the clear-screen character only under `~Vi|~Search`, so the key clears a transient log notice if one is showing and otherwise clears the screen. This is the binding model's expressiveness on display in the defaults themselves, and it is the reason the table has more rows than there are distinct keys.

Chapter 17 – Debug

[`debug`] exists to troubleshoot the emulator. Its fields can change or be removed without warning, so they are not stable API, but they are the windows into the core. Each field:

- `render_timer = true | false` — overlay the time to draw each frame. Default `false`.
- `persistent_logging = true | false` — keep the log file after quitting. Default `false`.
- `log_level = "Off" | "Error" | "Warn" | "Info" | "Debug" | "Trace"` — log verbosity. Default `"Warn"`. The `ZTERM_EXTRA_LOG_TARGETS` variable adds extra libraries (`ZTERM_EXTRA_LOG_TARGETS="winit;vte" zterminal -vvv`).
- `renderer = "glsl3" | "gles2" | "gles2pure" | "None"` — force a renderer backend; `"None"` picks the highest available. Default `"None"`.
- `print_events = true | false` — log all window events. Default `false`.
- `highlight_damage = true | false` — highlight the changed-cell damage regions. Default `false`.
- `prefer_egl = true | false` — use EGL where allowed (transparency may not work with EGL on Linux/BSD). Default `false`.

The command-line `-v/-q` flags and `--print-events` reach the same machinery; the control panel's Logs tab (Chapter 19) renders the log this produces.

Reading the log

The log at `~/.zterminal/zterminal.log` is the terminal's own account of what it did, and `debug.log_level` decides how much of that account it keeps. The levels form the usual ladder — `Off`, `Error`, `Warn` (the default), `Info`, `Debug`, `Trace` — each including everything louder than it, so `Warn` records errors and warnings but not the routine `Info` chatter, while `Trace` records essentially everything the terminal considers worth a line. The command-line `-v` flags map onto this ladder (`-vvv` is the loudest), and `-q` moves the other way for a quieter run. Two refinements matter in practice. `debug.persistent_logging` decides whether the log survives quitting — off by default, so a clean exit does not leave a file behind, on when you are chasing a bug across restarts. And `ZTERM_EXTRA_LOG_TARGETS` widens the net to *other* libraries' logs — `winit` for window-system events, `vte` for the parser — which is how you distinguish “the terminal mishandled a sequence” from “the window system delivered a strange event” when a symptom could be either. The Logs tab (Chapter 19) renders this same file live, so the log is readable without leaving the terminal, and everything the panel shows is text that already went to disk.

The renderer and EGL fields, and when to touch them

Two of the [`debug`] fields are not logging controls at all; they select graphics backends, and they are the fields most likely to be the actual fix when a symptom is visual rather than behavioural. Both default to letting the program choose, and the correct posture is to leave them alone until a specific symptom points at them.

`debug.renderer` forces one of `"glsl3"`, `"gles2"`, or `"gles2pure"`, with `"None"` — the default — picking the highest available. The reason a manual override exists is that “highest available” is a claim made by the driver, and drivers occasionally claim support for a path they implement badly. The symptom that indicts the automatic choice is a rendering fault that is *consistent and structural* rather than intermittent: glyphs drawn at the wrong offset, the atlas appearing as visible tearing across cells, colours transposed. Stepping down to a lower backend is a one-field experiment that either fixes it or eliminates the hypothesis, and it does so

without a rebuild. If a lower backend fixes it, the finding is about the driver, not about the terminal, and it is worth recording that in the report.

`debug.prefer_egl` selects EGL where the platform allows it, and this chapter's field list carries the caveat that matters: transparency may not work with EGL on Linux and the BSDs. That caveat is the whole decision procedure. If `window.opacity` is configured and the window is stubbornly opaque, this field is one of the small number of things that can cause it, alongside the compositor questions Appendix G raises. Turning it off is the test. Conversely, if the terminal fails to create a window at all on a system whose graphics stack prefers EGL, turning it on is the test. It is a two-value field with one documented interaction, which makes it unusually easy to reason about.

The reason both are grouped under `[debug]` rather than under `[window]` or a hypothetical graphics table is the stability disclaimer at the top of the chapter: these fields can change or be removed, because they expose an implementation choice rather than a user-facing behaviour. A configuration that depends on one of them is depending on something the program does not promise to keep, which is fine as a workaround for a specific machine's driver and not fine as a line you copy into a shared base file (Chapter 8). Treat them as diagnostics that happen to be persistent, not as settings.

Chapter 18 – Vi Mode and Search

zterminal's core includes a keyboard-driven mode for moving through scrollback, selecting text, and following links without a mouse, plus a search mode for finding text in the buffer. Both are inherited and modal in the vi sense.

Entering and leaving

`Ctrl+Shift+Space` toggles vi mode. A second cursor appears, decoupled from the terminal cursor, and the keyboard drives it. `i` or `Ctrl+C` leaves and returns to the bottom; `Escape` clears the active selection. The terminal keeps running underneath — vi mode is a reading overlay on a live buffer, not a frozen snapshot.

Motion

The motions are vi's, on a terminal grid: `h/j/k/l` and arrows move by cell; `0/$/^` go to first column / last column / first non-empty cell; `H/M/L` jump to top/middle/bottom of screen; `w/b/e` move by *semantic* word (bounded by `semantic_escape_chars`) and `W/B/E` by whitespace word; `{/}` move by paragraph; `%` jumps to the matching bracket; `g/G` go to top/ bottom of scrollback; `z` centers on the vi cursor. Scrolling: `Ctrl+b/Ctrl+f` page, `Ctrl+u/Ctrl+d` half-page, `Ctrl+y/Ctrl+e` line.

Selection and opening

`v` starts character selection, `V` line, `Ctrl+V` block, `Alt+V` semantic; pressing the same key toggles it off. `y` copies and clears. `Enter` runs `Open`, launching the URL under the vi cursor.

Inline, semantic, and buffer search

Inline search within the line: `f/F` to a character, `t/T` stopping short, `;/,` to repeat. Semantic search: `*/#` jump to the next/previous occurrence of the word under the cursor. Buffer search: `/` forward, `?` backward; matches highlight live using `colors.search`; `n/N` step through them; `Enter` confirms (or, outside vi mode, focuses the next match). In search mode, `Ctrl+U` clears, `Ctrl+W` deletes a word, `Ctrl+P/Ctrl+N` and the arrows walk the search history, `Escape/Ctrl+C` cancel. The footer bar shows the regex, coloured by `colors.footer_bar`.

The vi-mode key reference

Collected in one place, the vi-mode keymap — every key an action from the vi group of Chapter 16, so the prose above and the binding table agree:

Key	Effect
<code>h j k l</code> , arrows	move the vi cursor one cell

Key	Effect
<code>0 \$ ^</code>	first column / last column / first non-empty cell
<code>H M L</code>	top / middle / bottom of the visible screen
<code>w b e</code>	forward / back / end by <i>semantic</i> word
<code>W B E</code>	the same by <i>whitespace</i> word
<code>{ }</code>	previous / next paragraph
<code>%</code>	jump to the matching bracket
<code>g G</code>	top / bottom of scrollbar
<code>z</code>	center the view on the vi cursor
<code>Ctrl+b Ctrl+f</code>	page up / down
<code>Ctrl+u Ctrl+d</code>	half-page up / down
<code>Ctrl+y Ctrl+e</code>	line up / down
<code>v V Ctrl+V Alt+V</code>	start character / line / block / semantic selection
<code>y</code>	copy the selection and clear it
<code>Enter</code>	Open — launch the URL under the vi cursor
<code>f F t T</code>	inline search to / before a character, forward / back
<code>; ,</code>	repeat / reverse the last inline search
<code>* #</code>	next / previous occurrence of the word under the cursor
<code>/ ?</code>	start a buffer search forward / backward
<code>n N</code>	next / previous buffer-search match
<code>i</code> or <code>Ctrl+C</code>	leave vi mode, return to the bottom
<code>Escape</code>	clear the active selection

The table is the same action set the Keys tab (Chapter 19) exposes as rebindable rows, so nothing here is hard-wired: a user who prefers a different motion key rebinds it, and the change lands live like any other config. The two searches to keep distinct are *inline* search (`f/t`, within the current line, the classic vi character-find) and *buffer* search (`/`, across the whole scrollbar with live `colors.search` highlighting); they read similarly but operate at different scopes, and the semantic search (`*/#`) is a third, jumping by whole occurrences of the word the cursor sits on.

Why a terminal has a vi mode at all

It is a fair question why the emulator implements a modal editor's keymap rather than leaving scrollbar navigation to the scroll wheel and the search box. The answer is that the scrollbar is a body of text you frequently need to *operate* on — find a line, select a region precisely, copy exactly the right span, follow a URL — and every one of those operations is faster and more exact with a keyboard than with a pointer. A mouse selection of forty lines of log output means a drag that must scroll, which means a drag whose endpoints are

hard to place. `V` at the top line, `G`, `y` places them exactly.

The second reason is that the alternative is a separate program. The historical way to get keyboard scrollbar navigation was to run a multiplexer and use its copy-mode, which is the assembly tax of Chapter 1 in its smallest form: a whole extra process in the pipeline so that you can move a cursor through text the terminal already has in memory. Building the mode into the emulator removes the process, and it does so at essentially no cost, because the scrollbar and the selection machinery (Chapter 38) already exist for other reasons — `vi` mode is a keymap over data structures the terminal maintains regardless.

Vi mode versus tmux copy-mode

The two look similar and read from different places, and Chapter 31 gave the general rule; here is the specific one, since this is where users most often notice it.

`zterminal`'s `vi` mode navigates `zterminal`'s scrollbar for the focused pane — the buffer sized by `scrolling.history` (Chapter 13), the one that OSC 133 marks anchor into (Chapter 27) and that cross-pane search greps. In a native pane (Chapter 24) that buffer is the pane's entire history, so `vi` mode reaches everything the pane has ever printed within the configured depth.

Inside a `tmux` pane, the history belongs to the `tmux` server, and what `zterminal`'s grid holds is what `tmux` has painted. Entering `vi` mode there and pressing `g` walks `zterminal`'s buffer, not the pane's real history, which is why the correct instrument inside a `tmux` pane is `tmux`'s own copy-mode. This is not a bug in either program; it is two scrollbars existing because two programs each maintain a grid.

The practical consequence is a small habit: `vi` mode for native panes, copy-mode for `tmux` panes. The keymaps are similar enough — both are `vi`-flavoured — that the switch costs little, and the features that *do* span both (Exposé, cross-pane search, the process tree, broadcast) are the extension layer's answer to the underlying split.

Vi mode as the substrate for other features

Two things elsewhere in the book are `vi` mode wearing a different interface, and seeing them that way explains their behaviour. The `Open` action bound to `Enter` launches the URL under the `vi` cursor with the platform opener — the same opener a hint uses (Chapters 15, 39) — so `vi` mode is a keyboard path to the same targets hints label. And `action = "MoveViModeCursor"` on a hint (Chapter 15) runs the relationship the other way: a hint match *positions* the `vi` cursor, so you can jump to a regex match by label and then operate on it with motions and selections. The two features share the cursor and the selection model, which is why they compose instead of colliding.

The mode guards of Chapter 16 are the third connection. Because a binding can require `Vi` or forbid it with `~Vi`, the same physical key can be a `vi` motion while the mode is active and pass through to the shell when it is not — which is how a whole second keymap can exist over the terminal's keys without stealing a single one from the programs running under it.

Chapter 19 – The Control Panel

Here the book crosses the line drawn in Chapter 2. Everything to this point is the inherited, faithfully-documented core. The control panel is the first piece of the original extension layer, and the most architecturally distinctive: zterminal's *entire* configuration and inspection surface is a single-binary, in-process WebView application built only from the shared `zgui-core` component library and served over a custom `zterminal://` protocol.

The architecture

The control panel is not a separate process, not a remote web page, and not a native dialog set. It is a WebView hosted inside zterminal's own process. The frontend assets — `settings/frontend` plus the `zgui-core/webui` component library — are embedded into the binary with `include_dir!` and served over a custom `zterminal://` scheme. A JavaScript bridge (`window.__ztermInvoke`) posts messages to the Rust side, which dispatches them. The same fuzzy matcher, `ZGui.fzf`, powers the command palette, the shell-history palette, and cross-pane search — one component, reused, not three reimplementations. The repository frames this as a first: a terminal whose whole control plane is a reusable design-system app rather than native dialogs or a TUI (Chapter 34).

The glassy overlay

The panel and palette WebViews composite as translucent glass over the still- visible GPU terminal beneath. On macOS this is done by driving the native window's `alphaValue` directly through AppKit (winit's own transparency crashes the view), and a live opacity slider — backed by the `window.overlay_opacity` config field — sets it. The terminal keeps rendering underneath; the control plane floats over it.

One action, three front doors

Every macOS menu item carries the same key-equivalent as its keybinding and dispatches the *identical* action object through the same `run_palette_action` path as the key press and the command palette. Menu, key, and palette are three views of one action set, with one source of truth. This is what keeps the non-collision rule of Chapter 16 honest and what makes the command palette (Chapter 23) a complete index of everything the terminal can do.

The tabs

The tab strip is far broader than a settings dialog. Observed in the running app: Settings, Dashboard, Commands, Sessions, Layouts, tmux, tmux config, tmux buffers, tmux keys, Profiles, Env Vars, Snippets, Triggers, Hints, Processes, Recent Dirs, Keys, and Logs (plus an About panel). Each is a manager for one facet of the environment, and each has its own chapter or section ahead: Settings (Chapter 20), Profiles (Chapter 21), Dashboard (Chapter 22), Commands and the palettes (Chapter 23), Sessions and Layouts (Chapters 24, 28), the four tmux tabs (Chapter 28), Env Vars / Snippets / Triggers / Processes / Recent Dirs (Chapter 32), Hints (the graphical editor for the `[hints]` table of Chapter 15), Keys (the rebinding table from Chapter 16), and Logs (the `debug.log_level` output from Chapter 17). The README's older

six-tab list (Settings, Dashboard, tmux, Keybindings, Logs, About) describes an earlier, smaller panel; the running app is the authority.

The bridge, concretely

The claim “an in-process WebView that reads and writes the live config” rests on a specific, small mechanism, and naming it removes the magic. The frontend — the `settings/frontend` assets plus the `zgui-core/webui` component library — is embedded into the binary at compile time with `include_dir!`, so there is no directory of web files to install and nothing to fetch over the network; the whole control plane ships inside the one executable. It is served to the WebView over a custom `zterminal://` scheme rather than `http://`, so it is not a server listening on a port that another process could reach. Communication runs one way through a JavaScript bridge — `window.__ztermInvoke` posts a message from the frontend to the Rust side — and the Rust side dispatches each message to the handler that reads or writes the relevant state: a Settings-tab change becomes a `toml_edit` write to `~/.zterminal/zterminal.toml` (Chapter 20), a tmux-tab action becomes a wire command to the server (Chapter 28), a palette action becomes a `run_palette_action` call (below). The pattern is the same in every tab — the frontend renders and posts, the Rust side owns the state and acts — which is why one component library can present config, tmux, telemetry, and process management without each being a bespoke integration.

One dispatch path, three front doors

The single most load-bearing architectural choice in the panel is that a menu item, a keybinding, and a palette entry are three *views* of one action object, not three code paths. Every macOS menu item carries the same key-equivalent as its binding and dispatches the identical action through the same `run_palette_action` path the key press and the palette use. This is what keeps the non-collision rule of Chapter 16 honest — there is one place an action is defined, so there is one place to reason about which key it consumes — and it is what makes the command palette (Chapter 23) a *complete* index of what the terminal can do, because an action that exists is, by construction, an action the palette can dispatch. Add a capability once and it appears in all three front doors; that is the dividend of routing them through one object.

The tabs, at a glance

Gathered as a table, the panel's tabs and the chapter that documents each — the map of the whole control plane:

Tab	What it manages	Chapter
Settings	the live <code>zterminal.toml</code> , plus panel theming	20
Dashboard	live telemetry (PTY, render, tmux, system)	22
Commands	the command palette's action index	23
Sessions	tmux session save / restore	28
Layouts	native split-layout save / restore	24
tmux	the live session → window → pane tree	28
tmux config	the server's options, edited live	28
tmux buffers	the server's paste buffers	28
tmux keys	the server's key tables, rebound live	28
Profiles	whole-environment snapshots	21
Env Vars	<code>[env]</code> plus hot-inject into live shells	32
Snippets	reusable command snippets	32
Triggers	per-pane output-reaction regexes	32
Hints	a graphical editor for <code>[hints]</code>	15

Tab	What it manages	Chapter
Processes	the per-pane CPU/MEM process tree	30
Recent Dirs	the harvested working-directory list	32
Keys	the rebindable keybinding table	16
Logs	the <code>debug.log_level</code> output, live	17
About	version and identity	19

The breadth of that list is the point of the chapter: this is not a settings dialog with a few panes but a full management surface for the terminal, its multiplexer, and its environment, every tab a manager over live state and every one built from the same `zgui-core` components.

Chapter 20 – The Settings Tab

The Settings tab is the control panel's front page and the friendly face of the TOML from Chapters 8–17. Its header reads `ZTERMINAL · SETTINGS · AUTO-SAVES`, and a status line at the bottom reads `Editing ~/.zterminal/zterminal.toml` — it edits the canonical config in place, and because the config hot-reloads, every change lands in the live terminal immediately. A search box (“Search settings...”) and a profile selector sit above the content; a Reload button is in the header. Observed in the running app, the tab opens with two panel-theming blocks — Appearance and Effects — and then a full, searchable editor for the config file itself, one labelled section per table of Chapters 8–17.

Appearance

A **Light Mode** toggle, and a grid of eight named colour-scheme presets that theme the panel itself:

- **Cyberpunk** — hot pink + cyan neon (the default).
- **Midnight** — deep blue + electric purple.
- **Matrix** — terminal green on black.
- **Ember** — warm amber + orange.
- **Arctic** — cool whites + icy blue.
- **Crimson** — rose-red + teal.
- **Toxic** — acid-lime + magenta.
- **Vapor** — vaporwave pastel pink + cyan.

Below the presets is a full **palette editor** — Accent, Cyan, Magenta, Green, Yellow, Orange, Red, the four background tiers (Primary, Secondary, Card, Hover), Text, Text-Dim, Text-Muted, Border, and Link — with a *Save custom as preset* button. `zterminal` also carries a global `hsv` (hue/saturation/value) adjustment in its UI config, so a whole scheme can be rotated rather than edited swatch by swatch.

Effects

Four toggles for the panel's own visual treatment, the cyberpunk house style rendered in the WebView: **CRT scanlines**, **Bezel vignette**, **Neon glow**, and **Animations**.

The configuration editor

Below the two theming blocks the tab becomes a graphical front end for nearly the whole of `zterminal.toml`, laid out as one labelled section per table and each field a typed control — a toggle, a number stepper, a picker, a hex swatch, or a command-line box — that writes its value straight back to the file. The sections, in order: **General** (the `general.live_config_reload`, `general.ipc_socket`, and `general.working_directory` fields); **Font**; **Cursor**; then the full **Colors** family — Primary, the Normal / Bright / Dim sixteen-colour ANSI sets, Selection, Search, Vi-mode cursor, Line indicator, Footer bar, Hint labels, and an **Indexed (256-palette)** list editor that overrides any single slot 16–255 via `[[colors.indexed_colors]]`; **Window** (including startup **position** and the X11/Wayland **app class**); **Scrolling**; **Bell** (with a *Command on bell* box that runs a program on every ring); **Selection**; **Mouse & Terminal** (including the **Shell** each new terminal launches); and **Debug**. Two field kinds are worth naming: the command-line boxes for `bell.command` and `terminal.shell` are split into a `{ program, args }` on save by their own backend rather than stored as

a bare scalar, and the indexed-colour editor owns its own save path, appending array-of-tables rows. The thing to take from the detail is the architecture, not the list: a GUI that is part of the terminal process, themed in the stack's cyberpunk style, editing the live config file, and covering essentially every table the man page documents.

How a change reaches the file

The status line reading `Editing ~/.zterminal/zterminal.toml` is a literal claim, and the mechanism behind it is worth naming because it determines what happens to a file you also edit by hand. A control changed in the tab posts a message over the JavaScript bridge (Chapter 19); the Rust side applies it to the config file with `toml_edit`; the file changes on disk; the live-reload watch (Chapter 8) notices and the running windows apply the new configuration. The GUI is not a parallel configuration store that occasionally syncs to a file — the file is the store, and the GUI is one of two editors for it.

`toml_edit` is the load-bearing choice in that chain, because it is a *format-preserving* editor rather than a parse-and-reserialize round trip. The difference matters to anyone with a real config file: a parse-and-rewrite would read your TOML into data structures, write those structures back out, and in doing so discard your comments, your ordering, your blank-line grouping, and your formatting choices, replacing a file you had curated with a machine-generated one carrying the same values. Editing in place changes the value you asked to change and leaves the rest of the document byte-for-byte as you wrote it. This is what makes the two doors of Chapter 8 — hand-editing and the GUI — genuinely interchangeable rather than one silently destroying the other's work.

Two fields in the tab have their own save paths for structural reasons, and the chapter named them without explaining why. `bell.command` and `terminal.shell` are each either a bare string or a `{ program, args }` table (Chapters 13, 14), and a GUI text box collects one line of text; the backend splits that line into the program and its arguments on save, which is why they are described as having their own handling rather than writing a scalar straight through. The indexed-colour editor is the other: `[[colors.indexed_colors]]` is an array of tables (Chapter 12), so adding an override *appends a table* rather than setting a key, and removing one deletes an element from an array — a different edit shape from every other control in the tab, and one that has to preserve the other entries around it.

The search box, the profile selector, and what they scope

Two controls sit above the content and are easy to misread as decoration.

The search box (“Search settings...”) filters the sections and fields, which matters because the tab is a front end to essentially the whole schema of Chapters 8 to 17 — a surface large enough that scrolling to find `unfocused_hollow` is slower than typing it. It is a filter over the editor, not a search over your file's contents.

The **PROFILE** dropdown is the more consequential one, because it changes what “save” means at a larger scale. A profile (Chapter 21) captures the whole configuration, the panel's look, and the tmux server's state; the selector here is where you create, switch, and delete them. The running app shows it reading `– NO PROFILES` — before any exist, which is the honest default: until you make one, there is exactly one configuration and the tab edits it directly. Once profiles exist, the selector is the control that decides which whole environment you are in — and because a switch reaches the live tmux server over the wire, it is the one control in the tab whose effect extends outside the terminal process entirely.

Panel theming is not terminal theming

The Appearance and Effects blocks at the top of the tab theme the *panel*, and the Colors sections below theme the *terminal*, and conflating them produces a predictable confusion. The eight presets, the palette editor, the *hsv* adjustment, the CRT scanlines, the bezel vignette, the neon glow, and the animations are all properties of the WebView's own presentation — they change what the control panel looks like and have no effect on the grid behind it. The `[colors]` sections further down write `zterminal.toml` and change the cells (Chapter 12).

The reason both live in one tab is that both are things you look at, and the reason they are visually separated into their own labelled blocks is that they are stored differently: the panel's look is GUI preference state the control panel manages (Appendix E), while the terminal's colours are config keys in your TOML. A profile captures both, which is the one place they are treated as a unit — and that is deliberate, because “my environment looks like this” reasonably includes the panel.

Chapter 21 – Profiles

Profiles are the first of zterminal's three claimed in-repo firsts (Chapter 34). A profile is a named snapshot of the *entire* configuration that you switch between in one click — and it captures far more than the terminal config.

What a profile holds

- the full terminal config (`zterminal.toml`);
- the GUI look — colour scheme, light/dark, custom scheme, effects;
- and the **tmux server state** — the server's options, paste buffers, and key bindings.

Switching a profile restores the whole set, including reconfiguring the *live tmux server* over the wire protocol (Chapter 28). The repository's framing is direct: no emulator has profile-switchable tmux configuration. This is why profiles are listed as a first and not merely a convenience — a config-snapshot feature is ordinary, but one that snapshots and restores a running tmux server's state alongside the terminal's own is not.

Using them

The Settings header carries a **PROFILE** dropdown with **New profile** and **Delete** controls; the running app shows it reading – **NO PROFILES** – before any are created. Create a profile to capture the current whole-environment state; switch to it later to restore it. Because the restore reaches the live tmux server, switching a profile mid-session reconfigures tmux without a detach.

What switching restores, step by step

The claim that makes profiles a first is that a switch restores three normally- separate things at once, so it is worth walking what actually happens when you pick a profile. First, the terminal config is applied — the whole of `zterminal.toml` the profile captured — and because the config hot-reloads (Chapter 8), every open window re-themes and re-configures in place without a restart. Second, the GUI look is restored: the panel's colour scheme, light/dark, custom palette, and effects (Chapter 20), which are the WebView's own state rather than the terminal's. Third, and this is the novel part, the tmux server state is restored *over the wire* (Chapter 28): the profile carries the server's options, paste buffers, and keybindings, and the switch pushes them to the running server with the same `set-option`, `paste-buffer`, and `bind-key` commands the live tmux editors use. A config-snapshot feature that restored the first two would be ordinary; restoring the third — reconfiguring a running multiplexer to match a saved environment, without detaching the session — is what no other emulator does, and it is only possible because the terminal is a genuine tmux client rather than a program that shells out to `tmux`. The three restores land together, so switching a profile is switching your *whole* working environment, terminal and multiplexer alike, in one click.

Chapter 22 – The Dashboard

The Dashboard is the second of the three claimed firsts: a live, in-app telemetry surface built from a real component library. `^Z` opens it.

What it shows

The Dashboard polls and renders, using `zgui-core`'s gauges, sparklines, donuts, meters, tables, and stat strips:

- **PTY throughput** — bytes flowing through the pseudo-terminals.
- **Render timing** — the frame-draw time the renderer tracks (Chapter 4).
- **Scrollback** — buffer occupancy.
- **The full tmux summary** — clients, sessions, windows, panes, and the server identity, read over the wire protocol (Chapter 28).
- **System metrics** — CPU, memory, and the rest of a `sysinfo` snapshot.
- **Command history** — the commands recorded from OSC 133 marks (Chapter 27), each with its text and exit code.

The repository claims no terminal emulator ships a custom dashboard of this kind, and notes that it doubles as a showcase of the component library it is built from. Whether or not the “first” holds against every niche tool, the concrete fact is verifiable in the running app: a polling, component-built telemetry surface inside the terminal, fed by the same PTY, renderer, and tmux-client the rest of the program uses.

Where the numbers come from

The Dashboard is not a separate monitoring agent; it is a reader of counters the terminal already keeps, which is why it costs so little and why its numbers are the real ones. The PTY throughput is the byte volume flowing through the pseudo-terminals the process owns (Chapter 35) — the same reads the parser consumes, counted. The render timing is the per-frame draw time the renderer already tracks for `debug.render_timer` (Chapters 4, 17), surfaced without the debug flag. The scrollback occupancy is the buffer the grid already maintains (Chapter 13). The tmux summary — clients, sessions, windows, panes, server identity — is read over the one wire client the process holds (Chapter 28), not scraped from a subprocess. The system metrics come from a `sysinfo` snapshot, the same crate the process tree uses (Chapter 30). And the command history is the record built from the OSC 133 marks the scanner already recovers (Chapter 27). Every gauge is a view onto a counter the program keeps for its own reasons, which is the honest sense in which the Dashboard is a *showcase* of the component library rather than a new measurement system: the data was always there; `zgui-core`'s gauges, sparklines, and donuts just draw it.

The cost of looking

A telemetry surface that polled continuously would violate the performance discipline of Chapter 37, so it does not. The Dashboard samples on an interval while it is open and stops when it is closed — its cost is paid only when you are looking at it, and it is a periodic read of existing counters rather than a continuous instrument on the hot path. This is the same “cost proportional to use” choice the Processes tree, Recent Dirs, and the tmux

tab all make (Chapter 37): a manager that reads live state does so on a refresh cadence, so an idle-but-closed tab is free and an open one is cheap.

Chapter 23 – Command and History Palettes

The control panel is the broad surface; the palettes are the fast one. zterminal has three searchable overlays, and together they mean almost nothing in the terminal requires remembering a keybinding.

The command palette

`⌘K` opens it: a searchable list of every window, tab, terminal, panel, and pane action zterminal can perform, including every control-panel tab and a family of `tmux`: actions (new window, split horizontal/vertical, next/previous window or session, zoom/rotate/break pane, next layout, choose-tree switcher, new/kill session, detach). Because every action is *also* a rebindable shortcut (the Keys tab) and a menu item (Chapter 19), the palette, the keyboard, and the menu are three views of one action set. This is the mechanism that lets the extension layer add dozens of capabilities without adding dozens of mandatory keybindings.

The palette is also *user-programmable*. Above zterminal's built-in entries it merges a personal command set — `ZGui.userCommands`, the shared `zgui-core` feature the whole GUI suite carries — that you add from the app-shell settings. Each entry opens a URL, runs a snippet of JavaScript, applies a colour scheme, or dispatches an event, and giving it a keyword makes it argument-taking: typing `<keyword> <arg>` runs it with the argument substituted. Because the store is one JSON file in the shared config directory, a command you add here appears in every zgui app, and your own entries always rank above the terminal's.

The shell-history palette

`⌘R` opens a fuzzy search over your shell history that *inserts* the chosen command at the prompt rather than running it, so you review it before pressing Enter. Its distinctive engineering — a claimed first (Chapter 34) — is that it searches the *actual* history file of the focused shell, discovered by reading that child process's live `HISTFILE` environment variable, and parses zsh-extended (`: <ts>:<dur>;cmd`), bash, and fish formats, rather than assuming a default path. It uses the shared `ZGui.fzf` matcher.

Prompt navigation, MRU, and Quake

With OSC 133 sourced (Chapter 27), `⌘↑/⌘↓` jump to the previous/next prompt and `⌘⇧0` selects the output of the most recently finished command — navigation by the *semantic* structure of the session, not by text search. `⌘E` is the MRU quick-switch to the most-recently-used other tab. And `⌘`` summons the Quake dropdown: a global-hotkey terminal that slides down from the top of the screen, available from any application. Every one of these is in the command palette too.

One fuzzy matcher, reused everywhere

The three palettes feel consistent because they are, literally, the same matcher. `ZGui.fzf` — the fuzzy matcher from the shared `zgui-core` library (Chapter 19) — ranks the command palette's actions, the history palette's shell commands, and the cross-pane search's scrollbar (Chapter 30), and it is one component

reused rather than three search boxes that happen to look alike. That reuse is why typing behaves identically in all of them: the same subsequence matching (the query characters must appear in order but need not be contiguous, so `nw` finds “New Window”), the same scoring that rewards matches at word boundaries and against shorter candidates, and the same highlight of the matched characters in the results. When the book says the palettes mean “almost nothing in the terminal requires remembering a keybinding,” this matcher is why: you do not recall the exact name, you type a few characters of what you want and the ranking surfaces it. Building that matcher once and wiring three surfaces through it is the same single-source discipline the action dispatch of Chapter 19 follows — find the first implementation, reuse it, do not reimplement.

The palette as an open surface

The command palette is not a closed list of built-ins. Above `zterminal`'s own entries it merges `ZGui.userCommands`, the shared personal command set the whole GUI suite carries, so an entry you add — open a URL, run a snippet of JavaScript, apply a colour scheme, dispatch an event — appears here and in every other `zgui` app, ranked above the terminal's entries. Giving an entry a keyword makes it argument-taking: typing `<keyword> <arg>` runs it with the argument substituted, which turns the palette from a menu into a tiny command line. Because the store is one JSON file in the shared config directory, the palette is a place you *extend* rather than only a place you *browse*, and the extension travels across the suite with you.

Chapter 24 — Native Splits and Layouts

A multiplexer like `tmux` draws multiple panes by repainting regions of a single grid. `zterminal` does it the other way: it tiles *natively*, with its own i3-style binary split tree, one independent shell and PTY per pane, each pane GL-scissored into its own sub-viewport. There is no multiplexer in the loop and no single grid being subdivided — each pane is a real terminal.

The split tree

Any pane splits vertically (`⌘D`) or horizontally (`⌘⇧D`), and splits nest arbitrarily, so the window is a binary tree whose leaves are panes and whose internal nodes are splits. Each leaf owns its own shell, PTY, and scrollback, and renders in its own scissored viewport so the GPU draws them independently in the one frame. Because the panes are the emulator's own, inline images (Chapter 26) and OSC 133 marks (Chapter 27) work in a native pane exactly as in a full window, with no passthrough dance and no multiplexer redraw latency.

Moving around the tree

The navigation set is i3's: `⌘]/⌘[` focus next/previous; `⌘←↑↓→` focus the pane in a spatial direction; `⇧⌘←↑↓→` move (swap) the focused pane; `⌘⇧←↑↓→` resize along the split; `⌘⇧↔` zooms a pane to fill the window and toggles back; `⌘⇧W` closes the focused pane. Every one is in the command palette.

Layouts

`⌘⇧L` saves and restores a window's split tree. Saving captures the split geometry plus each pane's working directory and foreground command to `~/.zterminal/layouts/<name>.json`. Restoring rebuilds the tree, respawns each pane in its saved directory, and *replants* the saved command at the prompt — without running it; you review and press Enter. Live process state is not restored, because no tool can restore it; panes return in the right geometry and directory as fresh shells. This is the native counterpart to the `tmux` session save/restore in Chapter 28, and the two share the same honest limitation.

The scissor: why native panes are cheap

The mechanism that makes native panes different from a multiplexer's is worth seeing directly. A multiplexer like `tmux` owns *one* grid and paints several logical panes into regions of it, which means every pane's output flows through the multiplexer's own parser and redraw before it reaches the terminal — a layer of latency and a place where escape sequences (images, OSC 133) get filtered. `zterminal` instead gives each leaf pane its own real terminal — its own PTY, its own grid, its own scrollback — and draws them all into the one GL surface by *scissoring*: before drawing a pane, the renderer restricts drawing to that pane's sub-rectangle of the window, draws that pane's damaged cells there, then moves to the next. The GPU clips to the scissor rectangle, so a pane physically cannot draw outside its bounds, and the frame loop of Chapter 5 coalesces all the panes' damage into the one frame. The consequences are the ones Chapter 24 opened with: because

each pane is a genuine terminal, an inline image (Chapter 26) or an OSC 133 mark (Chapter 27) works in a pane exactly as in a full window — there is no multiplexer between the application and the emulator to strip it — and because they share one frame and one process, splitting costs a scissor rectangle and a PTY, not a subprocess and a redraw pipeline.

The tree, precisely

The split tree is a binary tree, and every operation is a tree edit. A split replaces a leaf with an internal node whose two children are the old leaf and a new pane; closing a pane removes a leaf and collapses its parent, promoting the sibling into the parent's place so no empty space is left. Focus is a pointer to the current leaf, and the spatial navigation (`\C + arrow`) walks the tree geometry to find the nearest leaf in a direction, while the ordinal navigation (`C]/C[`) walks the leaves in order. A resize adjusts the split ratio stored at an internal node, which is the single number that says how the node's area divides between its children — which is exactly what a saved layout serializes (Chapter 24's `layouts/<name>.json`) so a restore can rebuild the same proportions. Seeing the tree as the data structure makes the whole keybinding set legible: every split key adds a node, every close key removes one, every focus key moves the pointer, and every resize key edits a ratio.

Chapter 25 – Exposé, MRU, and the Quake Dropdown

Three window-level features sit between the splits and the tmux client, because they span both native panes and tmux panes.

Exposé

`^⌘E` opens Exposé: a tile grid of every zterminal window *and* every tmux pane at once; click a tile to focus it. This is the unifying view that treats the native split tree and the reparented tmux server as one spatial surface. The repository lists the blend — native i3 splits plus tmux, tiled together in one Exposé, with inline-image protocols decoded even through tmux passthrough — as a combination candidate (Chapter 34).

MRU quick-switch

`⌘E` jumps to the most-recently-used other tab — the terminal equivalent of an editor’s “switch to last buffer,” so two tabs you are alternating between are one keystroke apart.

The Quake dropdown

`^⌘`` is a global hotkey: a dropdown terminal that slides down from the top of the screen over whatever application is focused, and slides back. It is the always-one-keystroke-away terminal, independent of which window has focus.

What Exposé is actually solving

The tile grid is easy to describe and its significance is easy to miss, so it is worth saying what problem it addresses that a window switcher does not. By the time a working session has any depth, the panes you care about are distributed across two structurally different containers: the native split trees inside zterminal’s own windows and tabs (Chapter 24), and the session-window-pane tree owned by the tmux server (Chapter 28). The operating system’s window switcher can see the first kind, badly — it sees *windows*, and a window containing six native panes is one tile with no indication of what is in it. It cannot see the second kind at all, because a tmux pane is not a window; it is a region of a grid inside a process the window server has never heard of.

Exposé’s contribution is that it is built from the terminal’s own model rather than the window server’s. It enumerates what the process knows: every window, every tab, every leaf of every split tree, and — through the wire client — every pane the tmux server owns (Chapter 28). Those are exactly the things you think of as “my terminals,” and they are presented as one flat spatial surface with one selection gesture, regardless of which of the two ownership models a given tile came from. Chapter 31 spends a chapter on why that distinction is real and consequential; Exposé is one of the places the extension layer works to make it not *matter* for the operation you are actually performing, which is “look at everything, pick one.”

This is also why the inventory files the unified Exposé as a **low**-confidence combination rather than a first (Chapter 34), and the label is fair: tiling overviews exist, and so do tmux pane pickers. What does not obviously exist elsewhere is one overview whose contents span a terminal's native panes and a reparented multiplexer's panes together, and the honest claim is the combination rather than either piece.

MRU, and why alternation is the common case

⌘E looks trivial — jump to the last-used other tab — and it is included because the access pattern it serves is overwhelmingly the most common one. Real work is rarely a uniform tour of many contexts; it is a rapid alternation between two, with occasional excursions. Editing and running. Code and logs. A local shell and a remote one. An ordinal navigation set (**⌘1–9**, **⌘↑** **]**/**[**) makes every tab equally far away, which is exactly wrong for a workload where one of them is where you were half a second ago.

The editor world settled this long ago with “switch to last buffer,” and the terminal equivalent is the same idea with the same payoff: the pair you are alternating between collapses to one keystroke each way, and the cost of having ten other tabs open drops to zero because they are not in the rotation. It composes with Exposé rather than competing with it — MRU for the alternation you are in the middle of, Exposé for the moment you have lost track and need to see everything.

The dropdown, and what “global” means

The Quake dropdown differs from every other feature in this chapter in one respect: its hotkey is registered with the operating system rather than with the terminal's own binding table (Chapter 16). That is what “global” means, and it is the whole point — the binding must fire while a browser, an editor, or a video call has focus, which requires the OS to deliver the key to a process that is not in the foreground.

The consequence worth planning around is that a global hotkey is a scarce, shared resource. It is registered against the system, so it is unavailable to every other application while zterminal holds it, and if another application already holds it the registration is the one that loses. **⌘`** is chosen to be unlikely to collide, and the non-collision discipline of Chapter 16 applies here with extra force: this is the one binding zterminal takes that is not scoped to its own windows.

The interaction model is deliberately different from a window, too. The dropdown slides over whatever is focused and slides back, which makes it the right instrument for the interruption-shaped task — check a value, run one command, read one line of output — and the wrong one for the task you will still be doing in ten minutes. That work belongs in a window or a pane, and the dropdown is at its best when you treat it as somewhere you visit rather than somewhere you stay.

Chapter 26 — Inline Graphics

zterminal renders images inline in the grid, on the GPU, in all three common terminal image protocols, plus a config-set background image. This is original extension work built on the inherited renderer's compositing path (Chapter 4), and it is gated by an architectural decision worth stating first: a single scanner runs *ahead* of the VT parser.

The scanner ahead of the parser

The `vte` parser zterminal inherits discards the escape sequences these protocols use — it is a faithful VT parser, and these are non-VT extensions. So zterminal places one scanner in front of the parser that peels image sequences out of the PTY byte stream before the parser sees them: Kitty's APC sequences, Sixel's DCS strings, and iTerm2's OSC 1337. The *same* scanner also extracts the OSC 133 marks of Chapter 27 — one pre-`vte` stage recovers four protocols `vte` would drop. Sequences *wrapped by tmux* (`ESC Ptmux;...`, with `set -g allow-passthrough on`) are unwrapped and decoded, so images survive a multiplexer. The repository lists this unified pre-`vte` recovery — incl. OSC 133 and the `tmux-unwrap` — as a candidate (Chapter 34).

The three protocols

- **Kitty graphics protocol** — the most complete: transmit (`a=t`), transmit-and-display (`a=T`), put-by-id (`a=p`), support query (`a=q`, so capable tools auto-detect zterminal), delete (`a=d`); RGB/RGBA/PNG (`f=24/32/100`); direct and file (`t=f`) transmission; zlib compression (`o=z`); chunked payloads; stacking order (`z=`); Unicode placeholders (`U=1`) for grid-anchored placement; and animation (`a=f` frames, `a=a` play/stop/loop) played on a timer. Placeholder placement works through `tmux`.
- **Sixel** — `ESC P ... q ... ST`, with RGB and HLS colour registers, run-length encoding, and raster attributes.
- **iTerm2 inline images** — `OSC 1337;File=...` (the `imgcat` protocol), in any format the decoder handles (PNG/JPEG/GIF).

In all three, images are uploaded as GPU textures and blitted over the cells they occupy. They scroll with the buffer, clip at pane edges, draw behind (`z < 0`) or over the text, and are freed when they scroll out of the scrollbar.

The bundled icat

zterminal bundles a zero-dependency display tool, `extra/zterminal-icat`, which emits the Kitty protocol from any image using only `base64` (and `sips` on macOS for non-PNG input). Uniquely for an `icat`-style tool, it enables `tmux allow-passthrough` itself and places the image via Kitty *Unicode placeholders*, so the picture survives `tmux` redraws where a plain display escape would be erased on the next repaint (Chapter 34 lists this). Symlink it onto `PATH` and it stays current with the repo:

```
ln -sf "$PWD/extra/zterminal-icat" /usr/local/bin/zterminal-icat
zterminal-icat picture.png
```

Any other inline-image tool works too — `kitten icat`, `timg`, `chafa`, `img2sixel`, `imgcat`.

The background image

Distinct from inline images, zterminal draws an image *behind* the cells, set in config (`window.background_image`, `window.background_image_opacity`) rather than by an escape sequence — so it works everywhere, including inside tmux:

```
[window]
opacity = 0.85
background_image = "~/Pictures/wall.png"
background_image_opacity = 1.0
```

The image is uploaded once as a GPU texture and drawn each frame (cover-fitted) behind any cell whose background is the default colour; lowering `opacity` reveals more of it. It reloads live when the path changes. The config-survives- tmux angle and the negative-z behind-text behaviour are the parts the repository calls distinctive (Chapter 34).

How a Kitty transmission is read

The scanner-ahead-of-the-parser design is easiest to believe once you see the bytes. The Kitty graphics protocol wraps its control data in an APC envelope — `ESC _ G ... ESC \` — carrying a set of `key=value` pairs and, after a `;`, a base64 payload. A minimal “transmit RGBA and display now” looks like:

```
ESC _ G a=T,f=32,s=64,v=64,t=d ; <base64 RGBA bytes> ESC \
```

Read left to right: `a=T` is the action, transmit-and-display; `f=32` says the pixels are 32-bit RGBA (`f=24` is RGB, `f=100` is PNG); `s` and `v` give the image width and height in pixels; `t=d` says the payload is *direct* (inline base64) rather than `t=f` (a file path). The `vte` parser zterminal inherits would discard this whole envelope as an unknown APC string — which is exactly why the pre-`vte` scanner (above) exists to peel it out first, decode the payload into pixels, upload them as a GPU texture, and blit the result over the cells the cursor occupies. A large image is split across several chunks with `m=1` on all but the last (`m=0`), so the transmission survives the terminal’s own read batching; the scanner reassembles the chunks before decoding. A capable tool sends `a=q` first — the support query — and only commits to the Kitty path if zterminal answers, which is the runtime capability detection of Chapter 40 applied to graphics.

Placement, ordering, and the lifecycle

Once decoded, an image is not a character; it is a textured quad with a position, a size, a z-order, and a lifetime, all resolved against the grid. Placement anchors the image to the cell the cursor was on when it arrived, or — with Kitty Unicode placeholders (`U=1`) — to explicit placeholder cells the application prints, which is the mode that survives a tmux redraw because the *placeholders* are ordinary text tmux will faithfully repaint (Chapter 26’s `icat` uses exactly this). The z-order (`z=`) slots the image into the same compositing pass as the text (Chapter 4): a negative `z` draws it behind the cells, a positive `z` over them. The lifetime is tied to the scrollbar — the image scrolls with the buffer it was placed in, clips at pane edges so it never bleeds into a neighbour (Chapter 24), and its texture is freed when the anchoring lines scroll out of the scrollbar and can no longer be reached. This is why inline images do not accumulate as a memory leak over a long session: they are owned by the grid region they belong to, and they die with it.

The other two protocols, briefly

Kitty’s protocol is the most complete, but the same scanner-decode-blit path serves the other two, and their envelopes are worth recognizing. Sixel wraps its data in a DCS string — `ESC P ... q ... ST` — where the payload is a run-length-encoded raster built from colour registers, six pixels tall per band:

```
ESC P q "1;1;100;100 #0;2;0;0;0 #1;2;100;100;100 ... ST
```

The `q` introduces the sixel data, the `#n;...` sequences define colour registers in RGB or HLS, and the raster attributes give the image geometry. iTerm2's inline-image protocol is simpler still — an OSC 1337 carrying a base64 file, the `imgcat` form:

```
ESC ] 1337 ; File=inline=1;size=<bytes>;width=auto;height=auto : <base64> ST
```

Both decode to the same thing a Kitty transmission does — pixels uploaded as a GPU texture and blitted over the cells (Chapter 4's compositing pass) — and both are peeled out by the same pre-`vte` scanner (above), which is the whole point of that one interceptor: three image protocols and OSC 133 recovered by a single stage rather than four bolt-ons. Which protocol a tool uses is the tool's choice; the terminal decodes all three, and the support query (`a=q`, Chapter 40) lets a capable tool prefer Kitty when it detects it.

Chapter 27 — Shell Integration

OSC 133 is a small escape-sequence convention with a large payoff: if the shell emits a mark at each prompt, before each command, and around each block of command output, the terminal learns the *semantic* structure of the session — where every prompt, typed command, and output block begins and ends. zterminal reads OSC 133, and the features that fall out of it are this chapter.

Why a scanner, again

The inherited `vte` parser discards OSC 133, just as it discards the image protocols. So the same pre-`vte` scanner from Chapter 26 peels the OSC 133 marks out of the PTY stream and anchors them to *absolute scrollbar line numbers*. Anchoring to absolute lines rather than screen positions is what lets prompt navigation survive scrolling and reflow.

What the marks enable

- **Prompt jumping** — `⌘↑/⌘↓` (`^↑/^↓` on Linux/Windows) jump to the previous/next prompt: navigation by command, not by line.
- **Output selection** — `⌘⇧0` selects the entire output of the most recently finished command, ready to copy or pipe.
- **Failed-command gutter marks** — a thin bar in the left margin flags every prompt line and turns red when that command exited non-zero, driven by the recovered OSC 133 `D;exit` mark. A claimed addition (Chapter 34); the exit status rides in the mark.
- **Command history** — each command, with its text and exit code, is recorded and shown in the Dashboard (Chapter 22); it is also the data the `⌘R` history palette reads.

Turning it on

Source the snippet for your shell from `extra/shell-integration/` (bash, zsh, fish):

```
source /path/to/zterminal/extra/shell-integration/zterminal-shell-integration.zsh
```

The marks are zero-width and other terminals ignore them, so it is safe to source everywhere — your `.zshrc` needs no per-terminal guard. Inside tmux, enable `set -g allow-passthrough on` so the marks survive the multiplexer, the same passthrough the image protocols use.

The four marks, precisely

OSC 133 is four marks, and knowing which is which makes the features above legible. The shell integration emits them at the moments their letters name:

- `OSC 133 ; A` — *prompt start*. Emitted just before the shell prints its prompt. This is the anchor prompt-jumping lands on (`⌘↑/⌘↓`), and the gutter mark's position.
- `OSC 133 ; B` — *prompt end / command start*. Emitted after the prompt, before the command you type. The span from `B` to the next mark is your typed command line.

- `OSC 133 ; C` — *command output start*. Emitted after you press Enter, before the command's output. This begins the output block.
- `OSC 133 ; D ; <exit>` — *command end*. Emitted when the command finishes, carrying its exit code. The span from `C` to `D` is the command's output — which is exactly what `⌘⬆0` selects (Chapter 38) — and the `<exit>` is what turns the gutter mark red on failure.

The elegance is that four zero-width marks give the terminal the whole semantic structure of the session: where every prompt, command, and output block starts and ends, and whether each command succeeded. Anchoring them to absolute scrollbar line numbers (rather than screen rows) is what lets that structure survive scrolling and reflow (Chapter 3), so a `D` mark's exit status stays attached to the right prompt no matter how the buffer re-wraps. Everything the chapter lists — prompt jumping, output selection, gutter marks, command history — is a query over these four marks, which is why sourcing the one snippet lights all of them up at once.

What the snippet has to guarantee

The terminal's half of OSC 133 is easy: recover the marks and anchor them. The shell's half is where the correctness lives, and understanding what the snippet is responsible for explains both why it is a snippet rather than a setting and what goes wrong when a hand-rolled version misbehaves.

The snippet has to emit each mark at the right *moment*, which means hooking the shell's own lifecycle rather than printing from the prompt string alone. `A` must be emitted before the prompt is drawn and `B` after it, which is a statement about where in the prompt the marks sit. `C` must be emitted after you press Enter and before the command's output begins, which is a pre-execution hook. And `D` must be emitted when the command finishes, carrying the exit status — which means it runs in the shell's post-command position and must capture the status *first*, before any other command in the hook overwrites it. That last point is the classic error in a hand-written integration: read `$?` after running anything else in the hook and you report the hook's status rather than the command's, and every gutter mark turns the wrong colour.

Emitting the marks *unconditionally* is the second requirement, and it is the one that makes the shipped snippet safe to put in a shared dotfile. The marks are zero-width control sequences; a terminal that does not implement OSC 133 discards them as an unknown OSC string, exactly as `vte` would discard them if `zterminal`'s scanner did not run first (Chapter 26). So there is nothing to guard: no `if [[ $TERM == ... ]]`, no per-terminal branch, no capability probe. This is a meaningful improvement over the integration scripts these replace, which historically did carry such guards and consequently broke whenever the guard's assumption drifted.

The third requirement is idempotence with respect to *other* prompt machinery. A shell's prompt is usually already assembled by something — a prompt framework, a theme, a version-control status helper — and the integration has to coexist with whatever hooks are already installed rather than replacing them. This is why the snippet is sourced rather than assigned: it adds to the shell's hook arrays instead of overwriting the prompt.

Why the marks are anchored, not stored positionally

Chapter 27 stated that the marks anchor to absolute scrollbar line numbers; the reason repays a moment because it is a small design decision that everything above it depends on.

A mark's job is to say “a prompt began *here*.” The obvious implementation is to record the screen row, and it is wrong for two independent reasons, both already established. Scrolling moves content relative to the screen, so a row-anchored mark would drift the moment a single line was printed. Reflow (Chapter 3) is worse: when the window is resized, a logical line that had wrapped across three rows is rejoined and re-wrapped to a different number of rows, so the mapping from rows to content changes wholesale, and every row-anchored mark in the buffer would be pointing at different text than it was a moment ago.

Anchoring to an absolute line number in the scrollbar survives both, because the logical line a mark points at is the same logical line regardless of where it currently sits on screen or how many rows it currently occupies.

This is what lets a `D` mark's exit status stay attached to the correct prompt across a resize, and what lets `⌘↑` walk backwards through prompts in a buffer that has scrolled thousands of lines since they were emitted.

What OSC 133 does not carry

Two useful things are absent from the convention, and the book has already named both consequences without naming the cause.

There is no *working directory* in the marks. This is why Recent Dirs (Chapter 32) harvests each pane's cwd from the live foreground process rather than reading it out of the mark stream — the information was never in the stream to read. OSC 7 is the separate convention that carries a directory, and the recent-directories tracker exists precisely because neither it nor OSC 133 is a reliable universal source.

There is no *command text* in the `C` mark either; the mark says “output starts here,” not “the command was `cargo build`.” The command text the Dashboard shows (Chapter 22) is the span between `B` and `C` — the region of the grid where your typed command line sits — recovered from the buffer rather than transmitted. This is why the recorded command is exactly what was on screen, including any prompt editing you did before pressing Enter, and it is a good example of the marks' economy: four boundary markers are enough to reconstruct the structure without the shell having to report its state a second time in a different form.

Chapter 28 — The tmux Client

The most ambitious extension is the tmux client. zterminal does not run a `tmux` subprocess and scrape its output; it speaks tmux's client/server *wire protocol* directly to the server's Unix socket — `imsg` framing, protocol version 8 — resolving the target from `$TMUX` or the default socket, through the `ztmux-core` crate. It is a native tmux client living inside the terminal, and it is the basis for the repository's strongest claimed first: live-editing a running tmux server.

The wire, concretely

It is worth being precise about what “speaks the wire protocol directly” means, because the phrase does a lot of work. tmux is a client/server program: the long-lived `tmux` server owns the sessions and panes, and each `tmux attach` is a thin client that connects to the server's Unix-domain socket and exchanges length-prefixed messages framed with OpenBSD's `imsg` library — a compact binary envelope of a type tag, a peer id, and a payload, the same framing tmux uses between its own processes. zterminal opens that socket itself and becomes such a client: it resolves the target socket from `$TMUX` (whose value carries the path, the server pid, and the session id) or the default location, performs the version- 8 handshake, and thereafter sends commands and reads the server's replies and notifications as first-class messages. There is no `tmux` binary in the loop, no pseudo-terminal scraped for text, no fragile screen-parsing of `list-panes` output — the structured data the server sends to its own clients is the data zterminal reads. That is the difference between *integrating with* tmux and *shelling out to* it, and it is what makes the live editors of this chapter trustworthy rather than a race against a subprocess's stdout.

Reparenting and the one-surface goal

Because zterminal is a genuine client, the panes the server owns can be surfaced alongside the native split panes (Chapter 24) as one spatial thing: Exposé (Chapter 25) tiles both, cross-pane search (Chapter 30) greps both, and the process tree (Chapter 30) descends into the server's reparented children as if they were the terminal's own. The ownership genuinely differs — a native pane's PTY belongs to zterminal, a tmux pane's belongs to the server — but the wire client is the seam that lets the two read as a single surface to the extensions built above them. This is the combination the inventory files as a candidate rather than an outright first (Chapter 34), and the honesty of that label is the same as everywhere: the pieces exist elsewhere in parts, the unification is the claim.

The live pane manager

The control panel's `tmux` tab is a live view of the full session → window → pane tree, each pane carrying a `capture-pane` text preview. Click a pane to switch the attached client to it; toolbar and per-session buttons create, split, zoom, rotate, and kill windows, panes, and sessions. It refreshes while open.

Why the structured data beats screen-scraping

It is worth dwelling on what the wire client buys, because the alternative is what almost every “tmux integration” in other tools actually is. Screen-scraping tmux means running `tmux list-panes -F '...'` as a subprocess,

reading its stdout, and parsing the text — which works until a pane title contains the delimiter you split on, or the format string drifts between tmux versions, or two `list-panes` calls race a pane being created between them and you parse a tree that no longer exists. The wire client sidesteps all of it by reading the same structured messages the server sends its own attached clients: a pane is a record with fields, not a line of text to be re-parsed, and a change to the tree arrives as a *notification* the server pushes rather than a poll the client has to time correctly. `set-option`, `bind-key`, `capture-pane`, `paste-buffer` and the rest are issued as command messages and their results read as reply messages, framed with `imsg` (the version-8 handshake described earlier in this chapter). The live editors of this chapter are trustworthy for exactly this reason: editing a running server's options over the wire is a request/reply against the authoritative state, not a hopeful `sed` of some text representation of it.

Live editing the server: options, buffers, keys

This is the claimed first (Chapter 34). Three separate control-panel tabs edit a *running* tmux server over the wire:

- **tmux config** — reads every `show-options` scope (server, session, window) and edits any option in place with `set-option`, applied instantly.
- **tmux buffers** — lists, views, edits, creates, pastes, and deletes the server's paste buffers.
- **tmux keys** — lists every binding across the key tables and rebinds or unbinds them live with `bind-key/unbind-key`, the command re-tokenized for the wire.

The repository states the claim plainly: no other terminal emulator edits a live tmux server's options, buffers, and keybindings from its own UI. This is also what makes profiles (Chapter 21) able to snapshot and restore tmux server state.

Sessions: native save and restore

`^⌘S` saves and restores the full tmux layout — a native replacement for `tmux-resurrect` and `tmux-continuum`, done over the wire rather than by a plugin (Chapter 34). Saving snapshots every session → window → pane, with each pane's exact `window_layout`, working directory, and *full captured command line*, to `~/.zterminal/snapshots/`. The command line is read natively from the pane process (`libproc KERN_PROCARG52` on macOS, `/proc` on Linux — no `pgrep`). Restoring rebuilds the tree over the wire, including exact split geometry and the active window/pane, starting a server if none is running. It defaults to a plain shell per pane; opt-in *relaunch* re-runs each pane's full command with arguments; *save pane contents* captures and replays each pane's scrollback. A `tmux_restore_processes` preference — edited right in the Sessions panel — gives the resurrect-style whitelist (`name` or `name->replacement`, e.g. `nvim->nvim -S Session.vim`); empty restarts everything except shells and `sudo`. Marking a snapshot *Set startup* auto- restores it at launch. As always, live process state is not restored.

Chapter 29 — Session Restore and Persistence

Persistence is scattered across several earlier chapters — the native layouts of Chapter 24, the tmux session snapshots of Chapter 28, the profiles of Chapter 21, and the state directory of Appendix E — because each grew out of the feature it serves. This chapter gathers them, because they share one architecture, one honest limitation, and one design decision that recurs in every restore path zterminal ships. Reading them together is the fastest way to understand what “restore” can and cannot mean for a terminal, and why zterminal draws the line exactly where it does.

Four things that persist

zterminal writes four distinct kinds of durable state under `~/.zterminal` (Appendix E), and they are worth separating because they capture different scopes and restore through different mechanisms:

- **Native split layouts** (`layouts/<name>.json`, Chapter 24) — the geometry of one window’s split tree, plus each native pane’s working directory and foreground command.
- **tmux session snapshots** (`snapshots/<name>.json` and the sibling `<name>.contents/`, Chapter 28) — the full session → window → pane tree of a *tmux server*, captured and restored over the wire protocol.
- **Profiles** (Chapter 21) — a whole-environment snapshot: the terminal config, the GUI look, and the tmux server’s options, buffers, and keybindings.
- **Harvested lists** — recent directories, command history, triggers, snippets (Chapter 32), each a JSON file the control panel manages.

The first two restore a *shape*: they rebuild panes in the right places running the right shells in the right directories. The third restores a *configuration*. The fourth restores *data*. The interesting engineering is in the first two, because rebuilding a live arrangement of processes is where every terminal and multiplexer has to decide what is honestly recoverable.

The one limitation, stated once for all of them

No restore path in zterminal claims to bring back a running program’s live state, and the book has said so at each site (Chapters 24 and 28); here is the reason, once, because it is the same reason everywhere. A pane’s live state is the memory of a running process — the unsaved buffer in your editor, the position in your pager, the half-typed pipeline, the open connection. That state lives in the process’s address space and the kernel’s file tables, and nothing outside the process can serialize it. tmux-resurrect cannot; a checkpoint/restore-in-userspace tool can freeze a process tree but not a network connection or a pty peer; and zterminal does not pretend to. What a terminal *can* honestly restore is the arrangement — where the panes were, how big, in what directory, running what command *line* — and then respawn fresh processes into that arrangement. That is the promise zterminal makes and keeps, and refusing to over-promise is the same honesty the rest of the book applies to its invention claims (Chapter 34).

Capturing the command line, natively

Restoring the *arrangement* means one hard sub-problem: discovering what command was running in each pane, so it can be replanted or relaunched. The naïve approach shells out to `ps` or `pgrep` and parses text, which is slow, racy, and fragile. `zterminal` reads the command line natively from the pane's process instead — on macOS via `libproc` with `KERN_PROCARG52`, on Linux via `/proc/<pid>/cmdline` (Chapter 28) — recovering the full argument vector, not a truncated process name. The distinction matters for a faithful restore: a pane running `nvim -S Session.vim +Gstatus` should come back as that whole line, arguments intact, not as a bare `nvim`. Reading the kernel's own record of the process arguments is how the snapshot gets the real command instead of a lossy approximation, and it is the same native-over-scraping discipline the `tmux` client applies to the wire protocol in Chapter 28.

Replant versus relaunch

Given a captured command line, a restore has two defensible things to do with it, and `zterminal` offers both because the right choice depends on the command:

- **Replant** (the default for native layouts, Chapter 24) — respawn a plain shell in the saved directory and *type the saved command at the prompt without running it*. You see exactly what was there and press Enter yourself. This is the safe default: it never re-executes something with side effects — a `git push`, a `rm`, a deploy — behind your back, and it lets you review a command that may no longer be appropriate in the restored context.
- **Relaunch** (opt-in for `tmux` snapshots, Chapter 28) — actually re-run each pane's full command with its arguments. This is what you want for the editors, pagers, and long-running monitors that are the *point* of a saved session, and it is gated behind an explicit toggle plus the `tmux_restore_processes` whitelist so it re-runs what you named and leaves the rest as shells.

The whitelist is the resurrect model made precise. Each entry is a process name, optionally with a replacement (`name->replacement`), so `nvim` can be restored as `nvim -S Session.vim` — reopening the editor *and* the saved editor session in one step. An empty whitelist restarts everything except plain shells and `sudo` invocations, the two things you almost never want blindly re-run. The replacement arrow is what turns a dumb “re-run the command” into a smart “re-run the command the way it should come back,” and it is edited right in the Sessions panel rather than a config file.

Pane contents: replaying scrollback

An arrangement restore brings back fresh processes with empty scrollback, which loses the *output* you may have wanted to keep reading. The opt-in *save pane contents* mode (Chapter 28) closes that gap by capturing each pane's scrollback at save time into the `<name>.contents/` directory beside the snapshot, and replaying it into the restored pane so the history is visible above the fresh prompt. This is honestly a *replay of text*, not a restoration of process state — the replayed lines are inert scrollback, not a live program's output — and the book is careful with the distinction, but for the common case of “I want to see what that build printed yesterday,” inert replayed text is exactly what is wanted.

Startup auto-restore

A snapshot marked **Set startup** (Chapter 28) is restored automatically when `zterminal` launches, which turns the save/restore feature into a persistent working environment: quit at the end of the day, relaunch in the morning, and the session's shape — panes, directories, and (with relaunch) the editors and monitors — comes back. Combined with the `--daemon` model of Chapter 35, where one resident process spawns windows on demand, the startup snapshot is how a login session brings up a fully-formed multi-pane environment without a shell script orchestrating `tmux new-session` calls. The arrangement is data in `~/.zterminal/snapshots/`,

versioned as JSON, restored over the wire — not a pile of imperative setup commands that drift out of sync with what you actually use.

Why this belongs to the single process

The persistence layer, like nearly everything in the extension suite, leans on the single-process model of Chapter 35. A layout save enumerates the split tree of a window the process owns; a tmux snapshot reads the server the process's one wire client is attached to; a profile switch reconfigures that same live server and re-themes every window in the process at once. Restore is cheap for the same reason the dashboard and the broadcast overlay are cheap: the state being captured and rebuilt is all reachable from one address space, so a snapshot is a walk over in-process structures and a few wire commands, not a distributed query across a fleet of per-window processes. Persistence is another dividend of keeping the panes together.

Chapter 30 – tmux Broadcast, Search, and Process Tree

Three further tmux capabilities go beyond what native tmux offers, treating native and tmux panes as one surface.

Cross-session broadcast

`^⌘B` opens a broadcast overlay that sends keystrokes — or a saved snippet (Chapter 32) — to an arbitrary *checked set* of tmux panes spanning any windows and sessions at once. Native tmux cannot do this: `synchronize-panes` is scoped to a single window. Check the panes, type the command, Send; the per-window `synchronize-panes` toggle is available from the same overlay. A claimed addition (Chapter 34).

Cross-pane search

`^⌘P` fuzzy-matches (the shared `ZGui.fzf`) the recent scrollbar of *every* tmux pane at once: type a query, see panes ranked by score with the matched characters highlighted, and press Enter or click to jump to that pane. Combined with Exposé (Chapter 25), the native and tmux panes are one searchable, tileable surface.

The per-pane process tree

The **Processes** tab renders a live CPU/MEM process tree rooted at every pane's shell — including descending into the *tmux server's* reparented children — built from a `sysinfo` snapshot and a ppid → children map, with roots seeded from the native panes' shells and the tmux panes' pids (labelled `tmux <s>:<w>.<p>`). It can signal or kill, but only pids that are descendants of its own panes — the kill path is gated to the seen descendant set. The repository frames it as an inspector spanning native panes and the reparented tmux server, Unix-only (Chapter 34).

The kill gate, precisely

A process tree that can signal is a process tree that can do damage, so the kill path is deliberately narrow, and the narrowness is the point. The tree is built from a `sysinfo` snapshot and a parent-to-children map, with its roots *seeded* from a known set: the pids of the native panes' shells and the pids of the tmux panes (labelled `tmux <s>:<w>.<p>`). From those roots it descends, so the tree it shows is exactly the set of processes reachable as descendants of panes the process owns or is attached to — your shells and what they launched, not the whole system's process table. The kill and signal actions are then gated to that same descendant set: a pid the tree did not reach from a seeded root cannot be signalled through it. This means the inspector can end a runaway process *in one of your panes* but cannot be turned into a general `kill` for arbitrary system pids, which is the correct boundary for a tool embedded in a terminal — powerful over your own sessions, inert everywhere else. It is Unix-only because the ppid-to-children mapping and the native pid seeding depend on Unix process semantics (Chapter 34).

Chapter 31 — Native Panes versus tmux Panes

zterminal ships two ways to put several shells on one screen, and it is deliberately not trying to make you choose. Chapter 24 documented the native split tree; Chapters 28 through 30 documented the tmux client and everything built on it. A reader who has both available needs a decision procedure, and the honest one is not “native is better” — it is that the two have genuinely different ownership models, and ownership determines which properties you get. This chapter is that comparison, made concrete, because getting it wrong is the difference between losing an afternoon’s work to a closed window and not.

One question decides most of it

Everything below reduces to a single property: **who owns the PTY**. A native pane’s pseudo-terminal belongs to the zterminal process (Chapter 35). A tmux pane’s belongs to the tmux server, a separate long-lived process that zterminal is merely a client of (Chapter 28). Chapter 35 stated the consequence as an asymmetry — native panes die with the process, tmux panes outlive it — and that asymmetry is not one bullet among many. It is the root from which nearly every other difference grows.

So the first question to ask of any pane you are about to create is: *does the work in it need to survive this terminal process?* A build you will watch for thirty seconds does not. A long-running remote session you will reconnect to tomorrow from a different machine does. If the answer is yes, the pane belongs to tmux, and no amount of native-pane convenience changes that. If the answer is no, the native pane is the better instrument for the reasons that follow.

The comparison, property by property

Property	Native pane	tmux pane
PTY owner	the zterminal process	the tmux server
Survives quitting zterminal	no	yes
Survives an SSH disconnect	not applicable	yes, if the server is remote
Parses applied to the output	one (zterminal’s)	two (tmux’s, then zterminal’s)
Inline images (Chapter 26)	work directly	need <code>allow-passthrough on</code>
OSC 133 marks (Chapter 27)	work directly	need <code>allow-passthrough on</code>
Scrollback lives in	zterminal, <code>scrolling.history</code>	the tmux server’s history
vi mode (Chapter 18) reads	that pane’s real scrollback	only what tmux repainted
Restore mechanism	layouts (Chapter 24)	snapshots over the wire (Chapter 28)
Cost of one more	a PTY and a scissor rectangle	a pane in another process
Reachable by Exposé, cross-pane search, the process tree	yes	yes

The last row is the one that makes the table interesting rather than a scorecard. Most of the differences are real trade-offs you must live with; the unifying features of Chapters 25, 30, and 31 are zterminal’s attempt to

make the choice cost you less by treating both kinds as one surface once they exist.

The double-parse, and what it costs

The “two parses” row deserves unpacking, because it is the property most often asserted vaguely and least often explained. When a program writes to a native pane, its bytes travel one path: the PTY, the scanner, the `vte` parser, the grid, the frame (Chapter 37’s six steps). When a program writes inside a multiplexer, its bytes are first consumed by the multiplexer, which maintains *its own* grid for that pane, and the multiplexer then re-emits whatever the attached client needs to see onto the client’s own connection, where the terminal parses it a second time into a second grid. The content is rendered twice into two data structures by two independent parsers.

The latency implication is straightforward and modest: an extra process, an extra parse, an extra redraw decision between the program and the screen. This is what Chapter 24 meant by “multiplexer redraw latency,” and for interactive work it is usually imperceptible; it becomes perceptible under exactly the load Chapter 5 describes, when a program is producing output faster than a frame, because now two independent coalescing decisions are being made in series.

The *fidelity* implication is the one that actually bites, and it is why the images-and-marks rows read the way they do. A multiplexer that maintains its own grid must decide what to do with escape sequences it does not itself implement. The safe answer — the one `tmux` takes by default — is to consume and discard them, because passing arbitrary sequences through to the outer terminal breaks the multiplexer’s model of what is on screen and, as Chapter 39 notes, hands every program in every pane a channel to the outer terminal. So a Kitty graphics transmission or an OSC 133 mark emitted inside a `tmux` pane vanishes unless `allow-passthrough on` is set, whereas in a native pane there is nothing between the program and the emulator and the sequence simply arrives. This is not a defect in `tmux`; it is the correct default for a program that has to keep an accurate model of a screen it does not own.

Two scrollbacks, and the vi-mode trap

The scrollbar rows describe a distinction that catches people, so it is worth being explicit. In a native pane, the lines that scroll off the top go into `zterminal`’s own scrollbar, sized by `scrolling.history` (Chapter 13), and every `zterminal` feature that reads scrollbar — `vi` mode, search, hints, output selection, the OSC 133 marks — reads that buffer.

In a `tmux` pane, the scrolled-off lines go into the *tmux server*’s history for that pane, governed by `tmux`’s own history limit, not by `scrolling.history`. What `zterminal`’s grid holds is only what `tmux` has painted into the visible region. The practical consequence: entering `zterminal`’s `vi` mode (Chapter 18) inside a `tmux` pane and scrolling up does not walk that pane’s history, because that history is not in `zterminal`’s buffer. It walks whatever `zterminal` happens to hold, which is the outer client’s view. The correct instrument inside a `tmux` pane is `tmux`’s own copy-mode, which reads the server’s history — and the correct expectation is that you now have two scrollbar systems with two keymaps, which is precisely the assembly tax of Chapter 1 showing up in miniature. A native pane has one of each.

This also explains a sizing subtlety. Chapter 13 noted that `scrolling.history` is memory *per pane* and that deep history across many panes adds up. That accounting covers native panes. `tmux` panes are charged to the `tmux` server’s memory under `tmux`’s own limit, so raising `scrolling.history` does not deepen your `tmux` panes’ history and raising `tmux`’s limit does not deepen your native panes’. Two systems, two dials.

What each is actually good at

Stated as guidance rather than a table:

Reach for native panes when the work is local to this machine and this sitting: a build in one pane and an editor in another, a log tailing beside the service producing it, a scratch shell next to a long-running watch. You get the single parse, images and prompt marks with no passthrough dance, one scrollbar with one keymap, the i3 navigation set, and a layout you can save and rebuild (Chapter 24). The cost of another pane is a PTY and a scissor rectangle — cheap enough that splitting is not a decision you deliberate.

Reach for tmux when the work must outlive something: the terminal process, an SSH connection, a laptop lid. This is the case native panes structurally cannot serve, because their PTYs are held by a process that is going away. It is also the right answer when the session must be reachable from *elsewhere* — a server-side tmux you attach to from two machines — since the session lives on the remote host and the client is incidental.

Use both together is the case zterminal is actually designed for, and it is why the unifying features exist. Native splits for the local, ephemeral, image- and mark-heavy work; tmux for the durable and the remote; and then Exposé (Chapter 25) tiles both, cross-pane search (Chapter 30) greps both, the process tree (Chapter 30) descends into both, and the broadcast overlay reaches the tmux side of it. The argument of the whole extension layer is that you should not have to pick your multiplexing model based on which one your terminal's features happen to understand.

What the unification does not paper over

Honesty requires the limits, and Chapter 28 already conceded the main one: the ownership genuinely differs, and the wire client is a seam, not an erasure of the distinction. Concretely, three things do not unify.

The scrollbar split above is real; no feature makes tmux's history appear in zterminal's vi mode. The passthrough requirement is real; enabling it is a security decision (Chapter 39) that the terminal cannot make for you. And the persistence mechanisms remain distinct — a native layout (`layouts/<name>.json`) records a split tree the zterminal process will rebuild, while a tmux snapshot (`snapshots/<name>.json`) records a server's session tree to be rebuilt over the wire (Chapter 29) — so “save my workspace” is two operations if your workspace spans both. The book's position throughout has been that a clearly-stated seam is worth more than a pretended unity, and this is one of the places that position costs something.

Chapter 32 – Env Vars, Snippets, Triggers, Recent Dirs

Four more control-panel managers round out the environment, each backed by a JSON file under `~/.zterminal` and each, in its own way, an original addition.

Env Vars – hot-injected into running shells

Editing an env var in the control panel does two things at once: it persists the variable to `[env]` in `zterminal.toml` (via `toml_edit`), and it *live-broadcasts* the change — `export NAME=value` or `unset NAME` — into the PTY of every already-running shell across all panes, so it takes effect without relaunching. The export is POSIX-single-quoted and prefixed with a leading space to dodge `HISTCONTROL=ignorespace`. The caveat is honest: it injects a shell command, so it affects shells sitting at a prompt, not arbitrary child programs. A claimed addition (Chapter 34).

Snippets

Saved command snippets, reusable from the command palette and — notably — sendable through the cross-session broadcast overlay (Chapter 30), so a snippet can be typed into a chosen set of panes at once.

Triggers – per-pane output reactions

User-defined regexes match each pane's freshly *rendered* output as it streams, firing a desktop notification, a bell, or a shell command (with `$ZT_TRIGGER_TEXT` and `$ZT_TRIGGER_NAME` in its environment). Only lines completed since the last wakeup are scanned, with a cooldown, backed by `~/.zterminal/triggers.json`. The repository is candid that iTerm2's "Triggers" is direct prior art — this is *not* claimed as a first; the only distinguishing angle is the per-pane, zterminal-pane-model framing (Chapter 34).

Recent Dirs

Because OSC 133 and OSC 7 do not carry the working directory, zterminal harvests each open pane's live foreground-process cwd — from its own panes *and* every tmux pane (via `#{pane_current_path}`) — into a pinnable, most-recent-first Recent Dirs list at `~/.zterminal/recent_dirs.json`, sampled on tab refresh. A claimed addition (Chapter 34), Unix-only.

The four managers share a shape

Read together, the four managers of this chapter are variations on one pattern, and seeing the pattern explains why they were cheap to build and why each is backed by its own small JSON file under `~/.zterminal` (Appendix E). Each is a control-panel tab that reads some live state, presents it in `zgui-core` components, writes an edit back, and persists the result — the same frontend-posts, Rust-owns-state shape the whole

panel follows (Chapter 19). What differs is *what* they read and write. Env Vars reads and writes `[env]` in the config and additionally broadcasts an `export` into every live shell (the hot-inject). Snippets reads and writes a saved command list and additionally feeds the broadcast overlay (Chapter 30). Triggers reads and writes a regex-and-action list that a per-pane scanner then consults on each wakeup. Recent Dirs is read-mostly — it *harvests* each pane’s live cwd on refresh rather than taking an edit — and writes a pinnable list. The through-line is that none of them is a new subsystem; each is a thin manager over a JSON file and a hook into machinery the process already has (the config, the broadcast path, the scanner, the pane cwd), which is exactly why the extension layer could add four distinct capabilities without four distinct architectures.

Designing a trigger that does not fight you

Triggers are the manager most likely to be configured badly, because a regex over a live output stream has failure modes that a regex over a file does not. Three properties of the implementation shape what a good trigger looks like.

Only completed lines are scanned, and only since the last wakeup. The scanner does not re-examine the buffer; it looks at what finished arriving since the loop last woke (Chapter 36). This is a performance decision — it keeps the cost linear in output volume rather than quadratic in buffer size — and it has a semantic consequence worth knowing: a trigger matches against a *line*, so a pattern that spans a line boundary will never fire, and a pattern that needs the surrounding context of previous lines cannot see it. Write patterns that are complete within one line.

The cooldown exists because output is bursty. A build that fails prints its error line, and often prints related lines immediately after that also match a loosely-written pattern. Without a cooldown, a single conceptual event produces a volley of notifications. The cooldown collapses the volley into one reaction, which means a trigger’s job is to detect an *event*, and the mechanism assumes you would rather be told once.

The action forks a program with the match in its environment. `$ZT_TRIGGER_TEXT` carries the matched text and `$ZT_TRIGGER_NAME` the trigger’s name (Appendix J), which is what makes one script serve several triggers — branch on the name, act on the text. And because it forks a program, the ordinary discipline applies: this is a user-authored code-execution surface (Chapter 39), it runs with your privileges, and a trigger whose command is slow is a program you have arranged to launch on a schedule you do not fully control.

The pattern-writing advice that follows from all three is to anchor tightly. A trigger on `error` fires on the word appearing anywhere, including in a path, a package name, or the phrase “no errors.” A trigger on `^error:` or on the specific shape your tooling actually emits fires when the thing happened. The bell-versus- trigger distinction of Chapter 13 is the other half of this judgement: if the program already rings the bell to announce itself, `bell.command` is the more reliable hook, because it is reacting to an explicit signal rather than to text that happens to look like one.

Snippets, broadcast, and the composition worth noticing

Snippets read as the least interesting of the four managers until you notice what they compose with. A snippet is saved command text; the command palette can run one (Chapter 23); and the broadcast overlay can send one to a checked set of tmux panes spanning windows and sessions (Chapter 30).

That third path is the one that changes what the feature is for. A snippet sent to one pane is a convenience — you have saved yourself typing a long command. The same snippet sent to fourteen panes across four sessions is a different kind of operation: it is applying one change to a fleet, from a checkbox list, without a configuration-management tool in the loop. The mechanism is the keystroke injection of Chapter 7 — the text is written into each pane’s input — so the honest limits are the same ones the env-var inject carries: it reaches shells sitting at a prompt, and it is text delivery rather than remote execution with results collected. Within those limits it covers a real and common need, and it needed no new machinery because the snippet store, the broadcast path, and the pane enumeration all already existed for their own reasons.

Chapter 33 – IPC

`zterminal` exposes an inherited IPC mechanism — `zt msg` — that controls a running instance from the command line over a Unix socket, without touching the window. `general.ipc_socket` (default `true`, Unix only) enables it.

The messages

`zterminal-msg(1)` documents three:

- `create-window` — open a new window in the *same* `zterminal` process, optionally with `--hold`, `--working-directory`, `--title`, `--class`, `-o/--option` overrides, and a trailing `-e/--command`.
- `config` — push a live configuration change. `zt msg config 'cursor.style="Beam"'` applies the override; `-w/--window-id` targets a window (default `$ZTERM_WINDOW_ID`, `-1` for all); `-r/--reset` clears all runtime overrides. This is the command-line door to the same hot-reload the control panel drives through the GUI.
- `get-config` — read the runtime config back, per-window or global (`-w -1`).

The socket path defaults to the per-instance socket but can be set with `--socket` on both ends, which is how you address a specific instance among several. `zt msg` is the seam between `zterminal` and the rest of the shell-driven workflow: a script, a keybinding in another tool, or a `zshrs` function can open windows, retheme a running terminal, or query its config, all without a GUI.

Worked `zt msg` recipes

The three messages compose into small, useful commands. A few worked examples show the range, all against a running instance over its socket:

```
# Open a window in the running process, in a directory, running a command:
zt msg create-window --working-directory ~/src/zshrs --title build -e cargo build

# Switch every window to a beam cursor for one session (no file edit):
zt msg config -w -1 'cursor.style="Beam"'

# Retheme the running terminal by pushing a background colour to one window:
zt msg config -w "$ZTERM_WINDOW_ID" 'colors.primary.background="#0d1117"'

# Read back what the running instance currently believes, globally:
zt msg get-config -w -1

# Clear every runtime override, returning to the on-disk config:
zt msg config -r
```

Two things make these more than a curiosity. First, `config` pushes the *same* override machinery that the control panel and the `-o` command-line flag use (Chapter 8), so a `zt msg config` change is a live reconfiguration indistinguishable from one made in the GUI — it hot-loads into the running window. Second, the `-w` targeting (`-1` for all windows, `$ZTERM_WINDOW_ID` for the current one) means a script can retheme the specific window it is running in, which is how a `zshrs` function might, say, tint the terminal red while a production deploy is in progress and restore it after. The socket is per-instance by default and selectable with `--socket` (Chapter 33,

Appendix A), which is how a script addresses one instance among several. `zt msg` is what turns “configure the terminal” from a GUI action into a scriptable one.

Overrides are runtime state, not file edits

The most important property of `zt msg config` is what it does *not* do: it does not write your configuration file. An override pushed over the socket is runtime state layered on top of the loaded config, which is why `-r/--reset` exists and means something — clearing every override returns the running instance to what is on disk, with no file to clean up afterwards.

This is a different operation from the Settings tab, which edits `~/.zterminal/zterminal.toml` in place with `toml_edit` (Chapter 20) and therefore persists. Both converge on the same live-reload machinery, so their *effects* are indistinguishable while the process runs; their *durability* is not. The rule to keep is that the GUI and a hand edit change what the terminal will be next time, while `zt msg config` and the `-o` flag change what it is now.

That distinction is what makes the mechanism safe for the scripted uses it is good at. Tinting a window red for the duration of a deploy and restoring it afterwards should not leave a red background in your config file if the script is killed halfway; with overrides, the worst case is a running window that looks wrong until `-r` or a restart, and nothing on disk to repair.

Targeting, and the two window scopes

The `-w/--window-id` argument has three usable forms and they cover distinct intents. Omitted, it targets `$ZTERM_WINDOW_ID` — the window the calling process is running in, which the terminal exports into every child’s environment (Appendix J). This is the form a shell function wants: a script tints *its own* window without needing to know anything about the instance’s topology. Given an explicit id, it targets that window, which is the form a supervising script uses when it opened the window itself. Given `-1`, it targets every window in the instance, which is the form for “re-theme everything.”

`get-config` takes the same targeting, and its most useful application is diagnostic rather than scripted. Appendix G’s “a configuration change did nothing” entry lists three causes, and two of them — the wrong file being read, or a runtime override shadowing the file — are settled immediately by reading back what the running instance actually believes with `zt msg get-config -w -1` rather than reasoning about what it ought to believe.

The socket, and addressing the right instance

`general.ipc_socket` defaults on and is Unix-only, and the socket it creates is per-instance. That last word carries the complication: if you run several zterminal processes — a `SpawnNewInstance` fork rather than a `CreateNewWindow` peer (Chapter 35) — each has its own socket, and a bare `zt msg` addresses the default one. The `--socket <path>` argument on both ends is how you pin the relationship deliberately: launch an instance with a known socket path and address that path from scripts, which turns “the running instance” from an ambient assumption into a name.

This pairs directly with the `--daemon` model of Chapter 35. A daemon comes up with no window, opens its socket, and waits; every `create-window` after that materializes a window in a process whose fonts, config, and tmux connection are already warm. A login session that starts one daemon on a known socket and has its launcher scripts address that socket gets a terminal that opens instantly and a single, well-defined target for every scripted interaction with it.

The security boundary is filesystem permissions and nothing else, as Chapter 39 states: the socket lives under your user-owned state directory, so anyone who can write it can spawn a window in your session or push

overrides into it. On a single-user machine that is the same protection every per-user socket has. On a shared machine it is a thing to know about, and the field turns the capability off entirely if you would rather not have it.

Chapter 34 – The Inventions

This chapter gathers what `zterminal` claims as new, with the honesty the rest of the book has kept: the base VT/grid/search/hints are *not* claimed — they are the Alacritty inheritance of Chapter 2. Everything below is the extension layer's own, and each entry names its source basis so a reader can check it. Three are the repository's own documented firsts; the rest are candidates the broader stack inventory records, with their confidence and caveats.

The three in-repo firsts

These are stated in `zterminal/docs/INVENTIONS.md` as firsts “to our knowledge — no shipping terminal emulator did it before”:

1. **Live-editing a running tmux server.** Reading and editing the server's options, paste buffers, and keybindings from the terminal's own UI, over the native wire protocol with no `tmux` subprocess (Chapter 28). Basis: `crates/ztmux-core` plus the `tmux config/buffers/keys` tabs.
2. **tmux state in switchable profiles.** Profiles that snapshot the whole config — terminal, GUI look, *and* tmux server state — and restore it, including reconfiguring the live server, in one click (Chapter 21). Basis: the profile manager.
3. **A custom live telemetry dashboard.** A polling telemetry surface built from the `zgui-core` component library — PTY throughput, render timing, scrollback, the tmux summary, system metrics, command history — that no emulator ships (Chapter 22).

The catalogued candidates

The stack's inventory (`INVENTIONS.md`) records these additional entries, each with a confidence label and a caveat. They are reproduced here as the inventory frames them — candidates, not settled firsts, and several explicitly *not* firsts:

- **First-class tmux client over the native wire protocol (high)** — the umbrella for the three above: a terminal that is a first-class tmux client and the first to live-edit options/buffers/keys, capture tmux state into profiles, and ship the dashboard. Basis: `ztmux-core`, dashboard on `zgui-core`.
- **Unified Exposé + scrollback search across native and tmux panes (low)** — the blend of i3 native splits with tmux so Exposé (`⌘E`) tiles both and `⌘P` greps every tmux pane, with image protocols decoded through tmux passthrough. Caveat: the pieces exist individually; the combination is the candidate.
- **Per-pane process/activity monitor (med)** — the Processes tree spanning native panes and the repaired tmux server, with kill gated to descendants (Chapter 30). Basis: `src/event.rs pane_process_tree()`. Caveat: Unix-only, an inspector not a `top`.
- **GUI env-var editor that hot-injects into running shells (med)** — Chapter 32. Basis: `src/settings.rs save_env_var()` then `EventType::BroadcastInput`. Caveat: affects shells at a prompt.
- **Native tmux session save/restore over the wire (med)** — Chapter 28. Basis: `crates/ztmux-core/src/snapshot.rs, proc.rs` native command-line capture. Caveat: resurrect-as-concept is prior art; doing it natively over the wire from an emulator is the novelty.
- **Cross-session/cross-window tmux broadcast (med)** — beyond `synchronize-panes` (Chapter 30). Basis: `ztmux-core/src/ops.rs broadcast_list()`.
- **Whole control plane as an in-process zgui-core webview app (med)** — Chapter 19, served over `zterminal://`. Basis: `src/settings.rs include_dir!`

- the `IPC_BRIDGE`.
- **Shell-history palette resolving the focused shell's real HISTFILE (low)** — Chapter 23. Basis: `src/daemon.rs shell_histfile(pid)`. Caveat: history pickers exist; the per-shell HISTFILE-from-process-env resolution is the distinctive bit.
- **Recent-directories tracker from live pane cwd (low)** — Chapter 32. Basis: `src/recent_dirs.rs`. Caveat: sampled on refresh, Unix-only.
- **Single pre-vte interceptor recovering four protocols (low)** — Chapters 26–27, including OSC 133 and tmux-unwrap. Basis: `zterminal_core/src/graphics/scanner.rs`. Caveat: WezTerm supports the image protocols; the unified recovery incl. OSC 133 is the combination.
- **Failed-command gutter marks from OSC 133 exit codes (low)** — Chapter 27. Basis: `zterminal_core/src/shell.rs`. Caveat: exit-status decorations exist (fish/iTerm2).
- **Config-driven background image surviving tmux (low)** — Chapter 26. Basis: `src/config/window.rs`, `src/renderer/graphics.rs`. Caveat: bg images exist; the config-survives-tmux and behind-text angles are distinctive.
- **Zero-dependency icat self-enabling tmux passthrough (low)** — Chapter 26. Basis: `extra/zterm-icat`, `placeholder.rs`. Caveat: a narrow helper.
- **Native menu, keybindings, and palette sharing one dispatch path (low)** — Chapter 19. Basis: `src/macos/menu.rs`. Caveat: an architectural single-source-of-truth, not a user-visible first.
- **Glassy translucent webview overlays over the live GL terminal (low)** — Chapter 19, with a live opacity slider. Basis: `src/settings.rs apply_overlay_alpha()` via `NSWindow.alphaValue`. Caveat: macOS-only, cosmetic.
- **Per-pane output triggers (low)** — Chapter 32. Caveat: iTerm2 “Triggers” is direct prior art — explicitly **not a first**; included for completeness.

The honest reading of this catalogue is the same as the book's thesis. A handful of the entries are genuine firsts the repository documents and stands behind (the tmux live-editing, the profiles, the dashboard); many are low-confidence combinations or are openly marked as prior art; and the whole list sits on top of an inherited core that is not claimed at all. That is exactly the shape a faithful product should present — a clear foundation, a clearly-bounded set of real additions, and no rounding up.

Reading the confidence labels

The catalogue's `high` / `med` / `low` labels are not decoration; they encode how much of a claim survives scrutiny, and reading them correctly is the difference between a marketing list and an honest inventory. A `high`-confidence entry is one the repository stands behind as a genuine first — the first-class tmux client and the three firsts it umbrellas (live editing, profiles, dashboard) — where the authors could not find prior art and the capability is concrete. A `med` entry is a real, distinctive capability with a plausible-but-unverified originality claim: the env-var hot-inject, the native session save/restore, the cross-session broadcast, the whole-control-plane-as-webview — each does something the authors have not seen elsewhere, but the label admits they have not exhaustively proven no one else does it. A `low` entry is either a combination whose pieces exist separately (the unified Exposé, the pre-vte interceptor) or a narrow helper (the self-enabling icat, the glassy overlays), included for completeness rather than asserted as a landmark. And at the bottom, per-pane output triggers are labelled explicitly *not a first* — iTerm2's Triggers is named as direct prior art — which is the strongest signal that the catalogue is honest, because a list willing to print “this one is not ours” next to a feature it ships is not a list rounding up. The labels are a calibration, not a sales pitch: they tell you exactly how hard to squint at each claim, and the book reproduces them rather than laundering the `lows` into `highs`.

Chapter 35 – The Process Model

The book has now documented both halves — the inherited core in Chapters 3–18, the original extension layer in Chapters 19–34. The remaining chapters step back from individual features to the behaviour of the whole running program: how one process holds many windows, how it spends its microseconds, how it copies text, what it trusts, and how it announces its capabilities. This chapter is the first of those, and it is deliberately first, because the process model is the quiet reason so many of the preceding extensions can exist at all. A feature that must reach *every* pane — the dashboard, the command palette, the broadcast overlay, Exposé — is trivial when every pane lives in one address space and nearly impossible when it does not.

One process, many windows

zterminal is single-process, multi-window. `CreateNewWindow` (`⌘N`) opens another window *inside the same running process*; `SpawnNewInstance` forks a brand-new `zterminal` process that shares nothing. The distinction is not cosmetic. When a window is a peer inside the one process, everything the process holds is shared across all of them: the registry of open PTYs, the tmux client and its single socket to the server, the loaded configuration, the profile set, the command history harvested from OSC 133, the recent-directories list, and the control panel's state. That shared world is what makes the cross-cutting extensions cheap. The Dashboard (Chapter 22) polls one PTY registry and sees every pane in every window; the command palette (Chapter 23) can index actions that target any pane; the broadcast overlay (Chapter 30) can enumerate the panes to send to; Exposé (Chapter 25) can tile every window and every tmux pane at once. A per-window-process terminal would have to build an IPC layer to do any of this, and most do not bother, which is why most terminals do not have it.

Windows, tabs, and panes – three nestings

Three distinct levels of containment are easy to conflate, and the keybindings act on different ones, so it is worth naming them precisely:

- **Windows** are operating-system windows, each with its own title, decorations, and placement. `⌘N` makes another; `⌘W`/`⌘Q` close and quit.
- **Tabs** (macOS) are multiple grids stacked in one window behind a native tab bar. `⌘T` opens one, `⌘1–9` select by index, `⌘⇧ ]/[` and `⌘Tab` cycle, and `⌘E` jumps to the most-recently-used other tab (Chapter 25). Tabs are the OS window server's feature, not a multiplexer's.
- **Panes** are the leaves of the native split tree *within* a single tab or window (Chapter 24), each its own shell and PTY, GL-scissored into a sub-viewport. `⌘D`/`⌘⇧D` split; the i3 navigation set moves among them.

A tmux pane is a fourth thing again — a pane owned by the reparented tmux server, surfaced through the wire client (Chapter 28) — and part of zterminal's ambition is to let the native and tmux panes read as one surface in Exposé and cross-pane search, even though their ownership differs.

The daemon

`--daemon` starts zterminal with no initial window: the process comes up, opens its IPC socket, and waits. `zt msg create-window` (Chapter 33) then materializes windows in that already-running process on demand — the fast path, since the process, its fonts, its config, and its tmux connection are already warm. This is the

model a login session or a launcher uses to keep one terminal process resident and spawn windows into it instantly, and it is only coherent *because* the process is the shared host described above.

Why single-process is load-bearing

It is tempting to read the single-process design as a mere implementation convenience. It is closer to a precondition. Nearly every claimed addition in Chapter 34 assumes a shared address space: the live tmux client is one socket the whole process talks over; profiles snapshot a state that is process-global; the env-var hot-inject broadcasts to every PTY the process owns; the process tree seeds its roots from every pane the process knows about. Move to one-process- per-window and each of these becomes a distributed-systems problem with a socket protocol, a leader election, and a consistency story. zterminal declines that complexity by keeping the panes together, and the extension layer is the dividend.

The PTY, concretely

Under every pane is a pseudo-terminal, and its behaviour explains several things users notice. A PTY is a kernel object with two ends: the *primary* end, which zterminal holds and reads and writes, and the *secondary* end, which the shell has open as its stdin, stdout, and stderr. When the shell prints, it writes the secondary end and the bytes arrive at the primary end for zterminal to parse; when you type, zterminal writes the primary end and the bytes arrive at the shell's stdin. The kernel's line discipline sits between them, which is why `Ctrl+C` at a naive prompt sends `SIGINT` to the foreground process group — the discipline, not zterminal, translates the byte into a signal. A window resize propagates through this object too: when the grid's dimensions change (Chapter 3), zterminal sets the PTY's window size, the kernel delivers `SIGWINCH` to the foreground process, and a full-screen application repaints to the new size. This is the plumbing beneath the whole book — the grid on one side, the shell on the other, the PTY the pipe between — and it is per-pane, which is what “each pane is a real terminal” (Chapter 24) means at the kernel level.

Lifecycle: what closes when

The containment levels of this chapter have a matching teardown, and knowing it avoids surprises. Closing a pane (`⌘ W`) closes that pane's PTY, which sends the shell an end-of-file on its input and a hang-up (`SIGHUP`) — the shell exits, its children get the hang-up too, and the split tree collapses the leaf (Chapter 24). Closing the last pane in a tab or window closes the window. `⌘ Q` quits the whole process, closing every window and PTY it owns and dropping the tmux client's socket — the tmux server keeps running, because it is a separate process the client was merely attached to (Chapter 28), which is exactly why a tmux session survives quitting zterminal and a native pane does not. That asymmetry — native panes die with the process, tmux panes outlive it — is the single most important thing to internalize about the two kinds of pane, and it falls directly out of who owns the PTY: zterminal for a native pane, the tmux server for a tmux pane.

Chapter 36 – The Event Loop

The phrase “one event loop” has appeared in nearly every chapter of this book — in the pitch, in the architecture of the control panel, in the tmux client, in the performance discipline — always as an assurance and never as a subject. This chapter makes it the subject. Chapter 5 described the *draw* half of the loop, the damage set and the frame clock. Chapter 35 described the *process* the loop runs inside. This chapter describes the loop itself: what wakes it, in what order it handles what it finds, what it refuses to do, and why the single-loop design is a correctness property before it is a performance one.

What wakes it

A loop that spun continuously would be a busy-wait, burning a core to discover that nothing happened. This one sleeps, and it can sleep because every source of work is either a file descriptor the kernel can signal or a timer with a known deadline. The full set of wake sources, gathered from where the book introduced each:

- **A pane’s PTY becomes readable.** One per pane, in every window and every tab the process owns (Chapters 24, 35). This is the dominant source under load.
- **The window system delivers an event.** A key, a mouse button, a resize, a focus change, a scale-factor change when a window moves between monitors (Chapters 7, 10).
- **A frame is due.** The refresh-cadence timer that gates the coalesced draw (Chapter 5).
- **A feature timer expires.** Cursor blink, the visual-bell animation curve, an animated Kitty image’s next frame (Chapter 5), and the refresh intervals of the Dashboard, the Processes tree, the tmux tab, and Recent Dirs while their tabs are open (Chapters 22, 30, 32).
- **The tmux socket has a message.** A reply to a command the client issued, or a notification the server pushed because its session tree changed (Chapter 28).
- **The IPC socket has a message.** A `zt msg` from a script or another tool (Chapter 33).
- **The WebView bridge posts.** A `window.__ztermInvoke` message from the control panel or a palette (Chapter 19).
- **The config file changed.** The live-reload watch that makes a saved edit land in running windows (Chapter 8).

Two observations about that list carry the chapter. First, every entry is edge-triggered: something arrives, or a deadline passes. Nothing on it is a poll the loop performs speculatively, which is what makes an idle window genuinely idle rather than merely quiet. Second, the list spans the entire feature set — the inherited core’s PTYs and input, and the extension layer’s tmux client, IPC, control panel, and managers — and all of them are *sources for one loop* rather than each having a loop of its own. That is the architectural claim this chapter exists to make concrete.

Why one loop and not a thread per pane

A terminal with many panes invites the obvious design: give each pane a thread that reads its own PTY and updates its own grid. zterminal does not, and the reasons are worth stating because the alternative looks so reasonable.

The first reason is that the work does not decompose the way the design assumes. Reading a PTY and mutating a grid is cheap — Chapter 37’s step 3 is a table lookup and a cell write. The expensive, serializing step is the *draw*, and there is exactly one of those per frame for the whole window regardless of how many

panes contributed damage (Chapter 24's scissoring). Parallelizing the cheap half to contend on the expensive half buys nothing.

The second reason is shared state. Chapter 35 catalogued what the process holds in common: the PTY registry, the single tmux client socket, the loaded config, the profile set, the harvested command history and recent directories, the control panel's state. A thread-per-pane design makes every one of those a synchronization problem — locks around the registry, a lock or a channel around the tmux socket, a consistency story for a config reload that must reach every pane. The single-loop design makes them ordinary data, because there is only one place they are touched.

The third reason is ordering, and it is the one that matters most for correctness. In a single loop, a config reload and a batch of PTY bytes cannot interleave: the loop finishes applying the batch, then applies the reload, or the reverse, but never half of each. A profile switch (Chapter 21) that re-themes every window and pushes options to the tmux server happens as a sequence of steps with no other work wedged between them. A resize that reflows the grid (Chapter 3) is not racing a parser mid-sequence. These are not performance properties; they are the absence of a category of bug that a concurrent design would have to earn back with careful locking.

The order within one iteration

Within a single wake, the loop's ordering follows from the latency argument of Chapter 37: input first, drawing last.

Input events are handled early because the path from a keystroke to the echoed character on screen runs through the whole loop (write to the PTY, shell echoes, read, parse, draw), and any delay in accepting the input is added directly to a latency the user perceives. PTY reads follow, in batches, mutating grids and accumulating damage. Socket and bridge messages are handled as they arrive. Timers fire when due. And the draw is *last*, once, after everything else in the iteration has had its chance to mark damage — which is exactly the coalescing of Chapter 5 expressed as an ordering rule rather than a policy. Everything that wants to be seen in this frame must have marked its damage before the loop reaches the draw; whatever arrives after waits for the next frame, which is at most one refresh interval away.

This ordering is why a burst of activity across many panes still costs one draw. A firehose in pane A, a cursor blink in pane B, an animated image in pane C, and a tmux notification that changed the session tree can all land in the same iteration, and they produce one frame containing all four effects.

The blocking discipline

A single loop has one catastrophic failure mode: something in it blocks. A synchronous call that waits — for a socket reply, for a subprocess, for a WebView to finish painting, for a file — stalls every pane, every keystroke, and every frame, because there is no other thread to keep the terminal responsive. The extension layer is large, and much of it is exactly the kind of code that would naïvely block. So the discipline of Chapter 37 is not a performance preference here; it is the condition under which the single-loop design is viable at all. Each major extension pays it a different way:

- **The control panel** is a WebView doing its own rendering on its own schedule (Chapter 19). The loop never waits on it. The compositing trick — driving the native window's alpha so the panel floats over a still-live GL terminal — is what makes “the panel renders independently” visually coherent rather than a frozen terminal behind a live panel.
- **The tmux client** is a socket the loop reads when it is readable, and a place it writes commands to. It does not synchronously wait for a reply mid-iteration; the reply is a later wake (Chapter 28). This is why the phrase “folds its events into the same loop” recurs — folding in means becoming another readable descriptor, not becoming a blocking call.
- **The output triggers** scan only the lines completed since the last wakeup, under a cooldown (Chapter 32). A trigger that rescanned the whole buffer on every wake would turn a linear cost into a quadratic

one under exactly the firehose conditions where the loop can least afford it.

- **The managers** — Dashboard, Processes, Recent Dirs, the tmux tab — sample on an interval and only while their tab is open (Chapters 22, 30, 32), so they are timers with bounded work rather than continuous instrumentation.
- **The pre-vte scanner** is a linear pass over bytes already in hand (Chapters 26, 27). It cannot block because it does no I/O; it only inspects a buffer.

The pattern is uniform: every extension is either a descriptor the loop can wait on alongside everything else, a timer with a deadline, or a bounded computation over data already read. Nothing in the list is permitted to be a synchronous wait.

Reload, IPC, and profile switches as loop events

Three things that feel like they should be “outside” the loop are usefully seen as events within it, because that framing explains their behaviour.

A **config reload** is an event. The file changes, the loop wakes, the new config is parsed and applied, and — because a re-theme or a font change alters every cell — full damage is marked and the next frame redraws the whole grid (Chapter 5). Nothing about this is special-cased; it is a wake source that happens to mark a lot of damage. That is also why the restart-only fields of Appendix B are restart-only: they are consumed when a window is *built*, not when the loop applies a change, so re-applying them mid-life has nowhere to take effect.

A **zt msg** (Chapter 33) is an event on the IPC socket. **zt msg config** reaches the same override machinery a GUI edit or a **-o** flag does, which is why the book can say the three paths are indistinguishable in effect: they converge on one handler inside one loop. **zt msg create-window** builds a window in the running process because the loop is the thing that owns windows.

A **profile switch** (Chapter 21) is a burst of events with a guaranteed ordering: apply the terminal config, restore the GUI look, push the tmux server state over the wire. Because these happen in sequence within the loop rather than concurrently, the switch is atomic from the user’s perspective — there is no frame in which the config has landed but the theme has not. Chapter 21 called the three restores “landing together”; the loop is the reason they can.

The dividend, stated plainly

Chapter 35 argued that single-process is load-bearing for the feature set. The single loop is the same argument at a finer grain. A dashboard that sees every pane, a broadcast that reaches a chosen set of them, an Exposé that tiles native and tmux panes, a config change that lands everywhere at once — each of these is, implemented in the loop, a walk over structures that are already in hand and already quiescent, because nothing else is running while the loop is doing it. In a design with per-pane threads or per-window processes, each would need a protocol, a synchronization point, and a story about partial failure. The loop does not eliminate that complexity by being clever; it eliminates it by never creating the concurrency that would require it.

Chapter 37 — Performance

The pitch in Chapter 1 said speed is the floor, not the ceiling — the inherited core is fast and the value is the integration on top. But a floor still has to hold, and a terminal that stutters under load makes the whole shell-centric workflow of Chapter 57 stutter with it. This chapter examines the floor: where the microseconds go between a byte arriving on the PTY and a pixel landing on the screen, and the specific techniques that keep that path short.

The read loop

Bytes do not arrive one at a time to be handled one at a time; that would spend more time in syscalls than in work. `zterminal` reads from the PTY in large batches and feeds each batch to the `vte` parser as a run. The parser is a table-driven state machine — the classic VT500-series design — so the per-byte cost of the common case, a printable character, is a state lookup and a write into the current grid cell, with no allocation and no branching pyramid. The expensive events are the rare ones: a resize that reflows the grid, an alternate-screen switch that saves and restores, a scroll that shifts the scrollbar. The architecture is built so the frequent thing is cheap and the infrequent thing is allowed to be dear. This is why `cat`-ing a large file, the traditional terminal torture test, is bounded by how fast the grid can be mutated and the screen redrawn, not by parser overhead.

Damage and the frame cadence

The second half of the loop is drawing, and the governing principle is that a terminal does not redraw when nothing changed and does not redraw *more* than the display can show. The renderer tracks damage — the set of cells altered since the last frame (Chapter 4) — and coalesces however many PTY updates arrived since the last frame into a single redraw at the display's refresh cadence. A program that emits a thousand tiny writes in one frame's worth of time costs one frame, not a thousand. Synchronized output (private mode 2026) hands that batching decision to the application: between the begin and end marks the terminal holds the frame, so a complex full-screen repaint lands atomically instead of tearing through several frames. `debug.render_timer` and `debug.highlight_damage` (Chapter 17) are the two windows into this machinery — the first prints the per-frame draw time, the second tints the regions actually being redrawn, and watching the second while a TUI runs is the fastest way to see how little of the screen most updates touch.

The glyph atlas as amortization

The renderer's central performance idea is amortization: rasterizing a glyph is expensive, so it happens once, the first time a glyph is seen, and the bitmap is cached in the GPU atlas forever after (Chapter 4). Every subsequent appearance of that glyph is a textured quad — no CPU rasterization at all. The steady state of a running terminal, where the same few hundred glyphs recur endlessly, therefore does almost no rasterization; the cost is paid at warm-up and then not again. The only pathological case is an application that cycles through an unbounded set of distinct glyphs faster than the atlas can hold them, forcing eviction and re-rasterization, and that case is vanishingly rare in real terminal use.

The extension layer's performance discipline

The original layer is built to stay off the hot path, and each of its major pieces reflects a deliberate choice to do so. The pre-`vte` scanner (Chapters 26–27) adds one linear pass over the byte stream, but it is a cheap scan for a handful of sequence prefixes, not a second parser. The control panel is a `WebView`, which could have been a stall, but it composites over the still-live GL terminal by driving the window's alpha (Chapter 19) — the read-and-draw loop underneath never waits on it. The `tmux` client speaks over its own socket and folds the server's events into the same event loop rather than blocking on synchronous polls (Chapter 28). Output triggers scan only the lines completed since the last wakeup, under a cooldown, rather than re-scanning the whole buffer (Chapter 32). None of these is an accident; each is the extension layer paying rent to the principle that the terminal must stay fast under the features.

Measuring it

The instrumentation is built in and consistent with the rest of the program. `debug.render_timer` overlays frame time; the Dashboard (Chapter 22) renders live PTY-throughput and render-timing gauges from the same counters, so the numbers are visible without a debug flag; and `--ref-test` records a reference session for regression testing the renderer. The honest posture, as everywhere in this book, is that these are the real measurements the program exposes, not a benchmark table manufactured for a marketing page.

The end-to-end path, named

Pulling the pieces together, the life of a byte from PTY to pixel is a short list of cheap steps, and naming them in order is the clearest summary of where the microseconds go:

1. **Read.** A batch of bytes is read from the PTY in one syscall, not one byte at a time (the read loop above), amortizing syscall overhead across the whole batch.
2. **Scan.** The pre-`vte` scanner (Chapters 26–27) makes one linear pass to peel out image and OSC 133 sequences — a prefix check, not a second parser.
3. **Parse.** The remaining bytes feed the `vte` state machine, whose common case — a printable character — is a table lookup and a cell write, no allocation.
4. **Mutate.** Each parsed token applies to the grid, marking damage as it goes (Chapter 5).
5. **Coalesce.** All the damage since the last frame accumulates; the draw is deferred to the display's cadence (Chapter 5).
6. **Draw.** When a frame is due, the damaged cells become textured quads from the glyph atlas in a few draw calls, and the frame is presented.

Nothing in that path allocates or rasterizes in the common case: the atlas already holds the glyphs (step 6 is a lookup), the parser already knows the transition (step 3 is a lookup), and the read already batched the syscall (step 1). The expensive things — rasterizing a newly-seen glyph, reflowing on resize, switching screens — are pushed to the edges of the path where they happen rarely. This is the whole performance story in one list, and it is why the terminal's speed is a property of its architecture rather than a tuning pass laid over a slow core.

The cost of the features, honestly

The chapter would be dishonest if it claimed the extension layer is free. It is not free; it is *bounded*, and the bound is what matters. The scanner adds a pass whose cost is linear in the byte volume and tiny per byte. The control panel, when open, is a `WebView` doing its own rendering — but on its own, off the terminal's frame loop (Chapter 5), so it costs the terminal nothing while the panel is idle and never blocks a terminal frame even while it is busy. The `tmux` client holds a socket and processes notifications, which is real work, but event-driven work that sleeps when the server is quiet. The Processes tree, the Dashboard, and the Recent Dirs list sample on a refresh interval rather than continuously, so their cost is paid only while their tab is open and looking.

Each of these is a deliberate choice to make a feature's cost proportional to its use rather than a constant tax on the hot path, and the sum of those choices is a terminal that carries an unusually large feature set without the feature set showing up as latency at the prompt.

Latency, not just throughput

Throughput — how fast the terminal can consume a firehose — is the metric torture tests measure, but latency is the one a user feels. Latency here is the time from a keystroke to the character appearing, and it is a different path from the read loop: an input event arrives, translates to bytes written to the PTY, the shell echoes them back, and the echo travels the read-scan-parse-draw path to the screen. The governing constraint is the same frame cadence of Chapter 5 — the echoed character appears at the next frame — so keystroke latency is bounded below by the display's refresh interval and the input-to-frame alignment, not by any per-character work the terminal does, because that work is trivial. What the architecture avoids is *adding* latency: the extension layer's off-the-hot-path discipline means a keystroke is not waiting behind a WebView repaint, a tmux poll, or a trigger scan, because none of those sits between the key and the frame. The honest statement is that a terminal cannot beat the display's refresh for echo latency, and zterminal's goal is to not lose to it — to keep the input path as short as the frame loop allows, which is what the single event loop and the deferred-draw scheduler together buy.

Chapter 38 – Selection, Clipboard, and Copy

Copying text out of a terminal looks trivial and is not. A selection has a shape, a semantic unit, and a destination, and there are two clipboards on the free- desktop platforms with different rules. This chapter is the model, because it is the substrate several extensions build on and because the defaults reward understanding.

The selection model

A selection is a span of cells, and it has one of four shapes. **Character** selection (`v` in vi mode, a plain drag) runs cell to cell, wrapping at line ends. **Line** selection (`V`) takes whole rows. **Block** selection (`Ctrl+V`, or a drag with the block modifier) is rectangular — a column range across a row range, which is how you lift one column out of tabular output. **Semantic** selection (`Alt+V`, double-click) takes a “word” as bounded by `selection.semantic_escape_chars` (Chapter 13): the same field that defines what `w/b/e` traverse in vi mode, so the mouse and the keyboard agree on where a word ends. That agreement is the point of routing both through one configurable set of boundary characters rather than hard-coding “whitespace.”

The two clipboards

On X11 and Wayland there are two independent clipboards, and conflating them is the source of most “paste did the wrong thing” confusion. The **clipboard** proper is the explicit copy-and-paste buffer, written by `Copy` and read by `Paste`. The **primary selection** is the implicit one: any selection populates it, and a middle-click pastes it, with no explicit copy step — the traditional X model. `zterminal` keeps them distinct: `Copy/Paste` use the clipboard, `CopySelection/PasteSelection` use the primary selection (both Linux/BSD-only actions, Appendix C), and `selection.save_to_clipboard` (Chapter 13), when set, mirrors every selection into the clipboard as well so the two behave as one for users who want that. macOS has only the single system pasteboard, so the distinction collapses there and only the clipboard actions apply.

OSC 52 – the programmatic clipboard

An application can read or write the clipboard without a mouse via the `OSC 52` escape (Chapters 6, 14), which is what lets a remote editor over SSH copy to your local clipboard. That power is also the risk: a program that can *read* your clipboard can exfiltrate whatever you last copied, including a password. So `terminal.osc52` defaults to `OnlyCopy` — programs may write the clipboard but not read it — and the four settings (`Disabled`, `OnlyCopy`, `OnlyPaste`, `CopyPaste`) let you widen or close that gate deliberately. The default is the security-conscious choice, and Chapter 39 returns to why.

Bracketed paste

When you paste multiple lines into a naive shell, each newline runs as a command the instant it arrives — a classic way to execute something you meant only to review. Bracketed paste (private mode 2004) fixes this: a supporting application asks the terminal to wrap pasted text in `ESC [ 200 ~ ... ESC [ 201 ~`, so the

program can tell pasted bytes from typed bytes and decline to execute them immediately. `zterminal` implements the mode (Appendix D); modern shells and editors enable it, which is why a multi-line paste into `zsh` or `vim` lands as inert text rather than a runaway.

Selection meets the extensions

Two extensions produce selections without a mouse, and they reuse this same machinery. OSC 133 output selection (`⌘⇧O`, Chapter 27) builds a selection from the semantic prompt marks — the span from one command's output-start to its output-end — and hands it to the ordinary copy path, so a keystroke yields exactly the last command's output on the clipboard. Hints with a `Copy` or `Select` action (Chapter 15) build a selection from a regex match. In both, the *origin* of the selection is novel but the *destination* — the clipboard model of this chapter — is the shared, inherited one, which is the whole program's method in miniature.

Block selection and tabular data

The rectangular block selection deserves a paragraph of its own, because it solves a problem the other shapes cannot. Character selection wraps line to line; line selection takes whole rows; but neither can lift a *column* out of tabular output — the third field of every row of `ps`, the size column of `ls -l`, one column of a `|`-delimited table. Block selection (`Ctrl+V` in vi mode, or a drag with the block modifier) takes a rectangle: a column range across a row range, ignoring everything outside those columns. The result is that you can select the PID column of a process listing without dragging through the command names beside it, or pull one field from a wide log without the surrounding noise. This is not a cosmetic variant; it is the only selection shape that treats the grid as a grid rather than as a stream of wrapped lines, and it pairs naturally with the semantic word boundaries of the other shapes — where semantic selection respects *what a word is*, block selection respects *where a column is*. Both are ways of saying “select the meaningful unit, not the raw span,” which is the theme the whole chapter returns to.

Chapter 39 — Security and Trust Boundaries

A terminal emulator sits on a trust boundary that is easy to forget: it renders a byte stream produced by arbitrary programs, some of them remote and some of them hostile, and at the same time it holds the keyboard and the clipboard. Every escape sequence it answers is a small capability granted to whatever is on the other end of the PTY. This chapter names zterminal's trust boundaries and the defaults that keep them defensible. None of it is exotic; it is the standard terminal threat model, stated plainly.

The bytes are not trusted

The stream arriving from the PTY can come from a remote host over SSH, a file you `cat` by mistake, or a program that has been compromised, and none of those is entitled to control your machine merely by printing. This is part of why the REJECTED discipline of Chapter 6 is a *security* posture as much as a correctness one: a terminal that answers every sequence it can parse offers a wider attack surface than one that cleanly ignores what it does not need. The classic hazards are well known — a title-set-then-report sequence that echoes attacker-chosen text back onto your command line, a clipboard read that exfiltrates a secret, a cursor-position query used to smuggle a response — and zterminal's answer is the inherited one: implement what applications legitimately need, reject the rest, and never report back data an attacker could have planted.

Clipboard: the OnlyCopy default

The single most important security default is `terminal.osc52 = "OnlyCopy"` (Chapters 14, 38). Writing the clipboard is a convenience; *reading* it remotely is an exfiltration primitive, so the default withholds the read. A user who trusts their remote sessions can open it to `CopyPaste`, but the shipped posture assumes the bytes are hostile until told otherwise, which is the correct default for a tool that spends its life connected to other people's machines.

Passthrough is opt-in, on purpose

The inline-image protocols and OSC 133 marks survive tmux only when the user sets `allow-passthrough on` (Chapters 26–27). It is off by default in tmux for a reason: passthrough lets a program *inside* a multiplexer pane speak escape sequences directly to the outer terminal, bypassing tmux's own sanitization. That is exactly what zterminal needs to decode an image through tmux, and it is exactly the capability a careful operator might not want to hand every program in every pane. This book documents how to enable it and does not enable it for you; the choice is the user's, and the default-off is the safe one.

The IPC socket

`general.ipc_socket` (default on, Unix-only) opens a Unix-domain socket over which `zt msg` can create windows and, more consequentially, push configuration overrides and read the running config back (Chapter 33). Anyone who can write that socket can retheme your terminal, change its cursor, or spawn a window in

your session. The boundary here is filesystem permissions: the socket lives under your user-owned state directory, so the protection is the same as any per-user socket — adequate on a single-user machine, and a thing to be aware of on a shared one. If the capability is unwanted, the field turns it off entirely.

Injection and command execution are user-authored

Several extensions run programs or write to shells, and it is worth being exact about who authorized each. The env-var hot-inject (Chapter 32) writes `export NAME=value` into the PTY of shells sitting at a prompt — it is, honestly, a keystroke injection, which is why the book states plainly that it affects shells at a prompt and not arbitrary child programs, and why the export is single-quoted and space-prefixed to behave. `bell.command`, a hint's `command`, and a trigger's shell command all fork a program — but every one of those is a line *you* put in your own config or trigger list, run with your own privileges, not a path a remote program can reach on its own. The distinction the chapter wants to leave you with is that zterminal's code-execution surfaces are user-configured, not remotely reachable, and its remotely-reachable surfaces (the escape parser, the clipboard) are gated by the conservative defaults above.

Hyperlinks open with the platform opener

An OSC 8 hyperlink (Chapter 6) and a hint's URL (Chapter 15) are opened with the system opener — `xdg-open`, `open`, `cmd /c start`. That is the same trust decision a browser makes when you click a link: the target is attacker-influenced text, so the hint post-processing that trims stray trailing characters is a small correctness-and-safety aid, and the ordinary caution about where a link goes applies here as it does anywhere.

The boundaries, as a checklist

The chapter's argument reduces to a short checklist a careful operator can keep in mind, each item tied to a default this book documents:

- **The escape parser** answers only what applications legitimately need and cleanly rejects the rest (Chapters 5, 6). The REJECTED status is a smaller attack surface, not a missing feature.
- **The clipboard** withholds *reading* by default (`terminal.osc52 = "OnlyCopy"`, Chapters 14, 38): a remote program may set your clipboard but not read it, so a copied password cannot be exfiltrated by a sequence.
- **tmux passthrough** is off until you enable it (Chapters 26–27): images and OSC 133 marks work through tmux only when you accept that a program in a pane can speak escape sequences past tmux's sanitization.
- **The IPC socket** is a per-user socket under your state directory (Chapter 33): its protection is filesystem permissions, adequate on a single-user machine, and the field turns it off entirely if unwanted.
- **Code-execution surfaces** — `bell.command`, a hint's `command`, a trigger's command, the env-var inject — are all *user-authored*, run with your privileges, not reachable by the remote byte stream.

The dividing line the whole chapter draws is between what the *bytes* can do and what *you* can do. The bytes get the conservative defaults and the rejecting parser; you get the full set of code-execution features, because you are trusted and the byte stream is not. Every default in the list is chosen on that principle, and none of them is a prompt or a nag (the stack forbids those) — they are quiet, correct defaults that a knowing user can widen deliberately.

Chapter 40 – Terminfo and Capability Advertisement

The final system chapter is about how `zterminal` *tells the world what it can do*, which is a surprisingly load-bearing question: an application decides which sequences to send based on what it believes the terminal supports, and if that belief is wrong the symptoms are the familiar ones — missing colours, broken arrow keys, garbled box drawing. There are two mechanisms, a static database and a set of runtime queries, and `zterminal` participates in both.

TERM and terminfo

The static mechanism is `terminfo`: a compiled capability database keyed by the `TERM` environment variable, which every curses application consults to learn how to move the cursor, set a colour, or clear the screen for *this* terminal. `zterminal` ships its own entry, `extra/zterminal.info`, and installing it lets `TERM=zterminal` advertise the emulator's real capabilities. When the entry is not installed — on a fresh machine, or the far side of an SSH hop — `zterminal` falls back to identifying as `xterm-256color`, a nearly-universal entry that is correct about the common capabilities and silent about `zterminal`'s extras. Installing the entry (Chapter 41) is never required to run the binary, but it closes the gap between what the terminal can do and what applications believe it can.

The runtime alternative: queries

The static database is a belief; the runtime queries are a measurement. Rather than trust `TERM`, a modern application can ask the terminal what it supports and adapt to the answer. `zterminal` answers the standard probes — Device Attributes and the Device Status Report family (`CSI n`, Chapter 6), the identify sequence (`ESC Z`) — and, importantly, the Kitty graphics support query (`a=q`, Chapter 26), which is how an image-capable tool auto-detects that it can send Kitty graphics to `zterminal` without being told. Query-based capability detection is strictly more robust than `TERM`, because it survives the SSH hop that leaves the remote `terminfo` stale, and `zterminal` supports it precisely so tools that prefer to measure rather than assume get a truthful answer.

Keyboard protocol advertisement

Keys are their own advertisement problem. The legacy encoding cannot distinguish, say, `Ctrl+I` from `Tab`, and the modern fix is the Kitty keyboard protocol — the `CSI u` family (Chapters 6, 16) — which disambiguates keys and reports modifiers unambiguously. An application enables it by pushing a flag with `CSI > u` and querying the active mode with `CSI ? u`; `zterminal` implements the family, so an editor that wants unambiguous keys can negotiate them at runtime and pop the mode when it exits. This is capability advertisement for input rather than output, and it follows the same query-and-adapt shape as the graphics probe.

Over SSH, concretely

The practical failure mode is the SSH hop: you connect to a host that has never heard of `TERM=zterminal`, ncurses cannot find the entry, and applications either error out or fall back badly. There are two honest fixes. Copy the entry to the remote once — `infocmp zterminal | ssh host tic -x -` compiles it into the remote user's terminfo — and thereafter `TERM=zterminal` works there too. Or, for a host you do not control and will not revisit, set `TERM=xterm-256color` for that session and accept the common-denominator capability set. This book prefers to state the seam rather than pretend it is not there: a terminal that advertises a name the world does not know must either teach the world the name or fall back to one it does, and `zterminal` is built to do both.

What the entry advertises

It helps to know what a terminfo entry actually *is*, because it demystifies both the fallback and the SSH fix. The entry is a set of named capabilities — booleans (does it do colour?), numbers (how many colours?), and strings (what bytes move the cursor to row R, column C?) — that curses reads once at startup and uses to drive the terminal without knowing which terminal it is. The `xterm-256color` entry `zterminal` falls back to is correct about the capabilities almost every terminal shares — 256 colours, the standard cursor motions, the alternate screen — which is why the fallback works for the overwhelming majority of applications; what it omits is `zterminal`'s extras, the capabilities beyond the common set. Installing the `zterminal` entry replaces the common-denominator description with the accurate one. The reason this matters over SSH specifically is that the entry lives on the machine running the application, not the machine drawing the screen: a remote host has its own terminfo database, and if `zterminal` is not in it, `TERM=zterminal` names an entry the remote curses cannot find — hence the `infocmp zterminal | ssh host tic -x -` fix, which copies the description across so the name resolves on the far side too. The static database and the runtime queries are two answers to one question — “what can this terminal do?” — and an application that asks the second (Device Attributes, the graphics and keyboard queries above) is robust against the first being stale, which is the deeper reason `zterminal` answers the queries at all.

Chapter 41 – Building and Installing

zterminal builds from source with `cargo`. The build is vendored and self-contained — no external runtime dependencies — but the platform graphics and font libraries are needed at compile time, because the renderer is real OpenGL and the font stack is real system fontconfig.

Cargo and from source

```
cargo install zterminal          # binary only

git clone https://github.com/MenkeTechnologies/zterminal.git
cd zterminal
cargo build                      # debug binary at target/debug/zterminal
```

Pin the stable toolchain with `rustup override set stable`. The minimum build dependencies vary by platform; on Debian/Ubuntu:

```
apt install cmake g++ pkg-config libfontconfig1-dev libxcb-xf86dev libxkbcommon-dev
python3
```

`INSTALL.md` carries the per-distribution lists for Arch, Fedora, RHEL, openSUSE, Void, the BSDs, NixOS, Gentoo, Alpine, Windows, and more.

The macOS installer

`make pkg` builds a single `.pkg` that drops `Zterminal.app` into `/Applications` and the `zterminal-icat` helper into `/usr/local/bin`:

```
make pkg                        # -> target/release/osx/Zterminal-<version>.pkg
```

It builds a release binary, assembles and ad-hoc-signs the app bundle (stamping the version from `zterminal/Cargo.toml`), and packages both targets. The package is unsigned, so the first launch needs a right-click → Open. `make app` and `make dmg` build just the bundle or a disk image; an already-built binary can be packaged with `ZTERMINAL_BINARY=target/debug/zterminal extra/osx/make-pkg.sh`.

After the build

Install the terminfo entry from `extra/zterminal.info` so `TERM=zterminal` advertises the real capabilities (otherwise zterminal falls back to `xterm-256color`); install the man pages from `extra/man/`; install the shell completions from `completions/`. None is required to run the binary, but each closes a seam between zterminal and the surrounding system.

Vendored, self-contained, and durable

The build is vendored on purpose, and the purpose is durability. “No external runtime dependencies” means the shipped binary carries what it needs rather than reaching out to fetch it at run time, so there is no version of a

helper program to drift, no download to fail on a fresh machine, no curl-pipe-bash step to rot. The compile-time dependencies that remain are the *real* platform libraries the renderer and font stack genuinely use — OpenGL for drawing, fontconfig for face resolution on Linux/BSD — which is why the Debian dependency line names `libfontconfig1-dev` and the XCB and xkbcommon development headers: these are not conveniences the build could vendor away, they are the system's graphics and font services the terminal must bind to. The discipline is the stack's general one — a `cargo build` on a fresh machine should work years from now with whatever toolchain exists then — and it shows up here as a build that pins the stable toolchain, depends only on foundational system libraries, and ships everything else inside the one binary.

What “installing” actually places

It is worth separating the binary from its satellites, because only the first is required. The binary is the terminal; `cargo install` or `make pkg` places it (the macOS package additionally drops `zterminal-icat` into `/usr/local/bin`, Chapter 26). Everything else — the terminfo entry (Chapter 40), the man pages, the shell completions, the shell-integration snippets (Chapter 27) — is a *satellite* that closes a seam with the surrounding system but is not needed to run. This separation is why the book can say each of them is optional without hedging: a fresh binary runs and falls back gracefully (identifying as `xterm-256color`, sourcing no integration), and each satellite you install upgrades one interaction — capability advertisement, documentation, completion, prompt marks — from “works by fallback” to “works fully.” Install the binary to have a terminal; install the satellites to have it fully integrated.

Chapter 42 — Migrating an Existing Setup

Chapter 1 itemized the assembly tax: the multiplexer, the image tool, the shell- integration snippet, the restore plugin, the dropdown daemon, the trigger tool, and the glue holding them together. Every chapter since has documented the zterminal feature that occupies each of those slots. This chapter is the practical inverse — not “what does zterminal have” but “what do I do with what I already have” — and it is written for someone with a working, tuned environment who is correctly unwilling to throw it away on a promise.

The governing principle is that nothing here has to happen at once, and the order below is deliberately incremental: each step is independently useful, and each one leaves you with a working terminal if you stop there.

What replaces what, and how completely

The honest mapping is not one-to-one, and the differences are where the useful information is.

What you have	What takes over	How complete the replacement is
tmux for local splits	native split tree (Chapter 24)	full, for local work; see Chapter 31
tmux for durable sessions	the tmux client (Chapter 28)	not a replacement — tmux stays
<code>.tmux.conf</code> editing	tmux config tab (Chapter 28)	edits the <i>running server</i> , not the file
<code>kitten icat</code> , <code>imgcat</code> , sixel viewers	inline graphics (Chapter 26)	full; the tools still work too
iTerm2/FinalTerm integration plugin	the OSC 133 reader (Chapter 27)	the terminal half; you still source a snippet
tmux-resurrect / tmux-continuum	snapshots over the wire (Chapter 28)	full, with the same honest limit (Chapter 29)
a Quake-style dropdown daemon	the built-in dropdown (Chapter 25)	full
an output-trigger tool	the Triggers tab (Chapter 32)	full; prior art acknowledged (Chapter 34)
a directory-jumping helper	Recent Dirs (Chapter 32)	partial — a harvested list, not a ranker
several config files	one TOML plus managed JSON (Appendix E)	full for the above; your shell config is untouched

Three rows in that table are deliberately not “full,” and they are the three most important things to understand before you start.

tmux does not go away. Chapter 31 made the argument at length: native panes cannot outlive the process that owns their PTYs, so anything durable or remote stays in tmux. What changes is that tmux stops being your interface and becomes a service your terminal is a first-class client of. You will likely use it for fewer things and edit it far more conveniently.

The tmux tabs edit the server, not your file. This is the consequence of the wire-protocol design (Chapter 28) and it surprises people. A change made in the tmux config tab is a `set-option` against the running server: it takes effect immediately and it does not rewrite `~/.tmux.conf`. Restart the server and the file wins again. That is the correct behaviour for a live editor — it is editing state, not source — but it means the tab is for experimentation and for session-scoped adjustment, and durable changes belong either in your file or in a profile (Chapter 21), which is precisely the mechanism zterminal offers for capturing server state and restoring it.

Shell integration still needs a shell-side snippet. What zterminal replaces is the terminal-specific *plugin* half — the part that was iTerm2's or the multiplexer's. The marks themselves have to come from your shell, so you still source a line from `extra/shell-integration/` (Chapter 27). The improvement is that the marks are zero-width and terminal-agnostic, so unlike the setups they replace, the line needs no per-terminal guard and does no harm elsewhere.

The incremental path

Seven steps, each independently useful.

1. Install and run it as a plain terminal. No configuration at all. zterminal runs with defaults and seeds `~/.zterminal/zterminal.toml` with an auto-detected Nerd Font on first launch (Chapter 8). Use it for a few days as nothing more than a fast emulator with your existing tmux inside it. Everything you had continues to work, because a terminal that runs your shell is a terminal that runs your setup.

2. Install the terminfo entry. From `extra/zterminal.info` (Chapters 40, 41). This is the highest ratio of benefit to effort in the list: it closes the gap between what the terminal can do and what applications believe it can. Do the SSH side too if you have hosts you use daily — `infocmp zterminal | ssh host tic -x -` per host — or set `TERM=xterm-256color` for hosts you will not revisit and accept the common-denominator set. Appendix G's first entry is the symptom you are preventing.

3. Source the shell-integration snippet. One line in your shell rc (Chapter 27). This lights up prompt jumping, output selection, the failed-command gutter marks, and the command history the Dashboard and the `⌘R` palette read — four features from one line, and the marks are inert in every other terminal, so the line is safe in a shared dotfile.

4. Turn on tmux passthrough. `set -g allow-passthrough on`, having read Chapter 39 on what you are enabling. This is what makes images and OSC 133 marks survive a multiplexer pane. It is a genuine security decision — you are allowing programs inside panes to speak escape sequences past tmux's sanitization — and the book's position is to explain it rather than set it for you.

5. Try native splits for local work. Do not migrate your tmux sessions. Just start using `⌘D` and `⌘⇧D` (Chapter 24) for the throwaway local work you would otherwise have made a tmux window for: a build beside an editor, a log beside a service. The differences you will notice are the ones Chapter 31 predicts — images and prompt marks work without passthrough, one scrollbar with one keymap, and the panes are gone when you quit. Save a layout you like with `⌘⇧L`.

6. Move persistence to snapshots. If you run `tmux-resurrect` or `tmux-continuum`, `^⌘5` (Chapter 28) does the same job natively over the wire, with the same honest limitation about live process state (Chapter 29) that every such tool has. The `tmux_restore_processes` whitelist is the resurrect model with the `name->replacement` refinement, edited in the Sessions panel rather than a config file. Mark a snapshot **Set startup** and your environment comes back at launch. This is the step where you can retire a plugin outright.

7. Capture the result as a profile. Once the terminal config, the panel look, and the tmux server state are all where you want them, a profile (Chapter 21) snapshots the whole set so you can switch environments — a work profile and a writing profile, or a presentation profile with a larger font and a plainer theme — in one click, tmux server included.

Coming from Alacritty

Your configuration is nearly portable, and that is not a coincidence: the table structure and the field names are the inherited schema of Chapter 2. `[window]`, `[font]`, `[colors]`, `[cursor]`, `[keyboard]`, `[hints]`, and the rest carry the same fields with the same meanings and the same defaults, down to `option_as_alt` and `draw_bold_text_with_bright_colors`. Copy your file to `~/.zterminal/zterminal.toml` and run `zterminal migrate` (Chapter 8) with `--dry-run` first to see what the schema upgrade would change.

What is new to you is the extension layer and exactly three configuration keys: `window.background_image`, `window.background_image_opacity`, and `window.overlay_opacity` (Appendix B). Everything else the extension layer stores lives outside the TOML as managed JSON under `~/.zterminal` (Appendix E), which is why your config file will look familiar even after months of use — the profiles, layouts, snapshots, triggers, and snippets are not schema, they are state.

What you gain is the second half of this book. What you do not lose is anything from the first half: the escape-sequence support table, the vi mode, search, hints, and the binding model are the same behaviours documented at the same level of precision.

Coming from iTerm2

Three of your habits map, with a caveat each.

Triggers map directly to the Triggers tab (Chapter 32), and the book is candid that iTerm2's feature is direct prior art rather than something zterminal invented (Chapter 34). The framing differs — zterminal's are per-pane in its own pane model — but if you know iTerm2 triggers you know these.

Shell integration maps, but the mechanism is the standard OSC 133 convention (Chapter 27) rather than a vendor integration, so the snippet you source is zterminal's and the marks are portable.

Profiles do not mean the same thing, and this is the caveat worth internalizing. An iTerm2 profile is broadly a per-session appearance and startup configuration — which font, which colours, which command. A zterminal profile (Chapter 21) is a whole-*environment* snapshot: the entire terminal config, the GUI look, and the tmux server's options, buffers, and keybindings, restored together. It is a coarser, more powerful object, and it is switched globally rather than assigned per-session. If you used iTerm2 profiles to give different windows different looks, the closer analogue is a per-window `zt msg config` override (Chapter 33) or an imported theme file (Chapter 12); if you used them to snapshot whole working setups, zterminal profiles do considerably more than yours did.

Coming from kitty

The graphics protocol is the good news: zterminal decodes the Kitty protocol including transmit, put-by-id, the support query, chunked payloads, compression, z-ordering, Unicode placeholders, and animation (Chapter 26), so `kitten icat` and anything else speaking that protocol works. The bundled `zterminal-icat` is worth adopting anyway for the tmux case, since it enables passthrough itself and places via placeholders that survive a redraw.

The keyboard protocol is the other good news: the `CSI u` family is implemented (Chapters 6, 7, 40), so an editor that negotiates unambiguous keys gets them.

The honest bad news is ligatures. Chapter 11 states the position plainly and the reasoning is architectural rather than a to-do item: a per-cell grid renderer in this lineage does not fuse `!=` into a single glyph, because a ligature spans cells that must remain individually addressable, selectable, and overwriteable. If programming ligatures are load-bearing for you, this is a real and permanent difference, and the book would rather say so than let you discover it on day two.

What does not migrate, from anywhere

Three things, stated once so no step above implies otherwise.

Live process state. No restore path in any tool brings back a running program's memory — the unsaved buffer, the pager position, the open connection. Chapter 29 gives the reason in full. Restores rebuild *arrangements* and respawn processes; the replant-versus-relaunch choice (Chapter 29) is how you decide what that respawn does.

Ligature rendering, per the section above.

Your multiplexer's config file as an editing surface. The tmux tabs edit a running server (Chapter 28). Your `.tmux.conf` remains the thing that configures a freshly-started server, and profiles are the zterminal-side mechanism for making a server's state reproducible.

Everything else in the assembly-tax list of Chapter 1 has a home in this book, and the migration is finished when the list is shorter than it was.

Chapter 43 – The Line Discipline

Chapter 35 introduced the pseudo-terminal in a paragraph and named the line discipline in passing: the kernel layer that turns a `Ctrl+C` byte into a `SIGINT`. That paragraph was enough to explain pane teardown; it is nowhere near enough to explain the class of behaviour it governs. The line discipline decides whether your keystrokes reach a program one at a time or a line at a time, who echoes them back, which control characters become signals, which become flow control, and what happens to a background job that tries to read. It is not part of `zterminal`, and that is exactly why it deserves a chapter: a great deal of what users attribute to the terminal emulator is done by a kernel module the emulator never touches, and the two are debugged with entirely different tools.

Two ends and a filter

The PTY is a kernel object with a primary end that `zterminal` holds and a secondary end the shell has open as `stdin`, `stdout`, and `stderr` (Chapter 35). The line discipline sits on the secondary end, and it is a bidirectional filter. Bytes `zterminal` writes into the primary end pass through the discipline's *input* processing before a program's read can see them. Bytes a program writes pass through the discipline's *output* processing before they emerge at the primary end for `zterminal` to parse. Neither direction is a straight pipe, and every transformation is configurable per PTY through the `terminal-attributes` structure.

That structure is four flag words and an array of control characters. The input flags govern how arriving bytes are treated, the output flags govern how departing bytes are treated, the control flags govern the largely vestigial serial-line parameters, and the local flags govern the discipline's own behaviour — canonical mode, echo, signal generation. The control-character array maps individual bytes to the discipline's special functions: which byte is interrupt, which is suspend, which erases a character, which erases a word. Everything in the rest of this chapter is one of those settings.

Canonical mode, and why a shell turns it off

By default the discipline is *canonical*: it buffers bytes into a line, handles editing within that line itself, and delivers nothing to the reading program until a line terminator arrives. In this mode the erase character deletes the previous byte from the kernel's buffer, the word-erase character deletes back to a word boundary, and the kill character discards the whole pending line — all inside the kernel, invisible to the program, which eventually gets a finished line in one read. This is why a bare `cat` with no arguments lets you backspace over a typo: `cat` is not implementing line editing, the discipline is.

Any program that wants to see each keystroke as it happens must turn canonical mode off. Every interactive shell does — `zshrs` and its line editor, and every full-screen application — because line editing with history, completion, and syntax highlighting cannot be expressed as “erase one byte.” The consequence, and it is a large one, is that in the common case the discipline's line-editing machinery is switched off entirely and the shell is doing the work. The special characters that survive the switch are the ones the shell chooses to leave enabled, which is why `Ctrl+C` still interrupts at a `zshrs` prompt but the erase character no longer means anything to the kernel.

Who echoes

Echo is the second local flag and the second thing programs turn off. With echo on, the discipline writes each arriving byte straight back out, so it travels to zterminal and appears on screen without the program doing anything. With echo off, nothing appears unless the program prints it. Both arrangements are common and the difference explains a symptom every user has seen.

A password prompt shows nothing as you type not because the terminal is hiding the characters and not because the program is printing asterisks it then erases, but because the program cleared the echo flag before reading and set it again afterwards. Nothing was ever sent to the terminal to be displayed. This is also why a program that dies between those two steps leaves you at a shell prompt that appears not to accept input: echo is still off, your keystrokes are still reaching the shell, and typing the settings-restore command blind will fix it. The terminal is not broken and never was; a kernel flag on the PTY is in the wrong state.

The corollary for a shell in non-canonical mode is that the shell echoes what you type *itself* — which is precisely why it can colour it, re-render it after a completion, and redraw the whole line when you edit its middle. What you see at a `zshrs` prompt is not your keystrokes bouncing off the kernel; it is the shell's own rendering of its own buffer, sent as ordinary output through the path of Chapters 3 through 5.

Signals, and the foreground process group

The discipline's signal-generation flag is what turns three control characters into signals: interrupt, quit, and suspend. The crucial detail is the target. These signals do not go to “the program you were typing at”; they go to the PTY's *foreground process group*, a property of the terminal object itself. A session has a controlling terminal, the terminal has one foreground process group at a time, and the shell moves that designation around as it starts and stops jobs. When you run a pipeline, the shell puts every process in it into one new process group and makes that group the foreground; an interrupt then reaches the whole pipeline simultaneously, which is why interrupting a three-stage pipeline does not leave the first stage running.

The same mechanism governs what background processes may do. A process in a background group that tries to *read* from the terminal is stopped with a job-control signal rather than being allowed to steal input — there is exactly one foreground group and it owns the keyboard. Whether a background *write* is treated the same way is a flag: leave it clear and background output interleaves freely with whatever else is on screen, set it and background writers are stopped too. The default is to allow the writes, which is the direct cause of the familiar mess where a background job prints over your half-typed command line. Chapter 49 returns to that as an example of a problem the terminal cannot solve, because by the time the bytes arrive at the primary end there is no record of which process sent them.

Flow control, and the terminal that froze

Two more control characters are, in many default configurations, stop and start: one tells the discipline to suspend output, the other resumes it. This is software flow control, a survival from real serial lines, and it is still enabled by default in many places. Press the stop character by accident — it is adjacent to several common editor bindings — and the terminal appears to lock: your keystrokes are accepted, the program keeps running, and nothing is drawn, because the discipline is holding the output. Press the start character and everything that accumulated arrives at once.

This is a classic “my terminal hung” report that has nothing to do with the terminal. zterminal is not involved: it is sitting in its event loop (Chapter 36) with no bytes arriving to read, which is indistinguishable from a program that has simply gone quiet. The fix is a keystroke; the prevention is to disable the input flag that enables the feature, which costs nothing on a PTY because there is no slow serial line to protect.

Newline translation, and the raw-mode carriage return

The two most consequential translation flags concern line endings. On input, the carriage return your Enter key produces is by default translated to a line feed before the program sees it, which is why programs read a line feed from a terminal even though the terminal sent a carriage return. On output, a lone line feed is by default expanded to carriage-return-plus-line-feed, which is why printing a string with an embedded newline puts the second half at the start of the next row rather than under the end of the first half.

Both are off in raw mode, and both absences are a rite of passage for anyone writing a TUI. A program that clears the output translation and then prints a line feed gets a cursor that moves down one row without returning to column zero — the classic staircase — and must emit both characters explicitly. A program that clears the input translation and waits for a line feed on Enter waits forever, because what arrives is a carriage return. Neither is a terminal-emulator behaviour; zterminal transmits the byte the key produces (Chapter 7) and displays the cursor movements the grid receives (Chapter 3), and the translation between them belongs entirely to the kernel.

Window size, and the resize handshake

The PTY carries a window size — rows, columns, and optionally a pixel width and height — as part of its state, and this is the channel through which the grid's dimensions reach the program. The handshake has a fixed order and both halves matter. When zterminal's grid dimensions change (Chapter 3), it writes the new size onto the PTY *first*, then the kernel delivers the window-change signal to the foreground process group. A program that handles the signal queries the PTY for the size it will find already updated, and repaints.

The ordering is why a well-written application never repaints at a stale size, and the pixel fields are why an image-capable application can compute cell dimensions in pixels — the inline-graphics protocols of Chapter 26 depend on the program and the terminal agreeing on how many pixels a cell is. A program that caches the size it read at startup and never handles the signal is the reason some tools render at the wrong dimensions after a resize until restarted; nothing in the terminal can fix that, because the terminal has already done both halves of its job.

One discipline per pane

Every native pane has its own PTY (Chapter 24) and therefore its own line discipline with its own flags, its own control characters, and its own foreground process group. Nothing is shared. This is a stronger isolation than users usually assume, and several observed behaviours fall directly out of it.

An interrupt typed in one pane cannot reach a process in another, because the signal is generated by *that pane's* discipline and delivered to *that pane's* foreground group. A program that leaves echo off in one pane leaves the others untouched. A stop character that freezes output freezes one pane. And the cross-session broadcast of Chapter 30 is not a special signal-delivery mechanism at all — it writes the same bytes into several PTY primaries, and each pane's own discipline independently performs its own translation, so a broadcast interrupt character becomes as many independent signals as there are targets, each aimed at its own foreground group. The broadcast feature is doing something much simpler than it appears; the kernel is doing the interesting part, once per pane.

The inspection tools

Three commands cover nearly all of it. Printing the full terminal settings shows every flag and every control character for the current PTY, and reading it is the first diagnostic step whenever input behaves oddly — it will show at a glance whether echo is off, whether canonical mode is on, whether flow control is enabled, and which byte is currently bound to interrupt. Restoring a sane set of settings fixes the crashed-program case

without disturbing anything else. The full terminal reset goes further, re-initializing the terminal itself as well as the discipline, which is what you want when the *grid* is also in a strange state — the wrong character set selected, the cursor hidden, a mode left on — a situation Chapter 44 takes apart in detail.

The division of labour is worth stating once, plainly, because it is the chapter's whole point. The kernel owns line buffering, echo, signal generation, flow control, newline translation, and the window size. The emulator owns key-to-byte encoding (Chapter 7), escape-sequence interpretation (Chapter 6), the grid (Chapter 3), and the pixels (Chapter 4). The application owns everything above both. Nearly every confusing input symptom belongs to exactly one of those three layers, and knowing which one collapses the search space immediately.

Chapter 44 — Encoding, Malformed Bytes, and the C1 Range

The parser of Chapter 6 consumes bytes; the grid of Chapter 3 stores characters. Between the two sits a decoder, and the decoder deserves its own treatment because it is where a terminal meets input it cannot trust. Everything that arrives on the PTY is a byte stream produced by an arbitrary program, and a meaningful fraction of real terminal sessions include at least one moment where that stream is not what the terminal expected — a truncated file, a binary blob, a mis-encoded log, a mixed-encoding remote host. What the terminal does with that input determines whether the session recovers or the user reaches for a reset.

The decoder is incremental, and has to be

UTF-8 encodes a scalar in one to four bytes: the first byte's high bits declare the length, and every subsequent byte of the sequence is a continuation byte with a fixed two-bit prefix. This structure gives the property terminals depend on — any byte can be classified as “starts a sequence” or “continues one” without context — and it is what makes the encoding safely resynchronizable after damage.

The non-obvious consequence for a terminal is that the decoder cannot be a pure function of one read. Chapter 37 established that bytes are read from the PTY in large batches to amortize the syscall; nothing aligns those batches to character boundaries. A three-byte sequence can perfectly well have its first byte at the end of one batch and its remaining two at the start of the next, with an arbitrary delay in between if the writing program paused mid-character. The decoder therefore holds partial state across reads: an incomplete sequence at the end of a batch is retained, not discarded, and completed when the rest arrives. A decoder that reset per read would corrupt one character at every batch boundary under load — a bug that would be invisible on short output and constant on long output, which is the worst possible failure profile.

Malformed input becomes a replacement character

When a byte cannot continue the sequence in progress — a start byte where a continuation was required, a continuation byte with no sequence open, an encoding that is valid in shape but denotes a value the standard forbids — the terminal has no useful option except substitution. It cannot ask for a resend; there is no protocol underneath to request one, and the program on the other end has already moved on. It cannot stall waiting for more; the bytes that would complete a valid sequence may never come. And it must not silently drop, because a silent drop shifts every subsequent column and hides the fault.

So the malformed unit becomes the Unicode replacement character, occupies a cell, and the decoder resynchronizes at the next byte that can legally start a sequence. The visible result is the familiar run of replacement glyphs when a file in a different encoding is dumped to the screen, and the important property is that the run is *bounded*: after the malformed region ends, the very next well-formed sequence decodes correctly, because UTF-8's self-synchronizing structure lets the decoder find the boundary without any external framing. A terminal fed a megabyte of random bytes fills the screen with substitutions and then, the moment real text resumes, resumes with it.

Three specific classes of ill-formed input are worth naming because they are security-relevant rather than merely cosmetic. An *overlong* encoding spells a small scalar using more bytes than necessary; if accepted, it provides a second spelling for characters that filters check for by their canonical spelling. *Surrogate* code

points are reserved for a different encoding form and have no business in UTF-8. And values above the Unicode maximum are simply not characters. All three must be rejected as malformed rather than decoded to whatever the arithmetic produces, and the rejection is a correctness requirement, not a nicety — a decoder that accepts overlong forms is the classic way a validating layer upstream gets bypassed.

The C1 range, and why the seven-bit forms won

The control characters below the space are the C0 set — the tabs, newlines, carriage returns, and the escape character itself. There is a second control range immediately above the ASCII block, called C1, and in the original eight-bit encodings each of its members was a single-byte equivalent of a two-byte escape sequence. The control sequence introducer that Chapter 6 spells as escape followed by a bracket had an eight-bit spelling as one byte in that range, and so did the operating-system-command introducer and the string terminator.

In a UTF-8 stream those spellings are unreachable. Every byte in that range is a continuation byte by definition; it can only ever appear inside a multi-byte character, and a decoder that treated it as a control introducer would be misreading the middle of a character as a command. This is the reason every modern terminal and every modern application uses the seven-bit escape-prefixed forms exclusively, and it is a quiet piece of good news for safety: a binary file dumped to the screen cannot accidentally form an eight-bit control introducer, because those bytes are consumed by the decoder as malformed continuations and turned into replacement characters before any parser sees them.

What a binary dump *can* still do is contain a literal escape byte followed by bytes that happen to form a valid sequence. That is the real hazard, and it is the subject of the rest of this chapter.

The four ways a binary dump wedges a session

“The terminal is broken after I dumped a binary” is not one failure; it is usually one of four, and each has a specific, minimal repair. Knowing which is which turns a full reset into a one-line fix and, more usefully, teaches what state the grid actually carries.

The first is **charset selection**. Chapter 6 records that the charset-designation escapes are implemented for charsets B and O — ASCII and the DEC line-drawing set. A stray designation of the line-drawing set leaves every subsequent lowercase letter rendering as a box-drawing glyph, so your prompt turns into a fence. The grid is fine, the cursor is fine, the colours are fine; one register selecting which character set letters map through is wrong. Re-designating ASCII restores it instantly.

The second is **an unterminated string sequence**. The operating-system-command family (Chapter 6) is introduced by escape followed by a closing bracket and runs until a terminator arrives. A binary blob containing the introducer but no terminator puts the parser into a state where it is accumulating a string, and every subsequent byte — including your typing, echoed back — is swallowed into that string instead of being printed. The symptom is a terminal that accepts input and displays nothing, which looks identical to the flow-control freeze of Chapter 43 and to a program that has hung, and is distinguished from both only by what fixes it. A terminal must cap how much it will accumulate for such a sequence, because an attacker-controlled or merely corrupt stream would otherwise grow the buffer without bound; bounded accumulation is why the wedge is recoverable at all.

The third is **a left-on private mode**. Any of the modes in Chapter 6's private list can be set by a stray sequence. Cursor visibility off leaves you typing at an invisible cursor. Application cursor keys on makes your arrow keys send the alternate encodings of Chapter 7, so they no longer navigate your shell's history. Mouse reporting on makes every click emit gibberish into the command line. Each is a single bit, each has a specific clearing sequence, and each is individually harmless — the confusion comes from several arriving at once.

The fourth is **a scroll region**. A stray region-setting sequence pins scrolling to a band in the middle of the screen, so output appears to be trapped between two invisible lines while the rows outside stay frozen. Chapter 3 explains the mechanism as the feature it is; encountered by accident it reads as a display bug.

The full terminal reset fixes all four because it re-initializes the whole of that state at once, and that is the right first move when you do not care which happened. But the four-way split is worth internalizing, because it names exactly what a terminal's persistent state *is*: a charset register, a parser mode, a set of private-mode bits, and a scroll region — plus the grid and the cursor's pen (Chapter 3). That is the complete list of things a rogue byte stream can leave wrong, and it is short.

Where decoding sits relative to everything else

Two boundaries are worth fixing in mind. Decoding happens *below* the grapheme-cluster layer of Chapter 11: the decoder produces scalars, and the cluster logic then decides which scalars combine into one cell and which advance the cursor. A combining mark is a perfectly valid scalar that the decoder emits normally and the cluster layer then attaches to its neighbour; the two concerns do not overlap.

And decoding happens *above* nothing at all on the input side — it is the first interpretation applied to the raw stream, which is why the pre-parser scanner of Chapters 26 and 27 works on the byte level rather than on decoded text. A graphics payload is base64 and an OSC 133 mark is ASCII, so both can be recognized by their byte prefixes without waiting for character boundaries, and both are peeled out before anything downstream has to decide whether the payload was valid UTF-8. That ordering is not incidental: an image's binary payload would produce a screenful of replacement characters if it reached the decoder, and the scanner exists partly to ensure it never does.

On the outbound side, the encoding question is symmetric and simpler. When you type a non-ASCII character — through a compose sequence, a dead key, or an input method — the platform hands zterminal a string, and the input path of Chapter 7 writes its UTF-8 encoding to the PTY. The program on the other end receives the same bytes any file would have given it. There is no terminal-specific encoding on the way in, which is why an input method that works in any other application works here.

Chapter 45 – Backpressure and the Shape of a Firehose

Chapter 5 explained how the terminal avoids drawing more frames than the display can show, and Chapter 37 explained how it avoids spending its time in syscalls. Both are about the terminal's own efficiency. Neither answers the question a user asks when a build floods the screen and everything slows down: what governs the *rate* at which a program is allowed to produce output, and who pays when it produces more than can be shown? That is a question about backpressure, it involves the four buffers named below, and the answer is not intuitive.

The chain, buffer by buffer

Between a program's print call and a lit pixel there are several places bytes can accumulate, and they have different capacities and different overflow behaviours.

The program's own standard-output buffer is first, and it is the lever most often overlooked thinks about. When output goes to a terminal, the usual convention is line buffering: each newline flushes. When the same output goes to a pipe or a file, it switches to block buffering and flushes only when a several-kilobyte buffer fills. This is the entire explanation for the classic complaint that a program prints promptly on screen but appears to produce nothing when piped into a filter — the bytes exist, they are sitting in the producer's buffer, and the terminal never enters into it.

The PTY's kernel buffer is second, and it is finite and small. It is the only place in the chain where a real blocking boundary exists.

zterminal's read batch is third and is transient by construction: the loop reads what is available and consumes it immediately (Chapter 37), so it is a staging area rather than a queue.

The grid and scrollbar are fourth, and they are the interesting case, because they are not a buffer at all.

The grid is a lossy consumer, on purpose

A pipe preserves every byte written into it. The grid does not, and that difference is the structural fact that governs how terminal output behaves under load. When a program writes a line, the grid is mutated. When it writes a thousand lines faster than a frame interval, the grid is mutated a thousand times and drawn once (Chapter 5). The intermediate states are not queued for later display; they are simply overwritten. The terminal is decimating the stream, not buffering it.

That is the correct behaviour — nobody can read a thousand frames per second, and the states in between were never observable — but it changes how you reason about the pipeline. Output that scrolls is preserved in the scrollbar, so it is recoverable even though it was never drawn. Output that was *overwritten in place* — a progress bar rewriting its own line, a status display repainting a region — is gone the moment it is superseded, because nothing scrolled and so nothing entered the history. This is why you can scroll back through a build log but cannot scroll back through the intermediate percentages of a progress bar: one class of output becomes history and the other class is, by design, ephemeral.

The scrollbar itself is bounded rather than buffered. `scrolling.history` defaults to 10000 lines with a cap

of 100000 (Chapter 3); when the buffer is full the oldest lines are dropped. It is a fixed-size window over recent history, not a queue that anyone waits on.

Where the real backpressure is

Given all that, only one link in the chain can actually slow a producer down: the PTY's kernel buffer. If zterminal stops reading the primary end — because it is busy, because it is stopped, because the machine is loaded — the buffer fills, and the next write by the program on the secondary end blocks until space appears. That is genuine backpressure, propagated by the kernel, and it is the mechanism behind an effect every developer has noticed: a verbose build genuinely runs faster when its output is redirected to a file than when it is displayed.

The reason is now stateable precisely. Redirected to a file, the writes complete at the page cache's speed and the producer never waits. Displayed, every write must be consumed by a process that is also decoding, parsing, mutating a grid, and occasionally drawing a frame — and when that consumer falls behind, the kernel buffer fills and the producer blocks. The terminal is not “slowing the build” by being inefficient; it is slowing the build by being a slower sink than a file, which any screen-updating consumer necessarily is. Redirecting the output, or piping it through a filter that discards most of it, removes the terminal from the critical path entirely. That is the practical takeaway, and it costs nothing to apply.

The user-visible extreme of the same mechanism is the flow-control freeze of Chapter 43. Pressing the stop character tells the line discipline to hold output; the program's writes then block once the buffer fills, and the program is suspended in place — not killed, not signalled, just waiting on a write. Releasing flow control resumes it. Software flow control is backpressure with a keyboard shortcut.

Fairness across panes

A single event loop serving several panes (Chapter 36) has to answer a question a one-pane terminal never faces: if one pane is receiving a firehose, what stops it from starving the others? The structural answer is that the loop's expensive half is already bounded independently of input volume. Drawing happens at the display's cadence regardless of how many bytes arrived (Chapter 5), so a pane producing a million lines a second does not get a million times the draw budget of a quiet pane — it gets the same one frame, into which its accumulated damage is coalesced. The cheap half, reading and parsing, is linear in byte volume and has no per-pane minimum, so a busy pane does consume more CPU than a quiet one, but it does so in bounded batches rather than by holding the loop until its stream ends.

The consequence users can rely on is that a runaway pane degrades the whole window's responsiveness gradually rather than locking the other panes out, and that interrupting the runaway (Chapter 43 — in its own pane, with its own discipline, against its own foreground group) restores things immediately. The consequence they should not expect is that a firehose is free: it is not, and the Dashboard's PTY-throughput gauge (Chapter 22) is the honest instrument for seeing how much volume is actually moving.

The tmux hop

A tmux pane's output takes a longer road, and Chapter 31 has already counted the parse costs. Backpressure has its own version of the same observation. Output from a process in a tmux pane is written to a PTY the server owns, consumed by the server, re-emitted over the client connection, and consumed again by zterminal. That is two consumers and two buffers rather than one of each, and the server is itself maintaining a grid and its own history for the pane.

Two practical differences follow. The producer's backpressure boundary is now the server's read rate, not zterminal's, so a stalled or busy *client* does not immediately block the producer — the server keeps consuming and keeps its own history. And the scrollbar the user scrolls in a tmux pane is the server's, with the server's limit, which is the source of the “scrolling in a tmux pane shows the wrong text” symptom Appendix G addresses.

The extra hop buys process survival (Chapter 35) and costs a second copy of everything; that is the trade Chapter 31 frames, seen from the buffering side.

What to do about it

Three habits cover nearly every case. Redirect output you do not intend to read, and accept that reading it costs producer throughput. Prefer a filter that reduces volume early over one that reduces it late, because the reduction only helps downstream of where it happens. And when a program's output seems to have stopped, check whether it is buffered rather than stalled — the difference between a producer holding a partial block and a consumer refusing to drain is invisible from the screen but total in its implications.

Chapter 46 – Nesting: SSH, tmux, Containers, and Capability Attenuation

Chapter 40 covered how `zterminal` advertises its capabilities and what its terminfo entry contains. That is the single-hop story. The story users actually live is multi-hop: a terminal on a laptop, a connection to a build host, a `tmux` server on that host, a container the shell then enters, and an editor inside all of it. Each layer is a program that both consumes escape sequences and produces them, and every capability the innermost program can use must be supported by every layer between it and the screen. This chapter is about that composition — how capability sets attenuate as you nest, and how to find the layer that lost something.

The composition rule

The rule is simply stated and rarely applied: a capability is available at a given depth only if every layer from the outermost to that depth implements it. Not the outermost, not the innermost, every one. The terminal is a chain of filters, and the weakest link sets the result.

This inverts the usual debugging instinct. When `truecolor` does not work in an editor five layers deep, the reflex is to check the editor's configuration and then the outer terminal's. Both are usually fine; the loss is almost always in the middle, where a multiplexer or a remote host's terminfo entry describes a lesser terminal than the one actually present. The middle layers are the ones nobody configured deliberately, and they are where capability goes to die.

The attenuation is also *asymmetric between directions*. An escape sequence heading outward — the editor asking for `truecolor` — must be understood or at least passed through by every layer. A reply heading inward — the terminal answering a device query — must survive the same chain in reverse, and it must be routed back to the right requester among several possible ones. The reply direction is the fragile one, which is why capability *queries* are less reliable in deep stacks than capability *declarations*, even though queries are the more accurate mechanism when they work.

Who answers a query

This deserves its own statement because it surprises people. When a program emits a query that expects a reply — a device-attributes request, a cursor-position report, a palette query (Appendix I) — the reply comes from the nearest enclosing emulator, not from `zterminal`. If the program is running under `tmux`, `tmux` answers, because `tmux` is a terminal emulator from the program's point of view: it owns the PTY, it maintains a grid, it interprets the sequences. It may consult the outer terminal to form its answer, or it may answer from its own capability model, but the authority at that moment is the multiplexer.

The practical consequence is that a multiplexer's capability model caps everything above it, regardless of what the outer terminal can do. This is not a defect of any particular multiplexer; it is inherent to being a terminal emulator in the middle of a chain. And it is the sharpest technical version of Chapter 31's argument for native panes: a native split (Chapter 24) is not a nested emulator at all. Its PTY is `zterminal`'s, its grid is `zterminal`'s, its query replies are `zterminal`'s. The layer is not made faster, it is removed, and every capability that would have attenuated through it simply does not encounter it.

What attenuates, feature by feature

Working down the feature list of this book, each has a characteristic failure mode when a layer in the middle does not carry it.

Truecolor degrades rather than failing: a layer that only models 256 colours quantizes the 24-bit values, and the result is a scheme that is recognizably right and subtly wrong. Because it degrades quietly, it is the capability most often lost without anyone noticing which hop lost it.

Inline graphics (Chapter 26) fail outright, and they fail in the ugliest way: a layer that does not understand the protocol may pass the payload through as text, filling the screen with base64. This is the “the image tool’s escapes get eaten by the multiplexer” problem Chapter 1 itemizes as part of the assembly tax, and it is why passthrough configuration exists at all.

OSC 52 clipboard access (Chapters 14, 39) fails at whichever layer declines it, and since declining is the *safe* default in several of them, a clipboard write from a remote editor commonly reaches a layer that drops it deliberately. zterminal’s own default is **OnlyCopy**, which permits exactly the direction that is safe; a stack that loses the write is usually losing it further in.

OSC 133 shell integration (Chapter 27) is the interesting case, because the marks are emitted by the *shell*, at the innermost layer, and must travel outward to reach zterminal’s scanner. A middle layer that strips unknown sequences removes the marks and the feature silently does nothing — prompt jumping stops working, and there is no error anywhere, because from the terminal’s perspective the shell simply never sent any marks.

The Kitty keyboard protocol (Chapter 7) attenuates in the inbound direction: the enhanced encodings are produced by zterminal and must survive every hop to reach the program that asked for them. A middle layer that only understands the legacy encodings delivers legacy keys, and the program sees a terminal that agreed to speak a protocol and then did not.

Synchronized output (Chapters 5, 6) is unusual in that losing it is harmless: a layer that ignores the begin and end marks simply draws the intermediate states, so the repaint tears rather than breaking. It is the one capability whose absence degrades to “slightly uglier” instead of “wrong.”

Bracketed paste (Chapter 38) failing is the dangerous one, because the failure mode is that pasted newlines are executed rather than inserted. Every layer must carry it for the guarantee to hold.

Bisecting the stack

The method that follows from all this is straightforward, mechanical, and much faster than reasoning about it. Test the capability at each depth from the outside in, and the first depth at which it fails is the layer that lost it.

```
# depth 1: the terminal itself, no multiplexer, no remote host
printf '\033[38;2;255;100;0mTRUECOLOR\033[0m\n'

# depth 2: the same probe, over the connection, still no multiplexer
ssh host "printf '\033[38;2;255;100;0mTRUECOLOR\033[0m\n'"

# depth 3: the same probe, inside the multiplexer on the far side

# depth 4: the same probe, inside the container inside the multiplexer
```

The value of running it this way is that it converts an argument about configuration into an observation about which hop changed the behaviour. The same procedure works for every capability in the list above; only the probe changes.

Two supporting checks are worth adding when the bisection points at a remote host. First, what the far side believes it is talking to: the terminal type the connection propagated, and whether a terminfo entry for that type exists on the remote system at all. Chapter 40 explains why zterminal falls back to identifying as a

widely-present type when its own entry is not installed — the fallback exists precisely because a name with no entry behind it is worse than a slightly conservative name with a complete one. Second, whether the remote host's terminfo database is simply older than the capability in question, which is a common cause of "everything works locally and nothing works on the server."

Containers, and the database that is not there

A container image built for size usually ships no terminfo database. A shell started inside it inherits a terminal type from the environment, finds no entry describing it, and either falls back to something minimal or errors outright when a curses program starts. The symptom is distinctive: line-oriented tools work perfectly and every full-screen tool refuses to start or renders as if the terminal had no capabilities at all.

There are two repairs and they have different costs. Install a terminfo database in the image, which is correct and adds size. Or export a terminal type that the image's minimal database does describe, accepting a conservative capability set for the duration. The second is the pragmatic choice for a shell you are in for thirty seconds; the first is right for an image people work inside. Both are preferable to the third option people reach for, which is to declare a rich terminal type that the container cannot describe — that produces a program confidently emitting sequences against a capability list it invented, and the failures are far more confusing than a plain fallback.

Latency does not compose the way capabilities do

One closing distinction. Capability attenuates by intersection: the result is what every layer supports. Latency accumulates by addition, and the terms are not the same size.

Chapter 37 established that local echo latency is bounded below by the display's refresh cadence and that the terminal's own per-character work is trivial next to it. Add a network hop and that analysis is superseded: the round trip to a remote host is typically far larger than a frame interval, so the frame cadence stops being the dominant term and the network becomes it. A keystroke must reach the remote shell, be echoed by it (Chapter 43 — the shell echoes, not the kernel, once the shell has left canonical mode), and travel back. No terminal-side optimization touches that, because none of the time is spent on the terminal side.

This is worth knowing mostly so that the right thing gets blamed. A remote session that feels sluggish while a local one feels instant is not a rendering problem, and looking for one wastes time. The layers to examine are the ones between the keypress and the echo, and all of them are outside the terminal.

Chapter 47 — Colour, Contrast, and Legibility

Chapter 12 is a field reference: every key under `[colors]`, its type, and its default. This chapter is the other half — how the three colour models actually relate, what the defaults are doing that is not obvious from reading them, and how to choose values that stay legible under the conditions the rest of the book introduces: transparency, images behind text, selections over arbitrary content, and search matches that must be findable at a glance.

Three models, one cell

A cell's colour can come from three places (Chapter 3), and they are not alternatives at the same level of abstraction — they are a historical accretion, and their interactions are where the surprises live.

The sixteen ANSI colours are *names*, not values. When a program requests “red” it is requesting a palette slot, and what appears is whatever `colors.normal.red` holds. This indirection is the entire mechanism by which a theme works: every well-behaved program addresses colour by name, the user binds the names to values, and the whole ecosystem re-themes at once. It is also why a program that hardcodes an RGB value is mildly antisocial — it has opted out of the user's theme.

The 256-colour palette embeds those sixteen as its first sixteen slots and then adds two structured regions: a colour cube of six levels per channel, and a greyscale ramp. The cube's indexing is arithmetic — a base offset plus a weighted sum of the three channel levels — which is why programs can compute an index for an approximate colour without a lookup table, and why `colors.indexed_colors` (Chapter 12) overrides individual slots rather than redefining the structure. Six levels cubed is two hundred and sixteen; sixteen plus two hundred and sixteen plus a twenty-four-step grey ramp is two hundred and fifty-six, which is where the palette's size comes from and why there is no room for anything else in it.

Truecolor is the third and is not a palette at all: the SGR sequence carries the channel values directly (Chapter 6's worked example). Nothing indirects, so nothing themes. A truecolor value looks identical under every colour scheme, which is sometimes exactly what a syntax highlighter wants and sometimes exactly why a carefully-themed terminal has one tool in it that clashes.

The default black that is the default background

Read Chapter 12's complete theme carefully and one line stands out. `colors.normal.black` is `"#181818"`, and `colors.primary.background` is also `"#181818"`. They are the same value. Text printed in ANSI black on the default background is therefore invisible — not hard to read, not low contrast, but exactly the background colour.

This is not a mistake; it is what the base16 family does deliberately, and understanding why explains a convention you have seen everywhere without noticing. In a sixteen-colour scheme, the first slot of the normal palette is conceptually “the background,” and the first slot of the *bright* palette is the darkest colour that is still meant to be visible. That is why `colors.bright.black` is `"#6b6b6b"` — a mid grey — and why essentially every tool that wants to render de-emphasized text (comments, timestamps, inactive items) reaches for bright black rather than black. The convention exists because the alternative slot is reserved for the background.

The practical rules that follow are worth stating explicitly, because they are how you avoid shipping a theme with an invisible colour. Every one of the sixteen palette entries should be checked against `colors.primary.background` for adequate contrast, with `colors.normal.black` understood as the deliberate exception. If you invert the scheme — a light background — then `colors.normal.white` becomes the entry at risk, for exactly the same structural reason, and the theme needs `colors.normal.black` to be genuinely dark rather than inherited from a dark-background scheme. Themes that were built for one polarity and flipped by swapping foreground and background are the usual source of a light theme with one unreadable colour in it.

Bold, bright, and the field that conflates them

`draw_bold_text_with_bright_colors` defaults to `false` (Chapter 12), and the default is worth defending because the alternative was standard for decades.

Historically, hardware could not render a heavier weight, so the bold attribute was implemented as “use the brighter version of the current colour.” Software terminals inherited the convention, and a great many colour schemes were designed under it — which means, in those schemes, the eight bright colours are only reachable *via* bold and the bright palette is effectively the bold palette.

Turning the conflation off, as `zterminal` does by default, separates the two axes: bold selects a heavier font face (Chapter 10 — a real face, not a synthesized one), and the bright colours are reachable on their own through the high-numbered SGR parameters that Chapter 6’s partials table lists as handled. The result is a genuine sixteen-colour palette plus an independent weight axis, which is strictly more expressive. The cost is that a scheme designed around the conflation will look different: text the author expected to be bright red and heavy will be normal red and heavy. If you are porting such a scheme, the fix is to set the field rather than to edit the palette, because editing the palette breaks every program that addresses the bright colours directly.

The dim attribute is the third weight-adjacent axis and behaves differently again. `colors.primary.dim_foreground` and the whole `colors.dim` palette are auto-derived from `colors.normal` when unset (Chapter 12), so a theme that supplies sixteen colours gets a dimmed set for free. Dim is a separate palette rather than an alpha multiplier precisely because alpha would composite against whatever is behind the cell — which, under transparency or over a background image, is not the background colour at all.

Contrast under transparency

`window.opacity` makes the window’s background translucent, and `colors.transparent_background_colors` extends that translucency from the default background to every cell background (Chapters 9, 12). The second is a much bigger change than it reads as, and it interacts badly with several features if applied without thought.

With only `window.opacity` set, a cell that has an explicit background — a selection, a search match, a highlighted region in a TUI — is drawn opaque over the translucent default. Its contrast is exactly what the theme specifies, regardless of what is behind the window. With `colors.transparent_background_colors` on, that same selection is now translucent too, and its effective contrast depends on the user’s desktop wallpaper. A search match that was clearly the focused one against a solid amber becomes an ambiguous tint over whatever happens to be behind it.

The same reasoning applies to the background image of Chapter 26, which composites as a textured quad beneath the text. Text must remain legible against arbitrary pixels, and the terminal cannot help — it has no mechanism for adaptive contrast, because adapting would mean changing the colour the application asked for. The practical rule is that a background image intended to be lived with should be low-contrast and low-detail, and that heavy transparency and a busy image are the two settings most likely to make a beautiful screenshot and an unusable terminal.

Absolute colours and relative ones

There is a genuine design distinction visible in Chapter 12's defaults that repays being made explicit, because it tells you which kind of value to use when you write your own.

`colors.selection` defaults to the cell-relative references — text takes the cell's background, background takes the cell's foreground. A selection therefore *inverts* whatever it covers, and inversion is contrast-preserving by construction: if the content was legible before, the inverted version is legible after, whatever colours the content happened to use. A selection has to work over arbitrary content, so it needs a relative rule.

`colors.search.matches` and `colors.search.focused_match` default to fixed hex pairs, and so do the two hint colours. These are the opposite requirement. A search match must be *findable* — the eye should locate it by scanning, without reading — and findability requires a colour that is the same everywhere and distinct from everything, which a relative rule cannot provide. Two matches inverted against different backgrounds would look like two different things.

So the rule for writing your own values: use the cell references when the highlight must stay legible over unknown content, and use absolute colours when the highlight must be recognizable at a glance. Inversion has one failure mode worth knowing — when a cell's foreground and background are already close together, as they are for dim text, inverting them produces something equally low-contrast. That is the case where a selection over de-emphasized text can be hard to see, and it is a property of inversion rather than a bug in the implementation.

Keeping a theme portable

Because `[colors]` is large and a theme is a self-contained unit, Chapter 8's import mechanism is the right home for it: one file per scheme, pulled in by `general.import`, swapped by editing one line. Two habits make such a file portable across the rest of this book's features.

Supply the normal and bright entries and omit `colors.dim`, letting the derivation do its work — a hand-written dim palette is three times the maintenance for a set of colours users rarely see directly. And specify the search, hint, and footer pairs explicitly rather than relying on defaults, because those are the elements that must remain visible when the rest of the scheme changes, and a theme that omits them inherits a set of pairs tuned for the base16 default rather than for the scheme you actually wrote.

Chapter 48 — Writing a Program That Behaves Well Here

Most of this book explains the terminal to its user. This chapter explains it to its other audience: the person writing a program that will run inside it. Everything here follows from material already covered — the escape support of Chapter 6, the input path of Chapter 7, the line discipline of Chapter 43, the encoding rules of Chapter 44, the nesting behaviour of Chapter 46 — assembled into the form an application author needs. A program that follows these rules behaves identically in a native pane, under a multiplexer, and across a network hop, which is the only definition of terminal compatibility that means anything.

Detect capabilities in the right order

There are three ways to learn what a terminal can do and they should be tried in a specific order.

The terminfo database is first, because it is the mechanism every layer in a nested stack understands and because it is what the middle layers of Chapter 46 actually consult. It is conservative and occasionally stale, and both of those are acceptable failure modes — a program that under-uses a capable terminal is merely plain.

Runtime queries are second, and they are more accurate when they work: asking the terminal what it is gets an answer from the actual emulator rather than from a database that may be a decade old. The cost is that queries need a reply, replies traverse the chain in the fragile direction (Chapter 46), and a query with no reply must time out rather than hang. Every query needs a deadline and a fallback, and programs that omit the deadline are the ones that hang for several seconds on startup in an unusual environment.

Sniffing environment variables is last and should be treated as a hint, not a fact. A terminal-identifying variable tells you what program *launched* the chain, not what is at the other end of it, and in any nested stack those differ. Chapter 46's whole argument is that the outermost layer's identity does not determine the available capability set.

The fallback ladder should be explicit and should degrade in defined steps: truecolor if confirmed, 256 colours if confirmed, sixteen otherwise, and none at all when the output is not a terminal. That last case is the one most often forgotten, and it is why so many tools emit escape sequences into log files.

Restore what you change

Chapter 6's worked example of a full-screen application's setup is a list of state changes, and every one of them has a matching teardown. The discipline the example demonstrates is the whole of good citizenship: a program owns the terminal's state only for as long as it runs, and must hand it back exactly as received.

The failure to internalize is that *normal exit is the easy case*. A program that restores on its ordinary exit path but not on a signal leaves the terminal wedged whenever the user interrupts it, which for a long-running tool is most of the time. The correct arrangement is a single restore routine invoked from the exit path, from the handlers for the termination signals, and — where the language provides it — from whatever runs during unwinding on a panic or an uncaught exception. The restore should be idempotent, because it will sometimes run twice, and it should not allocate, because it will sometimes run from a signal context.

The state to restore is precisely the four-item list Chapter 44 arrives at: the charset register, any parser mode entered, the private-mode bits set, and the scroll region — plus the alternate screen, if entered, and the cursor's visibility and shape. That is the complete set. A program that restores those things cannot leave a terminal in a state requiring a reset.

Handle resize as a flag, not as work

The window-change signal arrives asynchronously and the handshake is the one Chapter 43 describes: the size is already updated on the PTY when the signal fires. Two rules follow.

Do not cache the size. Query it when you need it, or at minimum re-query on every signal, because a cached size is wrong from the first resize onward, and the class of bug it produces — a program that renders correctly until the window changes and then never again until restarted — is common enough to be recognizable on sight.

Do not repaint inside the signal handler. Set a flag and repaint from the main loop. A signal handler that calls into a rendering path is calling functions that are not safe to invoke from a signal context, and the resulting corruption is intermittent and extremely difficult to attribute. The handler should do the smallest possible thing.

The pixel dimensions in the size structure deserve a mention for anyone using the graphics protocols of Chapter 26: they are how a program computes the pixel extent of a cell, and therefore how it sizes an image to occupy a known number of columns and rows. A program that assumes a cell size will place images incorrectly at any font size but its author's.

Batch a repaint, and leave the history alone

Synchronized output (Chapters 5, 6) exists for exactly the case a full-screen program faces constantly: a repaint that takes many sequences and looks wrong in the middle. Wrap such a repaint in the begin and end marks and it lands atomically. The failure mode to avoid is nesting — begin and end are not a counted pair, so a nested region ends the outer one early, and a program that wraps both its whole frame and its individual widgets will hold frames for the wrong intervals. Wrap at one level, the outermost one that corresponds to a coherent visual state.

The other repaint discipline is about what a program must *not* touch. The scrollbar is the user's, and the erase-in-display sequence has a form that clears it (Chapter 3). A program that clears the scrollbar on exit — a surprisingly common habit, usually inherited from a snippet — destroys history the user may have been relying on, and destroys it invisibly, because the loss is only noticed later when they scroll up and find nothing. Clear the screen if you must; do not clear the history. The same applies to the scroll region: set it if you need it, clear it when you are done, and do not assume it starts at the full screen if you have already been running.

Mouse, paste, and the user's escape hatches

Enable the specific mouse tracking mode you need and the SGR reporting form, not every mode available (Chapter 6's private-mode list, Chapter 14's discussion). The older reporting encodings cannot express coordinates beyond a limited range and produce nonsense on a large window; the SGR form does not have that limit. Disable tracking on exit, without exception — a program that leaves mouse reporting on turns every subsequent click into garbage on the user's command line, and the user will blame the terminal.

Respect the fact that `Shift` is the user's override (Chapter 14). When a program captures the mouse, holding `Shift` suppresses reporting so the terminal's own selection works. A program cannot and should not try to defeat this; it is the escape hatch that makes mouse capture tolerable.

If the program reads a line of input, decide deliberately about bracketed paste (Chapter 38). Enabling it means pasted content arrives wrapped in markers the program must strip, and in exchange the program knows the

content was pasted rather than typed — which is the whole point, because pasted text containing a newline must not be treated as a submitted line. A program that enables bracketed paste and does not handle the markers is worse off than one that never enabled it, since the markers then appear as literal garbage in the input.

Measure width the way the grid does

Chapter 3 explains the width rules and Chapter 11 explains grapheme clusters. An application that lays out columns must apply the same rules, because the terminal certainly will. Counting code points is wrong — a combining mark advances the cursor by nothing. Counting bytes is more wrong. The requirement is to compute display width using the same East-Asian-width classification the grid uses, and to treat a cluster as one unit.

The consequence of getting it wrong is not local. A single mis-measured wide character shifts every subsequent cell on the line, so the program's model of where things are diverges from the terminal's, and every later positioning sequence lands one column off. Table borders develop a stair step; a status bar's right-aligned segment drifts. The bug reproduces only with content containing wide or combining characters, which is why it survives testing on ASCII and appears the first time a user has an emoji in a branch name.

Do not rely on the terminal to buffer

Chapter 45's analysis has a direct implication for output-heavy programs. The terminal is a lossy consumer and the only real backpressure is the PTY buffer, so a program that emits enormous volumes of output is not filling a queue that will drain — it is spending its own time blocked on writes while the terminal decimates most of what it produced. Output that is regenerated faster than a frame interval will mostly never be seen by anyone.

The design conclusion is to rate-limit intentional high-frequency output at the source. A progress display that redraws a hundred times a second is doing most of those redraws invisibly, at the cost of real producer throughput. Redrawing on a schedule the eye can follow costs nothing and is indistinguishable to the user.

The related discipline is flushing. Standard output is line buffered to a terminal and block buffered to a pipe (Chapter 45), so a program that is interactive when run directly may appear dead when piped. If a program is meant to be usable in a pipeline — and most are — it should flush at meaningful boundaries rather than relying on the buffering mode it happens to get.

Emit text the surrounding features can use

Two of zterminal's features read the program's output as data, and a program can make itself more useful to them at no cost.

Hints (Chapter 15) match text on screen with regular expressions and turn matches into targets. Paths and URLs printed in a single unbroken run are matched; ones the program has wrapped across a line break, or decorated with adjacent punctuation that the semantic rules pull in, are not. Printing a URL alone on its line is the most reliable way to make it clickable, and costs one newline.

Shell integration (Chapter 27) is the other, and it applies specifically to programs that act as a shell or wrap one. The marks are the mechanism by which the terminal learns where a prompt ends and output begins, which is what makes prompt jumping, output selection, and the command-history record possible. Chapter 27 covers what the snippet must guarantee; the point here is only that this is *cooperation*, not inference — a terminal cannot detect a prompt, and everything downstream of the marks exists because a program volunteered the information.

Chapter 49 – What a Terminal Cannot Do

Every honest book about a system needs a chapter on its limits, and a terminal's limits are unusually clean to state because they nearly all descend from one decision: the abstraction is a grid of fixed-size character cells. Chapter 3 presented that as the machine's design. This chapter presents it as the machine's ceiling, and walks the specific things that are impossible above it — not hard, not unimplemented, but ruled out by the model. The chapter matters commercially as well as intellectually, because several of zterminal's extensions are shaped the way they are precisely to work around items on this list, and that shape only makes sense once the constraint is visible.

Nothing happens between cells

The grid has a cell size derived from the font's metrics (Chapter 4), and every character occupies a whole number of them. There is no sub-cell horizontal position, no proportional advance, no kerning between adjacent cells, and no way to place text at an arbitrary pixel offset. A terminal cannot render proportional text as proportional text; it can only place proportional glyphs on a monospace lattice, which is why every attempt looks wrong.

This is also the deep reason ligatures are a difficult question rather than a feature toggle (Chapter 11 states the position). A ligature is a single glyph spanning what the grid regards as several independent cells, each of which can be independently addressed, coloured, selected, or overwritten by the next escape sequence. The grid has no representation for “these three cells are one glyph,” and any implementation must decide what happens when the middle one is repainted.

The inline graphics of Chapter 26 are the visible exception, and examining why they are an exception confirms the rule. An image is not text placed sub-cell; it is a textured quad drawn over a region of cells in the compositing pass, with its own z-index. It does not participate in the grid — it cannot be selected as text, it does not reflow with the paragraph it sits in, and its lifecycle is managed separately (Chapter 26). Graphics work by being *outside* the cell model rather than by extending it, which is why every terminal graphics protocol has a placement and lifecycle story to tell.

The terminal cannot see structure

The byte stream carries characters and cursor movements. It does not carry meaning. The terminal has no way to know that a region of the screen is a table, that a line is a prompt, that a run of output belongs to one command, or that a number is a file size. It sees cells being written.

Every feature that appears to understand structure therefore depends on cooperation, and this is a clarifying observation about zterminal's extension layer. OSC 133 (Chapter 27) exists because the terminal cannot detect a prompt — the shell must say where the prompt ends and the output begins, and everything downstream (prompt jumping, output selection, the command history the Dashboard renders, failed-command gutter marks) is built on that volunteered information rather than on inference. The tmux client (Chapter 28) exists because screen-scraping a multiplexer's rendered output is guessing, while the wire protocol is being told. The hints of Chapter 15 are the honest version of inference — regular expressions over visible text, with all the imprecision that implies, which is exactly why they are configurable rather than automatic.

A terminal that tried to infer structure from the byte stream would be building a heuristic on top of a lossy rendering of information the producing program had in structured form and threw away. Asking the producer to keep a little of it is cheaper and correct, and that is what shell integration is.

Overwritten is gone

The grid is a destructive state machine. Writing a cell replaces what was there; there is no undo, no journal, and no history of a cell's previous contents. The scrollbar (Chapter 3) is not an exception — it holds lines that *scrolled off the top*, which is a different thing entirely from lines that were overwritten in place.

This produces the asymmetry Chapter 45 describes from the buffering side and users encounter constantly: a build log is fully recoverable by scrolling, and a progress bar's intermediate states are irrecoverable, because the first scrolled and the second repainted its own line. It also means that a full-screen application's earlier screens are gone the moment it repaints, which is why scrolling back during a full-screen program shows you nothing useful — the alternate screen does not scroll (Chapter 3), so there is nothing behind it.

The consequence for recovery features is that they cannot be complete. Session restore (Chapter 29) replays what the scrollbar holds; it cannot replay what was overwritten, because nothing anywhere retained it.

Restoration is approximation

Chapter 29 states its one limitation once and this chapter is the right place to generalize it. A terminal can restore the *terminal's* state — the grid, the scrollbar it retained, the layout, the working directories, the command lines it captured. It cannot restore a process's internal state, because that state lives in a process that no longer exists.

Relaunching a command re-runs it from the beginning. Replanting a shell gives you a new shell in the right directory. Neither returns you to the middle of an editing session with unsaved changes, a debugger stopped at a breakpoint, or a paused download. The restore is faithful about what it can see and silent about what it cannot, and the honest framing — the one Chapter 29 uses — is that this is a limitation of the model rather than of the implementation. No terminal can do better without the cooperating program serializing its own state, which is the same cooperation pattern as OSC 133, applied to a much harder problem.

Two writers, one stream

Anything holding the PTY's secondary end can write to it, and the terminal receives one interleaved stream with no attribution. A background job that prints while you are typing produces output interleaved with your shell's rendering of your command line, and the terminal cannot separate them because by the time the bytes arrive there is no record of which process produced which.

This is not fixable at the terminal layer, and the workaround lives at the layer that *does* have the information: the line discipline's flag for stopping background writers (Chapter 43), which prevents the interleaving by preventing the write. That the only available remedy is a kernel flag rather than a terminal feature is a clear demonstration of where the boundary lies.

The same limitation explains why the terminal cannot attribute output to commands without help. It has no idea which process wrote a given byte, so the association between output and the command that produced it — which the command history of Chapter 22 displays — comes entirely from the OSC 133 marks that bracket it.

Reflow has a hard edge

Chapter 3 describes reflow as a feature of the primary screen: logical lines are rejoined and re-wrapped when the width changes. The limit is that reflow only knows about wrapping. Content that was laid out by *positioning*

— a table drawn with absolute cursor moves, a status bar with right-aligned segments, an ASCII diagram — has no wrap structure to rejoin, so narrowing the window truncates it and widening it leaves it at its old width. The information required to re-lay-out that content was never in the grid; it was in the program that drew it.

And the alternate screen does not reflow at all (Chapter 3), by design: it is the application's canvas, and the application is told the new size and repaints. Which means the quality of a resize under a full-screen program is entirely the program's doing. Chapter 48's resize rules are the application-side half of this; the terminal side is complete when the size is set and the signal delivered.

Refusal, not sandboxing

Chapter 39 treats the security surface properly; the limit worth naming here is structural. A program holding the PTY can emit any byte sequence it likes, and the terminal's only defence is to decline to act on the ones it considers dangerous. There is no sandbox, no capability handshake, no way to grant one program the right to set the title and deny it to another — the stream is undifferentiated, so policy can only be expressed per-terminal, not per-writer.

That is why the defences that exist take the shape they take: `terminal.osc52` defaults to permitting writes and not reads (Chapters 14, 39), because reading the user's clipboard is the operation a hostile stream wants and refusing it globally is the only enforcement available. Passthrough is opt-in for the same reason. Each of these is a refusal, not a containment, and refusals are coarse by nature.

The floor beneath latency and the ceiling above correctness

Two final limits, stated briefly because earlier chapters cover their mechanics.

Echo latency cannot go below the display's refresh interval (Chapter 37), because the character has to be drawn in a frame and frames arrive when they arrive. No architecture beats that, and over a network the term stops mattering at all next to the round trip (Chapter 46).

And a terminal cannot make an incorrect application correct. If a program miscounts the width of a wide character, its layout desynchronizes from the grid's and the result is wrong on any conforming terminal (Chapter 3). If it fails to handle the resize signal, it renders at a stale size. If it leaves mouse reporting enabled on exit, the user's next click produces garbage. The terminal's contract is to implement the sequences it documents, at the status it documents (Chapter 6); it cannot repair a program that uses them wrongly, and a terminal that tried — guessing at intent, “fixing” sequences it thought were mistakes — would be unpredictable in exchange for being occasionally forgiving. Fidelity to the documented behaviour is worth more than charity toward broken callers, which is the discipline the whole inherited core of this book's first half is built on.

Why the list is the argument

Read together, these limits explain the shape of everything in the book's second half. The terminal cannot see structure, so the shell tells it (Chapter 27). It cannot usefully screen-scrape a multiplexer, so it speaks the multiplexer's protocol (Chapter 28). It cannot place text sub-cell, so images are composited outside the cell model (Chapter 26). It cannot restore a process, so restoration is scoped honestly to what it can restore (Chapter 29). It cannot attribute a byte to a writer, so per-command features are anchored to volunteered marks rather than inferred.

Every one of those is a design that respects the ceiling rather than pretending to break it. That is the connection to the final chapter: the integration zterminal argues for is not a way around the terminal's limits, it is the recognition that working around them requires several programs to cooperate — and that cooperation is dramatically cheaper when they are one program.

Chapter 50 – Rasterization: Coverage, Gamma, and the Weight of a Stem

Chapter 4 explained where a glyph lives once it has been drawn: a rectangle in the atlas texture, packed with its neighbours, evicted when the atlas is rebuilt. Chapter 10 explained which face the glyph comes from and how the fallback chain is walked. Chapter 11 explained which cluster of code points maps to it. None of the three explained the step in the middle — how a font’s vector outline becomes the pixel values that get uploaded — and that step is responsible for a set of complaints that users almost always misfile as font-choice problems: text that looks too heavy on a dark background, thin strokes that vanish at small sizes, box-drawing characters that are a half-pixel out of alignment with each other, and the persistent sense that the same font “looks different here” than in an editor.

From outline to coverage

A font stores a glyph as a set of closed curves in a design-unit coordinate space. Rasterizing it means scaling those curves to the pixel size implied by `font.size` and the display’s scale factor (Chapter 10), then deciding, for every pixel the curves touch, how much of that pixel lies inside the outline. The answer is a *coverage* value: zero for a pixel entirely outside, one for a pixel entirely inside, and a fraction for the pixels the edge passes through. That fractional coverage is antialiasing. It is not a blur applied after the fact; it is the direct measurement of how much ink the pixel would have received.

The coverage map is what lands in the atlas. When the renderer draws a cell, the shader takes the coverage as an interpolation weight between the cell’s background and foreground colours: full coverage yields pure foreground, zero yields pure background, and the edge pixels yield a mix. This is why a glyph’s colour is not baked into its cached rasterization — one cached image of the letter `e` serves every colour that letter is ever drawn in, which is exactly the amortization Chapter 4 depends on. A colour emoji or a bitmap-strike font breaks that rule and must carry real colour data, which is one reason emoji occupy more atlas area per glyph than the letters around them.

Hinting, and why the argument has moved

Hinting is a font’s own set of instructions for distorting its outlines at small pixel sizes so that stems land on whole-pixel boundaries. Unhinted, a one-unit-wide vertical stem scaled to 1.4 pixels lands wherever the arithmetic puts it and is rendered as two columns of partial coverage — a stem that looks grey and slightly blurred. Hinted, the same stem is snapped to a single fully-covered pixel column and looks crisp. The cost is fidelity: the hinted glyph is no longer the shape the designer drew, letter widths distort unevenly, and at text sizes the accumulated snapping changes the rhythm of a line.

The reason this argument feels less urgent than it once did is the scale factor. On a display reporting 2.0, `font.size` of 11.25 resolves to a pixel size where stems are several device pixels wide, and a fraction of a pixel of error is invisible. Hinting exists to rescue the case where a stem is *about* one pixel, and that case has been receding. The practical consequence for a zterminal user is worth stating plainly: if text looks soft, the productive lever is usually the scale factor (Chapter 10) or a face with heavier stems, not a search for a hinting knob. The `[font]` table in Appendix B has no hinting field, no antialiasing field, and no gamma field; the levers the configuration actually gives you are the face, the size, the two offsets, and the built-in box-drawing switch.

Greyscale coverage and the subpixel alternative

An LCD panel's pixel is three physically separate coloured stripes. A rasterizer that knows this can compute coverage three times per pixel — once per stripe — tripling the effective horizontal resolution of the edge and producing text that looks sharper at the cost of coloured fringing on stroke edges. That technique assumes several things it is increasingly unsafe to assume: that the stripe order is known, that the panel is not rotated, that the text is not being scaled or composited by someone else afterwards, and that the surface will not be captured or transformed downstream.

The last assumption is the one a compositing terminal breaks. A window drawn with transparency (Chapter 9) or blurred by the compositor (Chapter 47) is not delivered to the panel unmodified; its pixels are blended with whatever is behind. Subpixel coverage values are only correct against the background they were computed for, so blending them against something else turns the fringing from an invisible artefact into visible colour on the edges of letters. Greyscale coverage has no such dependency — it is a single weight, correct against any background. A terminal whose documented features include per-cell translucency and a background image (Chapters 12, 26) is in the class of applications where the sturdier technique is the defensible default, and the absence of a subpixel switch from the configuration reference is consistent with that.

Gamma, and why light-on-dark looks fatter

Here is the effect that produces the most misfiled complaints. Take a face, render dark text on a light background, then invert both colours and render light text on dark. The second looks heavier — noticeably so at small sizes — even though the outlines are identical.

The cause is that coverage is linear and display encoding is not. A pixel with coverage 0.5 is being asked for half the light difference between background and foreground, but the numeric value halfway between two encoded values does not correspond to half the light; it corresponds to considerably less. On a light background the edge pixels of a dark glyph therefore come out lighter than a physically-correct blend would make them, thinning the letter. On a dark background the edge pixels of a light glyph come out lighter than correct too — but “lighter” now means *closer to the foreground*, which thickens the letter. The same error in the same direction thins one and fattens the other, which is why the asymmetry is so consistent and why no amount of switching fonts makes it go away.

Two consequences are useful. First, a face chosen for its weight against a light scheme will read as too heavy the moment you move it to a dark one, and the remedy is a lighter face or a smaller size rather than a configuration flag. Second, this interacts directly with `draw_bold_text_with_bright_colors` (Chapter 12): on a dark scheme the apparent-weight boost stacks with a brighter palette entry, so bold text can end up reading as considerably louder than the author of the colour scheme intended. Chapter 47's advice about how the bright palette should relate to the normal one has a rasterization reason behind it as well as a colour one.

What the built-in box drawing sidesteps

Chapter 4 introduced `builtin_box_drawing` as the way to get seamless lines and Chapter 10 mentioned it as a way around metric mismatch. Rasterization explains why it is the more reliable path in a way neither chapter needed to. A box-drawing character taken from a font is an outline, subject to everything above: scaled to the cell, antialiased at its edges, positioned by the same pen arithmetic as a letter. If the cell height is not an exact multiple of the stroke's design width, the horizontal line in one row lands on a slightly different sub-pixel offset than the one in the row above, and a long vertical rule assembled from many cells develops a visible waver. Fully-covered pixels in one row abutting half-covered pixels in the next is exactly what a hairline gap looks like.

The built-in font is not an outline font at all. It computes the geometry from the cell's own pixel dimensions, which means a horizontal line is placed at the same integer row within every cell by construction and a vertical rule occupies the same integer column all the way down. There is no rounding for adjacent cells to disagree

about because there is no scaling step. That is the whole trick, and it is why the default is on and why turning it off to get “the font’s real box characters” reliably makes the seams worse rather than better. The same reasoning covers the powerline separators: a triangle that must exactly meet the cell edge is a geometry problem, and geometry computed from the cell always meets the cell.

The one thing rasterization cannot amortize

The atlas caches a rasterization at a specific pixel size for a specific face. Every input to that rasterization — face, size, scale factor, the two offsets — is therefore a cache key in effect, and changing any of them invalidates the cached glyphs and forces them to be drawn again. This is the mechanism behind a behaviour users notice without explaining: the first frame after a live font-size change, or after dragging a window to a monitor with a different scale factor, does more work than the frames around it, because the atlas is being repopulated from nothing while the grid is also being reflowed (Chapter 3). It settles within a few frames and does not recur until the next change. It is the cost of the amortization, paid in a lump at the moment the amortization’s assumptions stop holding.

Chapter 51 – Presentation: the Swap, the Compositor, and the Idle Window

Chapter 5 is about deciding *when* to draw. It ends at the word “present” and does not open it. This chapter opens it: what happens between the renderer finishing its draw calls and the light changing on the panel, why that gap is where a terminal’s perceived latency can be lost after all the work of Chapters 4 and 37 has been done correctly, and what changes when the window is translucent, on a second monitor, or idle.

The swap is where the frame waits

The renderer does not paint the screen. It paints a *back buffer* — an off-screen surface — and then asks the graphics stack to make that buffer the one being scanned out. That request is the buffer swap, and it is the point at which the frame stops being the application’s problem and becomes the display pipeline’s.

The swap is also where the calling thread can block. If the previous frame is still being scanned out, the swap waits for the next vertical blanking interval before it can proceed. This is the mechanism that makes Chapter 5’s “refresh cadence” real — the frame scheduler does not need a clock of its own to be paced at the refresh rate, because the swap paces it. It is also why frame-time instrumentation has to be read carefully. `debug.render_timer` (Chapter 17) reports the time spent producing a frame; time spent parked in a swap waiting for the display is not the terminal being slow, it is the terminal being early. A render timer showing a small number while the window feels sluggish is pointing at the presentation path, not at the renderer.

Tearing, and why a terminal rarely shows it

If a swap were allowed to take effect immediately rather than at a blanking interval, the panel would scan out the top of the old frame and the bottom of the new one, and the seam between them is tearing. Games disable the wait deliberately, trading the artefact for latency. A terminal has no reason to make that trade, and the reason is structural rather than aesthetic: Chapter 5 already established that the terminal presents at most one frame per refresh and discards every intermediate grid state, so there is no queue of frames waiting to get out and nothing to be gained by cutting the line. Tearing buys latency only when frames are being produced faster than they can be shown. A coalescing terminal never is.

The one case where users report tearing in a terminal is a fast scroll under a compositor configured to bypass its own synchronization. That is the compositor’s decision, applied to every window on the desktop, and it is not something the terminal’s configuration reaches.

Two monitors, two cadences

A window has one swap chain and the panel it is on has one refresh rate, and the awkward case is a window spanning two monitors that do not agree. The window is presented on the schedule of whichever output the compositor considers it to be on, and the half hanging over the other monitor shows whatever was most recently presented, at that monitor’s own cadence. On mismatched panels — a 60 Hz external beside a high-refresh laptop display — the visible result is that the two halves of the same scrolling region update at slightly different rhythms.

This compounds with the scale-factor behaviour of Chapter 10, and the two are worth keeping separate when diagnosing. Crossing a scale-factor boundary changes the *cell* size and reflows the grid, which is a visible one-time re-layout. Crossing a refresh-rate boundary changes nothing about the grid and affects only the cadence of updates. A user who reports that “the terminal goes weird on the second monitor” is describing one of those two, and asking whether the text changed size separates them immediately.

Variable-refresh displays turn the cadence into a range rather than a number. For a workload that produces a frame only when something changed, that is close to ideal: the display can present the frame when it arrives rather than holding it for the next fixed interval, which removes up to a full refresh period of latency from a keystroke echo. It is one of the few cases where a terminal benefits from display technology built for games, and it needs no configuration to do so.

The compositor is a second consumer

On a modern desktop the terminal's presented buffer is not scanned out directly; it is handed to a compositor that assembles it with every other window into the final image. Most of the time this is invisible and costs one extra copy. Three of zterminal's documented features make it visible.

`window.opacity` (Chapter 9) means the terminal's buffer must be *blended* with what is behind it, so the compositor cannot take the shortcut of scanning an opaque window out directly. `window.blur` means the compositor must additionally filter the content behind the window, which is a real per-frame image operation over a region the size of the window, performed by a different process — and it is why Chapter 9 describes blur as a *request* that macOS and KDE Wayland honour and others ignore. The background image of Chapter 26 is the instructive contrast: it is composited by zterminal itself, inside its own frame, and costs the terminal a textured quad rather than costing the compositor anything.

The practical ordering, when a translucent zterminal feels heavier than an opaque one, is therefore to suspect the compositor before the terminal. The terminal's own work per frame is nearly identical in both cases; the desktop's is not.

The idle window, and what it costs

Chapter 5 established that the event loop sleeps and that an idle window presents nothing. Presentation is where that claim cashes out in power. A window that presents no frames holds no GPU work, keeps the compositor from recomposing the region it occupies, and — on a system that can do so — lets the panel's refresh drop or the GPU clock down. The self-terminating timers of Chapter 5 are what make this reachable rather than theoretical: a cursor that blinked forever would wake the loop, produce a small frame, and force a recomposite twice a second indefinitely, which is precisely why `cursor.blink_timeout` exists and defaults to stopping the blink.

This is also the most concrete argument for the design decision Chapter 5 states about the extension layer. A control panel that repainted on its own animation clock, or a dashboard that polled on a timer while hidden, would each convert an idle window into a permanently-presenting one. Chapter 22 describes the dashboard's cost as the cost of *looking* — paid while the tab is open and not otherwise — and presentation is why that distinction matters rather than being a nicety.

When presentation is the problem

Chapter 17 gives the decision procedure for `debug.renderer` and `debug.prefer_egl` and is the right place to go for it. What that chapter cannot supply, because it is organized by field rather than by layer, is the prior question: which of the two layers you are debugging at all.

The distinction is sharp enough to sort most symptoms in one step. A rendering fault produces *wrong pixels*: glyphs at the wrong offset, colours transposed, the atlas smeared across cells — the structural, consistent

faults Chapter 17 lists as the signature of a driver claiming a path it implements badly. A presentation fault produces *right pixels at the wrong time*: stutter under load, a frame of lag on echo, a window that updates only when you move it, two halves of a window at different rhythms. Nothing is drawn incorrectly; a correct frame arrives late or not at all.

The reason this matters is that the two have almost disjoint suspect lists. A rendering fault is the terminal or the driver, and the graphics-backend fields are the experiment. A presentation fault is the swap, the compositor, the monitor configuration, or a wake source that is not firing — none of which the backend fields touch, and none of which will change when you step down a backend. The one instrument shared across both is `debug.highlight_damage` (Chapters 4, 5, 17), and it answers a third question that belongs to neither: *not are the pixels right* and *not did they arrive on time*, but *how many of them did the terminal bother to draw*. A full-window flash on every keystroke means damage tracking is not doing its job; a small rectangle around the cursor means it is, and the sluggishness is somewhere below.

Chapter 52 — Locale, Ambiguous Width, and Text That Runs the Other Way

Chapter 3 established that the grid resolves each incoming scalar to a width using the East-Asian-width rules. Chapter 11 established what a grapheme cluster is and why the terminal does almost no shaping. Chapter 44 established what happens to bytes that are not valid UTF-8. Between them there is a topic none of the three covers: the environment those rules run in, the characters whose width the rules themselves do not settle, and the writing systems whose requirements a cell grid cannot express at all.

What the locale actually reaches

The locale environment variables are the traditional mechanism by which a program learns which character encoding, collation order, and formatting conventions to use. Appendix J lists the variables zterminal reads, and none of them is a locale variable. That is not an omission; it follows from the encoding model of Chapter 44. The terminal decodes UTF-8, unconditionally and incrementally, and treats anything that is not valid UTF-8 as replacement characters. There is no code path that would consult a locale to choose a different decoder, because there is no different decoder to choose.

The consequence is a clean division that is easy to state and frequently misapplied. The locale governs what the *programs* do — what encoding they emit, how they sort, how they format a date, whether a library believes a character is printable. It does not govern what the terminal does with the bytes those programs emit. A program configured for a legacy encoding will emit bytes that are not valid UTF-8, and the terminal will render replacement characters; the fix is in the program's environment and no terminal setting is involved. Correspondingly, a program that renders correctly under one terminal and as replacement characters under another is rarely revealing an emulator difference — it is more often revealing that one of them accepted non-UTF-8 bytes and the other did not.

Where this bites in practice is over a connection or into a container (Chapter 46). The locale is part of the environment, and an environment configured on the near side does not automatically exist on the far side. A shell that renders a prompt correctly locally and as replacement characters three hops away is the same program emitting different bytes, because the library it consulted has different information about what it may emit.

The characters whose width nobody agrees on

The East-Asian-width classification has a category that causes more layout damage than every other Unicode subtlety combined: *ambiguous* width. It contains characters conventionally rendered wide in East Asian typography and narrow everywhere else — a set that includes several Greek and Cyrillic letters, a number of mathematical and typographic symbols, and, most consequentially for a terminal, much of the box-drawing and block-element range.

The problem is that “ambiguous” is not a width. Something has to pick, and if the terminal picks one value while the program running inside it picks the other, every line containing such a character disagrees about

where the following cells are. The failure is exactly the one Chapter 3 warns about — a single mis-measured glyph shifts the rest of the line — but it arrives through a mechanism that is much harder to see, because most characters are handled correctly and only these are not.

Historically the resolution was locale-driven: a terminal in an East Asian locale treated ambiguous characters as wide, and one elsewhere treated them as narrow. Since `zterminal` reads no locale variable, the classification it applies is fixed rather than environment-dependent. That is a defensible choice with a real advantage worth naming: a fixed rule means the same content lays out identically for every user of the terminal, which is precisely the property a TUI author needs and precisely the property a locale-dependent rule destroys. It also means that when a mismatch happens it is between the terminal and a program that decided otherwise, and is therefore resolvable on the program's side.

The practical diagnostic is short. If a TUI's frame is correct but its content is shifted by one column per line, and the frame is drawn from characters in the ambiguous range, the two ends disagree about ambiguous width. If the same TUI is correct when its border style is switched to plain ASCII, that confirms it. And the durable fix for anyone writing such a program is Chapter 48's rule restated for this case: measure width the way the grid does, and where that cannot be guaranteed, do not use characters of disputed width for anything load-bearing.

Composition, dead keys, and the preedit problem

Chapter 7 notes that composition — dead keys, an input method's multi-keystroke sequences — is resolved by the platform before a character event reaches the terminal, and that this is why an input method that works elsewhere works here. That is accurate and it is also the whole of the terminal's involvement, which is worth drawing out, because it explains a limitation that users of East Asian input methods encounter immediately.

An input method has an intermediate state: the *preedit*, the partially-composed text shown to the user, underlined or highlighted, while they choose among candidates. In a text field, the preedit is displayed inline by the widget, which knows how to reserve space for it and remove it when composition finishes. A terminal has no widget. It has a grid whose contents belong to the program on the other end of the PTY, and that program knows nothing about a composition in progress — it has been sent nothing, because nothing has been committed.

Every terminal resolves this the same way in principle: the preedit is drawn by something other than the grid, over it, and only the committed result becomes key input. The consequences follow from the architecture rather than from any particular choice. The preedit is not in the scrollbar, because it was never in the grid. It cannot be selected, searched, or copied while it is in flight. Vi mode (Chapter 18) does not see it. And the cells underneath it are still owned by the program, so a program that repaints while composition is open repaints underneath the composition. None of this is fixable at the terminal layer; it is what “the grid is the contract” (Chapter 11) costs when the input is not yet ready to enter the grid.

Direction, stated honestly

Arabic, Hebrew, and the other right-to-left scripts require the renderer to reorder a logical sequence of characters into a visual sequence for display, and the reordering rules are among the most involved parts of the Unicode standard. A terminal in this lineage does not implement them, and Chapter 11 explains why in general terms without naming this case: reordering is shaping, and shaping breaks the guarantee that the character at logical position n occupies column n .

The specific incompatibility is worth stating precisely rather than leaving as a corollary. Every cursor-motion sequence in Chapter 6 addresses a *column*. If the terminal reordered text for display, a program that wrote a right-to-left string and then moved the cursor to column ten would have no way to know which character it had landed on, because the terminal's visual order and the program's logical order would have diverged and nothing in the protocol communicates the mapping between them. The escape-sequence interface has no vocabulary for bidirectional text, and adding reordering underneath an interface that cannot describe it produces a terminal that looks right and cannot be programmed against.

So right-to-left text is stored and displayed in logical order: the characters appear left to right in the order the program sent them. This is wrong as typography and correct as a grid, and it is one of the clearest examples in the whole book of the pattern Chapter 49 draws out — the terminal declines to do a thing well rather than doing it in a way that breaks the contract everything else depends on. Applications that genuinely need bidirectional display do the reordering themselves and emit the visual order, which works, because at that point the program and the terminal agree that column n holds what the program put there.

Chapter 53 – Windows: the Pseudo-console and the Chapters That Do Not Apply

zterminal runs on Linux, BSD, macOS, and Windows. Chapters 35 and 43 describe the process model and the line discipline in terms of a pseudo-terminal, a controlling terminal, process groups, and signals — a set of concepts that is precise, load bearing for everything those chapters conclude, and absent from Windows. This chapter is the seam. It exists because a reader who takes Chapters 35 and 43 as universal will misdiagnose every Windows symptom they encounter, and because the places where the platform substrate differs are exactly the places where a feature this book documents behaves differently or not at all.

What Windows has instead of a PTY

The Windows console was historically not a byte stream. A console application did not write escape sequences into a file descriptor; it called an API — set the cursor position, set the attributes, write a rectangle of character cells — against a console object the operating system owned. There was no stream between the program and the display for a terminal emulator to sit in the middle of, which is the structural reason terminal emulators on Windows were historically a different kind of program from their Unix counterparts.

The pseudoconsole is what changed that. It provides a pair of pipe handles that behave, from the emulator's side, like the two ends of a PTY: the emulator writes input into one and reads output from the other. Behind that interface the operating system keeps a console object, and it *translates*. A program that uses the console API is driven against the console object, and the system generates the escape sequences describing the resulting changes and pushes them out of the read handle. A program that writes escape sequences directly has them interpreted against the same console object and re-emitted.

The consequence is the single most important thing to know about the platform, and it is a consequence about composition rather than about any one feature. On Windows there is an emulator between the program and the emulator. The output zterminal parses is not the bytes the program wrote; it is the bytes the system generated to describe the state the program produced. That is the same structure as the double parse Chapter 31 takes apart for the multiplexer — two independent parsers and an intermediate grid in the middle — arrived at by a completely different route, and it attenuates capabilities for the reason Chapter 46 gives: anything the intermediate model cannot represent cannot survive the round trip, regardless of whether both endpoints support it.

The chapters that do not apply

Chapter 43 is about a kernel module Windows does not have, and nearly every mechanism in it is absent or differently located.

There is no line discipline as a configurable filter, so there is no terminal-attributes structure, no per-PTY flag words, and no control-character array to inspect. The inspection tools that chapter recommends as the first diagnostic step have no equivalent here, because there is nothing of that shape to inspect. Canonical mode is not a flag on the object; line editing for programs that want it is provided by a different console mechanism entirely.

There are no signals, so there is no interrupt character that becomes a signal, no foreground process group for it to be delivered to, and no job control in the sense Chapter 43 uses. Interruption is delivered as a console control event to the processes attached to the console, which is a genuinely different delivery model rather than a renamed version of the same one. The elegant property Chapter 43 identifies — that broadcast is not a special mechanism, because each pane's own discipline independently turns the same byte into its own signal — depends on the discipline existing. Writing an interrupt byte to several pseudoconsoles is not the same operation as writing it to several PTYs, and reasoning about the broadcast feature (Chapter 30) from the Unix model will mislead you here.

There is no window-change signal. The resize handshake of Chapter 43 — write the size, then the kernel signals the group, then the program queries and finds the new size already in place — is replaced by resizing the pseudoconsole object, which the system reflects to the attached programs by its own route. The ordering guarantee that makes a well-written Unix application never repaint at a stale size is a property of that specific handshake and does not transfer.

And there is no flow-control freeze of the kind Chapter 43 describes as “the terminal that froze”, which removes one of the most common Unix support questions from the platform entirely.

What the configuration says about the platform

The book's own configuration reference marks the platform seams where they exist, and reading them together gives an accurate picture without any guessing. The default `font.normal` is `Consolas/Regular` on Windows (Chapter 10), against `monospace` on Linux and BSD and `Menlo` on macOS. `window.class` is described as an X11 and Wayland concept with no macOS or Windows analogue (Chapter 9), so a field that is restart-only everywhere is inert here. `window.blur` is honoured by macOS and KDE Wayland and silently ignored elsewhere (Chapters 9, 47), which includes this platform. And `--embed` takes an X11 window ID and therefore has no meaning off X11. The state directory moves as well: Appendix E records the config living at `%APPDATA%\zterminal\zterminal.toml` here rather than under `~/.zterminal`, which matters for Chapter 8's resolution order, since `XDG_CONFIG_HOME` is a convention of the other platforms.

The pattern in that list is worth naming because it is the same pattern as Chapter 46's: the configuration surface is uniform and the *honouring* of it is not. A field a platform cannot implement is accepted and ignored rather than being an error, which keeps one configuration file valid across machines — Chapter 8's layering argument — at the cost of making “I set it and nothing happened” a symptom with a platform explanation as well as a restart-only explanation (Appendix G). When diagnosing on Windows, the platform explanation should be checked first, because it is the one the Settings tab cannot show you: the tab will happily save a value that nothing will ever read.

What to expect from the extension layer

The extension layer divides along a line that is easy to predict once stated. Features that are pure terminal-side state — profiles, snippets, triggers, recent directories, the palettes, the split tree, Exposé, the render-side counters behind the dashboard — depend on nothing the platform differs about, because they are operations on the terminal's own data structures.

Features that reach into the operating system are the ones to check. The per-pane process tree (Chapter 30) walks process relationships and the kill gate acts on them; process models differ. Shell integration (Chapter 27) needs the marks emitted by a shell that supports the snippet, and the snippet is shell-specific before it is platform-specific. The tmux client (Chapter 28) speaks to a tmux server, which is a Unix program. Session restore's replant path (Chapter 29) captures a command line and a working directory from a running process, which is process-introspection work. And the IPC socket (Chapter 33) is documented in this book by its path and its message set; what transport backs it on a given platform is an implementation detail this book does not document, and the honest statement is that the message surface is what is specified and the plumbing beneath it is not.

The summary a Windows reader should carry out of this chapter: the first half of this book — grid, renderer, damage, escape sequences, input encoding, configuration, colours, fonts — is platform-independent and describes what you have. The chapters built on the PTY and the line discipline describe a substrate you do not have, and their conclusions should be read as *what the Unix mechanism guarantees* rather than as what your machine does.

Chapter 54 – Conformance: How You Know an Emulator Is Right

Every chapter so far has described behaviour. This one asks the harder question underneath them: how does anyone establish that an emulator’s behaviour is correct, given that there is no single authority to check it against? The answer shapes the support model of Chapter 6 and explains a design decision that reads as pedantry until you see what it is protecting.

There is no specification to conform to

The obvious approach — read the standard, implement it, test against it — is not available. What a terminal must do is defined by an accumulation: a hardware terminal’s documented behaviour, the behaviours a widely-deployed emulator added on top, the sequences a graphics protocol introduced later, the modes a multiplexer expects, and a long tail of whatever particular applications happen to rely on. Some of that is written down. Some exists only as the observed behaviour of a program that other programs were tested against. None of it is versioned as a whole, and portions of it contradict each other.

So “correct” for an emulator means something weaker and more useful than “conformant”: *the applications people actually run behave the way their authors intended*. That is an empirical target, and it changes how you test. You are not proving a specification is satisfied; you are defending a compatibility surface against regression.

Why the three statuses are a testable contract

Chapter 6’s **IMPLEMENTED** / **PARTIAL** / **REJECTED** table looks like documentation. It is more usefully read as the test contract, because each status makes a different, checkable promise.

IMPLEMENTED promises a sequence produces its full documented effect, which is testable by sending it and inspecting the grid. **PARTIAL** promises that an enumerated subset works and — this is the part carrying the weight — that the rest is rejected rather than half-applied. That is also testable: send the unsupported parameter, assert that nothing changed. **REJECTED** promises no effect at all, which is the easiest assertion in the set.

Now consider the alternative a less disciplined emulator adopts: accept everything, implement what is convenient, silently ignore the rest. Nothing about that is testable. A sequence that is silently ignored and a sequence that is handled produce identical output whenever the handling happens to be a no-op for the current state, so a test cannot distinguish “we support this” from “we dropped it on the floor”. Worse, the application-side consequence is exactly what Chapter 6 identifies: programs probe and adapt, and a terminal that answers a probe for something it does not really implement has induced the application into taking a path that will not work.

The precision of Chapter 6’s partial list — SGR parameters **0–9**, **21–25**, **27–49**, **58**, **59**, **90–97**, **100–107** handled, **11–19** and **51–55** rejected — is therefore not documentation pedantry. It is the specification of a test suite. Every one of those boundaries is an assertion someone can write.

The self-tests the protocol already contains

Two features of the escape-sequence surface exist to make an emulator checkable, and both appear in Chapter 6's list.

`ESC # 8` is the DEC alignment test: it fills the entire screen with the letter `E`. Its purpose is visual verification of the things that are awkward to assert numerically. Every cell is covered, so the cell grid's pixel arithmetic (Chapter 4) is visible as a uniform field — any drift in cell width or height shows up as a column or row of `Es` out of step with its neighbours. It exercises the whole addressable area, so an off-by-one in the grid's dimensions is apparent at the right or bottom edge. And because the fill is uniform, a font-metrics problem, a scale-factor problem, and a glyph-offset problem each produce a visually distinct pattern. It is a one-sequence smoke test for the entire grid-to-pixel path, which is why a sequence from a hardware terminal of the 1970s is still worth having.

The second is the reply surface. `CSI n` (device status), `ESC Z` (identify), and the palette query forms of `OSC 4` and `OSC 10/11/12` all cause the terminal to write bytes back. That makes them scriptable assertions rather than visual ones: set a palette entry, query it, compare the reply to what you set. Chapter 40 presents the query mechanism as a runtime alternative to `terminfo`, which is its purpose in production. Its second use is that it makes a large part of the terminal's state externally observable, and an emulator whose state can be read back can be tested without a screenshot.

What reference tests catch, and what they do not

`--ref-test` (Appendix A, Chapter 37) records a reference session. The shape of that technique is worth understanding, because its strengths and weaknesses are characteristic of the whole class.

What it catches is regression, precisely and cheaply. A recorded session is a fixed input and a captured result; replay it after a change and any difference is a behavioural change somebody has to justify. This is the right tool for a large inherited compatibility surface, because the surface is too big to specify and the thing you actually want to know is not “is this right” but “did this change”.

What it does not catch is anything the recorded sessions never exercised. A reference corpus is a sample of the compatibility surface, and coverage of that surface is unknowable in a way coverage of a specification is not — you cannot enumerate the behaviours applications depend on, so you cannot compute what fraction you have captured. It also cannot tell you that a captured behaviour was wrong in the first place; if a bug was present when the reference was recorded, the reference now defends the bug. Every golden-output suite has this property, and the mitigation is the same everywhere: golden files are evidence of change, never evidence of correctness.

The regressions worth building tests around

Not all of the surface is equally fragile. Four areas concentrate the risk, and each is a place where an error is invisible in the common case and destructive in a specific one.

Width accounting is first, for the reason Chapter 3 gives and Chapter 52 extends: a single wrong width displaces the rest of the line, and the symptom surfaces in an application that did nothing wrong. Tests here are cheap — write a string, query the cursor position, assert the column.

The wrap edge is second. Chapter 3 states what `autowrap` does — a character printed in the last column wraps the cursor to the start of the next line and marks the line wrapped — and the detail a test has to pin down is the one that sentence does not settle: whether the wrap happens when the character is printed or is deferred until the *next* character arrives. Getting it wrong is invisible until a program writes exactly to the right margin and then emits a newline, at which point it either loses a line or gains a blank one. It is the classic emulator bug, it is trivially testable by construction, and it is the reason the *wrapped* flag Chapter 3 mentions

matters beyond reflow: the flag is the observable that distinguishes a line that ran off the edge from one the program deliberately broke.

Scroll regions are third, because they interact with everything: an index at the bottom margin scrolls the region rather than the screen, and a cursor-position sequence is relative to the region under origin mode and absolute otherwise. Full-screen editors depend on all of it.

Reflow is fourth and is the hardest to test, because its correctness is defined over a history rather than a state. Chapter 3 describes rejoining and re-wrapping logical lines; the assertions that matter are round-trip properties — narrow then widen, and the paragraph must return to its original shape — and stability properties, notably that the OSC 133 marks of Chapter 27 still point at the right logical lines afterwards. That last one is a cross-feature invariant, which is the category that survives longest without anyone noticing it has broken.

What the extension layer has to prove

The extension layer carries one obligation the inherited core does not, and it comes from the pre-**vte** scanner of Chapters 26 and 27. The scanner peels image and shell-integration sequences out of the byte stream before the parser sees them. The invariant it must preserve is that for every stream *not* containing those sequences, the parser sees exactly what it would have seen without the scanner, and the resulting grid is identical.

That is an unusually clean property to test, and it is the right one to insist on, because it is what lets every conformance conclusion about the inherited core continue to hold in a binary that has been extended. An extension that changed what the parser sees would invalidate the entire reference corpus at a stroke — not because the corpus would fail, but because passing it would no longer mean what it meant. The architectural discipline Chapter 5 describes for the frame loop has an exact counterpart on the parse path: the extensions express themselves as damage in the one frame loop, and as removed prefixes in the one parse stream, and in both cases the rule is that the inherited path's behaviour is unchanged for input the extension does not claim.

Chapter 55 – Accessibility, and the Surface a Screen Reader Cannot See

This chapter is short and it is honest, because the honest account is more useful than a longer one implying more than the program does. A terminal emulator is a structurally difficult host for assistive technology; zterminal does not solve that problem; and several of the settings documented in this book are nonetheless the difference between a usable terminal and an unusable one for a reader with low vision or a sensitivity to motion. Those are three separate claims and they deserve to be separated.

Why the grid is hostile to a screen reader

Assistive technology on every major platform works by traversing a tree of objects that describe an interface semantically: this is a button, this is a text field with this content and this insertion point, this is a list with these items and this one is selected. A screen reader announces that tree and lets the user navigate it. The tree is supplied by the application, usually by the toolkit it is built with.

A GPU terminal supplies no such tree. What it has is the grid of Chapter 3: a rectangular array of cells, each holding a cluster and its attributes, and a renderer that turns that array into pixels via an atlas (Chapter 4). There is no button, no field, no list, and no insertion point in any semantic sense — there is a cursor position, which is a coordinate rather than a caret in a document. This is the same limitation Chapter 49 states about structure, arrived at from the other side: the terminal cannot see structure in its own content, so it cannot describe structure to anything else.

The consequence is not that a terminal is unreadable but that the reading is mechanical. A screen reader with a terminal-specific integration reads cells, tracking the cursor and announcing what changed near it. Making that useful requires the emulator to publish its grid and its cursor through the platform's accessibility interface, and to do so on every update — a per-frame obligation on an interface designed for interfaces that change when a user does something, not for a surface that can rewrite itself entirely many times a second (Chapter 45).

zterminal, as this book documents it, publishes no accessibility tree. There is no field for it in the configuration reference, no chapter describing one, and no claim anywhere in the program's documentation that it is present. A reader who depends on a screen reader should treat that as the state of things here and evaluate accordingly. Stating it plainly is the same discipline Chapter 6 applies to escape sequences: a clean rejection is more useful than an ambiguous half-answer, because it lets someone plan around it.

The one surface with a structure

There is a structural asymmetry inside the program worth pointing out, because it is the seam where this could change. The terminal grid is drawn by the GL renderer and has no document behind it. The control panel (Chapter 19) is not — it is a WebView compositing over the terminal surface, built from the shared `zgui-core` component library, and a WebView's content is a document tree by construction.

That means the two halves of the program are in genuinely different positions with respect to assistive technology. The configuration editor, the tabs, the profile selector, the dashboard's controls — all of these are document elements in a component library shared across the stack's GUI applications, which is the kind of

surface platform accessibility interfaces are designed to traverse. The terminal grid is a texture. Nothing in this book states what the panel actually exposes, and this chapter does not claim it exposes anything; the point is narrower and structural. If accessibility work were ever done here, the panel and the grid would be two different projects with two different costs, and only one of them would be fighting its own architecture.

What the configuration already gives you

Several settings documented elsewhere in this book have accessibility effects their own chapters, written from a different angle, do not mention. Gathering them is the practically useful part of this chapter.

For low vision, `font.size` is in the live-reload set (Chapter 8) and the font-size bindings change it without editing a file, so magnification is immediate and reversible. `font.offset` increases line spacing, which is a distinct and often larger legibility win than size alone, because it attacks crowding rather than glyph size. Chapter 50's account of gamma is directly relevant as well: a face that reads as too light on a dark scheme is a legibility problem with a face-weight solution, and Chapter 47's contrast reasoning applies unchanged.

For contrast, the strongest single recommendation is a negative one already made in Chapter 47 for different reasons: `window.opacity` below one and `colors.transparent_background_colors` both make the effective contrast of text depend on the desktop behind the window, which is unknowable and not constant. A reader who needs guaranteed contrast should run opaque over a solid background. Similarly, the default `colors.normal.black` matching the default background (Chapter 47) is a contrast trap worth overriding explicitly rather than living with, since it makes one of the sixteen palette colours literally invisible.

For motion sensitivity, three things move on their own clocks and all three are addressable. The cursor blink is governed by `cursor.blink_interval`, `cursor.blink_timeout`, and the blinking mode itself (Chapter 13) — and the mode values that forbid an application from overriding the setting are what make “off” mean off rather than “off until a program asks”. The visual bell runs an easing curve over a flash colour for `bell.duration` (Chapter 13), and disabling it removes a full-window flash that is the most startling thing this program does by default. Inline-image animation (Chapter 26) plays on its own timer, which is content-driven rather than configuration-driven and therefore the one of the three you cannot switch off from the configuration file.

For motor accessibility, the terminal's strongest property is one it has by accident. Chapter 16's binding model and Chapter 23's palettes mean essentially every feature in the extension layer has a keyboard route — the control panel is keyboard-reachable, the split tree is navigated by binding, Exposé and MRU are keyboard-driven, and the command palette is a text-entry front door to the action catalogue. A program that can be driven entirely from the keyboard is a program that works with any input device capable of producing keystrokes, which is a large class of assistive hardware. This was built for speed rather than for access, and it delivers access anyway.

The honest position

The summary is short and none of it should be softened. Screen-reader support is not something this program documents, and the grid architecture makes it a substantial project rather than a configuration switch. The low-vision and motion-sensitivity story is genuinely decent, and it is decent because of settings that exist for other reasons — live font sizing, line spacing, an overridable palette, and the fact that nearly every self-driven animation in the program is bounded and switchable. And the keyboard story is strong, by accident of a design that treats the keyboard as the primary interface.

That is the state of it. Chapter 49 argues that the value of an honest list of what a terminal cannot do is that it tells you when to stop trying and route around; this chapter is that list for one more axis, and the routing-around is real — the settings above are worth configuring deliberately rather than discovering by accident.

Chapter 56 – Failure, and the Blast Radius

Chapter 35 argues that the single process is load-bearing: the shared address space is what makes the cross-cutting features affordable, and moving to a process per window would turn each of them into a distributed-systems problem. That argument is correct and this book has leaned on it repeatedly. It also has a cost, and no chapter has stated it, because the cost only becomes visible when you ask a question none of the architecture chapters asks: when something goes wrong, how much goes with it?

Shared address space means shared fate

The property that makes the extension layer cheap is the same property that makes the failure domain large. Every native pane in every tab of every window belongs to one process. Whatever ends that process ends all of them at once — not gracefully, in the ordered teardown Chapter 35 describes for `tmux`, but immediately, taking the PTYs with it and delivering hang-ups to every shell the process owned.

This is the honest counterweight to Chapter 35's dividend, and it deserves to be stated in the same plain terms: a design that lets one process see every pane is a design in which every pane depends on that one process staying alive. There is no partial degradation available. A window-per-process terminal loses one window to a fault of any kind; this one loses the session.

What makes that tolerable rather than reckless is the asymmetry Chapter 35 identifies for a different reason. Native panes die with the process; `tmux` panes do not, because their PTYs belong to the `tmux` server, which is a separate process the client was merely attached to. Chapter 35 presents that as a lifecycle fact about `tmux` and Chapter 31 presents it as a property in a comparison table. Read through the failure lens it becomes something more directive: it is the *durability control* the program offers, and it is the only one. Anything whose loss would genuinely cost you belongs on the far side of that boundary, and everything else is fine where it is.

That reframing is worth carrying into daily use, because it makes the native-versus-`tmux` decision of Chapter 31 easier rather than harder. The question is not which kind of pane is better — Chapter 31 already establishes that neither is — but which losses you are prepared to absorb. A build watcher, a log tail, a scratch shell, an editor on a file that is saved: losing these costs a keystroke to recreate, and they belong in native panes where they are cheap and fast. A long-running remote session, a job with hours of accumulated state, anything you would be annoyed to restart: these belong to the multiplexer, and the reason is not that `tmux` is more reliable but that its lifetime is independent of the process you are looking at.

The GPU seam, and why it is survivable in principle

There is one class of fault a terminal encounters routinely that is not a program bug: the graphics context going away underneath it. A driver update, a suspend and resume, a device reset, a display swapped for another — each can leave a process holding a GL context whose resources are gone. This is not hypothetical, it is a normal event on a laptop, and it is worth understanding what it can and cannot cost here.

The useful observation is that everything the renderer owns is *derived* state. The atlas (Chapter 4) is a cache of rasterized glyphs, regenerable from the fonts and the metrics by definition — Chapter 50 describes exactly this happening on every font-size change, where the atlas is repopulated from nothing within a few frames.

The vertex data for a frame is computed from the grid each time. Nothing on the GPU is a source of truth for anything.

The source of truth is the grid and the scrollbar, which live in ordinary process memory and are untouched by anything the graphics stack does. So a graphics fault is categorically different from a process fault: it destroys only reconstructible state. Whether any given fault is *recovered* rather than fatal depends on the code path that meets it, which this book does not document; the point worth taking is the architectural one, that the terminal's authoritative state does not live where the volatile resources are. The debug fields of Chapter 51 are the practical follow-up — if a machine meets this seam repeatedly, forcing `debug.renderer` to a lower path or changing `debug.prefer_egl` is the standard way to find out whether one particular graphics path is the offender.

The failures that are not the terminal's

Three common startup failures look identical from the outside — a window that is not usable — and have entirely different causes. Separating them is faster than debugging any of them.

The first is a configuration file the program cannot read. Chapter 8's live reload means a saved file is picked up while the terminal is running, so a file that has become malformed announces itself the moment you save it rather than at the next launch. The observable is the log (Chapter 17), and the default `debug.log_level` of `Warn` is exactly the level at which a complaint about a file the program could not use is worth emitting — which is why “a configuration change did nothing” (Appendix G) has reading the log as its first step and not its last. The interaction with the Settings tab is the part worth knowing: the tab writes the file, so a change made there is not the suspect; a change made in an editor, in an imported file (Chapter 8), or in a file that is layered per-machine is where a malformed value comes from.

The second is a child that exits immediately. If `terminal.shell` names a program that is not there, or names one that fails on startup, the pane's PTY reaches end-of-file at once, and a terminal that closes a pane when its child exits closes this pane as fast as it opened it — the classic “the window flashes and disappears”. This is precisely the case `--hold` exists for (Appendix A): it keeps the window open after the child exits, so whatever the child printed on its way out is still on the screen to read. Reaching for `--hold` first, before any other diagnosis, converts an invisible failure into a visible error message, and it is the single most useful flag in Appendix A for anything that fails at launch.

The third is not a failure at all but is reported as one: a restart-only field (Appendix B) that was changed and saved and had no effect. Chapter 53 adds a second member to that family — a field the platform cannot honour — and the two together cover most of the “I set it and nothing happened” reports. The diagnostic order is worth fixing in mind because it costs nothing: check whether the field is restart-only, check whether the platform honours it, then read the log.

What a bad restore can and cannot do

Session restore (Chapter 29) writes state to disk and reads it back, which makes that file an input to the program, and inputs deserve a moment's thought about what they can cause. Chapter 39's boundary is that zterminal's code-execution surfaces are user-configured rather than remotely reachable: a hint's command, a trigger's command, `bell.command` are lines you put in your own configuration. A restore file sits slightly differently, because its contents were not typed by you — they were captured from your running session by the program.

The bound on what that can do comes directly from Chapter 29's own limitation. Restore brings back the *arrangement* — where the panes were, how big, in what directory, running what command line — and respawns fresh processes into it. It does not restore live process state, and it has no mechanism to. So the failure mode of a corrupted or edited restore file is bounded to that surface: a pane that reopens in the wrong directory, at the wrong size, or running a command line that is not the one you left. That last one is the only one with any weight, and it is the reason the restore file deserves the same care as the configuration file — it is in the same

category as `bell.command`, a place where a line of text becomes a process. Appendix E is where to look for it: the saved arrangements are the JSON under `layouts/` and `snapshots/`, plain text in the `one` state directory, which is also what makes them reviewable before they are trusted.

Designing around it

The chapter's practical residue is short, and it is the kind of thing that is obvious once stated and easy never to state.

Put durable work behind the multiplexer boundary and keep disposable work native. Use profiles (Chapter 21) rather than manual arrangement for anything you would be irritated to rebuild, since a profile is a description that survives everything. Keep the config file under version control if you layer it across machines (Chapter 8), because a malformed file that reloads live is a fast failure and a fast revert. And treat the log as the first stop rather than the last, since it is the one surface that records what the program thought was happening at the moment it stopped doing what you expected.

The design commitment underneath all of it is the one Chapter 35 makes and this chapter has priced: `zterminal` chose a large failure domain in exchange for an integration story that is not otherwise affordable. That is a defensible trade and it is the trade the entire second half of this book is built on. It is not a free one, and a reader who knows exactly which of their panes would survive the process ending is a reader who has understood the architecture rather than merely the features.

Chapter 57 — Where It Sits in the Stack

zterminal is not a standalone product in the MenkeTechnologies world; it is the window the rest of the stack lives in, and the final chapter places it there.

The host

The stated development workflow is shell-centric to an unusual degree: the shell is the IDE, the build system, the deploy tool, and the orchestrator, and many synchronized shell instances run at once. In that world the terminal is the substrate every other tool sits on, and anything that slows it slows the whole pipeline. That is why the core is the fast inherited one and the extensions are integrated rather than shelled out: a multiplexer subprocess, an image-viewer subprocess, and a session-manager plugin are three more things between a keystroke and a result, and zterminal's thesis is to collapse them into the terminal itself.

The neighbours

zterminal hosts the stack's own tools. `zshrs` — the Rust `zsh` — runs inside it. `zpw` paints its prompts, status bars, and TUIs inside it, and the built-in box drawing (Chapter 4) and truecolor (Chapter 3) are what make a `zpw` status bar render seamlessly. The control panel is built from `zgui-core`, the same shared component library the stack's GUI applications use, so the design system is one across the suite. The screenshot that grounds Chapter 20 shows exactly this neighbourhood: a `zpw`-themed session running behind the zterminal control panel, the two halves of the stack in one window.

The honest summary

zterminal earns its place in the MenkeTechnologies catalogue not by being a faster terminal — that would be the `awkrs` trap, dead on the project's own rules — but by adding, on top of a fast inherited GPU core, a set of genuine firsts (live `tmux` editing, `tmux`-aware profiles, the telemetry dashboard) and a broad, clearly-bounded layer of integration the bare emulator lacks. The first half of this book documented that core faithfully and completely, every setting walked. The second half documented the original layer, every feature and every claimed invention, with its basis. The product is the sum: a fast, inherited, faithfully-documented emulator core, wrapped in an original, integrated extension suite, hosting the stack it was built for. That is what zterminal is, and that is where it sits.

The one binary, as an argument

If there is a single sentence that captures the design, it is that all of this is one binary and one process. The core emulator, the split tree, the `tmux` wire client, the inline-graphics decoders, the OSC 133 scanner, the control panel, the palettes, the dashboard, the profiles, the persistence layer — none of them is a subprocess, a plugin, a daemon, or a remote page. They share one address space, one event loop, one config file, and one GPU surface (Chapters 1, 35). The book has argued throughout that this is not merely tidy but *enabling*: the cross-cutting features — a dashboard that sees every pane, a broadcast that reaches every `tmux` pane, an Exposé that tiles native and `tmux` panes together, a profile that restores terminal and multiplexer at once — are cheap because the state they touch is all reachable from the one process, and would each become a

distributed-systems problem in a per-window-process design. The integration is the pitch (Chapter 1) and the single process is how the integration is affordable. A reader who takes one structural idea from this book should take that one: the features are not bolted to the side of a terminal, they are what becomes possible when the terminal, the multiplexer, the image viewer, the session manager, and the control plane stop being separate programs and become one.

Chapter 58 – The Configuration Derive Macro

Chapter 8 described what the configuration model does: one TOML file, imports, per-key overrides, a live reload that does not lose your comments. This chapter describes the machinery that makes that model possible, because it is not written by hand. Almost every struct and enum in the configuration tree gets its deserializer *generated* by a pair of procedural macros living in their own workspace crate, and the properties Chapter 8 presents as design decisions — an unknown key warns instead of failing, a bad value falls back to the default instead of aborting the launch, a renamed key keeps working under its old name — are properties of the generated code rather than of any hand-written parser.

Where it sits in the workspace

The workspace has five members and one deliberate exclusion:

```
[workspace]
members = [
    "zterminal",
    "zterminal_core",
    "zterminal_config",
    "zterminal_config_derive",
    "crates/zterminal-ledger",
]
# ztmux-core is its own submodule/repo (own workspace), consumed via path dep.
exclude = ["crates/ztmux-core"]
```

That is the whole build graph of the Rust side, and two of the five members exist only to serve configuration. `zterminal_config` is 101 lines and holds one trait plus its implementations for the primitive types. `zterminal_config_derive` is 503 lines of proc-macro across five files and generates everything else. The workspace pins edition 2024 and a minimum Rust of 1.85.0.

The split is the conventional one for procedural macros — a proc-macro crate cannot export ordinary items, so the trait it generates implementations *for* has to live somewhere else — but the size ratio is worth noting. Five hundred lines of macro stand in for the several thousand lines of deserializer that the configuration surface of Chapters 9 through 17 would otherwise require, and more importantly they stand in for a *consistent* one. Every config struct behaves the same way on an unknown key because there is exactly one code path that decides what an unknown key does.

Two macros, four attributes

The public surface is small:

```
#[proc_macro_derive(ConfigDeserialize, attributes(config))]
pub fn derive_config_deserialize(input: TokenStream) -> TokenStream {
    config_deserialize::derive(input)
}

#[proc_macro_derive(SerdeReplace)]
pub fn derive_serde_replace(input: TokenStream) -> TokenStream {
    serde_replace::derive(input)
}
```

`ConfigDeserialize` is the one the configuration tree uses; the string `ConfigDeserialize` occurs 61 times across the `zterminal` and `zterminal_core` sources, counting both the import lines and the derive sites. It dispatches on the shape of the input: a struct with named fields goes to the struct generator, an enum goes to the enum generator, and anything else — a tuple struct, a unit struct — produces a compile error reading `ConfigDeserialize must be used on an enum or struct with fields`. That is the first of several places where the macro refuses rather than guesses.

The `#[config(...)]` attribute carries five directives, and their usage counts across the configuration tree are lopsided in a way that says something about how the config has aged:

Attribute	Uses	Effect
<code>#[config(skip)]</code>	10	Field or enum variant is not deserialized at all
<code>#[config(deprecated)]</code>	5	Key still works, logs a warning when present
<code>#[config(alias = "...")]</code>	2	Old key name still resolves to this field
<code>#[config(flatten)]</code>	1	Field absorbs every key no other field claimed
<code>#[config(removed = "...")]</code>	1	Key is gone, logs a warning when present

`skip` dominating the list is the expected shape: it marks the runtime-only fields that live in the same struct as the user-facing ones but are never read from a file. `flatten` appearing exactly once is enforced rather than coincidental — the macro rejects a second one with `At most one instance of #[config(flatten)] is supported`, because the generated code drains the unclaimed-key table into the flattened field and there is no defensible way to split that table between two claimants.

The generated struct deserializer

For a struct, the macro emits a visitor type named after the struct with `Visitor` appended, implements `serde::de::Visitor` for it with `expecting` returning the string `a mapping`, and implements `Deserialize` by calling `deserialize_map` with a default-constructed visitor. The body of `visit_map` is the interesting part, and it is short enough to read whole:

```

let mut config = Self::Value::default();

// Unused keys for flattening and warning.
let mut unused = toml::Table::new();

while let Some((key, value)) = map.next_entry::<String, toml::Value>()? {
    match key.as_str() {
        #match_assignments
        _ => {
            unused.insert(key, value);
        },
    }
}

#flatten

// Warn about unused keys.
for key in unused.keys() {
    log::warn!(target: #LOG_TARGET, "Unused config key: {}", key);
}

Ok(config)

```

Three structural decisions are visible in those fifteen lines. The struct starts as its own `Default` and is *mutated* by whatever keys the file supplied, which is why a config file can be a partial document rather than a complete one — every absent key keeps the default that was already there, with no `Option` wrapper and no separate “partial config” type. The iteration is over the map’s entries rather than over the struct’s fields, so an unknown key is a value the loop simply fails to place, not an error the deserializer raises. And the unclaimed keys accumulate in a table that is used for two different purposes — flattening consumes it, and whatever is left over becomes the `Unused config key` warnings.

The per-field arm is where the second half of Chapter 8’s promise lives:

```

match serde::Deserialize::deserialize(value) {
    Ok(value) => config.#ident = value,
    Err(err) => {
        log::error!(
            target: #LOG_TARGET,
            "Config error: {}: {}",
            #literal,
            err.to_string().trim(),
        );
    },
}

```

A value that fails to deserialize logs an error naming the key and *leaves the default in place*. It does not abort the map visit, it does not poison the struct, and it does not propagate a `serde` error upward. One malformed value in a hundred-key config file costs you that one key. This is the mechanism behind the message-bar diagnostics of Chapter 8 rather than a startup failure, and it is generated identically for every field of every struct in the tree, which is precisely the consistency a hand-written deserializer would fail to maintain across that many call sites.

The log target for all of it is `env!("CARGO_PKG_NAME")` evaluated inside the macro crate, so every configuration diagnostic in the program is tagged with the derive crate’s name regardless of which crate the struct being deserialized lives in.

Deprecation, aliases, and the migration pointer

The `deprecated` and `removed` directives share one code path. Both append a warning to the field's match arm, and both compose their message the same way:

```
let mut message = format!("Config warning: {} has been {}", literal, parsed.ident);
if let Some(warning) = parsed.param {
    message = format!("{}", message, warning.value());
}
message.push_str("\nUse `zterminal migrate` to automatically resolve it");
```

The trailing sentence is a hard-coded pointer to the migration subcommand (Chapter 42), which means every deprecation warning the program can emit carries its own remedy without anyone having to remember to write one. The optional parameter is the human explanation — the integration test in the derive crate uses `#[config(deprecated = "use field2 instead")]` and `#[config(removed = "it's gone")]` — and it is appended after a semicolon rather than replacing the generated prefix, so the key name is always in the message even when the author of the attribute forgot to mention it.

An alias is handled entirely in the match pattern. Every `#[config(alias = "...")]` pushes another literal into the field's list, and the arm is emitted as an alternation:

```
field_streams.match_assignments.extend(quote! {
    #(#literals)* => { #match_assignment_stream },
});
```

So the old name and the new name are the same arm, not two arms that happen to do the same thing, and there is no ordering hazard between them.

One more transformation is applied to any field whose type path ends in `Option`:

```
if value.as_str().is_some_and(|s| s.eq_ignore_ascii_case("none")) {
    config.#ident = None;
    continue;
}
```

The string `"none"` in any case becomes `None` before the ordinary deserialize is attempted. This is what lets a config file `unset` an optional value that an imported file (Chapter 8) had set, which a plain serde deserializer cannot express — there is no TOML literal for “absent”. The detection is syntactic, on the last segment of the type path, so it fires for `Option<T>` regardless of how the type was imported.

The enum half

Enums get a much smaller generator. Variants are matched by their lowercased name, so `"THREE"` and `"three"` and `"Three"` all resolve to `Three`; variants carrying `#[config(skip)]` are filtered out of the match entirely; and the `expecting` string is built by accumulating every surviving variant name into a list of the form `one of `One`, `Two`, `Three``, with the trailing comma and space trimmed off by a `saturating_sub(2)`. An unrecognized string produces `unknown variant `X`, expected one of `One`, `Two`, `Three`` — the error names what it got and enumerates what it would have accepted, generated from the enum definition, so it cannot drift from the code the way a hand-maintained error message does.

The integration test pins the interaction between a rejected variant and the default: a struct field of an enum type given `"HugaBuga"` in the file keeps the struct's default value for that field, because the enum's `visit_str` returns an error and the struct's per-field arm swallows it. Two independently-generated pieces of code compose into “a bad enum value costs you that key and nothing else.”

SerdeReplace, and why override is not deserialize

The second macro exists for a problem Chapter 33 raises but does not resolve: the `-o` flag and the IPC config command apply a *fragment* — `cursor.style="Beam"` — to a config that is already fully constructed. Deserializing that fragment would produce a config where every key the fragment did not mention is back at its default, which is the opposite of what an override means.

`zterminal_config` declares the trait that expresses the difference:

```
pub trait SerdeReplace {
    fn replace(&mut self, value: Value) -> Result<(), Box<dyn Error>>;
}
```

For every leaf type the answer is trivially “deserialize and assign,” and the crate provides those with a macro over the primitives — every integer width from `u8` to `u128` and `i8` to `i128`, both floats, `bool`, `char`, `String`, `PathBuf`, and the log crate’s `LevelFilter`. `Vec<T>` replaces wholesale. `Option<T>` is the first interesting one: when it is already `Some`, the replacement recurses *into* the inner value rather than replacing the option, so overriding one field of an optional sub-table does not discard the rest of it. `HashMap<String, T>` is the second: it deserializes the fragment as a map and then inserts each pair into the existing map, so an override adds and updates entries instead of truncating the map to whatever the fragment listed.

For composite types the derive generates the recursion. A newtype or an enum gets the direct form — deserialize the whole value into `*self` — while a struct with named fields gets a walk over the fragment’s table keys:

```
match value.as_table() {
    Some(table) => {
        for (field, next_value) in table {
            match field.as_str() {
                #replace_arms
                - => {
                    let error = format!("Field \"{}\" does not exist", field);
                    return Err(error.into());
                },
            }
        }
    },
    None => *self = serde::Deserialize::deserialize(value)?,
}
```

Note the asymmetry with `ConfigDeserialize`, and note that it is deliberate. An unknown key in a *file* is a warning, because a file is a long-lived document that may be shared across versions. An unknown key in an *override* is an error, because an override is a command you just typed and a typo in it should tell you so. The two macros disagree on purpose, and the disagreement is the reason both exist rather than one.

The alias handling is duplicated on this side, with one extra check the deserialize side does not perform: an alias with an empty or whitespace-only value is a compile error, `Field "x" has an empty alias value`. A flattened field, when present, becomes the *last* match arm — a catch-all `_` that forwards the whole value into the flattened member — which is why the generator has to collect it separately and append it after every named arm rather than emitting it in field order.

The call site

Every one of those generated `replace` implementations exists to serve one function, in the CLI module:

```
pub fn override_config(&mut self, config: &mut UiConfig) {
    let mut i = 0;
    while i < self.config_options.len() {
        let (option, parsed) = &self.config_options[i];
        match config.replace(parsed.clone()) {
            Err(err) => {
                error!(
                    target: LOG_TARGET_IPC_CONFIG,
                    "Unable to override option '{option}': {err}"
                );
                self.config_options.swap_remove(i);
            },
            Ok(_) => i += 1,
        }
    }
}
```

An override that fails is reported *and removed from the list*, which matters because the list is re-applied on every config reload (Chapter 8). Without the `swap_remove`, a single bad `-o` would log its error again on every reload for the life of the process. The parse step ahead of it is equally forgiving in the same shape: each `-o` argument is parsed as a standalone TOML document, and one that does not parse prints `Ignoring invalid CLI option` and is dropped rather than aborting startup.

There is exactly one hand-written `SerdeReplace` implementation in the configuration tree, for the `Program` type that represents a command as either a bare string or a `{ program, args }` table. It uses the direct form — deserialize the whole value — because a `Program` given as a string and a `Program` given as a table are different variants, and recursing into the table's keys would be meaningless for the string form.

What the derive crate tests

The derive crate carries seven tests, all passing, and they are worth reading as a specification of the above rather than as coverage. The fixture struct is deliberately hostile: a field with both an alias and a deprecation, a second field whose deprecation warning must *not* fire because the key is absent, an `Option<u8>`, a generic nested struct, a flattened struct, three enum-typed fields including one given an invalid value, a removed key, and a field with two aliases. The test installs a capturing logger, deserializes a TOML document exercising all of it, and then asserts both the resulting values and the exact set of warnings emitted. That last part is what makes the suite meaningful: a regression that stopped emitting the deprecation warning would leave every value assertion passing.

Chapter 59 – The Command Ledger

Chapter 27 established the shell-integration marks and Chapter 29 gathered the persistence story, and between them they leave one thing unresolved that this chapter is entirely about. An OSC 133 command block is anchored to *absolute grid lines*. When those lines fall out of scrollback, the marks go with them, and every feature built on top of a block — the output diff, the metric trend, the column sort of Chapter 60 — stops being able to reach that run. “Diff this against the previous run” works until the previous run scrolls away, which for a busy pane is measured in minutes.

The ledger removes that bound. It is a separate workspace crate, 1,208 lines across two files, that stores a finished block’s captured output on disk keyed so those same features can reach a run from weeks ago.

What is stored, and what deliberately is not

The stored output is the grid-reconstructed text: the same `Vec<String>` the terminal already builds from cells for the diff, trend, and sort consumers. The crate’s module documentation is explicit about the consequence and about why it was accepted:

Escape sequences are gone by then, so **stored output carries no ANSI**. Preserving original styling would mean teeing raw PTY bytes on the read path, which is a per-byte cost paid by every session whether or not anyone ever queries the ledger. One append at block finish is the whole ingest cost here.

That is the central engineering trade of the feature, stated plainly. The alternative — tee the PTY stream — would put a cost on the hot path of Chapter 45, for every pane, in every session, in exchange for colour in a store most sessions never read. The chosen design puts the entire cost at block *finish*, once, on a thread that is not the parser’s.

The documentation also draws the line against the obvious prior art:

Persistent *metadata* search over shell history is not new (Atuin queries by text, directory, exit code, duration, host). What this stores that a history database does not is the output bytes, keyed by command block.

The on-disk format

The store is an append-only run of segments, each of which is two files written in lockstep. `NNNNNN.dat` holds the payloads — a `u32` length followed by that many bytes, repeated. `NNNNNN.idx` holds fixed-stride index records, one per payload. Both carry a 16-byte header of an 8-byte magic, a 2-byte little-endian version, and 6 reserved bytes. The magics are distinct per file kind, `ZTLEDGR1` for payloads and `ZTLEDIX1` for indexes, so an index file handed to the payload reader is rejected on the magic rather than misparsed.

The index stride is 48 bytes and it is load-bearing:

Offset	Width	Field
0	8	<code>id</code>
8	8	<code>offset</code> — byte offset of the payload’s length prefix
16	4	<code>length</code> — payload bytes, excluding the prefix

Offset	Width	Field
20	8	<code>started_unix_ms</code>
28	4	<code>duration_ms</code>
32	4	<code>exit_code</code>
36	1	<code>truncated</code>
37–47	11	reserved, zeroed

Eleven reserved bytes exist so a field can be added without changing the stride, and the stride cannot change because the index is read as a flat array. The crate’s own test says so in its doc comment: “The stride is load-bearing: the index is read as a flat array, so a field added past the stride would silently read the next record’s bytes.”

`exit_code` is an `Option<i32>` in the API and a fixed-width `i32` on disk, which requires a sentinel. The chosen value is `i32::MIN`, justified in the source rather than asserted: “A real exit code cannot be `i32::MIN` — wait(2) status codes are 8-bit, and even a shell reporting a signal stays far inside the range.” The test that pins it does not merely round-trip the sentinel; it round-trips `-1`, `0`, `1`, `127`, `255`, and `i32::MAX` and asserts each is distinguishable from it.

Reading a file that a crash was writing

Every decode path in the format module is built to fail rather than fault. The payload cursor yields `Option` from every accessor, checks bounds with `checked_add` before slicing, and refuses a length prefix that claims more bytes than the buffer holds. Its doc comment names the exact failure shape it exists for: “a truncated tail — the shape a crash mid-append leaves behind — is a skipped record rather than a panic.”

The tests exercise that directly. One encodes a full index record and asserts that every prefix of it shorter than the stride decodes to `None` — a loop over all 48 cut points, not a single hand-picked one. Another writes a length-prefixed string, asserts every truncation fails cleanly, then constructs a prefix claiming 999 bytes ahead of a five-byte body and asserts both the read and the skip refuse it. A fourth checks that a header with the version incremented is rejected, with the comment “a newer format is not guessed at.”

Segment loading composes those guarantees into a policy. A segment whose index header fails validation is skipped and the store carries on with the rest, so opening a store written by a newer build costs you the segments that build wrote rather than your entire history. The index walk stops at the first record that does not decode — `while pos + INDEX_STRIDE <= idx_bytes.len()` with a `break` on a failed decode — so a torn tail truncates the segment’s view at the last whole record.

Why the index carries strings

A `Segment` holds its index records and, alongside each, the command text and the working directory resolved once at open. Those two strings are not in the index file; they are at the front of the payload, which is why the payload’s layout puts metadata before output lines. Reading them at open costs one small read per record against an mmap that is already established, and it buys the property the whole crate is organized around:

```
fn select<F>(gself, q: GQuery, mut output_matches: F) -> Vec<LedgerRecord>
```

with the doc comment “This ordering is the crate’s whole performance argument, so it is expressed as one function rather than spread across callers: no path exists that can read output bytes for a record the metadata predicates already rejected.”

A `Query` is a conjunction of seven optional predicates plus a limit: `command_contains`, `command_exact`, `cwd_prefix`, `exit_code`, `since_unix_ms`, `until_unix_ms`, and `output_regex`. The first six are answered entirely from the in-memory entry — four from the fixed index fields, two from the strings resolved at open. Only `output_regex` touches the mapped payload, and it runs on the survivors. A regex query across a

two-gigabyte store therefore reads only the output of the records that already matched every other clause, and the test that pins this is named for exactly that property: `the_output_predicate_only_reads_records_the_metadata_scan_selected`.

The scan order is newest-first by construction rather than by sorting: segments are held in ascending sequence and entries within a segment are appended in order, so walking both in reverse is already the desired order. The limit is checked inside the inner loop, so a `LIMIT 20` over a large store stops after twenty matches rather than collecting everything and truncating.

Two caps, and what each one costs you

The store bounds itself twice. `MAX_BLOCK_BYTES` is 4 MiB per block; past it the output is cut and the record reports `truncated`. `MAX_STORE_BYTES` is 2 GiB total; past it the oldest segment is dropped. `SEGMENT_TARGET_BYTES` is 64 MiB, which makes segment rotation the granularity of both eviction and of the damage a single corrupt file can do.

Both caps are enforced in `append`, and both are enforced on the *stored* copy rather than trusted from the caller — the record written carries `truncated: truncated || rec.truncated`, so a caller that passed `false` for a 5 MiB block still gets a record flagged as cut. The per-block cut is on whole lines, with the reason given: “half a line is worse than a missing one for every consumer (diff, metric extraction, column parsing).” The byte accounting adds one per line for the newline the line stands for, so the cap bounds the text’s occupied size rather than only its characters.

Eviction is whole-segment and never partial:

Eviction is whole-segment on purpose: a partial trim would leave the index pointing at offsets that no longer hold what they claim, which is exactly the torn read this store must never serve.

The newest segment is never dropped even when it alone exceeds the cap, because dropping it would discard the block just appended. The consequence for a caller is clean: a dropped id reads back as *absent*, never as a garbled record, which is why `get` returns `Option` and every consumer already handles the `None`.

Durability, honestly scoped

The two writes per append are ordered and flushed independently. The payload is appended and flushed, then the index record is appended and flushed, with the reasoning in the source: “The index is what makes a payload findable, so it is flushed on every append. A crash between the two writes loses the newest block, never an older one and never the index’s alignment.”

That is a precise and modest claim, and it is the right one. It does not promise `fsync`-level durability, it does not promise the newest block survives a power cut, and it does promise that the failure mode is a lost tail rather than a corrupted store. The `mmap` used for reads is re-established after each append, with the justification that “blocks finish at human rate, so remapping per append costs nothing worth the complexity of a write-through mapping” — a rare case where the slow path is chosen explicitly because the event rate makes the fast path’s complexity unwarranted.

The application side

The host wrapper is 320 lines and adds three things the crate deliberately does not have: a policy about when to capture, a policy about whether to capture at all, and the addressing scheme the UI uses.

Capture does not happen at the `D` mark. The terminal only *queues* a finished block’s prompt line on the PTY thread, and the application drains that queue on its wakeup pass:

Materializing the output from grid cells for a large block on that thread would stall the parser, so the app drains the queue on its wakeup pass and builds the record there, while the block is still resident. A block whose lines scroll away before the drain is simply not stored — nothing blocks, nothing is retried.

The feature is off by default, and the reason given is not performance:

Storing output persists text that was previously ephemeral, which is a real change in what a terminal keeps. The store therefore stays closed until the `command_ledger` pref is turned on, and lives under `~/.zterminal/ledger/` with owner-only permissions.

The permission restriction is a `chmod 0700` on the store directory on Unix, with the reason stated at the call site: “it holds command output, which is exactly the material a shared machine should not expose.” A store that fails to open is logged and left closed rather than propagated, because “durable capture is an enhancement, never a reason to fail a terminal launch.” A block with no captured command text is refused, because it could never be grouped with its other runs and would only ever produce an un-queryable row.

The recorded working directory is the pane’s live foreground-process path — the same source the Recent Dirs tab of Chapter 32 harvests — and the module says so alongside what it therefore is *not*: “OSC 7 is not tracked, so this is a read of the process table at capture time rather than something the shell reported.” The recorded host comes from `gethostname`, falling back to an empty string “rather than a guess.”

Addressing: resident and stored are one code path

The three consumers take a `BlockRef` rather than a prompt line:

```
pub enum BlockRef {
    /// A block still anchored in the focused pane's scrollbar, by prompt line.
    Resident(i64),
    /// A block in the durable store, by record id.
    Stored(u64),
}
```

and resolve the two arguments an IPC command carries with a fixed precedence: an explicit `ledger_id` wins, “the caller only sends one when it already knows the resident copy is gone.” Both halves of that precedence are pinned by tests, and so is the case a careless implementation would break — a *negative* prompt line is a legal absolute line (the grid’s absolute-line space extends below zero into scrollbar, Chapter 3) and must not be mistaken for “absent.”

The most interesting test in the module is not about addressing at all. It appends five runs of one command with different printed elapsed times, interleaved with an unrelated command so the filter has real work, pulls them back with `runs_of`, extracts the numbers exactly as the trend path does, and then asserts that the peak and the trough both sit in the runs a buffer-scoped trend would have lost. It is a test of the *premise* of the feature rather than of its plumbing: a regression that quietly fell back to resident-only runs would produce a shorter series with wrong statistics, and the assertion is written to catch the wrong statistics rather than the shorter series.

The whole crate’s suite is 17 tests and the host wrapper adds 5; the run is clean:

```
Running unittests src/lib.rs (target/debug/deps/zterminal_ledger-...)
running 17 tests
test result: ok. 17 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out
```

Chapter 60 – Retroactive Relational Queries Over Command Output

The previous chapter made a command block’s output outlive its scrollbar. This chapter is about what the terminal then does with it, and it is the part of `zterminal` that is hardest to place in the taxonomy of Chapter 34, because the thing being built is not a terminal feature in any conventional sense. It is a small relational engine whose tables are your own already-printed command output, and it is 2,207 lines across four modules of `zterminal_core`.

The claim it rests on is one Chapter 27 already established: because every OSC 133 block is anchored to absolute lines, the terminal knows the exact output span of every run of every command. That span is a rectangle of monospace text. A great deal of the Unix command surface prints tables into rectangles of monospace text. The rest follows.

Recovering the table

`column.rs` is the base layer, and its detection rule is one sentence long:

A character position is a *gap* when every non-blank row is blank there (a whitespace character, or shorter than that position). Maximal runs of non-gap positions are the columns.

The implementation is a single pass over character positions, testing each against every row, accumulating spans between gaps. Blank rows are excluded from the scan “so a stray empty line between records does not erase every boundary” — without that exclusion a single blank line in the middle of `ps aux` output would make every position a gap and collapse the table to nothing.

Positions are `char` indices rather than byte indices, with the reason given at the type: “so multi-byte glyphs align the same way the terminal renders them.” That is the correct choice for the common case and it is worth being precise about its limit, because Chapter 52 covers the case it does not handle: a character index and a rendered column agree only while every character is single-width. Detection over output containing wide characters aligns on character positions, not on cells.

The sort on top of it is magnitude-aware — the same parse that makes `4.0K` order below `2.0G` — applied when the whole column parses as magnitudes and lexical otherwise, stable in both cases so ties keep source order. The module states its own scope against its two siblings: “This is orthogonal to the output diff (what text changed between two runs) and the metric trend (how one number moved across N runs): those compare a command against its own history; this reshapes a *single* run’s output into a sortable relational table.”

The grammar, and what it refuses

`query.rs` makes those tables addressable. A command block becomes a relation named by the command text that produced it and by how many runs back it was:

```
SELECT a.pid, a.command, b.name
FROM `ps aux` a
JOIN `lsof -i` @-1 b ON a.pid = b.pid
WHERE a.rss > 500M AND b.name LIKE '%LISTEN%'
ORDER BY a.rss DESC
LIMIT 20
```

The grammar is frozen and it is small:

```
SELECT '*' | item (',' item)*      item := colref | alias '.' '*'
FROM operand
[INNER] JOIN operand ON colref '=' colref      -- at most one
WHERE expr      -- AND / OR / NOT / parens over cell comparisons
ORDER BY colref [ASC | DESC]
LIMIT n

operand := `command text` ['@' ('latest' | '-n | n)] [AS] [alias]
colref  := [alias '.'] (header-label | 'c' n)
op      := '=' | '!=' | '<>' | '<' | '<=' | '>' | '>=' | [NOT] LIKE
```

The absences are enumerated rather than left to be discovered:

Deliberately absent in v1, and rejected with a parse error rather than half-supported: GROUP BY, aggregates, subqueries, UNION, OUTER joins, more than one join, expressions in the select list. A query that parses does what it says; a query that does not parse says so at the character position that broke it.

This is the Chapter 6 discipline — **PARTIAL** means the unsupported part is *rejected*, never half-applied — applied to a query language instead of to an escape sequence, and it is enforced by a test named **unsupported_clauses_are_rejected_not_ignored**. The error a user sees for a **GROUP BY** names the whole supported shape: “unexpected **GROUP** — this query subset is SELECT / FROM / one INNER JOIN / WHERE / ORDER BY / LIMIT”.

The **@** recency selector is what makes the language retroactive rather than live. **@latest** or **@0** is the most recent run of that command; **@-1** is the run before it. With the ledger of Chapter 59 behind it, that index reaches past the buffer. Without the ledger it reaches as far back as the scrollbar holds. Either way neither command has to be re-run, and neither had to emit structured data in the first place.

Typing without a schema

There is no schema, so a comparison decides its own semantics per pair. When both the cell and the literal parse as magnitudes, the comparison is numeric, using the same magnitude parse the column sort uses — so **500M**, **4.0K**, and **93%** behave as sizes rather than as text. Otherwise it is a case-insensitive lexical comparison. **LIKE** is always textual, with **%** and **_** wildcards.

Per-pair typing is the only workable answer for a source of data that has no declared types, and the tests pin both halves: **where_compares_magnitudes_not_text** and **text_comparison_is_case_insensitive**. A third, **like_wildcards_and_backtracking**, exists because the wildcard matcher is hand-written and **%** in the middle of a pattern requires backtracking, which is exactly the kind of code that works on the examples the author tried.

Columns are addressed by header label, matched case-insensitively, with a positional fallback:

```
// Positional fallback so a column whose header label collides with a
// keyword — or a block with no header at all — is still reachable.
let digits = name.strip_prefix('c').or_else(|| name.strip_prefix('C'))?;
```

c1 through **cN** name columns by position, which covers both the headerless case and the case where a header label happens to be **select** or **from**. The lexer helps here too: words absorb **-**, **_**, and **%** so that **%CPU** and

USE% lex as single tokens, while . stays separate because it qualifies a column reference.

Execution

The execution path is five ordered steps and each has a defensible reason for where it sits.

The join is a hash join, not a nested loop: the right side is indexed once by key and the left side streamed, “one pass each rather than the product.” Either operand order in the **ON** clause is accepted and normalized so the left key always belongs to the **FROM** relation, and a condition comparing two columns of the *same* table is an error rather than a silently-empty result. Duplicate keys fan out one row per pair, which is standard inner-join semantics and is pinned by a test because it is the behaviour a naive implementation gets wrong by keeping only the first match.

WHERE is planned before it is evaluated: every column reference in the expression tree is resolved to a flat cell index up front, so a typo fails once rather than once per row, and the per-row evaluation is pure comparison over an already-resolved plan.

ORDER BY delegates to the column sort, which is what keeps the ordering semantics identical between “sort this table in the UI” and “sort this query result.” **LIMIT** truncates after ordering — pinned by `limit_applies_after_ordering` for the obvious reason that the other order silently answers a different question.

Projection qualifies labels whenever more than one relation is in play:

```
// 5. Projection. Labels are qualified whenever a join is in play, because
//   two commands very often share a column name (NAME, PID) and an
//   unqualified header would hide which side a value came from.
```

and leaves them bare for a single-table query, both pinned by tests. The result set also carries a per-column **numeric** flag — true where every cell parses as a magnitude — which the UI uses to right-align, and two counters, **scanned** and **matched**, reporting how many rows existed after the join and how many survived the filter.

The schema you did not declare, shown back to you

The result set carries one field whose purpose is entirely about honesty:

```
/// What one operand contributed, so the UI can show the inferred schema next to
/// the result - the mitigation for whitespace column detection being a guess.
pub struct SourceSummary {
    pub alias: String,
    pub command: String,
    pub columns: Vec<String>,
    pub rows: usize,
}
```

Whitespace column detection *is* a guess. It is a good guess for aligned output and a bad one for output whose columns are separated by a single space and whose values sometimes contain spaces. The design does not pretend otherwise; it returns the inferred column names and row count for every operand so the user can see what the engine thought the table was. When the guess fails completely the failure is explicit rather than empty: a relation with no detected columns produces ``ps aux` produced no columns - its output is not an aligned table`, pinned by `non_tabular_output_is_reported`.

The error surface generally is written for a query bar rather than a log — the error type’s doc comment says so — and the tests treat it as part of the contract: `malformed_input_reports_instead_of_panicking` and `unknown_and_ambiguous_columns_are_errors` both assert on failure paths rather than success paths. Eighteen tests in total cover the module.

The two siblings

`diff.rs` implements Myers' O(ND) shortest-edit-script at line granularity, with the choice justified in three clauses: line granularity matches how terminal output is structured (one grid row per line), it keeps the edit script small enough to render inline, and it is the algorithm `git` and GNU `diff` use “so the hunks read the way users expect.” It carries a budget — 2,000 combined lines — past which it degrades to a coarse whole-block replacement, “so a pathological pair of large, wholly-different outputs cannot blow up the edit-graph trace.”

`metric.rs` extracts a single number from a run's output by *slot*: the k-th numeric token in reading order. Deterministic slot selection is what lets the same extraction be applied across every run of a command to build a trend. The tokenizer is a linear byte pass with no regex engine, and its one subtlety is a word-boundary guard:

To avoid lifting digits out of identifiers (`sha256`, `utf8`, `v2`), a token only starts at a word boundary — the byte before it must not be an ASCII letter, digit, or `_`. So `4.0K`, `1,024`, `-17`, and `3.14` are captured while the `256` inside `sha256sum` is skipped.

Thousands separators are consumed only when a digit follows, so `1,024` is one token but a comma-separated list is not. The token keeps both the parsed value and the verbatim source substring, so the trend can plot the number and label it with exactly the text that was printed.

The three together are the reason the ledger exists. Each is a function from “one or more runs of a command” to an answer, each was bounded by scrollbar residency, and each now takes a `BlockRef` (Chapter 59) that resolves to either a resident block or a stored one through the same code path.

Chapter 61 – Fan-Out Divergence

Chapter 30 described broadcast: type once, send to a checked set of panes. This chapter is about the question that immediately follows and that broadcast alone does not answer. You have just run the same command on eight machines. Seven of them printed the same thing. Which one is the eighth, and on which line did it disagree?

The conventional answer is to run the fan-out, capture each pane to a file, and diff the files afterwards. `zterminal` answers it live, while the command is still running, by hashing each pane's output as it arrives and folding the hashes into a per-line majority.

The hash

The unit of comparison is one output line, reduced to a 64-bit FNV-1a hash over its bytes with trailing blanks trimmed. The choice of a non-cryptographic hash is argued rather than assumed:

Non-cryptographic by design: this only ever decides “did two panes print the same line”. A collision costs a missed divergence (two different lines read as agreeing); it can never invent a divergence that isn't there, and it can never be more than one rendered variant wrong.

That is the right way to reason about a hash used as an equality proxy: name the failure mode, show it is one-sided, and bound its blast radius. A collision here produces a false negative on one line. It cannot produce a false positive, and it cannot corrupt the comparison of any other line.

Trailing blanks are trimmed before hashing because grid rows are fixed width and a line's cells past its text are blank; without the trim, two panes of different widths would disagree on every line.

The per-block hash vector is capped at 2,000 ordinals, mirroring the diff engine's budget from Chapter 60, and a block that exceeds it stops appending and reports `hashes_truncated` — so a pane dumping a hundred thousand lines cannot grow the vector without bound.

Ordinals, not line numbers

The comparison key is an *ordinal*: the index of a line within its own block's output, counting from the block's `C` mark. This is the detail that makes the feature work at all across real panes, and it is worth stating why.

Panes in a fan-out do not have aligned absolute line numbers. They have different scrollback depths, different prompt lengths, and different amounts of prior output. Comparing pane A's absolute line 4,182 against pane B's absolute line 4,182 compares unrelated text. Comparing pane A's third output line of *this* command against pane B's third output line of *this* command compares the thing a human would compare. There is a test named for exactly this — `ordinal_alignment_survives_prompt_line_skew` — and the source note attached to the arithmetic says “if it regresses, every divergence report is off by the prompt skew.”

The consensus rule

The fold is a pure function over a snapshot, which is what makes it testable without a live window. Its input is one `PaneHashes` per pane that is currently running a command; its output is a per-pane consensus row plus, for every ordinal where the panes did not all agree, a description of how they split.

Three rules govern it, and each exists to prevent a specific wrong answer.

First, only panes running the *same command text* are compared, and only the largest such group participates. A window where four panes ran `make test` and two ran `git status` compares the four and ignores the two. A group of fewer than two produces no consensus at all. Ties between equally-sized groups break on the command text ordering so that the report is stable across polls “rather than flipping with iteration order.”

Second, only panes that have actually reached an ordinal vote on it:

Only panes that have actually reached this ordinal vote. A pane still catching up is absent, not divergent — that is what makes the report meaningful mid-command instead of flagging every slower pane.

Without this rule the feature would be useless in its intended use — reading the report *while* the fan-out runs — because the slowest machine would be flagged as divergent on every line it had not yet printed. The test is `a_pane_lagging_behind_is_not_divergent`.

Third, a majority must be *strict*:

Within that group an ordinal has a majority only when one hash is held by *strictly more than half* the panes that reached it, so two disagreeing panes produce two variants and no divergence marking rather than an arbitrary “first pane wins”.

Two panes that disagree have no majority; neither is marked. Four panes splitting two-two have no majority; none is marked. In both cases the report still shows the split — the UI can render both variants side by side — but nothing is labelled the odd one out, because nothing is. Two tests cover it, `a_tie_between_two_panes_marks_neither` and `an_even_four_way_split_has_no_majority`.

Each pane’s consensus row also carries a `group_key`: a hash of that pane’s entire hashed prefix. Panes sharing it are identical so far, which is what lets the UI collapse a hundred agreeing panes into one row and show only the outliers individually. It is order-sensitive by construction, and there is a test asserting so — two panes that printed the same set of lines in different orders are not the same pane.

Cost when nobody is looking

The per-window state for this feature is one boolean:

```
/// Per-window fan-out divergence state. One boolean is the entire cost to an
/// unarmed window: every scan and report path short-circuits on it.
#[derive(Debug, Default)]
struct DivergenceState {
    armed: bool,
}
```

Every entry point tests it first and returns. An unarmed window hashes nothing, folds nothing, and allocates nothing. This is the same discipline Chapter 5 applies to damage and Chapter 37 to the frame loop: a feature that is off costs a branch.

Disarming does more than clear the flag — it resets every pane’s scan cursor to the never-scanned sentinel, “so re-arming starts from the panes’ current position rather than replaying stale output.”

The scan window

Hashing happens on the application’s wakeup pass, not on the PTY thread, and the range it may cover is computed by a small function whose documentation enumerates three properties the event loop depends on:

```
fn hash_scan_range(last_hash_line: i64, cursor: i64) -> Option<(i64, i64)> {
    if last_hash_line == i64::MIN {
        return None;
    }
    let from = last_hash_line.max(cursor.saturating_sub(crate::triggers::MAX_SCAN_LINES));
    (from < cursor).then_some((from, cursor))
}
```

The sentinel means the *first* scan of a pane hashes nothing and only initializes the cursor, “so screen content that predates arming never enters a comparison it was not part of.” The range is clamped to the same per-wakeup line budget the trigger scanner of Chapter 32 accepts, so a pane that dumped a huge burst between wakeups cannot stall the frame — and the clamp keeps the *newest* lines, “since those are the ones panes are still racing on.” A cursor that has not advanced yields `None` rather than an empty range, which keeps the caller’s “did anything change” answer honest and avoids a pointless consensus refold.

Four tests cover this function alone, including one for the underflow case near the start of the buffer, where `cursor - MAX_SCAN_LINES` would go negative.

Where the answer surfaces

The fold writes each pane’s divergent ordinals back into that pane’s own terminal, and the gutter reads them from there. The marking is derived in one place:

```
/// Every gutter-worthy absolute line in this buffer, ascending: one entry per
/// prompt line, plus one per output line that disagreed with the fan-out
/// majority for its ordinal.
///
/// This is the renderer's single snapshot query. Deriving the divergence rows
/// here rather than in the display layer keeps the ordinal→absolute-line
/// arithmetic in one place – the same arithmetic [Self::line_mark`] uses –
/// so a gutter mark can never name a different line than a point query would.
```

A prompt marking always wins over a divergence marking on the same line, because the prompt marking carries the exit status. An ordinal past the block’s hashed prefix names no line and is skipped — the setter can be handed a stale list when a block is re-hashed after eviction, and the guard means a stale list produces missing marks rather than wrong ones.

The overlay gets a richer view through a separate report that resolves each split into actual text, pulling every line from the pane that printed it. It is bounded at 200 consensus rows because “the overlay polls while a command runs, so the payload has to stay bounded even when a pane diverges on thousands of lines.”

The report’s failure cases are values rather than errors, each carrying a sentence the UI can display directly: `divergence tracking is not armed`, `need at least two panes running the same command to compare`, and `no two panes are running the same command right now`. Three different reasons for an empty result, distinguished, because “nothing to show” is unhelpful when the fix differs in each case.

Tested on both sides of the seam

The Rust side carries 14 tests in the divergence module. The browser side carries 11 more, in a headless harness (Chapter 63), and their subject is the one thing the Rust tests cannot check: whether the rendering agrees with the core’s rules. The test file names the properties it pins and why:

The properties pinned here are the ones that are wrong-but-plausible if the rendering drifts from the core’s consensus rules: a tie must not promote either side, a lagging pane must not read as divergent, and the arm toggle must carry the boolean the core expects.

Those are the same three rules the Rust tests assert, asserted again one layer up. That is not redundancy — a consensus algorithm that is correct and a UI that renders it incorrectly produce exactly the same user-visible bug as an incorrect algorithm, and only one of the two suites can catch each.

Chapter 62 — ztmux-core: the Wire, Byte by Byte

Chapter 28 said zterminal speaks tmux’s client/server wire protocol directly — `imsg` framing, protocol version 8 — and used that fact to argue why the live editors are trustworthy. This chapter opens the crate that does it. It is worth opening because “speaks the protocol directly” is a claim that a great many projects make about a great many protocols while shelling out underneath, and because the specifics of this one are unusually instructive about what porting a wire protocol actually costs.

The crate

`ztmux-core` is excluded from the zterminal workspace and consumed as a path dependency, because it is its own repository with its own workspace. It is 2,359 lines across five modules, and its dependency list is two crates: `libc` and `serde_json`. It describes itself as “Native tmux client engine — speaks tmux’s wire protocol directly (no subprocess); transport + tree/search/broadcast/snapshot ops for GUI hosts.”

The layering is explicit:

Module	Lines	What
<code>transport</code>	671	The wire client: <code>command(socket, argv)</code> , <code>socket_path()</code>
<code>proc</code>	149	The full command line running in a pane, no subprocess
<code>ops</code>	1,135	Session tree, search corpus, broadcast, options, buffers, keys
<code>snapshot</code>	379	Native session save/restore, parameterized by storage directory
<code>lib</code>	25	Re-exports

The transport is documented as ported from tmux 3.6’s `tmux-protocol.h`, `proc.c`, `client.c`, and `file.c`. That is the correct way to build this: the C source is the specification, and the message-type constants in the Rust file are the ones from the header, named the same.

The frame

The framing is OpenBSD’s `imsg` over `AF_UNIX/SOCK_STREAM`, and the header is sixteen bytes of four native-endian `u32` fields:

```
fn send(stream: &mut UnixStream, msgtype: u32, data: &[u8]) -> io::Result<()> {
    let len = (HEADER_SIZE + data.len()) as u32;
    let mut buf = Vec::with_capacity(HEADER_SIZE + data.len());
    buf.extend_from_slice(&msgtype.to_ne_bytes());
    buf.extend_from_slice(&len.to_ne_bytes());
    buf.extend_from_slice(&PROTOCOL_VERSION.to_ne_bytes()); // peerid = version
    buf.extend_from_slice(&u32::MAX.to_ne_bytes()); // pid = -1
    buf.extend_from_slice(data);
    stream.write_all(&buf)
}
```

Three things in six lines are worth pulling out. `len` is the *total* message length including the header, not the payload length — get that wrong and every message is sixteen bytes off. The byte order is native, not network: this is a local-socket protocol between processes on one machine, and the C struct is written straight to the socket, so a big-endian conversion would break it. And the `peerid` field is repurposed to carry the protocol version, which is how the handshake works — there is no separate version exchange, the version rides in every frame, and a server that disagrees replies with `MSG_VERSION`.

The receive side is the mirror, with one guard:

```
if len < HEADER_SIZE {
    return Err(io::Error::new(io::ErrorKind::InvalidData, "short msg"));
}
```

A length field smaller than the header would underflow the payload-size computation. It is one line and it is the difference between an error and a panic on a malformed frame.

Finding the server

`socket_path()` is 42 lines and every one of them is about a failure mode that a naive implementation hits. The naive implementation reads `$TMUX`, splits on the first comma, and uses that path. The documentation explains why that is not enough here:

Locate a *live* tmux server socket — every returned path is one we could actually connect to, so callers never report “no server” for a server that is in fact running (a GUI app’s minimal launched environment lacks `$TMUX` and may differ on `$TMPDIR`, so we probe every plausible location).

A GUI application launched from the Dock does not inherit a login shell’s environment. `$TMUX` is absent because the app is not running inside a pane, and `$TMPDIR` may differ from the one the tmux server used. So the search builds a candidate list — `$TMUX`’s socket first, then the `ztmux-<uid>` and `tmux-<uid>` socket directories under each of `$TMUX_TMPDIR`, `/tmp`, `/private/tmp`, and `$TMPDIR`, taking both the conventional `default` socket and the lone socket in a directory that contains exactly one — and then does the thing that makes the function correct rather than merely thorough:

```
// Return the first candidate we can actually connect to.
candidates
    .into_iter()
    .find(|p| UnixStream::connect(p).is_ok())
```

Existence is not the test; connectability is. A stale socket file from a server that died is a path that exists and refuses connections, and probing by `connect` skips it without any special-casing.

The `ztmux-<uid>` directory is probed before `tmux-<uid>` throughout, because `ztmux` — the native Rust port of the server — puts its sockets there. The same preference appears in the binary search: `ZTERMINAL_TMUX_BIN` overrides everything, otherwise `ztmux` is preferred over `tmux` across the `PATH` plus `/opt/homebrew/bin`, `/usr/local/bin`, `/opt/local/bin`, `/usr/bin`, and `/bin`. The preference logic is factored into a pure function over a directory list specifically so it can be unit-tested without a real installation, and it is: `find_bin_prefers_ztmux_over_tmux` creates both files in a temp directory, asserts `ztmux` wins, removes it, asserts

`tmux` is the fallback, removes that, asserts `None`.

The identify sequence, and a segfault avoided

Running one command means impersonating a client well enough that the server accepts it. The sequence is seven identify messages before the command:

```
let flags: u64 = 0;
send(&mut stream, MSG_IDENTIFY_LONGFLAGS, &flags.to_ne_bytes());
send(&mut stream, MSG_IDENTIFY_TERM, b"tmux-256color\0");
// Empty ttyname (we have no tty). Required: the server's DONE handler does
// `xstrdup(c->ttyname)` when ttyname is non-NULL-but-set, and segfaults on a
// NULL ttyname - so we must send the field even though it's empty.
send(&mut stream, MSG_IDENTIFY_TTYNAME, b"\0");
```

That comment is the single most instructive line in the crate. The client is not a terminal — it has no controlling tty, so the honest thing to send is nothing at all. Sending nothing crashes the server, because the server's identify-done handler dereferences the field unconditionally. So the client sends an empty NUL-terminated string: present, valid, and meaningless. This is the class of detail that separates a port from a reimplementation, and it is exactly the kind of thing that is discoverable only by reading the server's source, which is why the module header names the four C files it was ported from.

The command itself is `MSG_COMMAND` carrying an `i32` argument count followed by the arguments packed as NUL-terminated strings — no quoting, no shell, no escaping hazard. An argument containing a space, a quote, or a newline is transmitted verbatim because the framing is length-and-count based rather than whitespace-delimited. Every injection concern that applies to building a `tmux send-keys` command line for a shell simply does not arise.

Reading the reply, and the two-second bug that was not

Command output does not come back as a reply message. It comes back through *tmux's file protocol*: the server opens a stream with `MSG_WRITE_OPEN`, the client acknowledges with `MSG_WRITE_READY`, and then data arrives as `MSG_WRITE` chunks tagged with the stream id. The client tracks which stream ids are stdout and which are stderr from the file descriptor in the open message — anything not explicitly descriptor 2 is treated as stdout.

The termination logic is where the port had to diverge from a literal reading of the protocol, and the comment records both the behaviour and the symptom that forced it:

The server does NOT send `MSG_WRITE_CLOSE` for command output and never closes the connection (the *client* is expected to), and it sends all output *before* `MSG_EXIT` — so `MSG_EXIT` is the terminator. After it, we briefly drop the read timeout and drain any straggling `MSG_WRITE`, then stop (rather than waiting the full 2s, which made every command take ~2s and froze the `tmux` tab).

A client that waits for a close that never arrives blocks until its read timeout. With a two-second timeout and a `tmux` tab that issues several commands per refresh, that is a frozen UI. The fix is to treat `MSG_EXIT` as the terminator and then drop the read timeout to 30 milliseconds to catch any straggler, which bounds the per-command cost at the cost of a theoretical race the observed server behaviour does not exhibit. The reasoning is stated, the assumption is named, and the alternative was measured — which is the right way to document a protocol divergence.

A `MSG_VERSION` reply at any point aborts with `tmux protocol version mismatch`, which is the server's way of saying the pinned version 8 does not match what it speaks.

What the layers above it do with a synchronous command

Everything in `ops` is built on that one `command` function, and the module's shape is mostly about not calling it too often. The session tree polls roughly every two seconds, and each pane's text preview is a separate `capture-pane` round trip, so previews are cached per pane id with a five-second TTL while the tree structure refreshes at the faster rate. The cache prunes entries older than thirty seconds on each insert, so panes that have closed do not accumulate.

The entire module is `#[cfg(unix)]`, with non-Unix stubs beside every public function the application calls — `clients_attached` returns `false`, `run` does nothing, `panes` returns `{"running": false, "panes": []}`. This is why Chapter 30 can describe the process tree and broadcast as Unix-only without the Windows build needing conditional call sites: the stubs make the API total, and the caller's code compiles unchanged.

There is exactly one place a subprocess is used, and the crate is explicit that it is unavoidable:

```
/// A live tmux socket, starting a server via the binary if none is running.
pub fn ensure_socket() -> Option<std::path::PathBuf> {
    if let Some(s) = socket_path() {
        return Some(s);
    }
    let bin = tmux_bin()?;
    let _ = std::process::Command::new(bin)
        .args(["new-session", "-d", "-s", "__ztmux_bootstrap__"])
        .status();
    socket_path()
}
```

The wire protocol cannot fork a server — it is a client protocol, and a client requires a server to connect to. So bootstrapping runs the binary once to create one, then returns to the wire for everything afterwards. The README states the boundary in one line: “the only place a subprocess is used — the wire protocol cannot fork a server.”

Testing a protocol client

The transport carries seven tests, and they are unusual in shape because the subject is a conversation with another process. The suite stands up a throwaway tmux server on a dedicated socket, runs commands through the wire client, and asserts on stdout, stderr, and server survival. Three details in the harness are worth noting.

The tests serialize on a mutex, because “the wire tests each stand up a tmux server; running several at once races under load (a command can exceed the 2s read timeout).” The temporary directory is forced under `/tmp` rather than `$TMPDIR`, because “Unix socket `sun_path` is ~104 bytes, and macOS `$TMPDIR` is long enough to overflow it” — a constraint that has nothing to do with the code under test and everything to do with running the test on a Mac. And the suite skips when `tmux` is not installed, with the distinction spelled out: “tmux is only used to stand up the fixture; the client under test speaks the protocol directly with no subprocess.”

Chapter 63 – The Test Corpus, Concretely

Chapter 54 argued about conformance: that there is no specification to conform to, that reference tests are evidence of change rather than of correctness, and that four areas concentrate the regression risk. This chapter is the inventory. What actually exists, in what form, and what each part can and cannot catch.

The shape of the suite

The workspace contains 458 `#[test]` attributes across 53 files, plus a browser-side suite of 93 tests in Node's test runner. Distribution across the crates:

Crate	<code>#[test]</code> attributes
<code>zterminal_core</code>	249
<code>zterminal</code>	162
<code>crates/ztmux-core</code>	18
<code>crates/zterminal-ledger</code>	17
<code>zterminal_config_derive</code>	7
<code>frontend/lib/zgui-core</code>	4
<code>zterminal_config</code>	1

The attribute count and the number of tests `cargo test` runs are different numbers, and three effects separate them — two shrinking the count, one enlarging it.

Twenty-two of the 458 attributes are outside the cargo workspace: 18 in `crates/ztmux-core` and 4 in `frontend/lib/zgui-core/crate`, both separate workspaces consumed as path dependencies. `cargo test --workspace` never reaches them, which leaves 436 in the five members. Four more never compile on this machine: three behind `#[cfg(windows)]` in `zterminal_core/src/tty/windows/mod.rs` and `child.rs`, gated at the `#[cfg(windows)] pub mod windows;` declaration in `tty/mod.rs:16`, and one behind `#[cfg(target_os = "linux")]` at `zterminal/src/cli.rs:584`.

Pushing the other way, three attributes stand for far more than three tests, because they sit inside declarative macros rather than above functions: `ref_tests!` at `zterminal_core/tests/ref.rs:20`, expanded 45 times, and `test_clickstate!` and `test_process_binding!` at `zterminal/src/input/mod.rs:1363` and `:1428`, expanded eight times each. Three attributes, 61 tests.

The arithmetic closes exactly: 436 workspace attributes, less the 4 that do not compile, less the 3 that are macro bodies, plus the 61 those bodies expand into, is 490 — which is the sum of the six test binaries below, and 491 with the one doctest (`zterminal_core/src/term/mod.rs:2878`; the other fenced block in the crate, `grid/mod.rs:83`, is marked `notrust` and is not compiled).

The concentration in `zterminal_core` is the expected shape for this codebase and matches the architecture Chapter 2 describes: the core crate holds the grid, the parser, the selection model, vi mode, search, and the analysis modules of Chapter 60, all of which are pure data structures with no window, no GPU, and no PTY — which is to say, all of which are cheap to test. The heaviest single files are `term/search.rs` at 33, `term/mod.rs` at 30, `query.rs` at 23, and `settings.rs` at 22.

`cargo test --workspace` (which does not reach `crates/ztmux-core`, a separate workspace consumed as a path dependency) runs 491 tests across six test binaries plus one doctest: 175 in the `zterminal` binary, 245 in the `zterminal_core` library, 45 in the reference suite, 17 in the ledger, 7 in the config-derive integration test, and 1 in `zterminal_config`. 483 pass; the eight failures are all in the reference suite and are discussed below.

The reference suite

The reference tests are 45 recorded sessions, each a directory of exactly four files:

```
zterminal_core/tests/ref/<name>/
  zterminal.recording  the raw PTY byte stream, verbatim
  size.json           the terminal dimensions it was recorded at
  config.json         the scrollbar size in effect
  grid.json           the expected grid state, serialized
```

All 45 directories carry all four files, and the corpus is 46 MB — the bulk of it in the serialized grids, which run to hundreds of kilobytes each because a grid is every cell with its full attribute set.

The harness is 140 lines and its core is fifteen:

```
let mut terminal = Term::new(options, &size, Mock);
let mut parser: ansi::Processor = ansi::Processor::new();

parser.advance(&mut terminal, &recording);

// Truncate invisible lines from the grid.
let mut term_grid = terminal.grid().clone();
term_grid.initialize_all();
term_grid.truncate();

if grid != term_grid {
    // ... print every differing cell, then panic
}
```

No window, no renderer, no PTY. A `Term` constructed at the recorded size, an event listener that discards everything, the bytes fed through the parser, and the resulting grid compared to the stored one. The normalization before comparison — `initialize_all` then `truncate` — exists because the grid's storage lazily materializes rows, so two grids that are semantically identical can differ in how many rows have been touched.

The failure path is the part worth copying. Rather than reporting “assertion failed: `grid == term_grid`” against two multi-hundred-kilobyte structures, it walks every line and column, prints each differing cell with its coordinates and both values, and only then panics. A ref-test failure tells you which cells changed before you open a debugger.

The corpus names describe what it covers, and reading them as a group is a fair map of the compatibility surface Chapter 54 discusses:

- **Reset behaviour under state:** `alt_reset`, `colored_reset`, `grid_reset`, `row_reset`, `decaln_reset`, `deccolm_reset`, `delete_chars_reset`, `erase_chars_reset`, `insert_blank_reset`, `scroll_up_reset`.
- **Scroll regions and origin mode:** `region_scroll_down`, `vttest_scroll`, `vttest_origin_mode_1`, `vttest_origin_mode_2`, `origin_goto`, `newline_with_cursor_beyond_scroll_region`, `scroll_in_region_up_preserves_history`.
- **Attributes and colour:** `sgr`, `underline`, `clear_underline`, `colored_underline`, `indexed_256_colors`, `hyperlinks`, `selective_erasure`.
- **Cursor state:** `saved_cursor`, `saved_cursor_alt`, `vttest_cursor_movement_1`, `ll`.
- **Real application traces:** `vim_simple_edit`, `vim_24bitcolors_bce`, `vim_large_window_scroll`, `tmux_git_log`, `tmux_htop`, `zsh_tab_completion`, `fish_cc`.

- **Width and wrapping:** `zerowidth`, `wrapline_alt_toggle`, `tab_rendering`, `history`, `erase_in_line`, `delete_lines`, `csi_rep`, `vttest_insert`, `vttest_tab_clear_set`, `issue_855`.

The last group is the one Chapter 54 identifies as concentrating the risk, and the `vim` and `tmux` traces are the ones that make the suite an *empirical* target rather than a specification one — they are recordings of programs people actually run, which is precisely the definition of correct that chapter argues for.

The suite is generated by the binary itself: `--ref-test` records a session into this four-file shape. Adding a case is running the program and doing the thing.

What the suite says today

Thirty-seven of the forty-five pass. Eight fail, deterministically, across repeated runs: `clear_underline`, `colored_reset`, `delete_lines`, `row_reset`, `saved_cursor`, `saved_cursor_alt`, `selective_erasure`, and `sgr`.

The eight are one defect and it is not in the terminal. Their recordings — and only theirs, out of the forty-five — contain no carriage-return byte at all, so every linefeed in them moves the cursor down without returning the column, and everything written after the first newline lands progressively further right than the golden has it. Restoring the missing carriage returns in memory and replaying against the unmodified golden takes the suite to 45 of 45. Chapter 75 establishes that, Chapter 76 walks the eight individually, and the conclusion for this chapter is that the corpus is reporting a corrupt *input* rather than a divergence between golden and code — a third possibility Chapter 54’s framing does not have a name for.

Five of the eight fail twice over. The failure path described above indexes the *golden* grid across `0..grid.total_lines()`, and thirty-three of the forty-five golden serializes a `Storage` whose `visible_lines` is one smaller than the grid’s `lines` — `selective_erasure` has `lines: 3` with `visible_lines: 2`, `row_reset` has `lines: 47` with `visible_lines: 46`. Indexing such a grid at its last line trips `debug_assert!(requested.0 < self.visible_lines as i32)` at `grid/storage.rs:221` — live under `cargo test`, compiled out in release — so the diff printer panics partway through describing the mismatch it correctly detected. Twenty-eight of the thirty-three still pass, because the comparison itself never indexes: `PartialEq` for `Storage` compares `inner` and `len` only. Worse, for the five that fail the printed diff is misattributed by one row before it panics. Chapter 77 is that failure in full.

`selective_erasure` is worth one sentence of correction against Chapter 68’s reading of it: the parser genuinely has no arm for `CSI ? 2 J` or `CSI ? 2 K`, but the golden encodes that absence too, so the case has never measured selective erasure and its mismatch is entirely the missing carriage return.

The browser side

The control panel of Chapters 19 through 23 is HTML and JavaScript, and it has 93 tests that run in Node with no browser and no jsdom. They pass:

```
i tests 93
i suites 0
i pass 93
i fail 0
i duration_ms 332.016917
```

The mechanism is a hand-written shim. The tab modules are browser IIFEs that bail out when `document` is absent, so the harness loads them in a `vm` sandbox with a minimal DOM, a stub of the GUI widget library, and a *recording* invoke function standing in for the IPC bridge. Tests can then set fields, click buttons, and assert both what was rendered and which IPC calls fired.

Most of the shim is unremarkable, but one property in it is not, and its comment explains why:

```
// Real `textContent` assignment replaces all children, which is how the
// overlay apps clear their host before a re-render. Without this the shim
// silently accumulates children across renders and a test that asserts "the
// second render replaced the first" would pass against broken code.
Object.defineProperty(el, "textContent", {
  get() { return el._text; },
  set(v) { el._text = v == null ? "" : String(v); el.children.length = 0; },
});
```

That is the failure mode a hand-written DOM shim exists to have: a simplification that makes tests pass more easily than the real environment would. The shim is usable precisely because someone found that one and fixed it.

The five test files divide by subject: 31 tests on the tab framework, 19 on the settings form, 19 on the Commands tab's two block scopes, 13 on the query scope, and 11 on the divergence overlay. Each file opens with a paragraph naming the properties it pins and why they are the fragile ones — the ledger file's is representative:

The breakable part is addressing. A resident block is named by its prompt `line`; a stored one by its `ledger_id`. Sending the wrong one silently resolves to a different command or to nothing, and every Diff / Trend / Sort action goes through that decision — so each is pinned here, including the backwards-compatibility case where a pre-ledger caller must still send only `line`.

The query file's is more explicit about the division of labour between the two suites:

The breakable parts are all in the wiring rather than in the SQL (the engine is tested in Rust). Specifically: the scope must not fall through to the row list that the other two scopes render (nor fire their IPC), an empty editor must not send a query the engine would only reject, the editor's exact text must reach the backend unmangled, an engine error must be shown rather than swallowed into an empty grid, and the result must survive a round trip through another scope.

That is the correct boundary. The 23 Rust tests in `zterminal_core/src/query.rs` establish that the engine of Chapter 60 computes the right answer; these 13 establish that the right answer reaches the screen and that a wrong query reaches the user as an error rather than as an empty table.

The runner

`scripts/test.sh` runs both halves and reports them separately, extracting the JS counts from the Node runner's summary lines and the Rust counts by summing every `test result:` line across every suite. It exits non-zero if either side fails or either process returns non-zero — the double check matters, because a build failure produces zero test results and would otherwise read as zero failures.

What this corpus does not tell you

Chapter 54's caveat applies in full and is worth restating against the concrete numbers. Forty-five recorded sessions are a sample of a compatibility surface whose size is unknown, so 45 is not a coverage figure and cannot be turned into one. The stored grids are goldens: they defend whatever behaviour existed when they were recorded, including any bug present at that moment. And the entire suite runs without a window, a GPU, or a font — everything in Chapters 4, 10, 11, and 50 is outside it by construction, which is why the rasterization and presentation chapters describe verification in terms of visual inspection and the DEC alignment test rather than in terms of assertions.

What it does tell you is the thing that matters most for a codebase with an inherited compatibility surface: whether today's build produces the same grid as yesterday's for 45 real sessions, and whether 458 hand-written assertions about the parts you can assert on still hold. As of this writing eight of those 45 say no, which is the suite working — a golden corpus that never fails is a corpus nobody is testing against.

Chapter 64 – The Build Pipeline

Chapter 41 covered installing from the user’s side: `cargo install`, the platform dependency lists, the `.pkg`, and the distinction between the binary and its satellites. This chapter covers the machinery that produces those artifacts, because the assembly is more interesting than “run make” suggests and because several of the satellites Chapter 41 lists as optional are generated rather than committed.

Vendoring first

Every build target depends on one prerequisite:

```
vendor: ## Ensure required submodules are checked out (zgui-core webui is embedded by
  build.rs; ztmux-core is a path dep)
  git submodule update --init --recursive frontend/lib/zgui-core
  git submodule update --init --recursive crates/ztmux-core
```

Two submodules, for two different reasons. `zgui-core` is the GUI widget library the control panel is built from, and it is *embedded by* `build.rs` — the web assets are compiled into the binary rather than installed alongside it, which is why Chapter 19’s control panel needs no files on disk. `ztmux-core` is the crate of Chapter 62, consumed as a path dependency and excluded from the workspace because it carries its own.

Making `vendor` a prerequisite of the binary targets rather than a documented step is the difference between a clone that builds and a clone that fails with a confusing missing-crate error. The release script repeats the same two commands before its build, tolerating failure, so an already-vendored tree does not stall on a network hiccup.

The macOS bundle

The `.app` target does five things in order, and each is a small answer to a platform requirement.

Man pages are rendered with `scdoc` and gzipped into the bundle’s resources. The sources are five `scdoc` files totalling 2,217 lines: `zterminal.1` for the binary, `zterminal-msg.1` for the IPC subcommand, `zterminal.5` for the configuration file, `zterminal-bindings.5` for the binding syntax, and `zterminal-escapes.7` for the escape-sequence table. Four of the five are installed; the escapes page is a source-tree document. The choice of `scdoc` over hand-written `roff` is the conventional modern one — the sources are readable, and the 1,075-line configuration page is a document nobody would maintain in `roff`.

The build degrades rather than fails when `scdoc` is absent:

```
@if command -v scdoc >/dev/null 2>&1; then \
  scdoc < $(MANPAGE) | gzip -c > $(APP_EXTRAS_DIR)/Zterminal.1.gz; \
...
else \
  echo "scdoc not found - skipping man pages (install with: brew install scdoc)"; \
fi
```

Terminfo is compiled next, with `tic -xe zterminal,zterminal-direct`, also tolerating failure. The source is a 112-line file defining three entries: a `zterminal+common` fragment carrying the bulk of the capabilities, and two entries that `use=` it — `zterminal` with 256 indexed colours and a settable palette, and `zterminal-direct` with a 24-bit `RGB` capability and direct-colour `setaf/setab` strings. Chapter 40 covers what those

advertise and why; the structural point here is that the two entries share one fragment, so a capability added to the common block lands in both and cannot drift between them.

Completions are copied in — four files, 972 lines, one each for zsh, bash, and fish plus a zsh completion for the image helper. They are generated from the argument parser rather than hand-written, which is visible in their content: the zsh file's option list carries the exact help strings from the CLI definitions, including the bracketed defaults, and the fish file opens with a generated `__fish_zterm_global_optspecs` function enumerating every long and short option in `argparse` syntax. Generated completions are the only kind that stay correct across a flag rename, which is the failure Chapter 41's "install the completions" sentence quietly depends on not happening.

Finally the bundle is timestamped against the binary and ad-hoc signed:

```
@touch -r "${APP_BINARY}" "${APP_DIR}/${APP_NAME}"
@codesign --remove-signature "${APP_DIR}/${APP_NAME}" 2>/dev/null || true
@codesign --force --deep --sign - "${APP_DIR}/${APP_NAME}"
```

The remove-then-sign pair exists because re-signing a bundle that already carries a signature fails; removing first makes the target idempotent across rebuilds. The `-` identity is ad-hoc signing, which is what makes the bundle launchable at all on current macOS without a developer certificate — and is why Chapter 41 notes the first launch needs a right-click.

Universal binaries

The universal target builds twice and merges:

```
$(TARGET)-universal: vendor
    MACOSX_DEPLOYMENT_TARGET="10.12" cargo build --release --target=x86_64-apple-darwin
    MACOSX_DEPLOYMENT_TARGET="10.12" cargo build --release --target=aarch64-apple-darwin
    @lipo target/{x86_64,aarch64}-apple-darwin/release/$(TARGET) -create -output
    ~ $(APP_BINARY)
```

`MACOSX_DEPLOYMENT_TARGET` is set on both, and on the native build too, which is what determines the oldest system version the produced binary will run on. Setting it in the build rule rather than expecting it in the environment is the same discipline as making `vendor` a prerequisite: the reproducible thing is the one the makefile does, not the one the documentation asks you to remember.

Release packaging

`scripts/release.sh` produces the per-OS artifacts, and its first design decision is stated in its header comment: "There's no CI (private repo), so run this once on each OS to populate a release's assets." Every cross-platform choice in it follows from being run by hand on three machines rather than by a matrix.

The version is not passed in; it is read from the crate manifest with an `awk` program that tracks section headers so it cannot pick up a dependency's version by mistake, and the script aborts if it comes back empty. The tag is `v` plus that version. Architecture names are normalized — `arm64` and `aarch64` both become `aarch64`, `x86_64` and `amd64` both become `x86_64` — so the artifact names are stable regardless of which `uname` reported them.

The Linux path is the most interesting because it has to synthesize what a package manager would otherwise provide. It stages a directory tree with `bin`, `share/applications`, `share/icons`, `share/metainfo`, `share/terminfo-src`, and `completions`; copies in the binary, the image helper, the desktop entry, the AppStream metadata, the icon, the terminfo source, and all four completion files; and writes an `install.sh` into the staging directory that installs the lot into `~/.local` without root. Every copy of an optional file is `|| true`, so a tree missing an icon still produces a tarball.

The local loop

`scripts/localinstall.sh` is the one that matters day to day, and its header states exactly why it exists rather than `make install`:

Unlike `make install` (which builds a .dmg and asks you to drag), this copies the freshly built app straight into `/Applications` so the change is live now, then links the `zt` CLI (and copies the `zt-icat` helper) into `/usr/local/bin` so the terminal command tracks the fresh build instead of going stale.

The `zt` CLI is *linked* rather than copied, so it tracks subsequent rebuilds without re-running the script; `zt-icat` is copied because it is a script rather than a build product. The script refuses to run on anything but Darwin, and it checks the build's exit status through `PIPESTATUS` because the build output is piped through `tail` — a detail that is easy to get wrong and produces a script that reports success for a failed build, which is the worst possible failure for a deploy tool.

The scripts directory holds thirteen entries in total, and the ones that are not build steps are diagnostics that belong to earlier chapters: `24-bit-color.sh`, `colors.sh`, and `fg-bg.sh` print colour ramps for the verification Chapter 47 describes, and `create-flamegraph.sh` produces the profiles Chapter 37 discusses. They are shipped in the repository rather than described in prose for the same reason the reference recordings are: a check you can run is worth more than a check you can read about.

Chapter 65 – The Grid’s Storage: a Ring, a Zero, and Lazy Rows

Chapter 3 described the grid as a data structure: lines, columns, cells, a cursor, a scrollbar. This chapter is the allocation. What `Storage<T>` actually is, why scrolling is usually free, why it is sometimes not, and what the ring buffer costs in exchange.

Four fields

`zterminal_core/src/grid/storage.rs:33-53` declares the whole thing:

```
pub struct Storage<T> {  
    inner: Vec<Row<T>>,  
    zero: usize,  
    visible_lines: usize,  
    len: usize,  
}
```

`inner` is the allocation. `zero` is the index within it of the bottom-most line. `visible_lines` is the viewport height. `len` is how many rows are currently live — scrollbar plus viewport — which may be smaller than `inner.len()`.

The reason for `zero` is written directly above the type, at `storage.rs:15-19`:

```
/// A ring buffer for optimizing indexing and rotation.  
///  
/// The [Storage::rotate`] and [Storage::rotate_down`] functions are fast modular  
    ↪ additions on  
/// the internal [zero`] field. As compared with [slice::rotate_left`] which must  
    ↪ rearrange items  
/// in memory.
```

Scrolling a terminal by one line means every line moves up by one. Done literally, that is a memmove of the entire buffer on every newline — for a 10,000-line scrollbar of 200-column rows, tens of megabytes per line of output. Done with a ring, it is `self.zero = (self.zero as isize + count + len as isize) as usize % len;` (`storage.rs:180-185`), a single modular addition, and every subsequent index lookup pays a branch.

The price is that `Vec`’s API is no longer valid on this type. `storage.rs:21-26` spells out the consequence: `Index` and `IndexMut` are reimplemented, and no `Deref` to `Vec` is provided, so anything from `Vec` that should be exposed has to be written out by hand. That is the trade — a lot of small manual plumbing in exchange for making the most frequent operation in the program cost nothing.

Indexing without a remainder

`compute_index` at `storage.rs:220-234` translates a `Line` into an offset in `inner`. It does not use `%`. The comment at `:229` says why: “Use if/else instead of remainder here to improve performance.” A remainder instruction on the hot path of every cell access is measurable; a predictable branch is not.

The same function opens with the assertion that produced one of the two ref-test panics Chapter 63 now records:

```
debug_assert!(requested.0 < self.visible_lines as i32);
```

at `storage.rs:221`, followed by `debug_assert!(positive < self.len)` at `:225`. Both are `debug_assert!`, so they fire under `cargo test` and in development builds and vanish in release — which is the right choice for a check on the hottest indexing path in the program, and also the reason a malformed grid can be indexed silently in a shipped binary.

Rows are not allocated until they are needed

`with_capacity` at `storage.rs:67` allocates exactly `visible_lines` rows and comments at `:71` that “the scrollbar buffer is initialized dynamically.” A terminal configured with 100,000 lines of scrollbar does not allocate 100,000 rows at startup; it allocates a screenful and grows.

Growth goes through `initialize` at `storage.rs:126-138`, which has two paths. If `len + additional_rows` still fits inside `inner`, it only bumps `len` — this is the case immediately after a shrink, and it reuses rows that were logically dropped but never freed. Otherwise it reallocates to `inner.len() + max(additional_rows, MAX_CACHE_SIZE)`, where `const MAX_CACHE_SIZE: usize = 1_000` (`storage.rs:13`) is described as the “maximum number of buffered lines outside of the grid for performance optimization.”

Shrinking is the mirror image. `shrink_lines` at `storage.rs:107-114` decrements `len` and returns; it only truncates the allocation when `inner.len() > len + MAX_CACHE_SIZE`. A window that is being live-resized by a dragged corner therefore churns `len` up and down thousands of times without a single reallocation, as long as the excursion stays under a thousand lines.

The one place a memmove happens

`rezero` at `storage.rs:238-245` does `self.inner.rotate_left(self.zero)` and then sets `zero = 0`. This is the real rearrangement the ring exists to avoid, and it is called from exactly one place: `truncate` at `storage.rs:118-122`, which zeroes and then truncates `inner` to `len`.

That in turn is why `PartialEq for Storage<T>` (`storage.rs:55-63`) opens with two assertions that both sides have `zero == 0`. Comparing two rings whose zeros differ would require normalizing them, so equality is simply not defined for un-truncated storage. The ref-test harness of Chapter 63 calls `initialize_all` and `truncate` before comparing for exactly this reason.

The swap that counts instructions

`Storage::swap` at `storage.rs:153-176` is a hand-written four-word swap, guarded by `debug_assert_eq!(mem::size_of::<Row<T>>(), mem::size_of::<usize>() * 4)` at `:154`. The comment at `:150-152` explains the motivation: the default `swap` “generates 8 movups and 4 movaps instructions. This implementation achieves the swap in only 8 movups instructions.”

This is the kind of code that only earns its place if the surrounding call site is hot enough to notice four instructions, and it is: `Grid::scroll_up` uses swap-based movement whenever the scroll region does not start at line 0 (`grid/mod.rs:318-323`), which is what every full-screen application with a status bar does on every line of output.

When rotation is available and when it is not

The ring only helps when the whole buffer moves. `Grid::scroll_up` (`grid/mod.rs:270-329`) rotates only when `region.start == 0` (`:290-317`, rotation at `:311`), and otherwise falls back to swapping rows within the region. `Grid::scroll_down` (`grid/mod.rs:209-265`) is stricter still: it uses `rotate_down` only when `max_scroll_limit == 0` (`:227-252`), and the code at `:223-226` names the alternative “the slower, swap-based implementation.”

Reverse scrolling with scrollback enabled is therefore genuinely more expensive than forward scrolling, because moving the ring backwards would move scrollback lines into the viewport rather than pushing viewport lines into scrollback. The asymmetry is real and it is a consequence of the representation, not an oversight.

Rows

`Row<T>` (`zterminal_core/src/grid/row.rs`) carries an `occ` field — “maximum number of occupied entries” (`:20-24`) — so that operations which only need to touch live cells can stop early. `reset` at `:91-110` only resets `0..occ`, with a discriminant check at `:100-102`. `shrink` at `:71-87` truncates trailing empty cells at `:81-82` and returns `None` when nothing non-empty was removed, which is what lets the reflow code of Chapter 72 distinguish “this row lost content” from “this row lost padding.”

`Row::new` at `:37-56` is raw `ptr::write` plus `set_len`, described at `:42` as “a slightly optimized version of `std::vec::Vec::resize`.” A row is created for every line of scrollback that ever fills, so this runs once per line of output the terminal has ever seen.

What the layout looks like

The clearest single artefact in the grid module is the ASCII diagram at `grid/mod.rs:83-107`, which labels the five zones of `inner`: UNINITIALIZED, RESIZE BUFFER, SCROLLUP, VISIBLE, SCROLLEDOWN, with `zero` at the bottom. Reading it is faster than reading the four functions that maintain it, and it is the reason the invariants in this chapter are recoverable by someone who did not write them.

Chapter 66 – Damage, Line by Line

Chapter 5 argued for damage tracking as a policy: draw only what changed, and let the compositor know what changed so it can do the same. This chapter is the mechanism, which is smaller than the argument.

The unit of damage is a line and two columns

`zterminal_core/src/term/mod.rs:145-154:`

```
pub struct LineDamageBounds {
    pub line: usize,
    pub left: usize,
    pub right: usize,
}
```

That is the entire representation. Not a rectangle list, not a quadtree, not a per-cell dirty bit — one left bound and one right bound per screen line. An undamaged line is encoded by inverting the bounds: `undamaged` at :163-165 constructs `{ left: num_cols, right: 0 }`, and `is_damaged` at :179 is the test that `left <= right`.

The consequence is that damage on a line is always a single contiguous span. Two edits at opposite ends of a 200-column line damage the entire line. That is a deliberate under-approximation of precision in exchange for a fixed, tiny cost per line: the damage state for an 80×24 terminal is 24 structs of three `usizes`.

`TermDamageState` at `term/mod.rs:225-273` holds the `Vec<LineDamageBounds>` and the two entry points, `damage_point` (:259) and `damage_line` (:265). In `term/mod.rs` there are 10 call sites of `self.damage.damage_line` and 3 of `damage_point` — for a 3,944-line handler implementation, that is a very small number of places responsible for the correctness of the whole scheme, which is the argument for keeping the representation this coarse.

Two frames, always

`zterminal/src/display/damage.rs:16-28` holds `frames: [FrameDamage; 2]`, with `frame()` returning index 0 and `next_frame()` returning index 1 (`damage.rs:46-54`). `swap_damage` at :58-63 resets the current frame and swaps.

The reason for two is double buffering. With a swapped buffer chain, the region you must redraw this frame is the union of what changed this frame and what changed last frame, because the back buffer you are about to draw into is two frames stale. Anything that damages the screen for one frame — the visual bell, a moving cursor, a selection that just went away — has to be damaged twice. `display/mod.rs:866-868` does exactly that for the full-damage cases:

```
self.damage_tracker.frame().mark_fully_damaged();
self.damage_tracker.next_frame().mark_fully_damaged();
```

What the terminal reports and what the display adds

Damage arrives from the core as a `TermDamage` — either `Full` or `Partial` with an iterator of `LineDamageBounds` — and is folded in at `display/mod.rs:820-829`, followed immediately by `terminal.reset_damage()`.

The display then adds the damage the core cannot know about, because it belongs to zterminal's own UI rather than to the grid:

- The vi cursor, via `damage_vi_cursor` (`damage.rs:76`), which damages both the old position and the new one — the tracker keeps `old_vi_cursor` for precisely this (`damage.rs:18`).
- The selection, via `damage_selection` (`damage.rs:106`), with `old_selection` serving the same purpose (`damage.rs:20`).
- Hovered hyperlink underlines, damaged per cell inside the render loop at `display/mod.rs:934` as the hint underline flag is inserted.

There are 10 call sites of `mark_fully_damaged` in `display/mod.rs`. Each one is a case where the display gives up on precision: a font size change (`:696-698`, because “glyph size could change without cell size changing”), a visual bell, and so on. Giving up correctly is more important here than giving up rarely.

From damage to the compositor

`shape_frame_damage` (`damage.rs:93`) converts the accumulated line spans into `glutin::surface::Rects` in the coordinate system the platform surface wants, and `swap_buffers` at `display/mod.rs:624-638` chooses between two calls:

```
surface.swap_buffers_with_damage(context, &damage)
```

when the surface supports it and damage debugging is off, and plain `swap_buffers` otherwise. The guard at `:630` includes `!self.damage_tracker.debug` — when the `highlight_damage` debug option is on, the damage rectangles are drawn rather than reported, so passing them to the compositor as well would hide the thing you turned the option on to see.

The two coordinate conversions, `viewport_y_to_damage_y` and `damage_y_to_viewport_y` (`damage.rs:206, :211`), exist because the surface's Y axis points the other way from the terminal's. They are four lines each and they are the kind of function that is worth naming rather than inlining, because a sign error here produces a window that redraws the wrong half of itself under some compositors and looks perfect under others.

Chapter 67 – The Text Pipeline: Atlas, Instances, and Two Passes

Chapter 4 introduced the renderer and Chapter 50 covered rasterization. This chapter is what happens between them: how a rasterized glyph gets into GPU memory, how a screenful of cells becomes a draw call, and why there are two of them.

Choosing a renderer

`Renderer::new` at `zterminal/src/renderer/mod.rs:121-180` picks between two text renderers. The default rule is one line, at `:152`:

```
None => (shader_version.as_ref() >= "3.3" && !is_gles_context, true),
```

The comparison is a string comparison against the value of `GL_SHADING_LANGUAGE_VERSION`, so it is lexicographic rather than numeric. That is adequate for every GLSL version string that exists, and it is worth knowing about rather than assuming a parse happened.

The configuration can override it with three values — `Gles13`, `Gles2`, and `Gles2Pure` (`:149-151`) — where the third differs from the second only in disallowing dual-source blending. Whichever text renderer is chosen, the rect renderer and the graphics renderer are constructed with a matching `ShaderVersion` (`:156-167`), so a session never mixes shader dialects.

The atlas

`zterminal/src/renderer/text/atlas.rs:12`:

```
pub const ATLAS_SIZE: i32 = 1024;
```

One 1024×1024 RGBA texture. Not R8, not RGB — the comment at `atlas.rs:79-80` says “Use RGBA texture for both normal and emoji glyphs, since it has no performance impact,” which removes a format switch that would otherwise have to be made per glyph, and lets a colour emoji and a subpixel-antialiased letter live in the same atlas.

The packing is a shelf allocator, drawn as ASCII art at `atlas.rs:18-31` and implemented in three fields: `row_extent` (leftmost free pixel in the current row), `row_baseline` (the row's Y origin), and `row_tallest` (the tallest glyph so far, which becomes the next row's advance). `room_in_row` at `:225-231` is the whole test, and `advance_row` at `:234-245` is the whole promotion.

There is no repacking and no eviction within an atlas. When the shelf allocator runs out, `load_glyph` at `:251-287` increments `current_atlas` and, if that index is past the end, pushes a brand new 1024×1024 texture and recurses. The number of atlases is bounded only by how many distinct glyphs get rendered. Wholesale invalidation is the only reclamation: `clear_atlas` (`:290-295`) resets every atlas's shelf state and rewinds `current_atlas` to zero.

The failure case is worth noting because it does not fail. A glyph larger than 1024×1024 returns `AtlasInsertError::GlyphTooLarge` (`:124-126`), and `load_glyph` turns that into a `Glyph` whose width, height, and UV extents are all zero (`:274-285`) — an invisible glyph rather than a panic or an error dialog.

The glyph cache above it

`GlyphCache` (`renderer/text/glyph_cache.rs:46-48`) is a `HashMap<GlyphKey, Glyph>` and the file's own comment at `:42` calls it a “Naïve glyph cache.” The naïvety is the point: there is no LRU, no size cap, and no per-entry accounting, because the working set of a terminal is a few hundred glyphs and the atlas behind it has no eviction anyway.

`get` at `:200` does the obvious lookup, with one refinement at `:228-235`: a missing glyph is cached under the key `\0` so that a document full of unrepresented codepoints rasterizes the replacement once rather than once per character.

Invalidation is wholesale. `reset_glyph_cache` at `:272-277` clears the loader, replaces the map with a default, and calls `load_common_glyphs` (`:312`) to re-warm it. `update_font_size` at `:283` carries a note at `:281-282` that `reset_glyph_cache` must be called separately to reload the renderer's fonts — two steps rather than one, which is the kind of coupling that is worth documenting in the source because the compiler cannot enforce it.

One instance per cell

The GLSL3 renderer draws the screen as instanced quads. The element buffer is six indices, uploaded once (`glsl3.rs:65-73`): two triangles. Everything else is per-instance.

`InstanceData` (`glsl3.rs:296-330`) is 36 bytes: two `u16` for the cell coordinate, four `i16` for the glyph offset and size, four `f32` for the UV rectangle, three `u8` of foreground colour plus one `u8` of cell flags, and four `u8` of background colour with alpha. The attribute setup at `glsl3.rs:110-127` declares exactly five attributes, all with `VertexAttribDivisor(index, 1)`.

The colour packing is not stylistic. The comment at `glsl3.rs:120-123` records the reason:

```
// These are packed together because of an OpenGL driver issue on macOS, which caused a
// `vec3(u8)` text color and a `u8` cell flags to increase the rendering time by a
// huge margin.
```

`const BATCH_MAX: usize = 0x1_0000` (`glsl3.rs:27`) — 65,536 instances, or 2.25 MB of instance buffer, allocated once as `STREAM_DRAW` at `:79-84`. A 200×60 terminal is 12,000 cells, so an ordinary window fits in a fifth of one batch and issues one draw per pass.

The GLES2 path cannot use instancing, so it emits four vertices per cell and its batch limit is derived from the index type instead: `const BATCH_MAX: usize = (u16::MAX - u16::MAX % 4) as usize` (`gles2.rs:230`), with the reason at `:229`: `u16::MAX` “is the maximum possible index in `glDrawElements` in GLES2.”

Two passes, or four

Blending is set up once, at `glsl3.rs:51-52`:

```
gl::Enable(gl::BLEND);
gl::BlendFunc(gl::SRC1_COLOR, gl::ONE_MINUS_SRC1_COLOR);
```

`SRC1_COLOR` is dual-source blending: the fragment shader emits two outputs, a colour and a per-channel alpha mask, and the blend equation uses the second as the weight. That is what makes subpixel antialiasing possible in one pass — each of R, G, and B gets its own coverage.

`RenderingPass` (`renderer/text/mod.rs:35-47`) declares four variants: `Background = 0`, `SubpixelPass1 = 1`, `SubpixelPass2 = 2`, `SubpixelPass3 = 3`. The GLSL3 renderer uses two of them (`glsl3.rs:254-266`): one `DrawElementsInstanced` for backgrounds, one for glyphs. The GLES2

renderer uses two when dual-source blending is available and four when it is not (`gles2.rs:401-424`) — one background pass plus three separate subpixel passes, one per channel.

The fragment shader is a single file compiled for both dialects. `res/glsl3/text.f.glsl` is 73 lines, and its first 32 are `#if defined(GLES2_RENDERER)` aliasing: `gl_FragColor/gl_SecondaryFragColorEXT` versus `layout(location = 0, index = 0) out vec4 color` and `index = 1`. The `GL_EXT_blend_func_extended` extension is a hard `require` in the GLES2 branch — which is why the GLES2 renderer probes for it and falls back to three passes when it is absent. The GLES2-only shader (`res/gles2/text.f.glsl`, 57 lines) is what gets compiled in that case, selected at `gles2.rs:485`.

The background pass discards fully transparent cells (`bg.a == 0.0`) rather than blending them, which is what keeps a mostly-default screen cheap. Colour glyphs take a separate branch that reverts alpha premultiplication and forces alpha to 1.0; monochrome glyphs write the sampled RGB triple as the alpha mask and the flat foreground colour as the colour.

Rects

`renderer/rects.rs` handles everything that is a shape rather than a glyph: underlines, undercurls, strikeouts, the visual bell, the cursor, the OSC 133 gutter marks. `RectKind(:46)` has four members plus a count sentinel, and `RectRenderer::new(:258-317)` compiles four programs — normal, undercurl, dotted, dashed — into a fixed `[RectShaderProgram; 4]` at `:316`.

The dotted program is allowed to fail. `:266-272` catches a compile error, logs “Error compiling dotted shader: ... falling back to underline”, and substitutes the normal program. That is the only shader in the renderer with a fallback, and it exists because dotted underlines are drawn procedurally in the fragment shader — `res/rect.f.glsl` is 138 lines, by far the largest shader in the tree, and correspondingly the most likely to trip a driver.

`RenderLines::rects(:171)` maps cell flags to geometry and metrics (`:55-115`): `UNDERCURL` and `DOTTED_UNDERLINE` are given the entire descent area to draw in (`:98, :104`) rather than the underline thickness, because a wave and a dot pattern need vertical room that a straight line does not.

The order of a frame

`display/mod.rs:876-1000` is the draw order, and it is worth reading as a list because nothing else states it:

1. `make_current`, then `clear` with the background colour and window opacity (`:877-880`).
2. Background image, if configured (`:881-883`).
3. `sync_graphics` — apply pending texture uploads and frees (`:902`).
4. Inline graphics with `z < 0`, behind the text (`:903-909`).
5. `draw_cells` — the instanced text pass, during which underline/strikeout runs are accumulated into `RenderLines` and Unicode-placeholder tiles are collected (`:916-944`).
6. Inline graphics with `z >= 0`, on top of the text (`:948-954`).
7. Unicode-placeholder image tiles (`:956`).
8. Rects: underlines and strikeouts, then the cursor (`:973`), then the OSC 133 gutter marks (`:976-981`), then the visual bell (`:984-990`).

The placement of the bell last is not cosmetic — it is a full-screen translucent rectangle, and anything drawn after it would not be tinted.

Chapter 68 – The Parser You Did Not Write

Chapter 6 catalogued which escape sequences are supported. Chapter 54 argued about what conformance could even mean. This chapter is about where the parsing actually happens, which is mostly not in this repository.

The dependency

`zterminal_core/Cargo.toml:29:`

```
vte = { version = "0.15.0", default-features = false, features = ["std", "ansi"] }
```

`Cargo.lock` pins 0.15.0 exactly, with this checksum:

```
a5924018406ce0063cd67f8e008104968b74b563ee1b85dde3ed1f7cb87d3dbd
```

The crate is re-exported wholesale at `zterminal_core/src/lib.rs:26` as `pub use vte;`, and the `ansi` feature is what supplies the `Handler` trait, the dispatch tables, and the `Processor` that drives them. `zterminal_core` does not implement a state machine; it implements a trait.

The division matters for reading the codebase. `zterminal_core/src/term/mod.rs` is 3,944 lines and almost all of it is *semantics* — what `CUP` does to the cursor, what `ED` does to the grid, what `SGR 4:3` does to a cell's flags. The *syntax* — the DEC-style state machine that decides that `\x1b[1;2H` is a CSI with parameters 1 and 2 and final byte `H` — lives in `vte/src/ansi.rs`, 2,458 lines of it.

The size of the interface

The `Handler` trait begins at `vte-0.15.0/src/ansi.rs:495` and declares 71 methods. Every one has a default empty body, which is what allows a consumer to implement a partial terminal and what makes “unhandled” and “handled as a no-op” indistinguishable from the outside.

The dispatch tables are, in this version:

Dispatch	Arms	Location
CSI	44 arms covering 48 (<code>final byte</code> , <code>intermediates</code>) pairs	<code>ansi.rs:1529-1767</code>
ESC	14 arms, two of which are charset designators covering G0-G3 by intermediate	<code>ansi.rs:1799-1820</code>
OSC	11 arms	<code>ansi.rs:1329-1521</code>
SGR	64 arms	<code>ansi.rs:1834+</code>
Private modes (<code>NamedPrivateMode</code>)	17 named values	<code>ansi.rs:939-967</code>
Non-private modes (<code>NamedMode</code>)	2 named values	—

Dispatch	Arms	Location
----------	------	----------

Four of the CSI arms carry an alternate final byte, which is why 44 arms cover 48 pairs: `B/e` (:1561), `C/a` (:1571), `G/`` (:1579), and `H/f` (:1593). Those aliases are VT-era compatibility, and they are the sort of thing a hand-rolled parser gets wrong by omission.

Two of the 44 are guarded rather than plain patterns, and they are easy to miss when counting by eye because the `if` clause pushes the tuple off the shape the other arms have: `('c', intermediates) if next_param_or(0) == 0` at :1572 (DA1, primary device attributes) and `('W', [b'?']) if next_param_or(0) == 5` at :1580 (DECST8C, reset tab stops to every eighth column). Both are real dispatch arms and both call a `Handler` method. Dropping them is what gives the 42/46 figure this table carried in an earlier edition; the guard narrows when an arm fires, not whether it exists.

The 17 private modes are the whole DECSET/DECRST surface: `CursorKeys = 1`, `ColumnMode = 3`, `Origin = 6`, `LineWrap = 7`, `BlinkingCursor = 12`, `ShowCursor = 25`, `ReportMouseClicks = 1000`, `ReportCellMouseMotion = 1002`, `ReportAllMouseMotion = 1003`, `ReportFocusInOut = 1004`, `Utf8Mouse = 1005`, `SgrMouse = 1006`, `AlternateScroll = 1007`, `UrgencyHints = 1042`, `SwapScreenAndSetRestoreCursor = 1049`, `BracketedPaste = 2004`, `SyncUpdate = 2026`. Unrecognized mode numbers are not errors; they pass through as `PrivateMode::Unknown` and the handler ignores them.

What is not in the table

The dispatch tables are also a precise statement of what the terminal does not understand, and two entries are worth naming.

The only erase arms are `('J', [])` at `ansi.rs:1615` and `('K', [])` at `ansi.rs:1629`. There is no `('J', [b'?'])` and no `('K', [b'?'])` — so DECSED and DECSEL, the selective-erase forms that skip characters marked protected by DECSCA, are not dispatched at all. Nor is DECSCA itself: the only `q` arms are `('q', [b' '])` (DECSCUSR, cursor style) at :1711; the DECSCA form `('q', [b'"'])` is absent. A program that protects a region and then erases around it gets no protection and no erasure.

The OSC table is equally revealing. Its 11 arms are `0/2` (window title), `4` (indexed colour), `8` (hyperlink), `10/11/12` (fg/bg/cursor colour), `22` (mouse cursor shape), `50` (font), `52` (clipboard), `104` (reset indexed colour), and `110/111/112` (reset fg/bg/cursor). OSC 7 (working directory) is not there. Neither is OSC 133, and neither is OSC 1337. That absence is not an oversight in zterminal; it is the reason Chapter 69's scanner exists.

The shape of a ref-test failure, today

Chapter 63 describes the 45-case reference corpus. Measured on the current checkout, 37 of those 45 pass and 8 fail — `clear_underline`, `colored_reset`, `delete_lines`, `row_reset`, `saved_cursor`, `saved_cursor_alt`, `selective_erasure`, and `sgr` — deterministically across repeated runs. The command and the tail of its output:

```
$ cargo test -p zterminal_core --test ref
failures:
    clear_underline
    colored_reset
    delete_lines
    row_reset
    saved_cursor
    saved_cursor_alt
    selective_erasure
    sgr

test result: FAILED. 37 passed; 8 failed; 0 ignored; 0 measured; 0 filtered out
```

Five of those eight fail in a second, distinct way that is worth understanding because it is a property of the harness rather than of the terminal. The diff printer at [zterminal_core/tests/ref.rs:124-134](#) walks `0..grid.total_lines()` on the *golden* grid and indexes it. Thirty-three of the forty-five goldens serialize a `Storage` whose `visible_lines` is one smaller than the grid's `lines` — `selective_erasure` has `lines: 3` with `visible_lines: 2`, `row_reset` has `lines: 47` with `visible_lines: 46` — and indexing such a grid at its last line trips the debug assertion at [storage.rs:221](#). The comparison itself never indexes (Chapter 65: `PartialEq` compares `inner` and `len` only), so the mismatch is detected correctly and then the *report* panics while describing it. Twenty-eight of the thirty-three still pass, which is why it has never been visible before. Chapter 77 shows the further consequence: on a skewed golden the printer compares row $k+1$ of the golden against row k of the replay, so every line it prints before the panic names the wrong cells.

The content mismatch underneath is not what it looks like, and this paragraph stood wrong in an earlier edition. For `selective_erasure` — a forty-one-byte recording that is nothing but `ESC [ H`, three characters bracketed by DECSCA, `ESC [ ? 2 J`, a linefeed, the same three characters again, and `ESC [ ? 2 K` — the current build produces `ABC` on line 0 and `ABC` on line 1, while the golden has the second run at columns 0-2. Neither selective erase did anything, which is what the dispatch table above predicts — but *the golden agrees*, since it also holds all three characters intact on both lines. The three-column displacement is not selective erasure at all. It is a missing carriage return, the recording has none, and the case passes once one is restored. Chapters 75 and 76 establish that for all eight.

So this is not the failure mode Chapter 54 warns about. The goldens and the code have not drifted; the recorded stimulus underneath both of them was corrupted before this repository's first commit, and a golden suite has no way to see that, because a changed input produces exactly the signal a changed implementation produces.

Chapter 69 — The Scanner in Front of the Parser

Chapter 26 described inline graphics as a feature. This chapter is the plumbing, and the plumbing is unusual: `zterminal` does not feed the PTY byte stream to `vte`. It feeds it to a scanner, and the scanner feeds `vte` what is left.

Why there is a scanner at all

`zterminal_core/src/graphics/scanner.rs:1-13` states the problem:

```
///! `vte` 0.15 discards APC sequences (Kitty graphics) and its DCS hooks are
///! no-ops (Sixel), and large OSC payloads (iTerm2 inline images) can exceed
///! `vte`'s buffer. So this scanner sits in front of `vte`: it copies ordinary
///! bytes straight through and peels off the three inline-image protocols
```

Three protocols, three carriers, three reasons the upstream parser cannot deliver them:

- **Kitty graphics** — `ESC _ G ... ST`, an APC sequence. `vte` parses APC and throws the payload away.
- **Sixel** — `ESC P ... q ... ST`, a DCS with final byte `q`. `vte`'s DCS hooks exist but do nothing.
- **iTerm2** — `ESC ] 1337 ; File = ... ST`, an OSC. `vte` parses OSC, but a base64-encoded PNG routinely exceeds its payload buffer.

The scanner's `Kind` enum (`scanner.rs:24-40`) enumerates what it can be in the middle of: `Apc`, `Sixel`, `Item2`, `Shell`, `Tmux`, `Passthrough`. All state is carried across calls, so a sequence may span read buffers — which it will, since a 1 MB image arrives in 64 KB chunks.

`Passthrough` is the important one. Any DCS or OSC that is *not* one of the three image protocols is accumulated and then forwarded to `vte` intact. The scanner is a filter, not a replacement parser; everything it does not claim reaches the real parser byte for byte.

Two riders

Two things travel through the same mechanism for reasons that have nothing to do with images.

`Kind::Shell` (`scanner.rs:32-34`) is OSC 133, “consumed by the scanner; never forwarded to `vte`.” Chapter 68 established that `vte`'s OSC table has no 133 arm, so shell integration would otherwise be invisible. Chapter 70 is what it does with it.

`Kind::Tmux` (`scanner.rs:35-37`) is the tmux passthrough envelope `ESC P tmux; ... ST`, in which inner escapes are doubled. The scanner un-doubles them and *re-scans* the unwrapped bytes, which is what makes an image or an OSC 133 mark emitted inside tmux work at all. `is_tmux_prefix` at `:43-45` is the four-state prefix matcher that decides whether a DCS body is a tmux envelope before enough of it has arrived to be sure.

Kitty

`graphics/kitty.rs` is 462 lines with 14 tests. The header comment at :4-10 enumerates the supported actions: transmit (`a=t`), transmit-and-display (`a=T`), place a previously transmitted image (`a=p` with `i=`), and the support query (`a=q`). Delete (`a=d`), animation frames (`a=f`), and playback control (`a=a`) are all present in the dispatch at :50-170 and in the response enum.

`KittyResponse` (`graphics/mod.rs:55-78`) is the parser's whole output vocabulary: `None`, `Image`, `Report` (the `a=q` handshake string, written back to the application), `Delete`, `Store` (for `U=1` placeholder images, keyed by protocol id), `Frame` (an animation frame with its inter-frame gap in milliseconds), and `Animate`. Deletion is typed rather than stringly: `GraphicDelete::All { free }` and `GraphicDelete::Id { id, free }` (`mod.rs:83-88`), where `free` corresponds to the uppercase `d=` forms and means "free the image data, not just the placement."

Placement, and what happens when the screen scrolls

`GraphicPlacement` (`graphics/mod.rs:117-128`) is the anchor:

```
pub struct GraphicPlacement {
    pub id: GraphicId,
    pub line: i64,
    pub column: usize,
    pub columns: u16,
    pub rows: u16,
    pub z: i32,
    pub source_id: Option<u32>,
}
```

`line` is an *absolute* line in `Grid::total_scrolled` space, not a viewport row. That single decision is what makes images scroll with the buffer for free: the renderer converts absolute anchor to viewport row at draw time (`renderer/mod.rs:195-200`), and nothing has to be touched when the grid moves.

`z` is the stacking order, negative meaning behind the text, which is why Chapter 67's frame order draws graphics twice.

`source_id` is `Option<u32>` with the comment "None for Sixel / iTerm2 which carry no id" — those two protocols have no addressing scheme, so a targeted delete cannot name them.

Eviction is `evict_above` (`mod.rs:403-423`), called with the smallest absolute line still in the buffer. It drops placements that have scrolled entirely past (`p.line + p.rows as i64 > oldest_line`), and then — only if something was actually dropped — garbage-collects images no placement references any more, pushing their ids onto `pending_frees` for the renderer to delete textures for.

The live set is deliberately larger than the placement set (`mod.rs:409-413`): images referenced by `virtual_images` (Unicode placeholders) and by `animations` survive eviction even with no placement, because a placeholder image is addressed by cells rather than by a placement and an animation's frames must outlive any one frame's placement.

Unicode placeholders

`graphics/placeholder.rs` is 90 lines implementing Kitty's placeholder protocol, and its header (:1-10) is the clearest available description of the mechanism: an image is transmitted with `U=1` and an id, then *placed* by printing cells that hold `U+10EEEE` (`PLACEHOLDER`, :13) whose foreground colour encodes the image id and whose combining diacritics encode which tile of the image's `c×r` grid the cell shows.

`ROWCOLUMN_DIACRITICS: [u32; 250]` at :18 is the fixed table, and the comment notes it is "Fetched from kitty's `gen/rowcolumn-diacritics.txt`" — a vendored constant from a reference implementation rather than a derived one, which is the right choice for a table whose only definition is another program's source.

The payoff is that placeholder cells are *ordinary cells*. They scroll, they get selected, they reflow, they land in scrollback, and they need no placement record at all — which is why the renderer collects them during the normal cell loop (`display/mod.rs:918-923`) and draws them in a separate pass afterwards (`:956`).

Sixel and iTerm2

`graphics/sixel.rs` is 271 lines with 4 tests, and its header (`:1-7`) lists the covered surface: raster attributes (`"Pan;Pad;Ph;Pv"`), colour register definition in both RGB and HLS, colour selection (`#Pc`), run-length (`!Pn`), graphical carriage return (`$`) and newline (`-`), and the sixel data bytes themselves, `0x3f..=0x7e` (`is_sixel, :14-16`), six vertical pixels per byte.

`graphics/iterm2.rs` is 70 lines with 2 tests and is the simplest of the three, because the protocol is: strip an optional `File=`, split at the first colon (`:19-21`, with the observation that base64 never contains one), filter out whitespace, base64-decode, and hand the bytes to the `image` crate. Every format that crate decodes is therefore supported, which is the reverse of the usual situation where the terminal enumerates formats.

Across the six graphics files there are 43 tests: `scanner.rs` 14, `kitty.rs` 14, `mod.rs` 6, `sixel.rs` 4, `placeholder.rs` 3, `iterm2.rs` 2. The concentration in the scanner is correct — it is the only one of the six that has to be right about partial input.

Chapter 70 – OSC 133 in Practice

Chapter 27 introduced shell integration. This chapter is what the marks turn into, which is a data structure that several later features are built on top of and that none of them could have been built without.

Four marks

`zterminal_core/src/shell.rs:1-16` names them:

```
///! * `OSC 133 ; A` - prompt start (a fresh prompt is about to be drawn)
///! * `OSC 133 ; B` - command start (end of prompt; user input begins)
///! * `OSC 133 ; C` - command executed (output begins)
///! * `OSC 133 ; D [; exit]` - command finished, with an optional exit code
```

This is the FinalTerm sequence, adopted by iTerm2 and now by most terminals that do anything semantic with the prompt. `vte` does not dispatch it (Chapter 68), so the scanner of Chapter 69 peels it out and drives the `ShellHandler` methods on the terminal directly. The module comment at `:14-16` is explicit about the consequence: marks are anchored to *absolute* grid lines — `total_scrolled` plus the viewport row — “exactly like inline graphics, so they scroll with the buffer and are evicted once they fall out of scrollbar.”

The block

`CommandBlock` (`shell.rs:58-96`) is what four marks accumulate into. Every `*_line` field is an absolute line:

- `prompt_line: i64` — set at `A`, and the block’s identity for the rest of its life.
- `command_line: Option<i64>` and `command_column: usize` — set at `B`.
- `output_line: Option<i64>` — set at `C`.
- `end_line: Option<i64>` and `exit_code: Option<i32>` — set at `D`.
- `command: Option<String>` — “captured from the grid at output start,” which is to say the terminal reads the typed command back off its own screen at the `C` mark rather than the shell reporting it.

Two derived predicates carry most of the UI. `failed()` at `:100-102` is `matches!(self.exit_code, Some(code) if code != 0)`; `running()` at `:105-107` is `output_line.is_some() && end_line.is_none()`. A block that is still at the prompt is neither.

Two timestamp fields are filled in at the `C` mark rather than at `A`: `started_unix_ms` and, at `D`, `duration_ms`. The comment at `:86-89` gives the reason — “Zero until the command actually runs, so a block abandoned at the prompt carries no bogus timestamp.” Someone who opens a shell, types nothing, and closes it produces a block with no time in it rather than a zero-length command at the epoch.

The hashes

`output_hashes: Vec<u64>` (`shell.rs:79`) is the field the fan-out divergence machinery of Chapter 61 reads, and its doc comment states the invariant that makes that feature possible:

```
/// [ `line_hash` ] of each completed output line of this block's `C`..`D`
/// span, index-aligned with the *ordinal*: entry `k` is the line at
```

```
/// `output_line + k`. Counting from the `C` mark rather than from the
/// prompt is what makes ordinals comparable across panes whose prompts
/// have different widths or wrap differently.
```

Counting from `C` rather than from `A` is the entire reason a divergence report can compare a pane on a host with a 30-character prompt against a pane on a host with a 12-character one.

`line_hash` (`shell.rs:31-41`) is FNV-1a 64 over the line with trailing blanks trimmed, and the choice is justified rather than assumed (`:24-30`):

```
/// Non-cryptographic by design: this only ever decides "did two panes print the
/// same line". A collision costs a missed divergence (two different lines read
/// as agreeing); it can never invent a divergence that isn't there
```

The bound is `pub const MAX_HASHED_LINES: usize = 2000 (:22)`, after which a block sets `hashes_truncated` and stops appending. A command that prints a million lines cannot grow a per-block vector without limit.

The queue that does not block the parser

`shell.rs` runs on the PTY thread. Chapter 59 explains why capture does not happen there — materializing a large block's output from grid cells would stall the parser — so `Term::take_finished_blocks` queues finished blocks and the app drains them on its wakeup pass.

That queue is bounded: `const MAX_PENDING_CAPTURES: usize = 256` at `zterminal_core/src/term/mod.rs:59`, with the reasoning at `:55-58` — the app drains every wakeup, so the queue only fills if the app has stopped draining entirely, “at which point dropping the oldest pending captures is the right trade against growing a queue on the PTY thread without bound.”

What the marks are used for

The `ShellIntegration` API (`shell.rs:152-385`) is short, and the shape of it is the shape of the features:

- `line_mark (:293)` and `gutter_marks (:331)` — the coloured prompt gutter, drawn as rects in Chapter 67's frame order (`display/mod.rs:976-981`).
- `adjacent_prompt (:362)` with a `PromptSeek` direction — jump to previous/next prompt, bound to `Cmd+arrow` on macOS and `Ctrl+Shift+arrow` elsewhere.
- `block_at_prompt (:236)` and `previous_run (:244)` — the resident half of the diff and trend features of Chapter 59.
- `current_hashes (:271)` and `set_divergent_ordinals (:281)` — the read and write sides of Chapter 61's consensus pass.
- `set_ledger_id (:219)` — the back-reference written after a block is stored durably.
- `evict_above (:375)` — the same absolute-line eviction as graphics.

The scripts

Shell integration only works if the shell emits the marks, so three scripts ship in `extra/shell-integration/`: 53 lines of bash, 51 of fish, 50 of zsh, plus a 54-line README. They are near-identical in structure, and the zsh one is representative.

The emitter handles tmux by wrapping (`zterminal-shell-integration.zsh:25-31`):

```
__zterminal_osc133() {
  if [[ -n ${TMUX:-} ]]; then
    printf '\ePtmux;\e\e]133;%(a\e\\' "$1"
  else
    printf '\e]133;%(a' "$1"
  fi
}
```

That is the other end of `Kind: Tmux` from Chapter 69: the doubled `\e\e` is the escaping tmux requires, and the scanner un-doubles it.

`A` and `D` are emitted from `precmd`, in that order — `D` for the command that just finished, then `A` for the prompt about to be drawn — with a guard so the first prompt of a session emits no `D` (:34-40). `C` comes from `preexec` (:43-45). `B` is not a hook at all; it is appended to the prompt string itself (:50):

```
PS1="$ {PS1}%{ $(__zterminal_osc133 'B' )%}"
```

The `%{...}%` wrapper is load-bearing and the comment says so: it tells zsh the bytes are zero-width so the cursor column stays correct. Without it every prompt is mis-measured by the length of the escape sequence, and line editing drifts.

Three ordinary guards open the file: interactive shells only, never load twice, and `autoload -Uz add-zsh-hook` before using it (:16-20). None of that is interesting except that all three are the kind of omission that turns a shell integration script into a bug report about scp hanging.

What is not tracked

OSC 7 — the sequence a shell uses to report its working directory — is not dispatched by `vte` (Chapter 68) and is not intercepted by the scanner. Chapter 59’s ledger records a cwd anyway, and `zterminal/src/ledger.rs:25-29` is explicit that it comes from somewhere else: “The recorded cwd is the pane’s live foreground-process path, the same source the Recent Dirs tab harvests. OSC 7 is not tracked, so this is a read of the process table at capture time rather than something the shell reported.”

That is a real difference in fidelity, and stating it is better than letting a reader assume the shell told the terminal where it was.

Chapter 71 – Selection and the Clipboard, Concretely

Chapter 13 covered selection as behaviour and Chapter 38 covered copy as a workflow. This chapter is the model underneath: four selection kinds, an anchor pair that is normalized lazily, and a clipboard policy that refuses more than it allows.

Four kinds and a struct

`zterminal_core/src/selection.rs:93-98:`

```
pub enum SelectionType {
    Simple,
    Block,
    Semantic,
    Lines,
}
```

`Selection` itself is a struct holding a `ty` and a `Range<Anchor>` (`selection.rs:118-122`), where `Anchor { point, side }` (:19-23). The doc comment above it still describes it as an enum and links to `enum.Selection.html#method.*` (:100-117) — inherited Alacrity documentation that stopped being true when `ty` was introduced, and a reminder that comments are not checked by anything.

`Side` is not its own type: `zterminal_core/src/index.rs:14` is `pub type Side = Direction;`, with the inversion at :27-28. A selection anchor therefore says “this point, on its left edge” or “this point, on its right edge,” which is what lets a drag that starts in the middle of a character behave the way a user expects rather than snapping.

The mapping from input to kind is in `zterminal/src/input/mod.rs:735-757`: one click is `Simple` (:746), one click with Control is `Block` (:744), two clicks is `Semantic` (:751), three is `Lines` (:755). The multi-click window is `CLICK_THRESHOLD: Duration = Duration::from_millis(400)` (`input/mod.rs:72`).

Normalization is not stored

A selection whose start is after its end is legal and common — every drag upward produces one. Rather than normalizing on update, the code swaps on read. There are four `mem::swap` sites: `rotate` (:148-150), `is_empty` (:197-199), `intersects_range` (:232-234), and `to_range` (:279-281).

The advantage is that the anchor the user is dragging stays the anchor, so extending a selection past its origin and back again does not lose the origin. The cost is that four functions have to remember.

`to_range` (:279) is where a selection becomes coordinates. It clamps with `Boundary::Grid` (:287-288) and returns `None` when `end.point.line < term.topmost_line()` (:284-286) — a selection entirely scrolled out of the buffer resolves to nothing rather than to a clamped stub.

Rotation, and when a selection dies

`rotate` (`selection.rs:137-191`) is what keeps a selection attached to its text while the buffer moves under it. It is called from three places: `scroll_down_relative` with `-(lines as i32)` (`term/mod.rs:888`), `scroll_up_relative` with `+lines` (:914), and on resize with `-delta` (:825).

It returns `None` — meaning “this selection no longer exists” — in two cases, each with its own comment: when the end is in the same region and the start rotates out (:156-159), and when the end has overtaken the start (:175-178). Clamping to region edges resets column and side for non-block selections only (:162-168, :181-187), because a block selection’s columns are independent of its lines and must survive vertical clamping intact.

Resize has a blunter rule. `term/mod.rs:817-819` drops the selection outright when the column count changed:

```
self.selection = None;
```

Reflow (Chapter 72) moves text between lines in ways a line-delta cannot express, so there is no honest way to rotate through it.

Semantic selection

The default word separators, `zterminal_core/src/term/mod.rs:47`:

```
pub const SEMANTIC_ESCAPE_CHARS: &str = ",|`|:|\"' ()[]{}<>\t";
```

Seventeen characters, including both a space and a tab, and — second in the list — U+2502 BOX DRAWINGS LIGHT VERTICAL. That last one is there so that double-clicking inside a `tmux` or `ranger` pane border selects the word rather than the word plus the border. It is configurable at `selection.semantic_escape_chars` (`zterminal/src/config/selection.rs:6-19`), plumbed to the core at `ui_config.rs:150`.

`range_semantic` (`selection.rs:298-317`) has a special case that is easy to miss: when start and end are the same point, it first tries `term.bracket_search`. The pairs are `BRACKET_PAIRS: [(char, char); 4] = [(('(', ')'), ('[', ']'), ('{', '}'), ('<', '>'))]` (`term/search.rs:19`) and the search is at `search.rs:472-513`. Double-clicking a bracket selects to its match; double-clicking anything else expands to word boundaries.

The word walkers (`semantic_search_left:517`, `semantic_search_right:533`) sit on top of `inline_search_left/inline_search_right (:541, :565)`, which stop at a *hard* line break — `column == last_column && !WRAPLINE (:550-552, :584-586)` — and skip wide-character spacers. A word that soft-wraps across two rows is one word; a word at the end of a genuinely terminated line is not joined to the next line’s first word.

Turning cells back into text

`selection_to_string` (`term/mod.rs:552`) branches on kind. Block selections `trim_end()` every row and join with `\n` (:560-567). `Lines` appends a trailing newline (:570). Simple and semantic go through unmodified (:573).

`line_to_string` (:595-656) is where the details live. It truncates to `min(grid_line.line_length(), cols.end + 1)` (:604) so trailing blank cells never reach the string; it expands a selected wide-char spacer backwards (:607-609); it collapses tab runs (:611-626); it skips spacers and appends zero-width combining characters (:628-636).

The condition that decides whether a newline is emitted (:639-644) is the one worth quoting, because getting it wrong is what makes copied output from other terminals unusable:

```

if cols.end >= self.columns() - 1
  && (line_length.0 == 0
    || !self.grid[line][line_length - 1].flags.contains(Flags::WRAPLINE))
{
  text.push('\n');
}

```

A newline goes in only when the row reached the last column *and* was not soft-wrapped. Copying a wrapped 300-character command line out of an 80-column window yields one 300-character line, not four fragments.

The clipboard is two buffers, and one of them often does not exist

`zterminal/src/clipboard.rs` is 84 lines total and has no tests. It holds a `clipboard` and an `Option<selection> (:15-18)`; `ClipboardType (term/mod.rs:2736-2739)` has exactly two variants. The backend is `copypasta`, declared as `0.10.1 (zterminal/Cargo.toml:35)` and resolved to 0.10.2 in the lockfile.

Per platform:

Platform	Clipboard	Selection
Wayland	from external display (<code>clipboard.rs:21-32</code>)	yes, primary selection
X11	<code>ClipboardContext</code>	<code>X11ClipboardContext<X11SelectionClipboard> (:46-50)</code>
macOS / Windows	<code>ClipboardContext</code>	<code>None (:43-44)</code>
other	<code>NopClipboardContext (:36-38)</code>	<code>None</code>

Storing to `Selection` where no selection provider exists is a silent no-op — (`ClipboardType::Selection, None`) => `return`, at `:61` — while *loading* falls back to the regular clipboard instead (`:71-74`). That asymmetry is right: on macOS there is no primary selection to write to, but a middle-click paste should still paste something.

OSC 52, refused by default

The clipboard escape sequence is the one place a remote program can reach into the user’s machine, and the policy is a four-valued enum (`term/mod.rs:389-405`): `Disabled`, `OnlyCopy` (marked `#[default]` at `:399-400`), `OnlyPaste`, `CopyPaste`. The doc comment at `:396-398` calls `OnlyCopy` “a compromise between entirely disabling it (the most secure) and allowing `paste` (the less secure).”

`clipboard_store (:2126)` is gated at `:2127-2130` and logs “Denied osc52 store” at `debug!` when refused. `clipboard_load (:2147)` is gated at `:2148-2151` and, with the default of `OnlyCopy`, is refused — so out of the box a program on the far end of an SSH session can write your clipboard but cannot read it.

There is a second guard in the frontend that is easy to overlook: `zterminal/src/event.rs:4399-4409` drops both directions when the window is unfocused. A background pane cannot touch the clipboard even with the permissive policy set.

`selection.rs` carries 16 tests. `clipboard.rs` carries none — it is thin platform glue, and `new_nop()` (:36) exists specifically so that everything above it can be tested without one.

Chapter 72 – Scrollback, Reflow, and Search

Chapter 18 covered vi mode and search as an interface. This chapter is the search engine, the reflow that moves the text it searches, and the bounds on both.

How much scrollback

`zterminal/src/config/scrolling.rs:7:`

```
pub const MAX_SCROLLBACK_LINES: u32 = 100_000;
```

enforced in `Deserialize` at `:45-48`, with the error string `"exceeded maximum scrolling history ({lines}/{MAX_SCROLLBACK_LINES})"`. The default is 10,000 (`scrolling.rs:32-36`), duplicated in the core at `term/mod.rs:379`. The scroll multiplier default is 3 (`scrolling.rs:19`).

The cap is a deserialization guard only. Nothing in `zterminal_core` enforces it, so a caller constructing a `Config` directly can ask for more — which the ref-test harness does, and which is the correct division: the config layer decides what a user may write in a file, the core decides what is representable.

Reflow

`zterminal_core/src/grid/resize.rs` is 389 lines and it does reflow wrapped lines. `Grid::resize(reflow, lines, columns)` (`:14-36`) does lines first, then columns, taking the cursor template out and putting it back around the whole operation (`:20, :35`).

Growing lines pulls from history before scrolling: `from_history = min(history_size, lines_added)` (`:55`), remainder by scrolling up (`:58-61`), both cursors moved down by `from_history` (`:64-65`). Dragging a window taller therefore reveals what scrolled off, rather than inserting blank lines below.

`grow_columns` (`:101-242`) is where the WRAPLINE flag is manipulated, and the sequence matters: the flag is *removed* from the previous row before appending (`:131-134`) and *re-inserted* only if the row it pulled from still has cells (`:204-207`). `shrink_columns` (`:245-388`) is the mirror: `row.shrink(columns)` returns the overflow (`:273`), WRAPLINE is inserted on every row that lost cells (`:316-319`), and a lingering WRAPLINE on carried-over cells is removed at `:327-330`. History is capped during the shrink at `reversed.truncate(self.max_scroll_limit + self.lines)`; (`:368`).

The `should_reflow` predicate (`:103-106`) is the whole condition: reflow is on, the row is non-empty, it is shorter than the new width, and its last cell carries WRAPLINE.

Wide characters get four special cases across the two functions (`:136-143, :149-159, :304, :309`) because a two-cell character cannot straddle a wrap boundary and has to be pushed whole to the next row with a spacer left behind.

The alternate screen never reflows, and the way that is expressed is worth reading twice (`term/mod.rs:812-814`):

```
let is_alt = self.mode.contains(TermMode::ALT_SCREEN);
self.grid.resize(!is_alt, num_lines, num_cols);
self.inactive_grid.resize(is_alt, num_lines, num_cols);
```

The two arguments are inverted relative to each other, so whichever of the two grids is the *primary* one gets reflow and whichever is the *alternate* one does not — regardless of which is currently active. `vim` resized mid-edit does not have its display reflowed underneath it; the shell scrollbar behind `vim` does.

The search engine

`zterminal_core/src/term/search.rs` is 1,251 lines with 33 tests, and the engine is `regex-automata`'s lazy hybrid DFA — declared as `0.4.3` in `zterminal_core/Cargo.toml:27`, locked at 0.4.16.

There are four DFAs per search (`search.rs:25-30`):

```
pub struct RegexSearch {
    left_fdfa: LazyDfa,
    left_rdfa: LazyDfa,
    right_rdfa: LazyDfa,
    right_fdfa: LazyDfa,
}
```

Searching a terminal is bidirectional, and a regex engine gives you one direction. Finding the previous match therefore needs a genuinely reversed automaton, not a re-scan: `LazyDfa::new` sets `thompson.reverse(true)` for `Direction::Left (:99-102)`, and the scan loop reads bytes backwards out of the buffer (`:325-328`):

```
let byte = match regex.direction {
    Direction::Right => buf[i],
    Direction::Left => buf[utf8_len - i - 1],
};
```

Each direction needs two automata because finding a match requires locating both of its ends: `regex_search_left (:228-239)` runs `left_fdfa` to find the start and then `left_rdfa` back to the origin to find the end, and `regex_search_right (:244-255)` is the mirror.

Smart case is `case_insensitive(!has_uppercase) (:39-40)` — a lowercase pattern matches either case, a pattern with any uppercase character is exact.

Bounds, because a regex can be adversarial

Two limits are set at build time (`:41-44`): `minimum_cache_clear_count(Some(3))` and `minimum_bytes_per_state(Some(10))` on the DFA config, and `nfa_size_limit(Some(max_size))` on the Thompson config, where `max_size` is the cache capacity. The comment at `:37-38` cites the rust-lang/regex meta engine as the source of the numbers rather than deriving them, which is the honest thing to do with tuning constants.

When those limits are hit, `regex_search (:260-269)` swallows the error and logs "Regex exceeded complexity limit" at `warn! (:264)`. A pathological pattern produces no match and a log line, not a hung UI. Two tests pin this: `nfa_compile_error (:1106-1109)` asserts `[0-9A-Za-z]{9999999}` fails to build, and `runtime_cache_error (:1111-1119)` asserts a 9,999-character term with `[0-9A-Za-z]{9999}` returns `None`.

Line budgets are layered on top:

Constant	Value	Location	Purpose
<code>MAX_SEARCH_LINES</code>	100	<code>zterminal/src/display/hint.rs:20</code>	clamps hint matching to ± 100 lines around the viewport
<code>MAX_SEARCH_WHILE_TYPING</code>	<code>Some(1000)</code>	<code>zterminal/src/event.rs:75</code>	incremental search bound
<code>MAX_SEARCH_HISTORY_SIZE</code>	255	<code>event.rs:78</code>	search history depth
<code>TYPING_SEARCH_DELAY</code>	500 ms	<code>event.rs:72</code>	delay before the unbounded re-search

The two-tier incremental search

`update_search` (`event.rs:4000-4024`) recompiles all four DFAs on *every keystroke* (`:4017`) and then searches with the 1,000-line budget. That is not as expensive as it sounds — a lazy DFA builds states on demand, so a pattern that matches immediately never materializes most of its automaton.

The interesting part is what happens on a miss. `goto_match(limit)` (`event.rs:4051-4102`) branches at `:4087-4098`: with a limit and no match, it schedules a delayed unlimited re-search on `Topic::DelayedSearch` after `TYPING_SEARCH_DELAY`; without a limit and no match, it resets the viewport.

So typing a pattern searches 1,000 lines instantly, and if you stop typing for half a second the terminal quietly searches the entire 100,000-line buffer and jumps if it finds something. Neither behaviour alone is right — an unbounded search on every keystroke stutters, a bounded search that never retries silently lies about what is in the buffer.

The budget is dropped entirely when the buffer is smaller than it: `limit.filter(|&limit| limit <= self.terminal.total_lines())` (`:4058`). `confirm_search` (`:3527-3541`) forces `goto_match(None)` if a delayed search was still pending, so pressing Enter never confirms a match the bounded search had not yet found.

Search and vi mode

The two are coupled in five specific places, and the coupling is asymmetric.

`SearchState.origin` (`event.rs:2964-2990`) is the vi cursor when vi mode is on, and the viewport top or bottom otherwise, depending on direction (`:3455-3471`). Viewport restoration on reset is vi-only, and the comment at `:4035-4040` says why: without vi mode “our origin is always at the current display offset instead of at the vi cursor position which we need to recover to.”

On a hit, `goto_match` branches at `:4067-4073`: vi mode moves the cursor with `vi_goto_point`, non-vi merely scrolls the match into view and highlights it.

`cancel_search` (`:3544-3555`) is where the difference shows. Outside vi mode, cancelling converts the focused match into a `SelectionType::Simple` selection (`:3552-3553`) and copies it to

`ClipboardType::Selection (:3554)`. Inside vi mode it restores the pre-search state instead. `confirm_search` outside vi mode is just a cancel (`:3527-3532`).

`start_seeded_search (:3481-3524)` — search for what is selected — escapes regex metacharacters one character at a time against the class `'$' | '(' | '..='+' | '?' | '[' | '..='^' | '{' | '..='}'` (`:3490-3492`), then does a two-hop `search_next` to land the cursor on the right occurrence rather than on the selection it started from.

Finally, a one-line interaction that would otherwise be a bug: `event.rs:3176` guards the selection/vi-cursor coupling with `66 !self.search_active()`, so dragging a selection while the search bar is open does not drag the vi cursor with it.

Chapter 73 – The Split Tree

Chapter 24 described native splits as a feature and Chapter 31 compared them to tmux panes. This chapter is the tree, the geometry, and the traversal — about 500 lines of `zterminal/src/pane.rs`, and one function in it that is more interesting than the rest.

Leaf and split

`pane.rs:239-241`:

```
enum Node {
    Leaf(PaneId),
    Split { axis: SplitAxis, ratio: f32, first: Box<Node>, second: Box<Node> },
}
```

A binary tree, i3-style. `SplitAxis (:42-46)` has two members with a naming convention that is worth stating because it trips everyone: `Vertical` divides the *width*, so panes sit side by side — the iTerm “Split Vertically” and `Cmd+D` result — and `Horizontal` divides the height so panes stack.

`Panes (:480-491)` holds `slots: Vec<Option<Pane>>`, the `root: Node`, a `focused: PaneId`, and a `zoomed: bool`. Panes live in a slot vector indexed by id rather than inside the tree, so the tree carries only structure and every operation on it is a small recursion over `PaneIds`.

The module comment (`:1-7`) is candid about scope: “[`Panes`] tiles a set of them inside one window along a single axis (M1); arbitrary nesting is a later milestone” — which the code has since outgrown, since `split_leaf (:246)` replaces a leaf with a `Split` containing it, producing arbitrary nesting, and `split_nests_and_lays_out_recursively (:787)` is the test that pins it.

Geometry

`Node::layout (:315-341)` assigns a rectangle to every leaf in one recursion:

```
SplitAxis::Vertical => {
    let avail = (rect.w - DIVIDER_PX).max(0.);
    let w1 = (avail * ratio).round();
    let w2 = avail - w1;
```

`pub const DIVIDER_PX: f32 = 1.0 (:28)` is subtracted from the available extent *before* the ratio is applied, and the second child gets `avail - w1` rather than `(avail * (1 - ratio)).round()`. Both details are there to make the arithmetic exact: the divider is never double-counted, and rounding error accumulates into one pane rather than leaving a one-pixel gap in the middle.

Each pane carries its own `SizeInfo` and `ScreenRect (:102-105)` — “The pane’s own grid geometry (its sub-rectangle, not the window’s)” — and `Pane::resize (:179)` is what pushes a new rectangle down into the pane’s `Term` and its PTY. Every pane is a full terminal: its own grid, its own PTY, its own I/O thread, its own search state, its own trigger scan watermark (`:87-116`).

Directional focus

`focus_dir (:615-624)` delegates to `neighbor (:418-448)`, which is the one function in the file worth reading closely, because “move focus right” has no obvious definition once panes are not a grid.

The algorithm is: for every other pane, compute a *gap* along the direction of travel and an *overlap* along the perpendicular axis. Reject any candidate whose gap is negative beyond `EPS` (it is behind you) or whose overlap is zero (it is not across from you at all). Among the rest, prefer the smallest gap; break ties on the largest overlap:

```
let better = match best {
  None => true,
  Some( (_, bgap, bov) ) => gap < bgap - EPS || ((gap - bgap).abs() <= EPS && ov > bov),
};
```

`const EPS: f32 = 1.0 (:419)` — one physical pixel of tolerance, which is what makes the comparison robust against the rounding in `layout`.

The tie-break on overlap is what makes it feel right. Moving right out of a tall pane that faces two stacked panes selects the one that shares more of your vertical extent, which is the one whose text is next to the text you were looking at.

`move_focused (:628-635)` reuses the same neighbour computation and then `swap_leaves` — moving a pane is finding the pane you would have focused and trading places with it, so the two operations can never disagree about what “right” means.

Resize and zoom

`resize_focused (:639-648)` maps a direction to an axis and a signed step of `const STEP: f32 = 0.04`, then calls `Node::resize (:391)`, which walks to the nearest ancestor split of the matching axis and nudges its ratio, clamped to `(*ratio + signed).clamp(0.05, 0.95) (:403)`. The clamp means a pane can be squeezed to five percent of its parent but never to zero — there is no minimum size in cells, only a minimum fraction.

`toggle_zoom (:653-660)` sets a boolean. The layout is not recomputed: the comment at `:714-715` explains that when zoomed, the focused pane fills the window and “the rest keep their tiled rects (they are simply not drawn) so unzoom is instant.” Splitting or closing calls `unzoom (:668)` first, because both change the layout the un-zoomed state is supposed to return to.

Closing

`close_focused (:570)` and `close_id (:583)` both funnel into the tree operation that `close_collapses_parent_into_sibling (:830)` pins by name: when a leaf is removed, its parent `Split` is replaced by its remaining sibling. There is no rebalancing and no ratio redistribution — the sibling simply inherits the parent’s rectangle, which is what makes closing a pane look like the divider was dragged all the way over.

`Drop for Pane (:450-455)` sends `Msg::Shutdown` to the pane’s I/O thread. That is the entire teardown, and it is in `Drop` rather than in `close_*` so that dropping a whole `Panes` — closing a window — cannot leak a thread.

Saved layouts

`LayoutNode (:51-65)` is the serializable mirror of `Node`: same shape, but each leaf carries a `cwd` and an optional `cmd` instead of a runtime id. That is the whole difference between a live tree and a saved one, and it is why `build_layout_node (:460)` can rebuild a layout by walking the saved tree and calling a spawn closure per leaf, returning `None` if any spawn fails rather than leaving a half-built window.

`leaf_count (:69-75)` exists so the layout picker can show how many panes a saved layout will open — a two-line recursion, and an example of the kind of thing that is cheap to add once the structure is a tree rather than a list.

`pane.rs` carries 10 tests.

Chapter 74 – Keys, Modifiers, and the Bytes They Produce

Chapter 7 covered the input path and Chapter 16 covered bindings. This chapter is the encoder: what happens between a key event and the bytes on the PTY, and why that is more than a lookup table.

Two layers

A key press is resolved in two stages. First `process_key_bindings` (`zterminal/src/input/keyboard.rs:178`) looks for a binding whose key, modifiers, and mode all match. Only if none fires does `build_sequence` (`keyboard.rs:295-362`) synthesize an escape sequence.

The comment at `bindings.rs:506-507` states the division: “key sequences which are not present here, like F5-F20, PageUp/PageDown codes are built on the fly in `input/keyboard.rs`.” Bindings cover the keys whose encoding depends on terminal *mode*; the builder covers the ones whose encoding is mechanical.

Modes as a bitmask

`BindingMode` (`zterminal/src/config/bindings.rs:875-883`) is a `u8` with seven flags:

```
const APP_CURSOR      = 0b0000_0001;
const APP_KEYPAD      = 0b0000_0010;
const ALT_SCREEN      = 0b0000_0100;
const VI              = 0b0000_1000;
const SEARCH          = 0b0001_0000;
const DISAMBIGUATE_ESC_CODES = 0b0010_0000;
const REPORT_ALL_KEYS_AS_ESC = 0b0100_0000;
```

A binding carries both a `mode` and a `notmode` (`bindings.rs:35, :38`), so it can require a mode, forbid a mode, or ignore it. `is_triggered_by` (`:54`) is the match, and the empty-set convention at `:71-72` — an empty `mode` means “all modes” — is what lets most bindings say nothing about mode at all.

This is what makes the arrow keys work. `bindings.rs:522-527` binds each arrow to its application-cursor form, guarded by `+APP_CURSOR, ~VI, ~SEARCH`:

```
ArrowUp,    +BindingMode::APP_CURSOR, ~BindingMode::VI, ~BindingMode::SEARCH;
    Action::Esc("\x1bOA".into());
```

so `ESC O A` in application cursor mode, and the builder’s default `ESC [ A` otherwise. `Home/End` get `ESC O H` / `ESC O F` the same way (`:522-523`).

The F1-F4 bindings (`:529-532`) carry four negations each — `~VI, ~SEARCH, ~REPORT_ALL_KEYS_AS_ESC, ~DISAMBIGUATE_ESC_CODES` — because the legacy `ESC O P..ESC O S` encodings must yield to the Kitty protocol when the application has asked for it. `Shift+Tab` (`:533`) is `ESC [ Z`, `Alt+Shift+Tab` (`:534`) is `ESC ESC [ Z`, `Backspace` (`:535`) is `\x7f` and `Alt+Backspace` (`:536`) is `ESC \x7f` — all with the same negations.

How many bindings

Counted from the source, at the four `bindings!` invocations in `zterminal/src/config/bindings.rs`:

Table	Entries	Lines
<code>default_mouse_bindings</code>	3	499-503
<code>default_key_bindings</code> (mode-independent core)	100	510-615
<code>common_keybindings</code> (non-macOS)	18	625-644
<code>platform_key_bindings</code> (macOS)	67	665-734

So a macOS build ships 167 default key bindings and 3 mouse bindings; a Linux build ships 118; a Windows build ships 119 (the common set plus `Alt+Enter` for fullscreen, :656). Under `#[cfg(test)]` the platform set is empty (:738-741) with the comment “Don’t return any bindings for tests since they are commented-out by default.”

The macOS set is where the application’s own surface lives: tab management (:668-672), pane splitting and directional focus, `Cmd+Shift+Enter` for zoom (:705), `Cmd+Ctrl+E` for Exposé (:712), `Cmd+Ctrl+G` for the divergence overlay of Chapter 61 (:717), `Cmd+`, for settings (:721).

The Kitty keyboard protocol

Five `TermMode` bits implement it (`zterminal_core/src/term/mod.rs:83-93`):

```
const DISAMBIGUATE_ESC_CODES = 1 << 18;
const REPORT_EVENT_TYPES     = 1 << 19;
const REPORT_ALTERNATE_KEYS  = 1 << 20;
const REPORT_ALL_KEYS_AS_ESC = 1 << 21;
const REPORT_ASSOCIATED_TEXT = 1 << 22;
const KITTY_KEYBOARD_PROTOCOL = /* the union of the five */
```

These are the five flags of the protocol’s progressive enhancement scheme, and they map one-to-one from `vte`’s `KeyboardModes` at `term/mod.rs:98-105`.

`build_sequence` (`keyboard.rs:295`) reads them:

- `kitty_seq` (:298-302) is true if *any* of disambiguate, report-all-keys, or report-event-types is on — the trigger for CSI-u style encoding.
- `kitty_encode_all` (:304) is report-all-keys specifically.
- `kitty_event_type` (:306-307) is report-event-types *and* the event is a repeat or a release, with the comment “The default parameter is 1, so we can omit it” — a press needs no event type because 1 is the default.

`associated_text` (:312-317) is filtered to exclude releases, empty text, and control characters, the last via `is_control_character` (:713-718), which includes `0x7f` with a note: DEL “has a dedicated control code (^?)” which generally does not match the reported text (^H).”

Building the payload

:319-324 is a five-way fallback chain, tried in order:

```

context.try_build_numpad(&key)
.or_else(|| context.try_build_named_kitty(&key))
.or_else(|| context.try_build_named_normal(&key, associated_text.is_some()))
.or_else(|| context.try_build_control_char_or_mod(&key, &mut modifiers))
.or_else(|| context.try_build_textual(&key, associated_text))

```

Numpad first, because the whole point of disambiguation is that numpad **5** is not **5**. Named keys in their Kitty form, then in their legacy form, then control characters, then ordinary text. If every one returns **None**, **build_sequence** returns an empty **Vec** (:328) and nothing is written.

Assembly (:331-359) is then mechanical: **\x1b[**, the payload, a modifier parameter if there are modifiers or an event type or associated text, a **:** and the event type digit if reporting events, then the associated text codepoints separated by **;** then **:**, then the terminator.

The modifier parameter

SequenceModifiers (**keyboard.rs**:684-691) is four bits — SHIFT 1, ALT 2, CONTROL 4, SUPER 8 — and the encoding is one line (:696-698):

```

pub fn encode_esc_sequence(self) -> u8 {
    self.bits() + 1
}

```

That **+ 1** is the entire xterm modifier convention: no modifiers is 1, Shift is 2, Alt is 3, Shift+Alt is 4, Control is 5, and so on. It has been the encoding since the 1990s and it is why **ESC [1 ; 5 A** is Control+Up.

The comment at :689-690 records what is missing: “Kitty protocol defines additional modifiers to what is present here, like Capslock, but it’s not a modifier as per winit.” A gap attributed to the windowing layer rather than silently omitted.

The alternate key code

try_build_textual (:378-421) is where **REPORT_ALTERNATE_KEYS** earns its bit. For a single-character key it computes both the shifted codepoint (**alternate_key_code**) and the unshifted one (**unicode_key_code**), lowercasing when Shift is held (:391-395). When Shift is held and the two are equal — which happens for keys like **1/!** where lowercasing does nothing — it consults **key_without_modifiers()** to recover the base (:400-404), and the comment at :397-398 names the case: “Try to get the base for keys which change based on modifier, like **1** for **!**.”

The payload becomes **unicode:alternate** when the mode is on and the two differ (:408-414), otherwise just the base. An application can therefore tell Shift+1 from **!** typed on a layout where **!** is unshifted, which is the entire purpose of the flag.

The honest limitation is right there too, at :406-407: “Base layouts are ignored, since winit doesn’t expose this information yet.” The Kitty protocol has a third field for the base-layout key; zterminal cannot fill it in, and says so.

When a sequence is built at all

should_build_sequence (**keyboard.rs**:144-172) is the gate, and its middle clause is the disambiguation rule in full:

```

let disambiguate = mode.contains(TermMode::DISAMBIGUATE_ESC_CODES)
&& (key.logical_key == Key::Named(NamedKey::Escape)
    || key.location == KeyLocation::Numpad
    || (!mods.is_empty()
        && (mods != ModifiersState::SHIFT
            || matches!(
                key.logical_key,
                Key::Named(NamedKey::Tab)
                | Key::Named(NamedKey::Enter)
                | Key::Named(NamedKey::Backspace)
            ))));

```

Escape, any numpad key, and any modified key are disambiguated — except that Shift alone does not count, because Shift+a is just A, unless the key is Tab, Enter, or Backspace, where Shift genuinely means something different. That single expression is most of what separates a terminal that implements the protocol from one that claims to.

Outside disambiguation the rule is the old one: named keys are encoded only when they have no textual representation (:169), and character keys only when the platform produced no text (:170).

Chapter 75 – The Eight, Part One: A Carriage Return That Is Not There

Chapter 63 inventories the reference corpus and reports that eight of its forty-five cases fail. Chapter 68 repeats the list and attributes it to drift: goldens recorded at one moment, code that moved afterwards. That attribution is wrong, and this chapter establishes what is actually true, because the answer is both simpler and more useful than drift. The recordings lost their carriage returns. The goldens are correct. The terminal is correct. The *input* is corrupt, and it has been corrupt since before the repository's first commit.

The chapter that follows takes the eight apart one at a time. This one establishes the common cause, because eight separate investigations that all land on the same byte are better told once.

The failure, reproduced

```
$ cargo test -p zterminal_core --test ref
failures:
    clear_underline
    colored_reset
    delete_lines
    row_reset
    saved_cursor
    saved_cursor_alt
    selective_eraser
    sgr

test result: FAILED. 37 passed; 8 failed; 0 ignored; 0 measured; 0 filtered out; finished
    in 5.76s
```

Deterministic across runs, and identical to the list Chapters 63 and 68 record. The suite builds in about a minute on a cold `target/` and runs in under six seconds, because nothing in it touches a window, a GPU, or a PTY.

The measurement that decides it

A ref-test recording is a raw PTY byte stream. A shell writing a line to a terminal in cooked mode produces `\r\n`, not `\n` — the line discipline's `ONLCR` translation is applied on the way out, so the master side sees both bytes. Chapter 43 describes that translation from the other direction. Every recording in the corpus should therefore carry roughly as many carriage returns as linefeeds.

Counting them across all forty-five:

```

$ cd zterminal_core/tests/ref
$ for d in */; do d=${d%/}; \
  printf "%s\t%s\t%s\n" \
    "${(tr -dc '\r' < $d/zterminal.recording | wc -c)}" \
    "${(tr -dc '\n' < $d/zterminal.recording | wc -c)}" "$d"; \
done | sort -n
0      2      colored_reset
0      2      selective_erasure
0     26      sgr
0     37      delete_lines
0      4      saved_cursor_alt
0      5      clear_underline
0    616      row_reset
0      8      saved_cursor
5      2      delete_chars_reset
6      1      zsh_tab_completion
...
1233   1226   vim_24bitcolors_bce

```

Eight recordings contain no carriage return at all. The other thirty-seven contain between five and 1,233. The eight with none are, exactly and without exception, the eight that fail. Nothing else in the corpus separates the two groups: none of the forty-five contains a NUL byte, so no text/binary classification distinguishes them, and the presence of shell-prompt text does not either — seven of the eight carry a prompt, and so do thirty-one of the thirty-seven that pass.

The consequence, on one line

Without the carriage return, a linefeed moves the cursor down and leaves the column where it was. Everything written after the first newline therefore lands further and further to the right, one staircase step per line, and the golden — recorded when the carriage returns were present — has it flush left.

`clear_underline` is the smallest example with a shell in it. Its 173 bytes are a prompt line, an `ESC [ 4 m` before the word `UNDERLINED`, a `clear` whose output is `ESC [ 3 J ESC [ H ESC [ 2 J`, a second prompt ending in `exit`, and the shell's own `exit` on the next line. That line — the prompt plus `exit` — is thirty-nine columns wide. The golden's second row holds `exit` at columns 0–3 while the current build puts it at columns 39–42, which is precisely thirty-nine columns of drift — the width of the line above, retained because no carriage return reset the column.

The transform that proves it

Restoring the missing byte is a four-line change to the input, applied in memory without touching the corpus: insert `\r` before any `\n` that does not already have one, then replay and compare against the stored golden with the same `PartialEq` the harness uses.

```

$ probe fix clear_underline
clear_underline: as-recorded=false with-CR-restored=true

```

Across the whole corpus:

Group	Count
Fails as recorded, passes with CR	8
Passes both ways	36
Passes as recorded, fails with CR	1

The eight are the eight. The one in the last row is `hyperlinks`, and it is not a counter-example — the transform

is a no-op on its bytes, since all fourteen of its linefeeds are already preceded by a carriage return. It fails because the probe replays it twice in one process, and hyperlink identity is a process-global counter. Chapter 83 takes that apart; for the purposes of this chapter, [hyperlinks](#) is untouched by the transform and its result is an artefact of the measuring program rather than of the input.

Seven hundred carriage returns are missing in total, across 58,078 bytes of recording. Restoring them takes the suite from 37/45 to 45/45.

Where the bytes went

They did not go anywhere in this repository. The corpus has been touched by exactly two commits:

```
$ git log --format='%h %ad %s' --date=short -- zterminal_core/tests/ref
6a9f9546cf 2026-06-27 more tmux
a86c952b04 2026-06-27 init
```

[a86c952b04](#) is the repository's first commit and it is a squashed import — 91 commits exist in total and the first one arrives with the whole tree already assembled. [6a9f9546cf](#), seven commits later the same day, renames every `zterm.recording` to `zterminal.recording` (the harness at `ref.rs:101` was already reading the new name, so before that rename all forty-five failed with `No such file or directory`) and rewrites a string inside about half of them. No `grid.json` has been modified since the import. There is no commit in which a golden was re-recorded, so there is no drift to bisect.

The eight recordings have no carriage returns at [a86c952b04](#) and none at HEAD:

```
$ git show a86c952b04:zterminal_core/tests/ref/sgr/zterm.recording | tr -dc '\r' | wc -c
0
```

So the loss predates the first commit and the mechanism is not recoverable from this history. The shape of it is familiar — a line-ending normalisation applied to an imported tree, of the kind an editor or a `dos2unix` sweep performs — but why it touched eight files and left thirty-seven alone is not answerable from anything in the repository. That is where the archaeology stops. CANNOT CHECK.

What this changes about the suite's verdict

Chapter 54 argues that a golden corpus reports *that* two things diverged, not which of them is right. That argument stands, and this is a good illustration of why it needs a third option. The two things the suite compares are the stored grid and the replayed grid; the assumption is that one of them is wrong. Here neither is. The recording sits underneath both, is not compared to anything, and is the thing that broke.

A golden suite defends its goldens. It does not defend its inputs, and it has no way to notice when they change, because a changed input produces exactly the signal a changed implementation produces. The correction is cheap — store a digest of each recording alongside it, so that a corpus whose *stimulus* moved is distinguishable from a corpus whose *subject* moved — and this failure is the argument for it.

The practical conclusion for anyone reading the eight failures as a statement about the terminal: they are not one. Every behaviour the eight recordings exercise — attribute reset across `ED`, saved-cursor interaction with the alternate screen, `DL` past the bottom of the screen, colon-separated SGR subparameters, background-colour erase — is correct on the current build, and the next chapter shows each of them being correct.

Chapter 76 – The Eight, One at a Time

Chapter 75 establishes the common cause. This chapter walks the eight individually, because the names describe eight distinct areas of the compatibility surface and the question “is that area actually broken?” deserves eight answers rather than one. For each: what the recording contains, what the golden holds, what today’s build produces, how much of the grid differs, and the verdict.

Two conventions. Recorded prompt text is elided as `<prompt>` — its content is irrelevant and its *width* is not, so widths are given where they matter. “Differing rows” counts rows under the comparison the harness actually performs: Row’s `PartialEq` at `zterminal_core/src/grid/row.rs:27-31` compares `inner` only and ignores `occ`, so a row whose cells all match counts as equal even when its occupancy bound does not.

Case	Rows differing	Linefeeds
<code>selective_erasure</code>	1 of 3	2
<code>clear_underline</code>	1 of 32	5
<code>colored_reset</code>	1 of 63	2
<code>saved_cursor_alt</code>	2 of 35	4
<code>saved_cursor</code>	9 of 35	8
<code>delete_lines</code>	12 of 31	37
<code>sgr</code>	31 of 35	26
<code>row_reset</code>	833 of 1247	616

The ordering is by damage, and the correlation with linefeed count is the whole story: each missing carriage return displaces everything after it, so a two-linefeed recording loses one row and a 616-linefeed recording loses two thirds of a 1,247-row buffer.

`selective_erasure`

Forty-one bytes, no shell, hand-authored: `ESC [ H`, then `A`, `ESC [ 1 " q`, `B`, `ESC [ 2 " q`, `C`, then `ESC [ ? 2 J`, a linefeed, the same three characters with the same two DECSCA settings, and `ESC [ ? 2 K`. A three-line, ten-column terminal.

The golden holds `ABC` on row 0 and `ABC` on row 1, both at columns 0–2. Today’s build produces `ABC` on row 0 and `ABC` at columns 3–5 on row 1. One row of three differs, and the difference is three columns of displacement.

Chapter 68 reads this case as a demonstration that DECSED and DECSEL are not dispatched, and half of that is right. The dispatch table genuinely has no `('J', [b'?'])` and no `('K', [b'?'])` arm, and no `('q', [b'"]')` arm for DECSCA either — Chapter 68’s inventory of `vte-0.15.0/src/ansi.rs` is accurate. But that is not what the diff shows. The golden *also* has all three characters intact on both rows, which means the golden was recorded on a build where selective erase did nothing either. Both sides agree that `ESC [ ? 2 J` is a no-op; they disagree only about the column, and the column is the missing carriage return. Restore it and the case passes.

So the correct reading is stronger than Chapter 68’s: this test does not detect the absence of selective erasure at all, and never did. It is a test named for a feature whose golden encodes that feature’s absence. If DECSCA and DECSED were implemented tomorrow, `selective_erasure` would start failing.

Verdict: recording corrupt. Behaviour unchanged and, as far as this test can tell, always was.

clear_underline

The `clear` command's escape sequence — `ESC [ 3 J`, `ESC [ H`, `ESC [ 2 J` — issued while `SGR 4` is in effect. The question the name asks is whether the underline attribute survives a screen clear into the cells written afterwards.

The answer, on both sides, is that it does. The golden's row 0 is `<prompt> exit` with `UNDERLINE` on every cell; the current build produces the same thirty-nine-column row with the same flag. The only differing row is row 1, where the golden has `exit` at columns 0–3 and the build has it at 39–42.

That is worth stating plainly, because it is the more interesting fact: the attribute behaviour the test is named for is identical between golden and build, and would still be identical if the test were re-recorded today.

Verdict: recording corrupt. ED does not clear SGR state, on both sides.

colored_reset

`printf "\e[41m"` — red background, no text — then `clear`, then `printf "\e[0m"`, then a partial command line ending in `ESC [ K`. A 116-column, 63-line terminal.

The golden's cells are `Cell { c: ' ', fg: Named(Foreground), bg: Named(Red), ... }` across the whole screen: background-colour erase, the property that `ED` fills with the cursor's current background rather than the default one. Today's build does the same — background-colour erase is intact, which is what the `vim_24bitcolors_bce` case (still passing) independently confirms.

This case is the sharpest illustration of why `occ` matters when reading a diff. Comparing rows including their occupancy bound, all sixty-three differ. Comparing them the way `Row: :eq` does, exactly one does. The bound moves because a displaced write touches a different number of columns; the cells do not.

Verdict: recording corrupt. BCE on ED is correct.

saved_cursor_alt

`ESC 7` (DECSC) issued on the primary screen, `ESC [ ? 1049 h` to enter the alternate screen, work done there, `ESC [ ? 1049 l` to leave, `ESC 8` (DECRC) to restore. The property under test is that the two screens keep separate saved cursors, and that `1049`'s own save/restore does not clobber the explicit one.

Two of thirty-five rows differ. The golden's 139-column rows are all default cells; the build's are too. The recording also exercises the DEC special character set (`ESC ( 0`), so the surviving content is line-drawing glyphs, and those match.

Verdict: recording corrupt. Alternate-screen cursor separation is correct.

saved_cursor

Three separate `echo -e` lines exercising DECSC/DECRC against `ESC ( 0` and `ESC [ ? 1049 h/l` in different orders. Nine of thirty-five rows differ; the recording has eight linefeeds, and the displacement compounds across them, so by the last of the three lines content that belongs at column 0 has walked past column 100.

The interesting detail is that the golden and the build disagree about *which rows* hold content, not just where in the row it sits: with the column never resetting, a 139-column row fills up and wraps, so a row that should hold one short line ends up holding the tail of the previous one. That is why nine rows move for eight linefeeds.

Verdict: recording corrupt. DECSC/DECRC and charset state are correct.

delete_lines

An `ls -lah` listing, then `ESC [ K` three times mid-line, then `ESC [ 10 H`, then `ESC [ 1000 M` — delete a thousand lines on a thirty-one-line terminal. The name's property is that `DL` with a count far past the bottom of the scroll region clamps rather than misbehaving.

Twelve of thirty-one rows differ. The listing itself is thirty-seven linefeeds of staircase, so most of the twelve are displacement in the output above the interesting part. The `ESC [ 1000 M` clamps correctly on both sides — after the transform the whole case matches.

Verdict: recording corrupt. DL clamping is correct.

SGR

Twelve `echo -e` lines, each setting `SGR 9` (crossed out) and `SGR 4` (underline) around a truecolor or indexed foreground/background specified two ways: with colons (`38:2:255:0:255`, `38:5:1`, and the five-argument `38:2:0:255:0:255` colourspace form) and with semicolons (`38;2;255;0;255`, `38;5;1`). This is the case that pins the subparameter handling Chapter 6 describes.

The golden's flag histogram is exactly forty-eight cells carrying `UNDERLINE | STRIKEOUT` — four per `TEST`, twelve `TEST`s. Today's build produces the same forty-eight; thirty-one of thirty-five rows differ, all of it displacement from twenty-six linefeeds.

Both colon and semicolon forms, both three-argument and five-argument colon truecolor, and both indexed forms are handled identically by golden and build.

Verdict: recording corrupt. Colon and semicolon SGR are both correct.

row_reset

The largest case: 53,649 bytes, 616 linefeeds, a 172-column by 47-line terminal with a 1,200-line scrollbar, producing a 1,247-row buffer. 833 rows differ.

There is nothing to add about the mechanism — it is the same staircase, applied 616 times, in a buffer large enough that two thirds of it moves. It is worth keeping only as the case that demonstrates the cost of the corruption is proportional to the recording's length, and as the one whose 19 MB `grid.json` accounts for a large share of the corpus's 46 MB.

Verdict: recording corrupt.

The corrected summary

Eight names that read like eight compatibility failures are one input defect. Every behaviour they exercise is correct on the current build, verified by replaying each recording with its carriage returns restored and comparing against the unmodified stored golden. The suite's verdict on the terminal, once the input is repaired, is 45 of 45.

What remains genuinely unhandled is what Chapter 68's dispatch-table reading says: DECSCA, DECSED, and DECSEL have no arms. That absence is real. It is simply not what `selective_erasure` measures.

Chapter 77 – The Reporter That Panics

Chapters 63 and 68 both note that some ref-test failures end in a panic inside `zterminal_core/src/grid/storage.rs` rather than in the harness's own `panic!("Ref test failed; grid doesn't match")`, and both put the count at two. The count is five, the state that causes it is present in thirty-three of the forty-five goldens rather than “roughly half”, and the consequence is worse than a cosmetic wart: the diff the harness prints before it dies is misattributed by one row, so for five of the eight failures every printed line names the wrong cells.

The assertion

```
/// Compute actual index in underlying storage given the requested index.
#[inline]
fn compute_index(&self, requested: Line) -> usize {
    debug_assert!(requested.0 < self.visible_lines as i32);

    let positive = -(requested - self.visible_lines).0 as usize - 1;

    debug_assert!(positive < self.len);
    ...
}
```

`zterminal_core/src/grid/storage.rs:218-234`. It is the single translation from a `Line` — which Chapter 65 describes as an index where 0 is the top of the viewport and negatives run into scrollbar — into a slot in the ring buffer. Both `Index<Line>` (:248-256) and `IndexMut<Line>` (:258-264) go through it, as does the hand-optimised `swap` (:153).

The failure text is always the same:

```
thread 'row_reset' (89883527) panicked at zterminal_core/src/grid/storage.rs:221:9:
assertion failed: requested.0 < self.visible_lines as i32
```

The state that reaches it

`Storage` derives `Serialize/Deserialize` (:32) and `visible_lines` (:44) is an ordinary serialized field. `Grid` holds both a `raw: Storage<T>` and its own `lines: usize` (`grid/mod.rs:121,127`), and in a live terminal the two are kept in step by the only code that changes either — `grow_lines` at `grid/resize.rs:51-52` and `shrink_lines` at :96-97, each of which updates the storage and then assigns `self.lines`.

In the stored goldens they are not in step. Thirty-three of the forty-five serialize a `Storage` whose `visible_lines` is exactly one less than the `Grid`'s `lines`:

```
$ python3 - <<'PY'
import json, os
for n in sorted(os.listdir('.')):
    g = json.load(open(n + '/grid.json'))
    if g['lines'] != g['raw']['visible_lines']:
        print(n, g['lines'], g['raw']['visible_lines'])
PY
alt_reset 34 33
clear_underline 32 31
...
zsh_tab_completion 43 42
```

Thirty-three names, every one of them off by exactly one, never by more. Twelve are self-consistent: `colored_underline`, `erase_in_line`, `hyperlinks`, `origin_goto`, `region_scroll_down`, `saved_cursor`, `saved_cursor_alt`, `scroll_in_region_up_preserves_history`, `sgr`, `underline`, `wrapline_alt_toggle`, `zerowidth`.

Where the skew came from is not recoverable — the goldens arrive in the squashed import commit of Chapter 75 and have never been rewritten — but its shape is characteristic of a serialized field whose meaning changed. `visible_lines` participates in the index arithmetic and in nothing else that is compared, so a convention change in `compute_index` would leave every stored grid silently holding the old convention's number, and no test would notice, because nothing compares it.

Why nothing notices

`Storage`'s equality, at :55-63:

```
impl<T: PartialEq> PartialEq for Storage<T> {
    fn eq(&self, other: &Self) -> bool {
        // Both storage buffers need to be truncated and zeroed.
        assert_eq!(self.zero, 0);
        assert_eq!(other.zero, 0);

        self.inner == other.inner && self.len == other.len
    }
}
```

`visible_lines` is not compared. Neither is it derived from anything that is. So a golden with the wrong `visible_lines` compares equal to a correct replay whenever the cells match — which is why thirty-three goldens carry the skew and twenty-eight of them still pass.

The comparison never indexes. The *report* does.

The report

The harness's failure path, `zterminal_core/tests/ref.rs:124-134`:

```

let history = grid.history_size() as i32;
for i in 0..grid.total_lines() {
    let line = Line(i as i32 - history);
    for j in 0..grid.columns() {
        let cell = &term_grid[line][Column(j)];
        let original_cell = &grid[line][Column(j)];
        if original_cell != cell {
            println!("{}", [{"j"}] {original_cell:?} => {cell:?}, line.0);
        }
    }
}

```

`grid` is the deserialized golden. `Dimensions` for `Grid` (`grid/mod.rs:543-558`) defines `total_lines` as `raw.len()` and `screen_lines` as `self.lines`, so the loop's last iteration is `Line(lines - 1)`. For a golden with `visible_lines == lines - 1`, that is `Line(visible_lines)` — one past what `compute_index` will accept — and the assertion fires.

Two things follow, and the second is the one that matters.

The panic is not a mystery about which grid is broken. `term_grid` is constructed by `Term::new` and mutated only through the resize paths, so its `visible_lines` always equals its `lines`. The panic is always on the golden.

Every line the report prints before it dies is attributed to the wrong row. For a skewed golden, `grid[Line(k)]` resolves to `inner[visible_lines - k - 1]` while `term_grid[Line(k)]` resolves to `inner[lines - k - 1]` — one slot apart. The printer is comparing the golden's row $k+1$ against the replay's row k , for every k .

`clear_underline` shows the damage. The report opens with

```

[0][0] Cell { c: 'e', ..., flags: Flags(UNDERLINE), ... } => Cell { c: '[', ..., flags:
  ~ Flags(UNDERLINE), ... }

```

and prints forty-two such lines across rows 0 and 1. Compared in storage order — which is what `PartialEq` does — exactly one row of thirty-two differs, and it is neither of those two. The report describes a mismatch that does not exist and omits the one that does.

For the eight failures, five panic and three do not:

Case	Golden self-consistent	Reporter
<code>clear_underline</code>	no	panics
<code>colored_reset</code>	no	panics
<code>delete_lines</code>	no	panics
<code>row_reset</code>	no	panics
<code>selective_erasure</code>	no	panics
<code>saved_cursor</code>	yes	prints
<code>saved_cursor_alt</code>	yes	prints
<code>sgf</code>	yes	prints

The three that print are exactly the three whose goldens have `visible_lines == lines`, and their reports are correctly aligned. So the harness produces trustworthy diffs for three of eight failures and misleading ones for five, with nothing in the output distinguishing the two cases beyond the panic at the end.

Can it fire outside the harness

Three questions, in order.

Is the assertion live in the shipped binary? No. `debug_assert!` compiles out whenever `debug_assertions` is off, and the workspace's `[profile.release]` in `Cargo.toml:18-21` sets `lto`, `debug`, and `incremental` and does not turn debug assertions back on. Every packaging path uses the release profile — `Makefile:43-46`, `scripts/release.sh:31`, `scripts/rebuild.sh:25`, `scripts/nuke.sh:26`. It is live in `cargo build`, which is what a development loop produces, and dead in the `.pkg`.

Can a running terminal produce a `Grid` whose `visible_lines` disagrees with its `lines`? Not by deserialization: `Grid<Cell>` is deserialized in exactly two places in the workspace, `zterminal_core/tests/ref.rs:107` and the round-trip unit test at `zterminal_core/src/term/mod.rs:3166`. Nothing in `zterminal/src` deserializes a grid — session restore, as Chapter 29 describes, replays scrollback as text rather than rehydrating a serialized buffer. The seventeen `serde_json::from_str` call sites in the binary crate decode IPC messages, triggers, recent directories, layout nodes, and settings; none of them decodes a `Grid`.

Can it fire for any other reason? Only by indexing a `Line` at or past the bottom of the screen, which is an out-of-range access of the ordinary kind — the assertion is doing its job there rather than reporting a stale field. Chapter 81 finds one path in `Grid::resize` that misbehaves with a one-column grid, and it does not reach this assertion; it loops instead.

So: a debug build of the test suite is the only place this fires today, and the sixty-second fix to the harness — iterate `0..grid.screen_lines()` offset by the golden's own history rather than trusting a field that is not compared — would also repair five of eight failure reports. Neither is done here, because a harness is a measurement tool and Chapter 78 explains why one does not quietly edit those.

Chapter 78 – What Forty-Five Recordings Actually Pin

Chapter 63 ends with a caveat about what the corpus does not tell you: that forty-five sessions are a sample of an unknown surface, that goldens defend whatever was true when they were recorded, and that everything with a window or a font in it is outside the suite by construction. All of that is true and none of it is specific. This chapter is specific, because the answer is narrower than the caveat suggests and it is derivable from three `PartialEq` implementations.

Three equality implementations

The harness's whole assertion is `grid != term_grid`. What that expression compares is fixed by:

`Grid`, at `zterminal_core/src/grid/mod.rs:465-473`:

```
impl<T: PartialEq> PartialEq for Grid<T> {
    fn eq(&self, other: &Self) -> bool {
        self.raw.eq(&other.raw)
        && self.columns.eq(&other.columns)
        && self.lines.eq(&other.lines)
        && self.display_offset.eq(&other.display_offset)
    }
}
```

`Storage`, at `grid/storage.rs:55-63`, comparing `inner` and `len`.

`Row`, at `grid/row.rs:27-31`:

```
impl<T: PartialEq> PartialEq for Row<T> {
    fn eq(&self, other: &Self) -> bool {
        self.inner == other.inner
    }
}
```

Four fields of `Grid`, two of `Storage`, one of `Row`. Everything else in the three structs is invisible to the suite.

What is pinned

Property	Pinned
Cell char, colours, flags, extra	yes
Column count	yes
Screen-line count	yes
Display offset	yes
Total buffer length	yes
Row occupancy bound (<code>occ</code>)	no
Ring-buffer origin (<code>zero</code>)	asserted 0
<code>Storage::visible_lines</code>	no

Property	Pinned
Cursor position	no
Saved cursor	no
<code>max_scroll_limit</code>	no

The cursor is not merely unpinned by `PartialEq`; it is not in the file. `Grid`'s `cursor` and `saved_cursor` carry `#[cfg_attr(feature = "serde", serde(skip))]` at `grid/mod.rs:112` and `:116`, so a golden has no cursor in it at all and a deserialized one is `Default`. Nothing above `Grid` is serialized either — `Term`'s modes, scroll region, tab stops, charset assignments, and title live in `Term`, and the harness compares grids.

The demonstration

Replaying a recording, appending one extra escape sequence, and comparing against the unmodified golden shows which regressions the corpus would let through. The appended bytes are the smallest thing that changes the named piece of state and nothing else.

```
$ probe pins underline
nothing appended
cursor moved to 1;1 (CUP)
cursor saved (DECSO)
autowrap disabled (DECRST 7)
origin mode set (DECSO 6)
insert mode set (SM 4)
all tab stops cleared (TBC 3)
scroll region set (DECSO 7)
G0 = line drawing
window title set (OSC 2)
application cursor keys (DECSO 1)
bracketed paste on (DECSO 2004)
one space printed
golden == replay: true
golden == replay: true
golden == replay: true
golden == replay: true
golden == replay: true
golden == replay: true
golden == replay: true
golden == replay: true
golden == replay: true
golden == replay: true
golden == replay: true
golden == replay: true
golden == replay: true
```

Thirteen mutations, thirteen passes, on `underline` and identically on `tmux_http`. A build that left the cursor in the wrong place after every recording, or came out of a session with autowrap off, origin mode on, insert mode on, no tab stops, a scroll region covering four lines, G0 mapped to the DEC graphics set, and a stale window title would pass all forty-five.

The last row is the mildest of the thirteen and worth reading carefully: printing a space at the final cursor position is a no-op in these two recordings because the cell there already holds a space. It belongs in the list not as a claim that content is unpinned — it plainly is pinned — but because `occ` is not compared, so the write that touched the row leaves no trace.

What this means for the four risk areas

Chapter 54 names four areas that concentrate regression risk and Chapter 63 maps the corpus onto them. The mapping is right, with one qualification that this chapter supplies: the corpus pins the *grid consequences* of those areas and none of their *state*.

A scroll-region regression is caught if it moves cells and missed if it only leaves `Term::scroll_region` wrong at the end. An origin-mode regression is caught if a `CUP` lands in the wrong place during the recording and missed if the mode is simply left set. A charset regression is caught for characters already written and missed for the mapping that would apply to the next one.

This is not an argument that the corpus is weak. Cells are the observable that matters, and pinning cells at the end of forty-five real sessions is a strong property. It is an argument that the corpus is not a substitute for the

hand-written tests — the 249 in `zterminal_core` — for anything expressed as terminal state rather than as glyphs, and that a change to `Term`'s mode handling should not be considered covered because the reference suite is green.

Why the harness is not being fixed here

Three defects in the harness are established by this chapter and Chapter 77: the report is misaligned for thirty-three goldens, the comparison ignores `visible_lines` in a way that lets those goldens exist, and the corpus stores no digest of its own inputs, which is how eight corrupt recordings survived from the import commit to now.

All three are one-file changes. None is made here, and the reason is worth stating once for the whole book: the reference harness is the instrument by which the terminal's behaviour is measured, and a chapter documenting what the instrument says is the worst possible moment to adjust the instrument. A reader who wants the numbers in Chapters 75 through 78 to mean anything has to be able to reproduce them against an unmodified checkout. Every count in these four chapters was produced either by an unmodified `cargo test` or by a separate program that links `zterminal_core` as a dependency and does not touch it.

Chapter 79 – Reads That Land in the Wrong Place

Everything in Chapters 75 through 78 replays a recording as one contiguous buffer. A real terminal never does. Chapter 45 describes the read loop's 64 KiB buffer and Chapter 36 describes the event loop that drives it; between them the guarantee that matters is that a PTY read returns whatever bytes are available at that instant, with no regard for where escape sequences or characters begin and end. An emulator has to be indifferent to the split.

zterminal is indifferent to it almost everywhere, and there is one place where it is not. The defect is in `vte`, it drops a character rather than mangling one, and it is reachable from ordinary text.

The sweep

The 45 recordings, replayed at eight different chunk sizes and compared against the contiguous replay:

```
$ for d in zterminal_core/tests/ref/*; do probe chunks $(basename $d); done
alt_reset: differing chunk sizes []
clear_underline: differing chunk sizes []
...
hyperlinks: differing chunk sizes [1, 2, 3, 5, 7, 13, 64, 4096]
...
zerowidth: differing chunk sizes [3]
zsh_tab_completion: differing chunk sizes []
```

Chunk sizes 1, 2, 3, 5, 7, 13, 64, and 4096. Forty-three of the forty-five produce a bit-identical grid at every size, including the 350 KB `vim_24bitcolors_bce` and the 51 KB `tmux_htop` fed one byte at a time. That is the result you want, and it covers escape sequences split at every internal position, since a one-byte chunking splits everything.

Two are not in that group and only one of them is a real result. `hyperlinks` differs at every chunk size because the probe replays it nine times in one process and hyperlink identity comes from a process-global counter — Chapter 83 takes that apart. Its bytes are parsed identically at every size; only the identifiers stamped into the cells move.

`zerowidth` is the real exception. Sweeping it more finely:

```
$ probe chunkscan zerowidth 300
zerowidth: chunk sizes 1..=300 that change the grid: [3, 4, 173, 229]
```

Four of three hundred. Splitting the recording at exactly one offset instead localises it:

```
$ probe splitscan zerowidth
zerowidth: 965 bytes; single-split offsets that change the grid: [687, 692]
  at 687: [13, 13, 10, 40, 32, 205, 161, 194] | [176, 32, 205, 156, 202, 150, 32, 205]
  at 692: [205, 161, 194, 176, 32, 205, 156, 202] | [150, 32, 205, 161, 194, 176, 41, 13]
```

Two offsets in 965 bytes, and the four bad chunk sizes are the four that hit one of them: $3 \times 229 = 687$, $4 \times 173 = 692$. Both offsets fall *inside* a two-byte UTF-8 character — 687 between `0xC2` and `0xB0` (U+00B0), 692 between `0xCA` and `0x96`, the two bytes of U+0296.

The character that disappears

The affected text is the emoticon (° º °), eleven bytes of alternating combining marks, degree signs, and spaces. Recording what the parser hands the `Handler` — a `Perform` implementation whose only method is `input` — isolates the fault from anything zterminal does with it:

```
$ probe chars
whole      : ['(', '\u{361}', '°', ' ', '\u{35c}', '°', ' ')']
split at 1 : ['(', '\u{361}', '°', ' ', '\u{35c}', '°', ' ')']
split at 2 : ['(', '\u{361}', '°', ' ', '\u{35c}', '°', ' ')']
split at 3 : ['(', '\u{361}', '°', ' ', '\u{35c}', '°', ' ')']
split at 4 : ['(', '\u{361}', '°', '\u{35c}', '°', ' ')']
split at 5 : ['(', '\u{361}', '°', ' ', '\u{35c}', '°', ' ')']
...
```

At one split point out of ten the space is never delivered. Not replaced, not reordered, not turned into U+FFFD — the handler is simply never told about it. The grid consequence follows mechanically: with the space gone, U+035C attaches to the preceding cell instead of to its own, and the rest of the line shifts one column left, leaving seven occupied cells where there should be eight.

The mechanism

`vte-0.15.0/src/lib.rs:668-718`. When `advance` returns with a truncated character in hand it stashes the bytes in a fixed four-byte buffer (`partial_utf8`, :68-69) and the next call begins in `advance_partial_utf8`:

```
let old_bytes = self.partial_utf8_len;
let to_copy = bytes.len().min(self.partial_utf8.len() - old_bytes);
self.partial_utf8[old_bytes..old_bytes + to_copy].copy_from_slice(&bytes[..to_copy]);
self.partial_utf8_len += to_copy;

match str::from_utf8(&self.partial_utf8[..self.partial_utf8_len]) {
  Ok(parsed) => {
    let c = unsafe { parsed.chars().next().unwrap_unchecked() };
    performer.print(c);
    self.partial_utf8_len = 0;
    c.len_utf8() - old_bytes
  },
  Err(err) => {
    let valid_bytes = err.valid_up_to();
    if valid_bytes > 0 {
      let c = /* first char of the valid prefix */;
      performer.print(c);
      self.partial_utf8_len = 0;
      return valid_bytes - old_bytes;
    }
    ...
  },
}
```

The return value is how many bytes of the new slice the function consumed; the caller resumes at that offset. The `Ok` arm is careful: it prints one character and returns `c.len_utf8() - old_bytes`, consuming only that character's bytes and leaving anything else in the buffer for the ordinary ground-state path.

The `Err` arm with a non-empty valid prefix is not. It prints the *first* character of the prefix and returns `valid_bytes - old_bytes` — the length of the *whole* prefix. Every character after the first in that prefix is consumed and never printed. The comment above it says as much: “Since we only care about the first character, we just ignore the rest.”

Reaching that arm requires the four-byte buffer to end up holding a valid prefix of two or more characters followed by something incomplete, which pins the trigger tightly:

Ingredient	Value
Stashed bytes	1, from a 2-byte character
Next byte	that character's continuation
Then	exactly one ASCII byte
Then	a lead byte needing more

The buffer becomes `[lead, cont, ascii, lead]`, `from_utf8` reports `valid_up_to() == 3` with `error_len() == None`, and the ASCII byte in the middle is discarded.

Five bytes are enough to reproduce it, and the neighbouring cases show how narrow it is:

```
$ probe vtebug
2-byte, ASCII, 2-byte: split at 1
  whole [' ', ' ', ' ']
  split [' ', ' ']    DIFFERENT
2-byte, 2 ASCII, 2-byte: split at 1
  whole [' ', 'a', 'b', ' ']
  split [' ', 'a', 'b', ' ']    same
3-byte, ASCII, 2-byte: split at 1
  whole ['-', ' ', ' ']
  split ['-', ' ', ' ']    same
3-byte, ASCII, 2-byte (split 2): split at 2
  whole ['-', ' ', ' ']
  split ['-', ' ', ' ']    same
2-byte, ESC CSI: split at 1
  whole [' ', 'X']
  split [' ', 'X']    same
2-byte, LF, 2-byte: split at 1
  whole [' ', ' ']
  split [' ', ' ']    same
```

Two ASCII bytes instead of one fills the buffer with four valid bytes and takes the `Ok` arm. A three-byte lead leaves no room for the trailing lead byte and takes the `Ok` arm. An `ESC` or a control byte in the ASCII slot is not lost, because a control byte in the prefix is dispatched by `ground_dispatch (lib.rs:720-729)` rather than dropped — the drop only claims printable characters.

How much this matters

The honest accounting has three parts.

The trigger is narrow. It needs a read boundary in a specific one-byte position of a two-byte character, followed by a specific three-byte pattern. Across 58,078 bytes of accented, box-drawing, and combining-mark-heavy recording, two offsets qualify.

The consequence is silent and permanent. The lost character is a printable one — a space, a letter, a digit — and nothing downstream can tell it was ever there. The grid is wrong, the scrollbar is wrong, a selection copied out of it is wrong, and the ledger of Chapter 59 will hash the wrong line.

The exposure is a function of how the bytes arrive. A 64 KiB read of a large `cat` produces one boundary per 64 KiB. A slow SSH link, a `tmux` pane relaying small writes, or a program printing a line at a time produces one boundary per write, and every one of them is a chance. Chapter 46's nesting discussion is the relevant context: the more layers between the writer and the grid, the smaller the writes and the more boundaries there are.

The fix belongs upstream and is one line — return `c.len_utf8() - old_bytes` in the `Err` arm as the `Ok` arm does, so that only the first character's bytes are consumed. Whether `vte` has changed this since 0.15.0 is past this book's verification: CANNOT CHECK. The pin in `zterminal_core/Cargo.toml:29` is exact and the checksum in `Cargo.lock` is `a5924018406ce0063cd67f8e008104968b74b563ee1b85dde3ed1f7cb87d3dbd`, so the version in use is not in doubt.

The corpus, meanwhile, would catch a regression here only by accident. Chunk-size invariance is not something the reference suite tests; it feeds every recording as one buffer. Adding a second pass at a one-byte chunking would cost about five seconds of test time and would have caught this.

Chapter 80 – The Parser's Limits

The dispatch tables of Chapter 68 answer “what does the terminal do with a sequence it knows”. This chapter answers the other question: what does it do with input that no golden file covers and no table has an arm for. Four cases, each with a limit that is a constant somewhere in `vte`, and each with a behaviour that is defensible but worth knowing.

An OSC that never ends

```
$ probe osc
--- unterminated-osc cursor=Point { line: Line(0), column: Column(0) } display_offset=0
--- unterminated-osc: lines=3 cols=20 history=0
  0 || Flags(0x0)
  1 || Flags(0x0)
  2 || Flags(0x0)
--- terminated-osc cursor=Point { line: Line(0), column: Column(5) } display_offset=0
--- terminated-osc: lines=3 cols=20 history=0
  0 |AFTER| Flags(0x0)
```

`ESC ] 0 ;` followed by 100,000 bytes and then `BCD`. Nothing reaches the screen — not the payload, not the text after it. The terminal is silent until a terminator arrives, and the second half of the probe confirms that a well-formed OSC lets the following text through normally.

The escape hatches are in `advance_osc_string`, `vte-0.15.0/src/lib.rs:407-435`: `BEL` (0x07) ends the string, `CAN` (0x18) and `SUB` (0x1A) cancel it, and `ESC` (0x1B) ends it and moves to the escape state — which is how `ST` works. Every other byte, including every printable character and including `LF`, is payload. There is no length cap and no timeout.

Memory is the part worth stating precisely, because it depends on a feature flag. The buffer is declared twice:

```
#[cfg(not(feature = "std"))]
osc_raw: ArrayVec<u8, OSC_RAW_BUF_SIZE>,
#[cfg(feature = "std")]
osc_raw: Vec<u8>,
```

`lib.rs:61-64`. Without `std` it is a fixed 1,024-byte array and `action_osc_put (:544-552)` drops bytes once it is full. With `std` — which `zterminal_core/Cargo.toml:29` enables — it is an unbounded `Vec` and every byte is retained. A process writing an unterminated OSC is therefore a memory leak proportional to what it writes, and a terminal that has gone quiet for no visible reason is the symptom.

That is not a vulnerability so much as a property to know when debugging: a pane that stops updating in the middle of a program that emits OSC sequences has probably lost a terminator, and the recovery is any of `BEL`, `CAN`, `SUB`, or `ESC`.

A DCS with a large payload

```
$ probe dcs
--- huge-dcs cursor=Point { line: Line(0), column: Column(5) } display_offset=0
--- huge-dcs: lines=3 cols=20 history=0
0 |AFTER| Flags(0x0)
```

`ESC P`, one megabyte of `x`, `ESC \`, then `AFTER`. The payload is consumed, the string terminator is recognised, and the following text prints in the right place. DCS passthrough is a per-byte callback rather than a buffer — the `dcs_put` path at `lib.rs:317-339` hands each byte to the handler and keeps no copy — so payload size is bounded by whatever the handler retains, and `zterminal_core`'s handler retains nothing for an unrecognised DCS.

This is the well-behaved case and the contrast with OSC is the point: two string states, one that accumulates and one that streams.

SGR with a hundred parameters

```
$ probe sgr100
--- sgr-100-params cursor=Point { line: Line(0), column: Column(1) } display_offset=0
--- sgr-100-params: lines=3 cols=20 history=0
0 |A| Flags(0x0)
```

`ESC [` followed by a hundred `1`s separated by semicolons, then `m`, then `A`. `SGR 1` is bold; a hundred of them should be bold a hundred times over. The `A` has no flags at all.

The cap is `MAX_PARAMS = 32` at `vte-0.15.0/src/params.rs:5`. The parser does not truncate to the first thirty-two and dispatch. Every action that would add a parameter checks `params.is_full()` first and sets a sticky `ignoring` flag instead — `action_param` at `lib.rs:498-505`, `action_subparam` at `:487-494`, `action_paramnext` at `:508-516`, `action_csi_dispatch` at `:453-462`. That flag is passed to the handler, and `ansi.rs:1545-1548` opens the dispatch with

```
if has_ignored_intermediates || intermediates.len() > 2 {
    unhandled!();
    return;
}
```

so the whole sequence is dropped before the forty-four-arm table is reached. A CSI with more than thirty-two parameters is not a partially-applied CSI but an absent one, and the parameter's name in `ansi.rs` mentions only intermediates even though the flag it receives is also set by parameter overflow. `unhandled!` is a `debug!` log line, so at default verbosity nothing is reported at all.

Thirty-two is generous for SGR in practice — the longest realistic sequence is a handful of attributes plus two truecolor triples — but it is reachable by generated output, and the failure mode is that a colour silently does not apply. The related caps, all in `vte`:

Limit	Value	Location
CSI parameters	32	<code>params.rs:5</code>
CSI intermediates	2	<code>lib.rs:44</code>
OSC parameters	16	<code>lib.rs:45</code>
OSC buffer, no <code>std</code>	1024	<code>lib.rs:46</code>
OSC buffer, <code>std</code>	unbounded	<code>lib.rs:63-64</code>
Partial UTF-8 buffer	4	<code>lib.rs:68</code>

Exceeding the intermediate limit sets `ignoring` too (`lib.rs:477`); exceeding the OSC parameter limit stops recording new parameters but keeps the string (`:529-530`).

Intermediates outside the forty-four arms

```
$ probe intermediate
--- unknown-intermediates cursor=Point { line: Line(0), column: Column(5) }
  ~ display_offset=0
--- unknown-intermediates: lines=3 cols=20 history=0
    0 |ABCDE| Flags(0x0)
```

Five sequences with no arm in the table, each followed by one letter: `ESC [ ? 2 J` (DECSER, absent per Chapter 68), `ESC [ ! p` (DECSTR, soft reset), `ESC [ $ q` (DECRQM, request mode), `ESC [ > 4 ; 2 m` (XTMODKEYS), and `ESC [ 3 SP q` — a DECSCUSR with a parameter the table's ('q', [b' ']) arm does accept.

`ABCDE` on one row, in order, at columns 0 through 4. Every unhandled sequence is consumed silently and completely; nothing leaks into the grid as text, nothing shifts the cursor, and no replacement characters appear. That is the correct behaviour for an emulator facing a sequence it does not implement, and it is the reason a program can probe for capabilities without corrupting the screen — the counterpart to Chapter 48’s advice about detecting capabilities before using them.

The cost is the one Chapter 68 names: silence is indistinguishable from success. `ESC [ ! p` should reset every mode and does nothing; `ESC [ $ q` should produce a reply and produces none. A program that assumes a soft reset happened will misbehave later, in a place with no obvious connection to the sequence that was ignored.

The pattern

Three of the four cases are the same design: a fixed limit, and beyond it the input is discarded rather than partially applied. That is the right trade — a half-applied SGR is worse than no SGR — and it is consistent with the “refusal, not sandboxing” position of Chapter 49.

The fourth, the unbounded OSC buffer under `std`, is the one that does not fit the pattern, and it is the one where the `no_std` build of the same crate is stricter than the build zterminal ships.

Chapter 81 – Resize, Reflow, and the Column That Never Comes

Chapter 72 describes reflow as a feature: shrink the window and wrapped lines re-wrap, grow it and they re-join. Chapter 3 gives the model. This chapter is the edges — what reflow does when there is nowhere to put the overflow, where the cursor ends up when the line under it disappears, what the alternate screen does instead, and one input that makes `Grid::resize` allocate until the process dies.

Every transcript below comes from driving `Term::resize` directly with a mock event listener. No window, no display, no PTY. The probe prints a `cursor=...` `display_offset=...` line before each grid dump; those lines are elided below except where the cursor is the point being made.

Round-tripping when there is room

A ten-column, four-line terminal holding one twenty-character wrapped line and two short ones, with the default scrollbar:

```
--- before: lines=4 cols=10 history=0
0 |0123456789| Flags(WRAPLINE)
1 |abcdefghij| Flags(0x0)
2 |SHORT| Flags(0x0)
3 |X| Flags(0x0)
--- shrink-to-5: lines=4 cols=5 history=2
-2 |01234| Flags(WRAPLINE)
-1 |56789| Flags(WRAPLINE)
0 |abcde| Flags(WRAPLINE)
1 |fghij| Flags(0x0)
2 |SHORT| Flags(0x0)
3 |X| Flags(0x0)
--- regrow-to-10: lines=4 cols=10 history=0
0 |0123456789| Flags(WRAPLINE)
1 |abcdefghij| Flags(0x0)
2 |SHORT| Flags(0x0)
3 |X| Flags(0x0)
```

Exact. The twenty characters split into four five-column rows, two of which move into scrollbar, and rejoin into the original two rows on the way back. The cursor stays at `Line(3)`, `Column(1)` throughout. This is the behaviour Chapter 72 describes, and it is the behaviour whenever the scrollbar can absorb the extra rows.

Round-tripping when there is not

The same content with `scrolling_history = 0`:

```

--- before: lines=4 cols=10 history=0
0 |0123456789| Flags(WRAPLINE)
1 |abcdefghij| Flags(0x0)
2 |SHORT| Flags(0x0)
3 |X| Flags(0x0)
--- cols=5: lines=4 cols=5 history=0
0 |abcde| Flags(WRAPLINE)
1 |fghij| Flags(0x0)
2 |SHORT| Flags(0x0)
3 |X| Flags(0x0)
--- back to cols=10: lines=4 cols=10 history=0
0 |abcdefghij| Flags(0x0)
1 |SHORT| Flags(0x0)
2 |X| Flags(0x0)
3 || Flags(0x0)

```

0123456789 is gone at the first step and does not come back at the second. With no scrollbar there is nowhere for the two extra rows to go, so they fall off the top; regrowing re-joins what is left and leaves a blank row at the bottom, and the cursor has moved from [Line\(3\)](#) to [Line\(2\)](#).

This is not a bug — a zero-scrollbar terminal has, by construction, agreed to lose whatever leaves the viewport — but it is worth naming, because the intuition that “resize is reversible” is only true when the scrollbar is deep enough to hold the difference. `scrolling.history = 0`, which Chapter 13 lists as a way to save memory, also makes every narrowing destructive.

Shrinking the height

```

--- lines=2 cursor=Point { line: Line(1), column: Column(1) } display_offset=0
--- lines=2: lines=2 cols=10 history=0
0 |C| Flags(0x0)
1 |D| Flags(0x0)
--- back to lines=4 cursor=Point { line: Line(1), column: Column(1) } display_offset=0
--- back to lines=4: lines=4 cols=10 history=0
0 |C| Flags(0x0)
1 |D| Flags(0x0)
2 || Flags(0x0)
3 || Flags(0x0)

```

With the cursor on the bottom row and no scrollbar, shrinking pushes [A](#) and [B](#) out the top — the “keep the cursor at the bottom” rule that `shrink_lines` documents at [grid/resize.rs:73-77](#). Regrowing appends blank rows *below*, so the content that was at the bottom is now at the top and the shell’s prompt has moved into the middle of the window. That asymmetry is visible in daily use as the familiar gap after dragging a window shorter and then taller again.

Move the cursor first and the rule inverts:

```

--- cursor home cursor=Point { line: Line(0), column: Column(0) } display_offset=0
--- cursor home: lines=4 cols=10 history=0
0 |A| Flags(0x0)
1 |B| Flags(0x0)
2 |C| Flags(0x0)
3 |D| Flags(0x0)
--- lines=2 cursor=Point { line: Line(0), column: Column(0) } display_offset=0
--- lines=2: lines=2 cols=10 history=0
0 |A| Flags(0x0)
1 |B| Flags(0x0)

```

With the cursor at [Line\(0\)](#), `shrink_lines` computes `required_scrolling = (0 + 1).saturating_`

`sub(2) = 0`, scrolls nothing, and the rows below the cursor are dropped instead. `C` and `D` are not in scrollback — `shrink_lines` reduces `Storage::len` (`storage.rs:107-114`), which discards them rather than archiving them. So which end of the screen survives a height shrink depends entirely on where the cursor is, and content below the cursor is destroyed outright.

The alternate screen

```
--- alt before: lines=4 cols=10 history=0
0 |0123456789| Flags(WRAPLINE)
1 |abcdefghij| Flags(0x0)
2 || Flags(0x0)
3 || Flags(0x0)
--- alt cols=5: lines=4 cols=5 history=0
0 |01234| Flags(0x0)
1 |abcde| Flags(0x0)
2 || Flags(0x0)
3 || Flags(0x0)
--- alt cols=10: lines=4 cols=10 history=0
0 |01234| Flags(0x0)
1 |abcde| Flags(0x0)
2 || Flags(0x0)
3 || Flags(0x0)
```

`Term::resize` passes `!is_alt` as the reflow flag to the active grid and `is_alt` to the inactive one (`term/mod.rs:812-814`), so the alternate screen never reflows. Narrowing truncates each row at the new width and widening does not restore it — the characters are gone, and the `WRAPLINE` flag is cleared along with them.

This is correct and deliberate: a full-screen application redraws after `SIGWINCH`, so preserving its content across a resize is pointless work, and reflowing a `vim` buffer's rendered rows would produce nonsense. It is worth knowing only because it explains why resizing during a full-screen program that has *stopped* redrawing — one that is blocked, or a `tmux` pane whose client has gone away — leaves a permanently truncated screen.

The scroll region does not survive

A six-line terminal with `DECSTBM` set to rows 2–4, filled, with the cursor at the region's bottom row; then a single newline:

```
--- region set, filled: lines=6 cols=10 history=0
0 |a| Flags(0x0)
1 |d| Flags(0x0)
2 |e| Flags(0x0)
3 |f| Flags(0x0)
4 || Flags(0x0)
5 || Flags(0x0)
--- one LF at region bottom cursor=Point { line: Line(3), column: Column(0) }
  ~ display_offset=0
--- one LF at region bottom: lines=6 cols=10 history=0
0 |a| Flags(0x0)
1 |e| Flags(0x0)
2 |f| Flags(0x0)
3 || Flags(0x0)
4 || Flags(0x0)
5 || Flags(0x0)
```

Rows 1–3 scroll; row 0 does not. Now the same sequence with a width-only resize inserted before the newline:

```

--- same LF after a width-only resize cursor=Point { line: Line(4), column: Column(0) }
~ display_offset=0
--- same LF after a width-only resize: lines=6 cols=11 history=0
0 |a| Flags(0x0)
1 |d| Flags(0x0)
2 |e| Flags(0x0)
3 |f| Flags(0x0)
4 || Flags(0x0)
5 || Flags(0x0)

```

Nothing scrolls, and the cursor moves from `Line(3)` to `Line(4)`. The region is gone. `Term::resize` resets it unconditionally at `term/mod.rs:836-837`:

```

// Reset scrolling region.
self.scroll_region = Line(0)..Line(self.screen_lines() as i32);

```

The reset does not check whether the height changed, so changing only the width discards the region too. For a full-screen application this is invisible — it re-issues `DECSTBM` as part of its redraw — but a program that sets a region once and then relies on it, such as a status-line script that scrolls only the body, will silently start scrolling the whole screen after any resize. Chapter 48’s advice to treat `resize` as a flag and redraw from scratch is the mitigation, and this is the concrete reason for it.

The column that never comes

Shrinking a grid that contains a double-width character to a single column does not finish:

```

$ probe narrow2
ascii only: 4 -> 2 ...
done, 4 lines
ascii only: 4 -> 1 ...
done, 6 lines
wide char: 4 -> 2 ...
done, 4 lines
wide char: 4 -> 1 ...
[no further output]

```

The last case runs until it is killed. It is not a spin — it allocates:

```

t=2s  rss=239.2MB
t=4s  rss=442.0MB
t=6s  rss=679.7MB
t=8s  rss=881.9MB
t=10s rss=1077.5MB
t=12s rss=1287.0MB

```

Roughly 100 MB per second, unbounded. The input is four columns holding `ab—c`, where `—` is U+4E00, and the destination is one column.

The mechanism is the wide-character branch of `shrink_columns, grid/resize.rs:245-388`. The inner `loop` splits a row at the new width and pushes the remainder back through; when the character at the last column is a `WIDE_CHAR` it replaces it with a `LEADING_WIDE_CHAR_SPACER` and prepends the wide character to the wrapped remainder (`:289-297`). With `columns == 1`, “the last column” is column 0, so every iteration takes the wide character out of the row and puts it at the head of a new row that is again one column wide and again begins with a wide character. There is no state that advances, and `new_raw` grows by one `Row` per iteration.

Two columns is enough for the loop to terminate, and two columns is what the running terminal enforces:

```

pub const MIN_COLUMNS: usize = 2;

```

`zterminal_core/src/term/mod.rs:38`, applied in `SizeInfo::new` at `zterminal/src/display/mod.rs:251` as `cmp::max(columns as usize, MIN_COLUMNS)`. Every route into `Term::resize` in the binary passes through a `SizeInfo` — the window's own at `display/mod.rs:738` and each split pane's at `pane.rs:184`, whose `SizeInfo` is built by the same constructor at `pane.rs:730` — so the shipped application cannot reach one column no matter how far a split divider is dragged.

The defect is therefore in `zterminal_core`'s public API rather than in the terminal: `Grid::resize` accepts a width its own reflow cannot handle, and the guard that saves it lives in a different crate. A caller embedding the library — which is what `zterminal_core`'s package description invites — has no way to know the constant exists. The one-line repair is a clamp inside `resize`; the reason to know about it now is that the failure is an out-of-memory kill with no diagnostic, which is the hardest kind of bug to attribute after the fact.

Chapter 82 – The tmux Client That Is Not a Client

Chapter 62 takes the `ztmux-core` wire apart byte by byte: the sixteen-byte `imsg` header, the identify sequence, the file protocol that carries command output, the segfault avoided by sending an empty `ttname`. Chapter 28 describes what the tmux tab does with the results. Neither answers the question a reader of both is left with, which is what the channel *cannot* carry — and the answer shapes several features.

Everything below was produced against a live `tmux 3.7b` on a dedicated socket, by linking `ztmux-core` and calling `transport::command` directly.

It is not control mode

The name “control mode” belongs to `tmux -CC`, in which the client’s stdout becomes a stream of `%output`, `%window-add`, `%layout-change`, and similar notifications, and the multiplexer’s state changes are pushed to the client as they happen. `ztmux-core` does not do that. There is no `-C` or `-CC` anywhere in the crate, and the transport is a synthetic *command* client: connect, identify, send one `MSG_COMMAND`, read the reply, disconnect.

That distinction is the root of everything else in this chapter. A command client asks questions and gets answers. It is never told anything.

What it forwards

The identify sequence sends seven messages and the command one (`transport.rs:135-159`): long flags, `TERM`, an empty `ttname`, the current directory, the client PID, a one-variable environment, `MSG_IDENTIFY_DONE`, then `MSG_COMMAND` carrying `argc` and packed NUL-terminated argv.

Coming back, exactly three message types are acted on:

Message	Handling
<code>MSG_WRITE_OPEN</code>	acknowledged, stream recorded
<code>MSG_WRITE</code>	appended to stdout or stderr
<code>MSG_EXIT</code>	exit status; ends the read
<code>MSG_VERSION</code>	aborts with a version error
everything else	dropped

The catch-all is at `transport.rs:225`:

```
_ => { /* MSG_WRITE_CLOSE, MSG_READY, etc: ignore */ }
```

That is a reasonable design for a request/response client, and it works:

```
["list-sessions", "-F", "#{session_name}"] -> exit=0 stdout="probe1" stderr=""
["display-message", "-p", "#{version}"] -> exit=0 stdout="3.7b" stderr=""
["list-panes", "-a", ...] -> exit=0 stdout="%0 80x12\n%1 80x11" stderr=""
["no-such-command"] -> exit=1 stdout="" stderr="unknown command:
  ~ no-such-command"
```

Protocol version 8 ([transport.rs:23](#)) is still current for tmux 3.7b, stdout and stderr are separated correctly by the file-descriptor number in [MSG_WRITE_OPEN](#), and exit status arrives. Large replies survive chunking: [list-keys](#) returns 35,007 bytes intact in 42 ms, [show-options -g](#) 5,811 bytes in 33 ms.

What it drops

Notifications, entirely. Because there is no control mode, nothing tells zterminal that a pane was created, that a window was renamed, that a layout changed, or that output arrived. Every number in the tmux tab and the dashboard is a poll. Chapter 22’s note about the cost of looking is the consequence: the panel is a snapshot taken when it was asked for, and it is stale the instant the server changes.

Pane output. [%output](#) is a control-mode notification. Live pane content reaches the screen because a real [tmux](#) client is running in a PTY inside a pane — the double-parse Chapter 31 describes — not through this socket. [ops::capture](#) ([ops.rs:111-116](#)) can pull a pane’s visible text on demand via [capture-pane -p](#), and that is a poll too.

Client identity. The synthetic client has no tty, so it is not a client in any sense tmux cares about:

```
["display-message", "-p", "#{client_tty}|#{client_name}|#{client_control_mode}"]
  -> exit=0 stdout="||"
```

Three empty fields. It also does not appear in [list-clients](#), which is what makes [ops::clients_attached](#) ([ops.rs:92-100](#)) correct — the function asks whether any client is attached and is not confused by the asker.

Anything scoped to “the current client”. This is the part with user-visible consequences. [switch-client](#) operates on the client issuing it, and there is never one:

```
switch-client -p          exit=1 err="no current client"
```

That holds with a real client attached to the same session, verified by attaching one over a pty and repeating the run. The two bindings that use it — [tmux_next_session](#) and [tmux_prev_session](#), [zterminal/src/event.rs:1590-1591](#) — cannot work through this transport at all.

The rest of the bound commands do work, because they resolve a target from the server’s own notion of current rather than from the client:

```
new-window          exit=0
split-window -h     exit=0
next-window         exit=0
previous-window     exit=0
resize-pane -Z      exit=0
detach-client       exit=1 err="no current client"    (no client attached)
detach-client       exit=0                          (client attached)
last-window         exit=0
break-pane          exit=0
rotate-window       exit=0
next-layout         exit=0
kill-pane           exit=0
```

Ten of twelve, unconditionally. [detach-client](#) needs a real client to exist but does not need the caller to be one. [switch-client](#) needs the caller to be one and therefore never succeeds.

There is a subtler point in the ten that work: they act on tmux’s *current* window and pane, which is whatever the attached client last focused. That is usually the pane the user is looking at, because there is usually exactly

one client. It is not guaranteed to be, and the transport has no way to say “the pane this keystroke came from” — the pane id is not sent.

Failure is not reported

```
/// Run a tmux command fire-and-forget against the resolved server.
#[cfg(unix)]
pub fn run(args: &[&str]) {
    if let Some(sock) = socket_path() {
        let _ = command(&sock, args);
    }
}
```

`ops.rs:81-87`. Every one of the seventeen `tmux_*` keybindings at `event.rs:1580-1596` goes through this, and it discards the exit status, the stderr text, and any transport error. So `tmux_next_session` fails with `no current client` on every press and the user sees nothing at all — not a message bar entry, not a log line at default verbosity, not a bell.

The other consumers are better: `clients_attached`, `capture`, `set_option`, `set_buffer`, `delete_buffer`, `paste_buffer`, `set_key`, and `unbind_key` all check `o.exit == 0` and return a boolean the caller can surface.

Two seconds is not an error

Both socket timeouts are two seconds (`transport.rs:132-133`), and the read loop treats `WouldBlock` as a normal end of stream (`:176-183`). `TmuxOutput` derives `Default` (`:44-49`), so its `exit` starts at zero. A command that does not reply within two seconds therefore produces an empty, successful-looking result:

```
["wait-for", "zt-probe-never-signalled"] -> exit=0 bytes(out)=0 bytes(err)=0
↳ elapsed=2.002521959s
```

Exit zero, no output, two seconds gone. That is indistinguishable from a command that succeeded and printed nothing — which most of the ones bound to keys do.

The blast radius is bounded by what the callers ask for. None of the bound commands is slow, and the timeout exists precisely because an earlier version of this code waited the full two seconds on every command; the comment at `:161-169` records that and the drain-after-`MSG_EXIT` fix. But the failure mode is worth naming, because “empty and successful” is the same shape as “timed out”, and a server under load or a socket on a slow filesystem turns one into the other silently.

What this argues for

Nothing in this chapter is an argument against the design. A synthetic command client is the right way to ask a tmux server structured questions without screen-scraping, and Chapter 30’s cross-pane search and process tree are only possible because of it.

It is an argument for two small things. `ops::run` should return the exit status and stderr so that a binding that cannot work stops pretending it did; that is four lines and it would surface the `switch-client` problem the first time someone pressed the key. And a timeout should be distinguishable from success, which means an explicit sentinel rather than a defaulted zero.

Chapter 83 – Things That Pass

Chapter 54 argues that a green suite proves less than it appears to. This chapter collects the specific instances found in this codebase where something reports success for a reason unrelated to the thing being asserted — a test that passes because of a global counter, a command that reports success while doing nothing, a report that describes cells it did not compare, and a string extraction that depends on state the screen does not show.

They are gathered together because the shape is the same in every case, and the shape is what to look for: an observable that is *correlated* with correctness in the common case is being used as a *proof* of correctness.

A test that passes because it ran first

`hyperlinks` is one of the forty-five, it passes, and it pins the OSC 8 handling Chapter 71 describes. Its golden contains four hyperlink identifiers:

```
ids: ['0_zterm', '1_zterm', '42', 'hello']
uris: ['http://example.com/naoheu;ntahoeu', 'https://example.com']
```

`42` and `hello` come from the recording, which sets them explicitly with `id=`. `0_zterm` and `1_zterm` do not. They are synthesized, at `zterminal_core/src/term/cell.rs:85-97`:

```
let mut id = HYPERLINK_ID_SUFFIX.fetch_add(1, Ordering::Relaxed).to_string();
id.push_str("_zterm");
id
```

from a process-global `AtomicU32` declared at `:40`. The golden holds the values that counter produces when it starts at zero, so `hyperlinks` passes if and only if it is the first code in its process to create an anonymous hyperlink.

Today it is, because it is the only one of the forty-five recordings that contains an OSC 8 at all — a `grep` for `ESC ] 8 ;` across the corpus matches one file — and because `cargo test` gives each test binary its own process. Both of those are accidents. Add a second recording with an anonymous hyperlink and the two tests race: whichever runs second sees a counter at 2 and fails, non-deterministically, depending on thread scheduling.

The effect is easy to see by replaying the same recording twice in one process:

```
hyperlinks: as-recorded=true with-CR-restored=false
```

Both halves feed identical bytes — the carriage-return transform of Chapter 75 is a no-op on this file, since all fourteen of its linefeeds already have one. The second replay fails anyway, because the counter has moved.

What is actually being asserted here is “hyperlink identifiers are allocated in a particular order from a particular starting value”, which is not a property anyone wanted to pin. The property they wanted is “two cells that were part of the same OSC 8 span share an identifier, and cells from different spans do not”. That is expressible without the absolute value, and it would not be order-dependent.

A command that succeeds by doing nothing

```
["choose-tree", "-Zs"] -> exit=0 bytes(out)=0 bytes(err)=0
```

`choose-tree` opens an interactive chooser in a client. There is no client, so there is nothing to open, and `tmux` reports success because the command itself did not fail. `tmux_choose_tree` is bound at `zterminal/src/event.rs:1593`, goes through the fire-and-forget `ops::run` of Chapter 82, and produces a keybinding that returns zero and changes nothing.

This is the cleanest example of the shape. The exit status is a true statement about the command's execution and a false statement about the user's intent, and nothing in between the two notices the difference.

A report that describes the wrong cells

Chapter 77 establishes the mechanism; it belongs in this list for the same reason. When a ref test fails against a golden whose `Storage::visible_lines` is one less than its `Grid::lines` — thirty-three of forty-five — the harness's diff printer compares the golden's row $k+1$ against the replay's row k , and prints the result as though both were row k .

The report is not empty and not obviously wrong. It is a well-formatted list of cell coordinates and values, and for `clear_underline` it lists forty-two cells across two rows. Compared correctly, one row differs, and it is neither of those. A reader who trusted the report would go looking for a bug in the first two rows of a screen whose first two rows are identical on both sides.

Then it panics, which is the only reason anyone looked.

An extraction that depends on invisible state

`Term::bounds_to_string` turns a rectangle of grid into text; Chapter 71 covers the selection model that feeds it and Chapter 38 the clipboard that consumes it. On a ten-column terminal holding `pad` followed by seven spaces and an `X` — so that the `X` wrapped to the next row — the screen is:

```
2 |pad| Flags(WRAPLINE)
3 |X|  Flags(0x0)
```

and the extractions are:

```
whole screen: "abcde\nshort\npad      X"
row 2 alone:  "pad      "
row 3 alone:  "X"
row 1 alone:  "short"
```

Selecting both rows joins them with no newline, which is right — the wrap is not a line break and Chapter 71 explains why the `WRAPLINE` flag exists. Selecting row 2 alone keeps its seven trailing spaces, while selecting row 1 — the same visual shape, a short line on a ten-column screen — trims them. The difference is the `WRAPLINE` flag, which is not drawn anywhere.

Both behaviours are defensible in isolation. Together they mean that whether a selection carries trailing whitespace depends on whether the line it came from happened to wrap, which the user cannot see and did not choose. Pasting one into a shell is harmless; pasting one into a file, or hashing it for the divergence comparison of Chapter 61, is not.

What the renderer does

The most interesting version of this question — whether a cell that is drawn disagrees with the cell in the model — is the one this chapter cannot answer. It requires building `zterminal`, which pulls in `wgpu` and `winit`, and no such build was made. **CANNOT CHECK.**

What can be said is where to look, because the seams are named elsewhere in the book. Chapter 67 describes one instance per cell and two passes; Chapter 66 describes damage as a line and two columns, which is the mechanism by which a correct model can be drawn stale. Chapter 4's atlas eviction is the other candidate: a glyph evicted between the frame that measured it and the frame that drew it. Neither has a test in the workspace, because as Chapter 63 notes, the entire suite runs without a window, a GPU, or a font.

The common repair

Four instances, one pattern, and the repair is the same in each: assert the property, not a proxy for it.

The hyperlink test should assert that spans share identifiers, not what the identifiers are. The tmux binding should assert that the chooser opened, which it can only do by having a client — or it should not be bound at all. The diff printer should index both grids by the same rule instead of trusting a field that the comparison ignores. The extraction should decide once whether trailing whitespace is part of a selection, rather than inheriting the answer from a wrap flag.

None of these is a large change and all four are worth more than another recording in the corpus, because a forty-sixth recording pins forty-six sessions and a repaired assertion pins a property.

Appendix A — Command-Line Reference

The flags, options, and subcommands of `zterminal(1)`. The binary is `zterminal` (commonly aliased `zt`).

Flags

`-h`, `--help` · `--hold` (stay open after the child exits) · `--daemon` (no initial window) · `--print-events` · `-q` (quieter, min `-qq`) · `-v` (louder, max `-vvv`) · `--ref-test` · `-V`, `--version`.

Options

`--class <general>` or `<general>,<instance>` (window class; default `Zterminal,Zterminal`) · `-e`, `--command <cmd>...` (run a command; must be last) · `--config-file <path>` · `--embed <parent>` (X11 window ID) · `-o`, `--option <opt>...` (override config, e.g. `-o 'cursor.style="Beam"'`) · `--socket <path>` · `-T`, `--title <title>` (default `Zterminal`) · `--working-directory <dir>`.

Subcommands

`msg` — send IPC messages (`create-window`, `config`, `get-config`; Chapter 33). `migrate` — upgrade a config file: `-c/--config-file`, `-d/--dry-run`, `-i/--skip-imports`, `--skip-renames`, `-s/--silent`.

Appendix B – Configuration Reference

A field index of `zterminal.toml`, by table. Full prose is in Chapters 8–17 and `zterminal(5)`; the `zterminal`-specific additions are marked **(zt)**.

[general]

`import` (paths) · `working_directory` (string | "None") · `live_config_reload` (bool, default `true`) · `ipc_socket` (bool, default `true`, Unix).

[env]

Arbitrary `KEY` = "value" pairs, injected into every spawned process.

[window]

`dimensions` {columns, lines} (restart) · `position` · `padding` {x, y} · `dynamic_padding` · `decorations` (Full/None/Transparent/Buttonless) · `opacity` · `blur` · `startup_mode` (Windowed/Maximized/Fullscreen/ SimpleFullscreen, restart) · `title` · `dynamic_title` · `class` {instance, general} · `decorations_theme_variant` · `resize_increments` · `option_as_alt` · `level` · **(zt)** `background_image` · **(zt)** `background_image_opacity` · **(zt)** `overlay_opacity`.

[scrolling]

`history` (default 10000, max 100000) · `multiplier` (default 3).

[font]

`normal/bold/italic/bold_italic` {family, style} · `size` (default 11.25) · `offset` {x, y} · `glyph_offset` {x, y} · `builtin_box_drawing` (default `true`).

[colors]

`primary` {foreground #d8d8d8, background #181818, dim_foreground #828482, bright_foreground None} · `cursor/vi_mode_cursor` {text, cursor} (default {CellBackground, CellForeground}) · `search` {matches #181818/#ac4242, focused_match #181818/#f4bf75} · `hints` {start #181818/#f4bf75, end #181818/ #ac4242} · `line_indicator` {None, None} · `footer_bar` {#181818, #d8d8d8} · `selection` {CellBackground, CellForeground} · `normal/bright/dim` (16 ANSI each) · `indexed_colors` [{index, color}] · `transparent_background_colors` · `draw_bold_text_with_bright_colors`.

Default **normal**: black #181818 red #ac4242 green #90a959 yellow #f4bf75 blue #6a9fb5 magenta #aa759f cyan #75b5aa white #d8d8d8. Default **bright**: black #6b6b6b red #c55555 green #aac474 yellow #feca88 blue #82b8c8 magenta #c28cb8 cyan #93d3c3 white #f8f8f8. Default **dim**: black #0f0f0f red #712b2b green #5f6f3a yellow #a17e4d blue #456877 magenta #704d68 cyan #4d7770 white #8e8e8e.

[bell]

animation (ten curves, default **Linear**) · **duration** (ms, default **0**) · **color** (default #ffffff) · **command**.

[selection]

semantic_escape_chars · **save_to_clipboard** (default **false**).

[cursor]

style {**shape** **Block**, **blinking** **Off**} · **vi_mode_style** (default **None**) · **blink_interval** (default **750**) · **blink_timeout** (default **5**) · **unfocused_hollow** (default **true**) · **thickness** (default **0.15**).

[terminal]

shell (string | {**program**, **args**}) · **osc52** (**Disabled**/**OnlyCopy**/**OnlyPaste**/ **CopyPaste**, default **OnlyCopy**).

[mouse]

hide_when_typing (default **false**) · **bindings** (array; see [keyboard], action adds **ExpandSelection**).

[hints]

alphabet (default "jfkdl;ahguriewp") · **enabled** [{**regex**, **hyperlinks**, **post_processing**, **persist**, **action**, **command**, **binding**, **mouse**}].

[keyboard]

bindings [{**key**, **mods**, **mode**, **action** | **chars** | **command**}].

[debug]

render_timer · **persistent_logging** · **log_level** (**Off**...**Trace**, default **Warn**) · **renderer** (**glsl3**/**gles2**/**gles2pure**/**None**) · **print_events** · **highlight_damage** · **prefer_egl**.

Restart-only fields

Most fields reload live (Chapter 8), but a few are read once at window creation and take effect only on a new window or a restart. Editing them in the Settings tab saves the value; it applies when the window is next created. The restart-only set:

Field	Why it is read once
<code>window.dimensions</code>	the initial grid size is set when the window is built
<code>window.startup_mode</code>	windowed/maximized/fullscreen is a creation-time choice
<code>window.position</code>	the startup position is applied as the window opens
<code>window.class</code>	the X11/Wayland window class is fixed at creation

Everything else in the tables above reloads live when `general.live_config_reload` is on: fonts, colours, cursor, scrolling, bell, selection, hints, keybindings, and the debug flags all apply to running windows the moment the file is saved. The `zt msg config` path (Chapter 33) and the `-o` override (Chapter 8) reach the same live-reload machinery for the non-restart fields; the restart-only fields above are the exceptions to keep in mind when a change “does nothing” (Appendix G).

The zterminal additions, gathered

The fields this product adds on top of the inherited schema, marked **(zt)** throughout the tables, gathered in one place:

Field	Effect	Chapter
<code>window.background_image</code>	a GPU image drawn behind the cells	26
<code>window.background_image_opacity</code>	how strongly it shows	26
<code>window.overlay_opacity</code>	alpha of the glassy control-panel overlays	19

The rest of the product’s additions live outside `zterminal.toml` — the profiles, layouts, snapshots, triggers, snippets, recent-dirs, and GUI preferences are their own JSON files under `~/.zterminal` (Appendix E), managed by the control panel rather than the config schema. The three fields above are the only extension surface that is a `zterminal.toml` key; everything else the extension layer stores is a managed file, which is why the config reference is otherwise the inherited schema documented faithfully.

Appendix C – Default Keybindings

The defaults from `zterminal-bindings(5)`. `~Vi/~Search` mean the mode must *not* be active. Vi-mode and search-mode bindings are in Chapter 18.

General

`Paste` → Paste · `Copy` → Copy · `Paste` (Vi|Search) → Paste · `Ctrl+L` → ClearLogNotice · `Ctrl+L` (Vi|Search) → clear screen.

Scrolling

`Shift+PageUp/PageDown` (~Alt) → ScrollPageUp/Down · `Shift+Home/End` (~Alt) → ScrollToTop/Bottom.

Mouse

`Right` → ExpandSelection · `Right+Control` → ExpandSelection · `Middle` (~Vi) → PasteSelection.

Windows / Linux / BSD

`Ctrl+Shift+V` → Paste · `Ctrl+Shift+C` → Copy · `Ctrl+Shift+F/B` (~Search) → SearchForward/Backward · `Shift+Insert` → PasteSelection · `Ctrl+0` → ResetFontSize · `Ctrl+=/+/NumpadAdd` → IncreaseFontSize · `Ctrl+-/NumpadSubtract` → DecreaseFontSize. Windows only: `Alt+Enter` → ToggleFullscreen.

macOS

`Cmd+K` → clear screen + ClearHistory · `Cmd+0/=/-` → font reset/up/down · `Cmd+V` → Paste · `Cmd+C` → Copy · `Cmd+H` → Hide · `Cmd+Alt+H` → HideOtherApplications · `Cmd+M` → Minimize · `Cmd+Q/Cmd+W` → Quit · `Cmd+N` → CreateNewWindow · `Cmd+T` → CreateNewTab · `Cmd+Ctrl+F` → ToggleFullscreen · `Cmd+F/B` (~Search) → SearchForward/Backward · `Cmd+Shift+] / [` → Next/ Previous Tab · `Cmd+Tab/Cmd+Shift+Tab` → Next/Previous Tab · `Cmd+1–8` → SelectTab1–8 · `Cmd+9` → SelectLastTab.

The MenkeTechnologies extension shortcuts

The extension layer’s default shortcuts, gathered from Chapters 19–32. Every one is also a command-palette entry and, on macOS, a menu item with the same key-equivalent (Chapter 19), so the table is a convenience rather than something to memorize. macOS accelerators are shown; Linux/Windows route the same actions through `^␣` variants or the palette per the non-collision rule of Chapter 16.

Shortcut	Action	Chapter
<code>⌘,</code>	open the control panel / Settings	19–20
<code>⌘D</code>	open the Dashboard	22
<code>⌘K</code>	open the command palette	23
<code>⌘R</code>	open the shell-history palette	23
<code>⌘↑ / ⌘↓</code>	jump to previous / next prompt (OSC 133)	23, 27
<code>⌘⇧O</code>	select the last command's output	23, 27
<code>⌘E</code>	MRU quick-switch to the last-used tab	25
<code>⌘⇧E</code>	open Exposé (native + tmux panes)	25
<code>⌘⇧P</code>	cross-pane scrollback search	30
<code>⌘⇧B</code>	cross-session broadcast overlay	30
<code>⌘⇧S</code>	save / restore tmux sessions	28
<code>⌘⇧`</code>	toggle the Quake dropdown	25
<code>⌘D / ⌘⇧D</code>	split the pane vertically / horizontally	24
<code>⌘] / ⌘[</code>	focus next / previous pane	24
<code>⌘⇧←↑↓→</code>	focus the pane in a direction	24
<code>⌘⇧←↑↓→</code>	move (swap) the focused pane	24
<code>⌘⇧←↑↓→</code>	resize along the split	24
<code>⌘⇧↔</code>	zoom a pane to fill, toggle back	24
<code>⌘⇧W</code>	close the focused pane	24
<code>⌘⇧L</code>	save / restore a native split layout	24

Appendix D – Escape Sequence Support

The matrix from `zterminal-escapes(7)`. Statuses: IMPLEMENTED, PARTIAL, REJECTED (Chapter 6).

ESC codes

`ESC ( ) * +` (charsets B and O only), `ESC => 7 8 # 8 D E H M Z` — all IMPLEMENTED.

CSI

`CSI ` @ A/a B/b C/c D/d E/e F/f G/g H I J K L M n P $ p ? $ p SP q r S s T u ? u = u < u > u X Z` — IMPLEMENTED. `CSI m` — IMPLEMENTED for SGR `0–9, 21–25, 27–49, 58, 59, 90–97, 100–107`; REJECTED for `11–19, 51–55`. `CSI h/l` — PARTIAL (modes 4, 20). `CSI ? h/? l` — PARTIAL (modes 1, 3, 6, 7, 12, 25, 1000, 1002, 1004, 1005, 1006, 1007, 1042, 1049, 2004, 2026). `CSI t` — PARTIAL (22, 23); REJECTED (1–13, 15, 19–21, 24).

OSC

`OSC 0` (no icon names) `2 4 8 10 11 12 50` (CursorShape only) `52` (clipboard + primary only) `104 110 111 112` — IMPLEMENTED. `OSC 1` — REJECTED (icon names unsupported).

DCS

`DCS = s` — REJECTED (`CSI ? 2026 h/l` is used instead).

ESC codes, described

Sequence	Meaning
<code>ESC 7 / ESC 8</code>	save / restore cursor and pen (Chapter 3)
<code>ESC D / ESC M</code>	index / reverse index (move down / up, scrolling at margins)
<code>ESC E</code>	next line (carriage return + index)
<code>ESC H</code>	set a tab stop at the cursor column
<code>ESC = / ESC ></code>	keypad application / numeric mode
<code>ESC # 8</code>	DEC alignment test (fill the screen with E)

Sequence	Meaning
<code>ESC () * +</code>	designate character set (B and O only)
<code>ESC Z</code>	identify terminal (Device Attributes)

CSI, described

Sequence	Meaning
<code>CSI A–H, a–f</code>	cursor motion (up/down/forward/back, position)
<code>CSI J / CSI K</code>	erase in display / in line
<code>CSI X</code>	erase N characters in place
<code>CSI L / CSI M</code>	insert / delete lines
<code>CSI P / CSI @</code>	delete / insert characters
<code>CSI S / CSI T</code>	scroll up / down
<code>CSI m</code>	SGR — set colour and attributes
<code>CSI n</code>	device status report
<code>CSI g</code>	clear tab stop(s)
<code>CSI r</code>	set the scroll region (top; bottom)
<code>CSI s / CSI SP q</code>	save cursor / set cursor style
<code>CSI u, ? u, = u, < u, > u</code>	Kitty keyboard protocol (Chapters 16, 40)
<code>CSI ? h / CSI ? l</code>	set / reset a private mode (see below)

Private modes, described

The `CSI ? h / CSI ? l` modes zterminal handles, and what each does — the set an application toggles during its startup handshake (Chapter 6):

Mode	Meaning
1	application cursor keys (arrows send SS3)
3	132-column mode
6	origin mode (address relative to the scroll region)
7	autowrap at the last column
12	cursor blink
25	cursor visibility
1000 / 1002	mouse reporting: clicks / clicks + drag
1004	focus reporting
1005 / 1006	mouse encoding: UTF-8 / SGR
1007	alternate-scroll (wheel sends arrows on the alt screen)
1042	urgency-hint bell

Mode	Meaning
1049	alternate screen (save/restore the primary)
2004	bracketed paste (Chapter 38)
2026	synchronized output (Chapters 5, 6)

OSC, described

Sequence	Meaning
<code>OSC 0 / OSC 2</code>	set the window title (icon-name form ignored)
<code>OSC 4</code>	set / query a palette colour
<code>OSC 8</code>	hyperlink (Chapter 15)
<code>OSC 10 / 11 / 12</code>	set foreground / background / cursor colour
<code>OSC 50</code>	set the cursor shape
<code>OSC 52</code>	clipboard access, gated by <code>terminal.osc52</code> (Chapters 14, 39)
<code>OSC 104 / 110 / 111 / 112</code>	reset palette / fg / bg / cursor colour

Appendix E — The ~/.zterminal Directory

All zterminal state and logs live under ~/.zterminal. The files and what writes them:

- `zterminal.toml` — the canonical config (Chapter 8); edited by hand or by the Settings tab.
- `zterminal.log` — the runtime log; level set by `debug.log_level`, kept on exit if `debug.persistent_logging` is on; shown in the Logs tab.
- `layouts/<name>.json` — saved native split layouts (Chapter 24).
- `snapshots/<name>.json` and `<name>.contents/` — saved tmux session snapshots and their captured pane scrollbar (Chapter 28).
- `recent_dirs.json` — the harvested recent-directories list (Chapter 32).
- `triggers.json` — per-pane output triggers (Chapter 32).
- plus the saved snippets, profiles, and GUI preferences (colour scheme, effects, light/dark, `tmux_restore_processes` whitelist) the control panel manages.

As a tree, the whole state directory:

```
~/.zterminal/
├── zterminal.toml          the canonical config (Chapter 8)
├── zterminal.log          the runtime log (Chapter 17)
├── layouts/
│   └── <name>.json        native split layouts (Chapter 24)
├── snapshots/
│   ├── <name>.json        tmux session snapshots (Chapter 28)
│   └── <name>.contents/    captured pane scrollbar (Chapter 28)
├── recent_dirs.json       harvested working directories (Chapter 32)
├── triggers.json          per-pane output triggers (Chapter 32)
└── (snippets, profiles, GUI preferences the panel manages)
```

The organizing principle is that everything zterminal persists is a plain file under one directory you own — no scattered dotfiles, no registry, no hidden state elsewhere. Back up ~/.zterminal and you have backed up the whole environment: the config, the saved layouts and sessions, the harvested lists, the profiles. The formats are TOML for the config and JSON for the managed state, both text, both diff-able, both versioned so an older file upgrades cleanly (Chapter 8's migration discipline applies to the config; the JSON managers tolerate older shapes).

On Windows the config lives at `%APPDATA%\zterminal\zterminal.toml`.

Appendix F — Glossary

Alternate screen — the non-scrolling secondary grid full-screen TUIs paint on; entered via private mode 1049. (Chapter 3.)

Cell — one grid position: a character plus its colours and attributes. (Chapter 3.)

Block selection — a rectangular selection of a column range across a row range, for lifting one column out of tabular output. (Chapter 38.)

Bracketed paste — private mode 2004; wraps pasted text so a program can tell it from typed input and not execute it immediately. (Chapter 38.)

CSI — Control Sequence Introducer, `ESC [`; the largest escape family. (Chapters 3, 6.)

Damage — the cells changed since the last frame; the renderer redraws only damage. (Chapter 4.)

Frame coalescing — deferring the draw so all damage since the last frame is drawn once at the display's refresh cadence. (Chapter 5.)

Glyph atlas — the GPU texture holding each rasterized glyph once, so drawing a cell is drawing a textured quad. (Chapter 4.)

Grapheme cluster — a base scalar plus its combining marks or joiners, stored against one cell (or a wide cell and its spacer). (Chapters 3, 11.)

Hint — a regex- or hyperlink-matched on-screen target that can be opened, copied, selected, or piped. (Chapter 15.)

`include_dir!` — the macro that embeds the control-panel frontend into the binary. (Chapter 19.)

`img` — the framing of tmux's wire protocol (version 8) the tmux client speaks. (Chapter 28.)

Ligature — a single glyph substituted for a multi-character sequence; not performed by the per-cell grid renderer, by design. (Chapter 11.)

OSC — Operating System Command, `ESC [`; titles, palette, hyperlinks, clipboard. (Chapters 3, 6.)

OSC 133 — the FinalTerm/iTerm2 semantic-prompt marks for prompt and output boundaries. (Chapter 27.)

Profile — a named snapshot of the whole config, GUI look, and tmux server state. (Chapter 21.)

PTY — pseudo-terminal; the kernel object pairing the emulator with the shell. (Chapters 1, 24.)

Scanner — the single pre-`vte` stage that peels image and OSC 133 sequences out of the PTY stream. (Chapters 26, 27.)

Scale factor — the monitor's DPI multiplier the window manager reports, used to turn point-based font metrics into pixels. (Chapter 10.)

Scroll region — the top/bottom margins (`CSI r`) within which scrolling happens, leaving rows outside them fixed. (Chapter 3.)

Scrollback — the buffer of lines scrolled off the primary screen; sized by `scrolling.history`. (Chapters 3, 13.)

SGR — Select Graphic Rendition, `CSI m`; sets colour and attributes. (Chapters 3, 6.)

Split tree — zterminal's native i3-style binary tree of panes, one PTY each. (Chapter 24.)

Synchronized output — private mode 2026; the application asks the terminal to hold the frame until a repaint is complete, so it lands atomically. (Chapters 5, 6.)

vi mode — the keyboard-driven scrollbar-navigation and selection mode. (Chapter 18.)

vte — the VT parser library the core uses. (Chapters 3, 4.)

Wire protocol — tmux's client/server imsg protocol the client speaks directly. (Chapter 28.)

zgui-core — the shared component library the control panel is built from. (Chapter 19.)

zterminal:// — the custom scheme the in-process WebView is served over. (Chapter 19.)

zt msg — the IPC command for controlling a running instance. (Chapter 33.)

Alternate scroll — private mode 1007; wheel notches become arrow keys on the alternate screen, so the wheel pages a program that has no mouse handling. (Chapter 7.)

Application cursor keys — private mode 1; arrows send SS3 (**E5C 0 A**) instead of CSI (**E5C [A**). An application that exits without resetting it is the usual cause of arrows printing escape text. (Chapters 6, 7.)

Coalescing — see *frame coalescing*.

Copy-mode — tmux's own scrollbar navigation, which reads the tmux server's history; the correct instrument inside a tmux pane, where zterminal's vi mode reads a different buffer. (Chapter 31.)

Cooldown — the interval that collapses a burst of matching output lines into one trigger firing. (Chapter 32.)

Focus reporting — private mode 1004; the terminal sends **CSI I** and **CSI O** as the window gains and loses focus. (Chapter 7.)

Kitty keyboard protocol — the **CSI u** family, which encodes every key event with explicit codepoint and modifier parameters and manages its flags as a push/pop stack, resolving the ASCII collisions of the legacy encoding. (Chapters 6, 7, 40.)

Override — a runtime configuration change pushed by **zt msg config** or **-o**, layered over the loaded config without editing the file; cleared with **-r**. (Chapters 8, 33.)

Passthrough — tmux's **allow-passthrough** option, which lets a program in a pane speak escape sequences past tmux to the outer terminal; required for inline images and OSC 133 marks inside tmux, and off by default for the reason Chapter 39 gives. (Chapters 26, 27, 39.)

Replant / relaunch — the two things a restore can do with a captured command line: type it at a fresh prompt without running it, or actually re-execute it. (Chapter 29.)

SGR mouse encoding — private mode 1006; mouse coordinates as decimal CSI parameters, with no 223-column ceiling and distinct press/release terminators. (Chapter 7.)

Wake source — anything that brings the event loop out of sleep: a readable PTY, a window event, an expiring timer, a socket message, a bridge post. (Chapter 36.)

Appendix G – Troubleshooting

A terminal has a small number of failure modes, and nearly every one of them has a named cause somewhere in this book. This appendix is the index from symptom to cause: what you see, why it happens, and which chapter's mechanism is responsible. Nothing here is a workaround for a bug; each entry is a place where the terminal's documented behaviour meets an environment that has not been told about it.

Colours are wrong or missing over SSH

The symptom is a remote application that renders in eight colours, or with no colour, or that errors with “terminal not fully functional.” The cause is almost always terminfo (Chapter 40): the remote host has never heard of `TERM=zterminal`, so ncurses cannot find the capability entry and either falls back badly or refuses. The two honest fixes are the ones Chapter 40 states — compile the entry onto the remote once with `infocmp zterminal | ssh host tic -x -`, or set `TERM=xterm-256color` for hosts you will not revisit and accept the common-denominator capabilities. If colours are wrong *locally*, the cause is different: check whether the application is emitting truecolor SGR that a 256-colour theme is quantizing (Chapter 3), or whether `draw_bold_text_with_bright_colors` (Chapter 12) is coupling bold to the bright palette in a way a colour scheme did not expect.

Arrow keys or function keys send the wrong bytes

The symptom is arrows that print `^[` in the shell, or an editor that does not recognize a modified key. The usual cause is application-cursor-keys mode (private mode 1, Chapter 6): an application that set it and exited without resetting it leaves the terminal sending SS3-prefixed arrows the shell does not expect. Resetting the terminal (`reset`, or `tput rmkx`) clears it. For modified keys that collapse together — `Ctrl+I` indistinguishable from `Tab`, `Ctrl+M` from `Enter` — the cause is the legacy keyboard encoding's ambiguity, and the fix is an application that negotiates the Kitty keyboard protocol (`CSI u`, Chapters 6, 16, 40), which zterminal implements; an application that does not ask for it gets the legacy encoding by design.

Box drawing has gaps, or icons are missing

If a TUI's borders show hairline gaps, the text font's box-drawing glyphs are the culprit and `font.builtin_box_drawing` (Chapters 4, 11) is the fix — it defaults on, so a gap usually means it was turned off. If powerline separators or Nerd Font icons render as tofu boxes, the primary face lacks them and the fallback chain (Chapter 10) has nothing to fall back to — install a Nerd Font, or let first-run seeding pick one (Chapter 8). If an emoji or a composed glyph is missing while its components render, that is the grapheme story of Chapter 11: the layout is correct and stable, but the fallback font has no glyph for the composed cluster, so it decomposes rather than shifting the grid.

Images do not display

If an inline image prints as a wall of base64 or simply nothing, walk the causes in order. Inside tmux, the image protocols require `set -g allow-passthrough on` (Chapters 26, 39) — off by default in tmux for the security reason Chapter 39 gives — and without it tmux swallows the sequences. Outside tmux, confirm the

tool is sending a protocol `zterminal` decodes (Kitty, Sixel, or iTerm2, Chapter 26) and, ideally, that it queried support first (`a=q`, Chapter 40) rather than assuming. The bundled `zterminal-icat` (Chapter 26) sidesteps both problems by enabling passthrough itself and placing via Unicode placeholders that survive a tmux redraw; if a plain display escape vanishes on the next repaint inside tmux, that placeholder behaviour is exactly what is missing.

Prompt jumping and failed-command marks do nothing

The `⌘↑/⌘↓` prompt jumps, the `⌘⇧0` output selection, the gutter marks, and the command history all depend on OSC 133 marks that the shell must emit (Chapter 27). If they do nothing, the shell integration snippet is not sourced — add the `source ../zterminal-shell-integration.<shell>` line from `extra/shell-integration/` — and inside tmux the same `allow-passthrough on` the images need (Chapter 39) is required for the marks to survive the multiplexer. The marks are zero-width and harmless in other terminals, so there is no reason to guard the source line per terminal (Chapter 27).

A configuration change did nothing

If editing `zterminal.toml` had no effect, three causes account for nearly all of it. The change may be to a restart-only field — `window.dimensions`, `window.startup_mode`, and the few others marked in Appendix B are read once at window creation (Chapter 8). Or `general.live_config_reload` may be off, in which case nothing reloads until restart (Chapter 8). Or the file being edited is not the file being read: the resolution order of Chapter 8 picks the first match, so a stale `$XDG_CONFIG_HOME` copy or a `--config-file` override can shadow the canonical `~/.zterminal/zterminal.toml` the Settings tab reports it is editing (Chapter 20). `zt msg get-config` (Chapter 33) reads back the runtime config so you can confirm what the running instance actually believes.

The window is opaque when transparency was configured

`window.opacity` below `1.0` with no visible transparency usually means the compositor is the obstacle, not the config. On Linux, EGL and transparency interact — `debug.prefer_egl` (Chapter 17) notes transparency may not work under EGL on Linux/BSD — and a compositor that is not running or not compositing this window shows a black background instead of the desktop behind it. `window.blur` (Chapter 9) additionally needs a compositor that honours the blur request (macOS, KDE Wayland); elsewhere it is silently ignored. If only *some* cells are transparent, that is `colors.transparent_background_colors` (Chapter 12) being off, which limits opacity to the default background rather than every cell.

Scrolling in a tmux pane goes nowhere, or shows the wrong text

If entering vi mode inside a tmux pane and pressing `g` does not walk that pane's history, nothing is broken: there are two scrollbacks and you are reading the wrong one. A tmux pane's history belongs to the tmux server, while `zterminal`'s vi mode reads `zterminal`'s own buffer, which holds only what tmux has painted (Chapters 18, 31). Use tmux's copy-mode inside a tmux pane, and `zterminal`'s vi mode in a native pane. The same split explains why raising `scrolling.history` does not deepen a tmux pane's history — that depth is tmux's own limit, on the server's side of the seam.

The mouse selects nothing, or the wrong thing

If dragging does not select text, an application has captured the mouse by enabling a reporting mode, and your drag is being delivered to it as an escape sequence rather than handled by the terminal (Chapters 7, 14). Hold **Shift** while dragging to suppress the reporting for that one action and get the terminal's own selection back. If clicks work on the left of a wide window and stop working past a certain column, the application negotiated the legacy mouse encoding rather than SGR (mode 1006), whose coordinate range is capped at column 223 — that is a limitation of what the application asked for, not of the terminal (Chapter 7). And if a double-click grabs more or less than you expect, that is `selection.semantic_escape_chars` deciding where a word ends (Chapter 13), which is a set you can tune.

Diagnosing anything else: the debug surface

When a symptom is not in this list, the `[debug]` table (Chapter 17) is the instrument panel. `debug.log_level = "Debug"` (or `-vvv` on the command line, optionally with `ZTERM_EXTRA_LOG_TARGETS` to include `winit` or `vte`) writes a verbose trace to `~/.zterminal/zterminal.log`, which the Logs tab (Chapter 19) renders live. `debug.highlight_damage` (Chapters 5, 17) tints what the renderer actually redraws, which diagnoses “the screen looks wrong” problems by showing which cells the terminal thinks changed. `debug.render_timer` (Chapters 5, 17) overlays per-frame draw time for “it feels slow” problems. `debug.renderer` forces a specific backend (`glsl3`, `gles2`, `gles2pure`) when the automatic choice misbehaves on a particular GPU or driver. And `--print-events` logs the raw window events when input itself is suspect. None of these is stable API (Chapter 17), but together they turn “something is wrong” into a specific, locatable fact — which is the same disposition the whole book takes toward the program.

Appendix H — Worked Configurations

The field-by-field walk of Chapters 8–17 and the reference of Appendix B describe every setting in isolation. This appendix assembles them into complete, realistic `zterminal.toml` files for common working styles. Each is a file you could drop into `~/.zterminal/` and use; each is annotated with the chapter that explains the choice. None uses a field not documented elsewhere in the book.

A minimal, fast config

The smallest useful file — a good starting point that changes only what most people change first:

```
[font]
normal = { family = "JetBrainsMono Nerd Font", style = "Regular" }
size = 13.0

[scrolling]
history = 20000

[cursor]
style = { shape = "Block", blinking = "Off" }
```

Everything else is the default of Chapters 9–17: the base16 palette, full window decorations, the OnlyCopy OSC 52 posture (Chapter 39), live reload on. This is the “I just want a good terminal” file.

A translucent, themed config

For a desktop with a compositor, leaning on the renderer’s compositing pass (Chapter 4):

```
[general]
import = ["~/zterminal/theme.toml"]      # the full [colors] block (Appendix)

[window]
opacity = 0.88
blur = true                             # macOS / KDE Wayland only (Chapter 9)
padding = { x = 10, y = 10 }
dynamic_padding = true
decorations = "Buttonless"              # macOS (Chapter 9)

[colors]
transparent_background_colors = true      # opacity on every cell bg (Chapter 12)

[font]
normal = { family = "FiraCode Nerd Font", style = "Regular" }
size = 12.5
offset = { x = 0, y = 2 }                # a little line spacing (Chapter 10)

[bell]
animation = "EaseOutExpo"                # visual bell instead of audible
duration = 120
color = "#f4b755"
```

The `transparent_background_colors` line is the one people miss: without it, `opacity` affects only the default background, so a cell with an explicit background colour stays opaque and the translucency looks patchy (Chapter 12). The visual bell replaces the audible one — `bell.duration` above zero enables it, and the easing curve is cosmetic (Chapter 13).

A tiling-heavy, keyboard-driven config

For a user who lives in native splits (Chapter 24) and drives everything from the keyboard, adding custom hints (Chapter 15) and a couple of bindings:

```
[window]
option_as_alt = "Both"                  # free the Option key for Alt bindings

[mouse]
hide_when_typing = true

[[hints.enabled]]
regex = "(https?|ftp|file)://[^\u0000-\u001F\u007F-\u009F<>\"\\s{-}\\^<>`]+\"
action = "Copy"
binding = { key = "U", mods = "Control|Shift" }

[[hints.enabled]]
regex = "[0-9a-f]{7,40}"                  # a git hash - copy on Ctrl+Shift+G
action = "Copy"
binding = { key = "G", mods = "Control|Shift" }

[[hints.enabled]]
regex = "(/[^\ ]+)"                       # a filesystem path - open in $EDITOR
command = { program = "sh", args = ["-c", "$EDITOR \"$1\"", "sh"] }
binding = { key = "E", mods = "Control|Shift" }

[keyboard]
bindings = [
  { key = "Return", mods = "Command|Shift", action = "ToggleViMode" },
]
```

The third hint is the general pattern worth stealing: a `command` hint receives the match as its final argument (Chapter 15), so wrapping it in `sh -c '... "$1"' sh` lets you do anything with the matched text — open it, grep it, feed it to a tool. Nothing here needs the GUI, but the Hints tab (Chapter 15) edits this exact `[hints]` table if you would rather click than type.

A remote-work config

For a machine used mainly to SSH elsewhere, tuned for the terminfo and clipboard realities of Chapters 39–40:

```
[env]
TERM = "zterminal"                # local; copy the entry to remotes too

[terminal]
osc52 = "CopyPaste"              # allow remote clipboard both ways...

[selection]
save_to_clipboard = true

[scrolling]
history = 100000                 # the maximum (Chapter 13)
```

The one deliberate risk here is `osc52 = "CopyPaste"`, which opens clipboard *reading* to remote applications (Chapter 39). It is convenient when you trust your hosts and want a remote editor to paste from your local clipboard; it is a choice to make eyes-open, because the default withholds read for the exfiltration reason Chapter 39 gives. Everything else on this file is about surviving the SSH hop: advertise `zterminal` locally, keep a deep scrollback for long remote sessions, and mirror selections to the clipboard so a copy on either side is available everywhere.

Appendix I – The Escape Sequence Cookbook

Chapter 6 and Appendix D catalogue what zterminal answers; this appendix shows the answers in use. Each recipe is the actual byte sequence for a common operation, written with `ESC` for the escape byte (`0x1B`) and `ST` for the string terminator (`ESC \`). These are the sequences your shell, editor, and prompt already emit — seeing them by hand demystifies the byte stream the grid consumes. You can try any of them with `printf`; where noted, a sequence expects a reply.

Cursor and screen

```
ESC [ H          move the cursor to the top-left (row 1, col 1)
ESC [ 10 ; 40 H  move to row 10, column 40
ESC [ 2 J        erase the whole screen
ESC [ K          erase from the cursor to the end of the line
ESC [ s          save the cursor position; ESC [ u restores it
ESC 7 ... ESC 8  save/restore cursor AND pen+charset (Chapter 3)
ESC [ 3 J        erase the scrollbar as well as the screen
```

Colour and attributes (SGR)

```
ESC [ 1 m        bold on
ESC [ 3 m        italic on
ESC [ 4 m        underline on
ESC [ 31 m       foreground = ANSI red (palette index 1)
ESC [ 91 m       foreground = bright red (index 9)
ESC [ 38 ; 5 ; 208 m foreground = 256-palette index 208
ESC [ 38 ; 2 ; 0 ; 175 ; 255 m foreground = truecolor #00afff
ESC [ 48 ; 2 ; 24 ; 24 ; 24 m background = truecolor #181818
ESC [ 0 m        reset all attributes and colours
```

The parameters accepted here are exactly those in the `CSI m` partials of Appendix D — `38;5;n` and `38;2;r;g;b` for foreground, `48;...` for background — and a parameter outside the accepted set (say, `53` for overline) is ignored rather than half-answered, per Chapter 6’s REJECTED discipline.

The alternate screen

```
ESC [ ? 1049 h    switch to the alternate screen, saving the primary
...full-screen app draws here...
ESC [ ? 1049 l    switch back, restoring the primary exactly
```

This is the round trip every TUI performs (Chapter 6). It is why `vim` leaves your scrollbar untouched when it exits: it painted on the alternate screen the whole time, and switching back restored the primary screen and cursor.

Titles, hyperlinks, and the clipboard

```
ESC ] 0 ; my session BEL          set the window/icon title
ESC ] 8 ; ; https://example.com ST begin a hyperlink...
click me
ESC ] 8 ; ; ST                    ...end the hyperlink (Chapter 15)
ESC ] 52 ; c ; SGVsbG8= ST        set the clipboard to base64 "Hello" (Chapter 39)
```

BEL (`0x07`) or **ST** terminates an OSC string. The hyperlink form (**OSC 8**) wraps the visible text between a begin with a URI and an empty end; zterminal's hints and the platform opener handle the click (Chapters 15, 39). The **OSC 52** clipboard write succeeds under the default **OnlyCopy** posture; a *read* request would be refused unless `terminal.osc52` is widened (Chapter 39).

Sequences that expect a reply

```
ESC [ c          Device Attributes – the terminal replies with its type
ESC [ 6 n        Cursor Position Report – replies ESC [ row ; col R
ESC [ ? 2026 h ... ESC [ ? 2026 l   synchronized output: hold the frame between
ESC [ > u        push a Kitty keyboard flag; ESC [ ? u queries the mode
```

The reply-bearing sequences are the runtime capability queries of Chapter 40: an application sends one and adapts to the answer rather than trusting **TERM**. The Cursor Position Report is also the classic hazard of Chapter 39 — a sequence that returns attacker-influenceable data — which is why a terminal reports only what it legitimately knows and never echoes planted text back onto the command line.

Appendix J – Environment Variables

zterminal reads and sets a handful of environment variables, scattered through the chapters where they matter. Gathered here with what each does and which chapter covers it, both the ones the terminal *reads* to configure itself and the ones it *sets* for the programs it spawns.

Read by zterminal

The “read at” column names the call site, and a variable is listed whether zterminal reads it itself or a dependency reads it on zterminal's behalf. The second case is marked, because it changes what happens when the variable is unset — the default is the dependency's, not zterminal's.

Variable	Effect	Chapter	Read at
<code>SHELL</code>	the shell to spawn when <code>terminal.shell</code> is unset	14	<code>zterminal_core/src/tty/unix.rs:149</code>
<code>TMUX</code>	resolves the tmux server socket, pid, and session for the wire client	28	<code>crates/ztmux-core/src/transport.rs:65</code>
<code>XDG_CONFIG_HOME</code>	the first two entries of the config resolution order	8	via <code>xdg 3.0.0: base_directories.rs:308</code> , reached from <code>zterminal/src/config/mod.rs</code> , lines 507–509
<code>WINIT_X11_SCALE_FACTOR</code>	overrides the reported DPI scale on X11	10	via <code>winit 0.30.13: platform_impl/linux/x11/util/randr.rs:112</code>
<code>HISTFILE</code>	history-file fallback, used only when the focused shell's own <code>HISTFILE</code> cannot be read	23	<code>zterminal/src/settings.rs:336</code>
<code>ZTERM_LOG</code>	the log file path; set at startup and read back when the debug bundle is assembled	17	<code>zterminal/src/logging.rs:254</code>
<code>ZTERM_EXTRA_LOG_TARGETS</code>	adds other libraries (<code>winit</code> , <code>vte</code>) to the log	17	<code>zterminal/src/logging.rs:43</code>
<code>ZTERM_SOCKET</code>	the IPC socket a <code>zt</code> subcommand connects to, tried after <code>--socket</code> and before the socket-directory scan	33	<code>zterminal/src/polling/ipc.rs:181</code>
<code>ZTERM_NO_TMUX_PROFILES</code>	suppresses tmux-backed profile mirroring	28	<code>zterminal/src/settings.rs:1749</code>
<code>ZTERM_WINDOW_ID</code>	default <code>-w</code> target for <code>zt msg</code> , read through clap's <code>env</code>	33	<code>zterminal/src/cli.rs:382, :397</code>

One variable earlier editions of this appendix listed here does not belong: `TERM`. It is written and never read. `zterminal_core/src/tty/mod.rs:105` calls `env::set_var("TERM", terminfo)` so the child inherits it, and there is no `env::var("TERM")` anywhere in the tree; it moves to the set table below. Anyone reading the old entry would have concluded that exporting `TERM=zterminal` before launch changes what the emulator advertises, and it does not — the value is chosen by `terminfo_exists("zterminal")` and overwritten unconditionally.

The two dependency-mediated rows above are worth stating precisely, because a `grep` of this repository for their names finds nothing but prose and would suggest deleting them. `XDG_CONFIG_HOME` appears in the Rust sources only in the doc comment at `zterminal/src/config/mod.rs:376–377` and the clap help at `zterminal/src/cli.rs:83` — but `find_config(:507–509)` calls `xdg::BaseDirectories`, and that crate reads the variable at `base_directories.rs:308`, defaulting to `$HOME/.config` when it is unset. The variable does reach the config search; it is just consumed one crate down, under the prefix `find_config` passes (`zterminal`), which is the level of detail Chapter 8's list flattens. `WINIT_X11_SCALE_FACTOR` is the same shape: its only appearance in this repository is `extra/man/zterminal.5.scd:77`, and the read is `winit's`, at `platform_impl/linux/x11/util/andr.rs:112`, behind `winit's` own X11 `cfg` — so it has an effect on X11 and none anywhere else, which is what the man page says.

Set by zterminal for spawned processes

Variable	Effect	Chapter	Set at
<code>TERM</code>	the terminfo entry the child consults: <code>zterminal</code> where that entry is installed, else <code>xterm-256color</code>	40	<code>zterminal_core/src/tty/mod.rs:105</code>
<code>COLORTERM</code>	set to <code>truecolor</code> , advertising 24-bit colour to the child	40	<code>zterminal_core/src/tty/mod.rs:108</code>
<code>ZTERM_WINDOW_ID</code>	identifies the window, the default target for <code>zt msg config</code>	29, 33	<code>zterminal_core/src/tty/unix.rs:230</code>
<code>ZTERM_SOCKET</code>	the IPC socket path, exported so <code>zt</code> finds this instance	33	<code>zterminal/src/polling/ipc.rs:42</code>
<code>ZTERM_LOG</code>	the log file this process is writing	17	<code>zterminal/src/logging.rs:297</code>
<code>ZT_TRIGGER_TEXT</code>	the matched text, in a trigger command's environment	32	<code>zterminal/src/triggers.rs:433, :443</code>
<code>ZT_TRIGGER_NAME</code>	the trigger's name, in its command's environment	32	<code>zterminal/src/triggers.rs:433, :443</code>
anything in <code>[env]</code>	injected into every spawned process, including the shell	8	—

Two of these repay a second look. `HISTFILE` is looked up in the *focused child shell's* live process environment first (Chapter 23) — which is what lets the history palette open the actual history file of the shell you are looking at, in the format that shell uses, rather than guessing a default path. Only when that lookup fails does `resolve_histfile` (`zterminal/src/settings.rs:332`) fall back to `zterminal's` own `$HISTFILE`, and then to `~/.zsh_history`, `~/.zhistory`, `~/.histfile` and `~/.bash_history` in that order. The variable is listed in the read table for that fallback and for no other reason; describing it as read purely from the child,

as earlier editions did, understates where the palette will look when the child's environment is unavailable. And `ZTERM_WINDOW_ID` is the seam that makes `zt msg config -w "$ZTERM_WINDOW_ID"` (Chapter 33) able to retheme the specific window a script is running in — the terminal tells each child which window it lives in, and the child can address that window back over the IPC socket. The `[env]` table is the general mechanism for the rest: anything you put there is exported into every process the terminal launches, and the Env Vars manager (Chapter 32) both writes it and hot-injects it into shells already running.

Appendix K — An Annotated Session

Every mechanism in this book has been described in isolation. This appendix runs one ordinary session end to end and names which mechanism is responsible at each step — launch, prompt, a command, a full-screen application, a resize, and quit. It introduces nothing new; it is the index of the whole book arranged by *when things happen* rather than by subject, and it is the fastest way to see how the pieces compose.

Launch

The process starts and resolves its configuration by the search order of Chapter 8, taking the first match — normally `~/.zterminal/zterminal.toml`, seeded on first run with an auto-detected Nerd Font. It reads `[font]` and resolves the four faces (Chapter 10), asks the window system for the monitor's scale factor, and multiplies the point metrics into pixels to get the cell size. The window opens at `window.dimensions` if set, in the `window.startup_mode` it names — both restart-only for the reason Chapter 9 gives: they are arguments to *creating* a window.

Dividing the content area by the cell size gives the column and row counts, and the grid of Chapter 3 is allocated at that size with `scrolling.history` lines of scrollbar behind it. A pseudo-terminal is created (Chapter 35), the shell from `terminal.shell` — or `$SHELL` — is spawned into its secondary end with `[env]` injected and `ZTERM_WINDOW_ID` set (Appendix J), and the event loop of Chapter 36 begins waiting. If a snapshot is marked **Set startup** (Chapter 28), the arrangement it records is rebuilt now, over the wire.

The first prompt

The shell initializes, sources your rc, and — because that rc sources the integration snippet of Chapter 27 — begins emitting OSC 133 marks. It writes its prompt to the PTY. Those bytes wake the loop.

The read is a batch, not a byte (Chapter 37, step 1). The pre-`vte` scanner makes one linear pass first (step 2), and recovers the `OSC 133 ; A` and `OSC 133 ; B` marks that bracket the prompt, anchoring them to absolute scrollbar line numbers so a later reflow cannot move them (Chapter 27). What remains goes to the `vte` parser (step 3): SGR sequences set the pen — bold, a truecolor foreground, a palette background, exactly the `CSI m` walk of Chapter 6 — and the printable characters land in cells, each resolved to a width first so a powerline separator or an emoji claims the right number of columns (Chapter 3).

Damage accumulates (step 4). The loop finishes the batch, reaches its draw stage, and coalesces everything into one frame (step 5). Each new glyph is rasterized once and packed into the atlas; every glyph already resident is a lookup. The frame is a batch of quads — backgrounds, then glyphs textured from the atlas — presented in a few draw calls (step 6, Chapter 4). The gutter shows a mark at the prompt line, uncoloured, because no command has run yet.

Typing and running a command

You type `cargo build`. Each key traverses the input path of Chapter 7: the window system delivers the event, the binding table is consulted and does not match, the encoder writes the character's UTF-8 byte to the PTY. The shell echoes it back, and the echo travels the whole read-scan-parse-draw path to appear on

screen — which is why Chapter 37 describes keystroke latency as bounded by the frame cadence rather than by per-character work.

You press Enter. The shell emits `OSC 133 ; C`, and the span from the `B` mark to this one is your command line — recovered from the grid rather than transmitted, which is why the recorded command is exactly what was on screen (Chapter 27). The build runs and prints. Its output is thousands of writes over several seconds; the loop reads them in batches and coalesces the damage, so the display shows one frame per refresh rather than one per write (Chapter 5). If a trigger's regex matches a completed line, it fires once and enters its cooldown (Chapter 32).

The build fails. The shell emits `OSC 133 ; D ; 1`, and the gutter mark for that prompt turns red (Chapter 27). `⌘⇧D` now selects the span from `C` to `D` — the whole of the build's output, exactly — and hands it to the ordinary clipboard path of Chapter 38. `⌘⇧` jumps to the previous prompt. The command and its exit code are in the Dashboard's history (Chapter 22) and reachable from the `⌘R` palette (Chapter 23).

Opening a full-screen application

You run your editor. It performs the handshake of Chapter 6: `CSI ? 1049 h` switches to the alternate screen, saving the primary and marking full damage because the visible contents are wholesale replaced (Chapter 5); `CSI ? 1 h` puts the cursor keys in application mode so arrows now send SS3 (Chapter 7); `CSI ? 25 l` hides the cursor while it paints; `CSI ? 1006 h` enables SGR mouse reporting, after which clicks are encoded as `ESC [ < b ; x ; y M` and delivered to the editor rather than selecting text — and `Shift` takes the mouse back for one action (Chapter 14). If the editor negotiates the Kitty keyboard protocol, it pushes flags with `CSI > u` and will pop them on exit (Chapters 7, 40).

While it runs it sets a scroll region with `CSI r` to keep its status line pinned, addresses rows inside that region under origin mode, and repaints between `CSI ? 2026 h` and `CSI ? 2026 l` so each update lands atomically instead of tearing (Chapters 3, 5). The alternate screen does not scroll and does not reflow: it is a fixed canvas the application owns.

Resizing mid-session

You drag the window wider. The window system reports the new size; the loop wakes. The new column and row counts are computed from the cell size, the grid is resized, and the primary screen *reflows* — logical lines that had wrapped are rejoined and re-wrapped to the new width (Chapter 3). Every OSC 133 mark survives, because marks anchor to logical lines rather than screen rows (Chapter 27). Full damage is marked, because every screen position may now hold different content, and the next frame redraws everything (Chapter 5).

The PTY's window size is set, the kernel delivers `SIGWINCH` to the foreground process (Chapter 35), and the editor repaints to the new dimensions. If `window.resize_increments` is on, the drag snapped to whole cells, so there is no partial row or column of pixels to account for (Chapter 9).

Quitting

You exit the editor. It reverses its handshake — pop the keyboard flags, disable mouse reporting, show the cursor, restore normal cursor keys, `CSI ? 1049 l` — and the primary screen returns exactly as it was, prompt marks and scrollbar intact. This round trip working is the reason Chapter 6 calls the support table a contract rather than bookkeeping.

`⌘Q` ends the process. Every window closes, every PTY it owns is closed, and each shell receives a hang-up (Chapter 35). The tmux client's socket drops — and the tmux *server* keeps running, because it is a separate process this was merely a client of, which is exactly why a tmux session survives and a native pane does not (Chapters 28, 31). The log at `~/.zterminal/zterminal.log` is removed unless `debug.persistent_`

`logging` is on (Chapter 17). Everything under `~/.zterminal` remains: the config, the layouts, the snapshots, the profiles, the harvested lists (Appendix E) — the environment, ready for the next launch.

Colophon

This book is generated from one source. `docs/book.md` is the manuscript; `scripts/book_pdf.sh` renders it to PDF through pandoc and LuaLaTeX with the project's own theme, and `scripts/book_epub.sh` renders the same manuscript — plus this apparatus — to a reflowable EPUB 3. The cover is not hand-placed either: `scripts/book_cover.sh` draws it at 1600×2560 from the same palette the PDF theme uses, so the cover on a storefront and the chapter headings inside the book cannot drift apart.

The display faces are Orbitron and Share Tech Mono; body text is set in Arimo. All three are licensed under the SIL Open Font License 1.1.

Every figure in this edition was measured against the program rather than recalled: the version, the test-attribute counts, the escape-sequence support levels, the default bindings, and the configuration fields. *Before You Begin* records the provenance — the zterminal commit the measurements were taken on, the `Cargo.toml` version (`v0.29.9`), the platform (macOS aarch64), and the `vte` release the parser figures came from. Where a number or a behaviour was provisional at the time of writing, the text marks it rather than rounding up, and Chapter 2 states plainly which parts of the program are inherited from the Alacritty lineage and which are original work.

Reproducing the Book's Claims

zterminal is proprietary MenkeTechnologies software and is **not** vendored into this repository, so the source-tree checks this book cites — the workspace test run in Chapter 63, the build pipeline in Chapter 64 — can only be run inside the zterminal repository itself, at the commit named in *Before You Begin*:

```
cargo test --workspace
```

Chapter 63 is explicit that this command does not reach every test attribute in the tree. `crates/ztmux-core` and `frontend/lib/zgui-core/crate` are separate workspaces consumed as path dependencies and are never visited; four attributes are gated to Windows and Linux and do not compile on macOS; and three attributes sit inside declarative macros and stand for far more tests than their count suggests. The chapter closes that arithmetic step by step, which is why the attribute count and the number of tests actually run are two different figures in the book rather than one.

The behavioural claims need no source access — only an installed zterminal. These are the checks behind the chapters most likely to be doubted.

The terminfo entry the emulator ships, and what it advertises (Chapter 40):

```
infocmp zterminal
```

The live configuration the running instance currently believes, read back over its own socket (Chapter 33):

```
zt msg get-config -w -1
```

Truecolor, probed at each layer of a nested session — the terminal itself, then the same probe over the connection, then inside the multiplexer, then inside the container. The first depth at which it fails is the layer that lost it (Chapter 46):

```
printf '\033[38;2;255;100;0mTRUECOLOR\033[0m\n'
ssh host "printf '\033[38;2;255;100;0mTRUECOLOR\033[0m\n'"
```

Appendix D records the support level for every escape sequence the parser answers, and Appendix I is a cookbook of sequences to send by hand; between them any claim in the escape-sequence chapters can be checked against the running emulator directly.

Also from MenkeTechnologies

zterminal is one program in a stack that is documented the same way — from source, with the numbers cited to the tools that produce them. Companion volumes cover the shell that runs inside this terminal, the multiplexer whose wire protocol it speaks, the bytecode VM the language frontends share, and the language ports built on it.

Titles and availability change; the current catalog is published at github.com/MenkeTechnologies.